

目录

1. 系统架构设计师-按照知识点导出历年真题（案例题）	2
2. UML 行为建模	2
3. 区块链架构	6
4. 架构评估方法	8
5. 微服务与云原生架构	27
6. 领域驱动设计	29
7. 类图与实体建模	32
8. 顺序图与交互建模	44
9. 业务流程与活动建模	46
10. 架构质量属性场景	51
11. 软件架构风格与模式	66
12. SOA 与企业应用集成	70
13. 分布式架构与协调机制	79
14. 云边端与智能架构	81
15. 可靠性与容错架构	101
16. 安全架构与威胁防护	104
17. 构件化与插件架构	109
18. 数据架构与数据治理	116
19. 数据库缓存策略	122

20. 分布式缓存与高并发	129
21. 关系数据库设计与规范化	150
22. 大数据处理与数据挖掘	152
23. 数据访问层与 ORM	158
24. 分布式搜索与 Elasticsearch	162
25. Web 表现层与 MVC	166
26. 响应式 Web 与多终端	168
27. Java Web 框架与应用服务器	170
28. 嵌入式开放式架构	176
29. 实时系统与 RTOS	178
30. 物联网与 CPS	183
31. 软件 FMEA 与故障诊断	186
32. 项目进度计划	192
33. 知识图谱与智能问答	195
34. Scrum 与敏捷项目管理	202

——

注：版权提醒：本资料已通过国家版权局登记（登记号：渝作登字-2025-A-00624260），（一旦发现资料恶意流出，直接封号不退款，后果严重的话会请求平台披露你的个人信息，并于互联网法院起诉，请珍惜账号权益和个人名誉，谢谢）。

系统架构设计师-按照知识点导出历年真题（案例题）

第1章——UML 行为建模

（2015年11月系统架构设计师真题-第2题）

阅读以下关于系统设计建模的说明，在答题纸上回答问题1至问题3。

【说明】

某公司拟研制一款高空监视无人直升机，该无人机采用遥控—自主复合型控制实现垂直升降。该直升机飞行控制系统由机上部分和地面部分组成，机上部分主要包括无线电传输设备、飞控计算机、导航设备等，地面部分包括遥控操纵设备、无线电传输设备以及地面综合控制计算机等。其主要工作原理是地面综合控制计算机负责发送相应指令，飞控计算机按照预定程序实现相应功能。经过需求分析，对该无人直升机控制系统纵向控制基本功能整理如下：

- （a）飞控计算机加电后，应完成系统初始化，飞机进入准备起飞状态；
- （b）在准备起飞状态中等待地面综合控制计算机发送起飞指令，飞控计算机接收到起飞指令后，进入垂直起飞状态；
- （c）垂直起飞过程中如果飞控计算机发现飞机飞行异常，飞行控制系统应转入无线电遥控飞行状态，地面综合控制计算机发送遥控指令；
- （d）垂直起飞达到预定起飞高度后，飞机应进入高度保持状态；
- （e）飞控计算机在收到地面综合控制计算机发送的目标高度后，飞机应进入垂直升降状态，接近目标高度；垂直升降过程中出现飞机飞行异常，控制系统应转入无线电遥控飞行；
- （f）飞机到达目标高度后，应进入高度保持状态，完成相应的任务；
- （g）飞机在接到地面综合控制计算机发送的任务执行结束指令后，进入飞机降落状态；
- （h）飞机降落过程中如果出现飞机飞行异常，控制系统应转入无线电遥控飞行；
- （i）飞机降落到指定着陆高度后，进入飞机着陆状态，应按照预定着陆算法，进行着陆；
- （j）无线电遥控飞行中，地面综合控制计算机发送着陆指令，飞机进入着陆状态，应按照预定着陆算法，进行着陆。

问题（1）

状态图和活动图是软件系统设计建模中常用的两种手段，请用200字以内文字简要说明状态图和活动图的含义及其区别。

思路解析

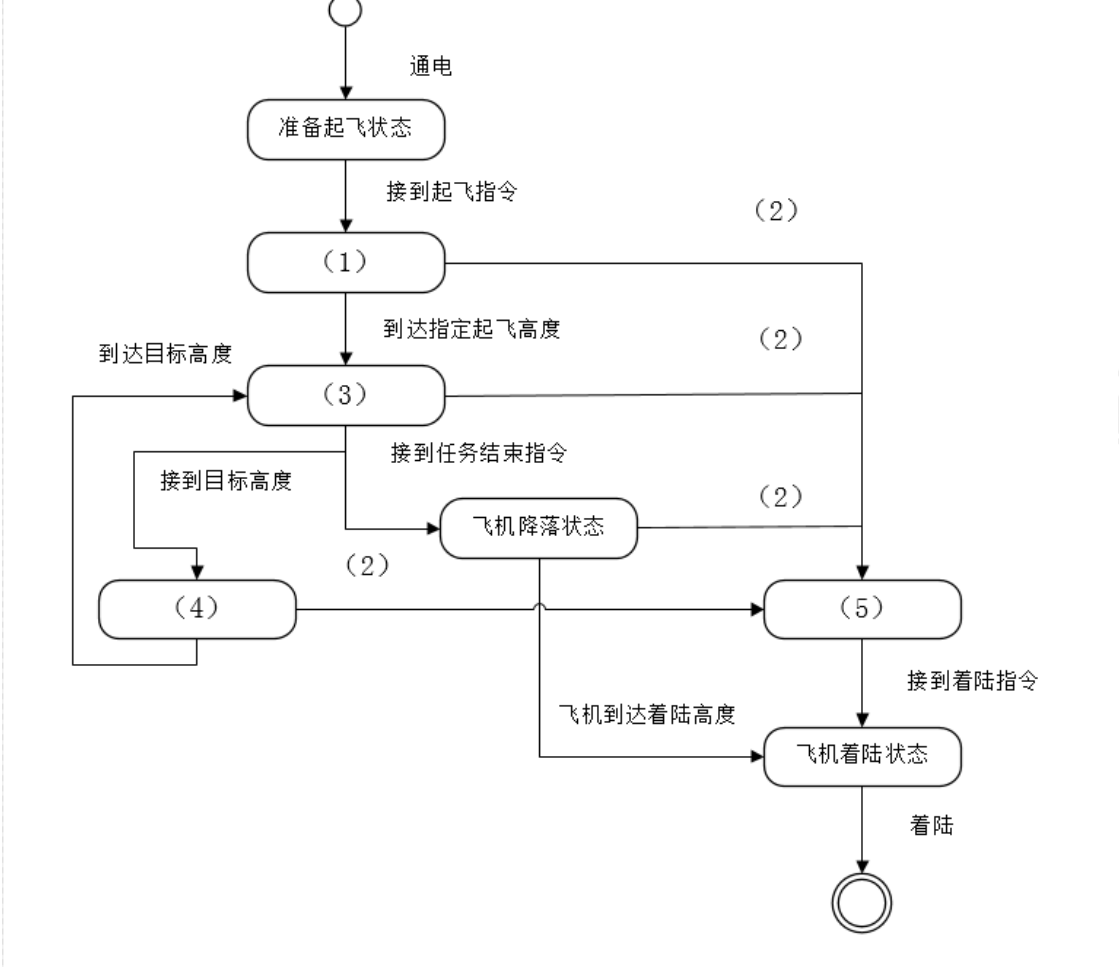
状态图（State Machine Diagram）关注的是某一特定对象（如“纵向控制子系统”或“飞控计算机中的高度保持控制器”）在整个生命周期内可能处于的离散状态及其之间的事件驱动转移，并在转移上标注守卫条件与动作（entry/do/exit/action）。其价值在于：当系统受到外部事件（如“收到起飞指令”）或内部事件（如“到达预定高度”）时，如何从当前稳定状态跃迁到下一稳定状态并执行相应动作（如“解锁油门、升力闭环接管”）。因此，状态图特别适合**协议、设备控制器、模式管理（如自主/遥控/告警）**等“状态主导”的对象建模。**关键点：离散状态、事件驱动、转移动作、守卫条件。**

活动图（Activity Diagram）则强调流程视角：从起点到终点的活动/动作流转，天然支持分叉与合并、并发与同步、泳道（责任主体）等结构，对于业务过程、用例执行轨迹、算法步骤的表达更直观。在活动图中，控制流/对象流决定执行先后，一个活动完成通常立即流向下一个活动，无需外部事件触发；同时可通过决策/合并/分叉/汇合结点表现条件分支与并发。因此，活动图更适合描述流程驱动的问题，例如“地面下发目标高度→飞控进入垂直升降→接近目标后转入高度保持→等待任务结束”。

本质差异在于：状态图=对象的结果性状态+事件触发的转移；活动图=动作序列+流程控制+并发。在无人机案例中，若要呈现模式切换（例如从“垂直起飞”到“高度保持”或“异常→R/C”），状态图更契合；若要呈现GCC与FCC的工作分工、指令下发与响应流程，泳道活动图更直观。

问题（2）

根据题干中描述的基本功能需求，架构师王工通过对需求的分析和总结给出了无人直升机控制系统纵向控制状态图（图2-1）。请根据题干描述，提炼出相应状态及条件，并完善图2-1所示状态图中的（1）~（5），将答案填写在答题纸中。



思路解析

本题要求依据（a）~（j）将纵向控制状态机补齐。梳理如下：

由（b）可知：**准备起飞**在收到**起飞指令**事件后进入**垂直起飞**，故（1）应为**垂直起飞状态**。此处是典型的**命令触发转移**，表示模式从地面准备切换至由FCC控制的起飞闭环。

关于（2），从（c）（e）（h）知：**垂直起飞/垂直升降/降落**中若发现飞机飞行异常（如传感器失真、姿态越界、功率不足），系统统一转入R/C，体现**故障—安全模式**的**异常分支条件**。因此（2）应为**飞机飞行异常**（或同义“飞行异常”）。将其抽象为**统一守卫条件**可减少图中重复，突出了“安全优先”的设计理念。

由（d）可知：**垂直起飞**达到**预定起飞高度**（条件满足）后，进入**高度保持**，因此（3）为**高度保持状态**。这是**目标达成触发**的模式稳定化，便于后续任务装订。

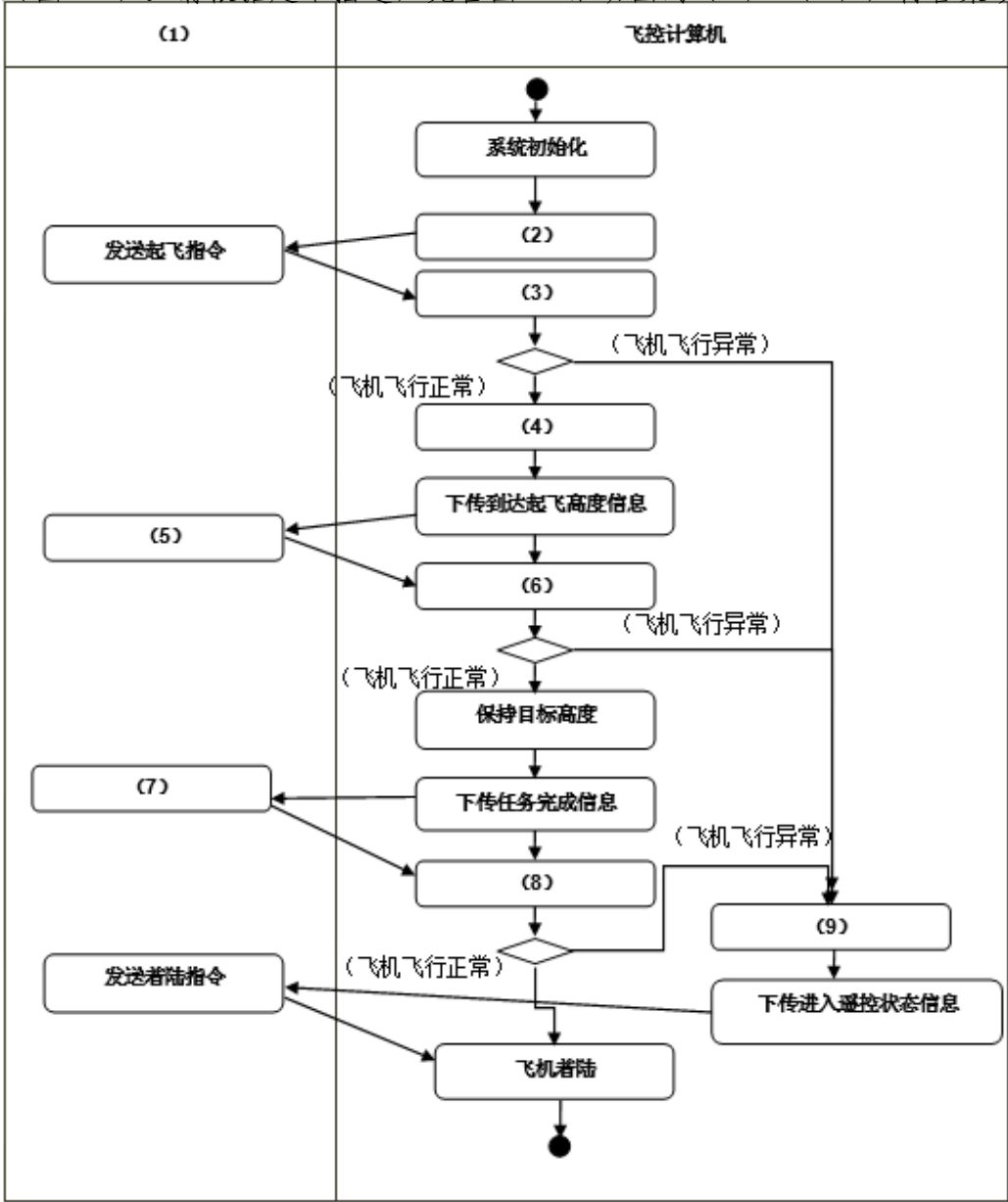
由（e）可知：在**高度保持**期间，一旦收到**目标高度**指令，应进入**垂直升降**逼近目标；因此（4）为**垂直升降状态**。这体现了**命令驱动—数值目标—连续控制**的链路。

对于（5），从（c）（e）（h）（j）串联可见：无论在哪个飞行阶段，一旦出现**飞行异常**，或在R/C中收到**着陆指令**，都汇聚到一个**统一的人回路遥控模式**。该模式下GCC直接下发控制量或终止命令，属于**安全冗余**。故（5）必须是**无线电遥控飞行状态**。

综上，（1）~（5）与题干陈述形成**完备的模式拓扑**：**准备起飞**→（**起飞指令**）→**垂直起飞**→（**达预定高度**）→**高度保持**→（**下发目标高度**）→**垂直升降**→（**到达目标**）→**高度保持**→（**任务结束**）→**降落**→（**达着陆高度**）→**着陆**；其间任一主流程状态在**异常**时均可**断点转入R/C**，并在R/C下可由**着陆指令**收束到**着陆**。

问题（3）

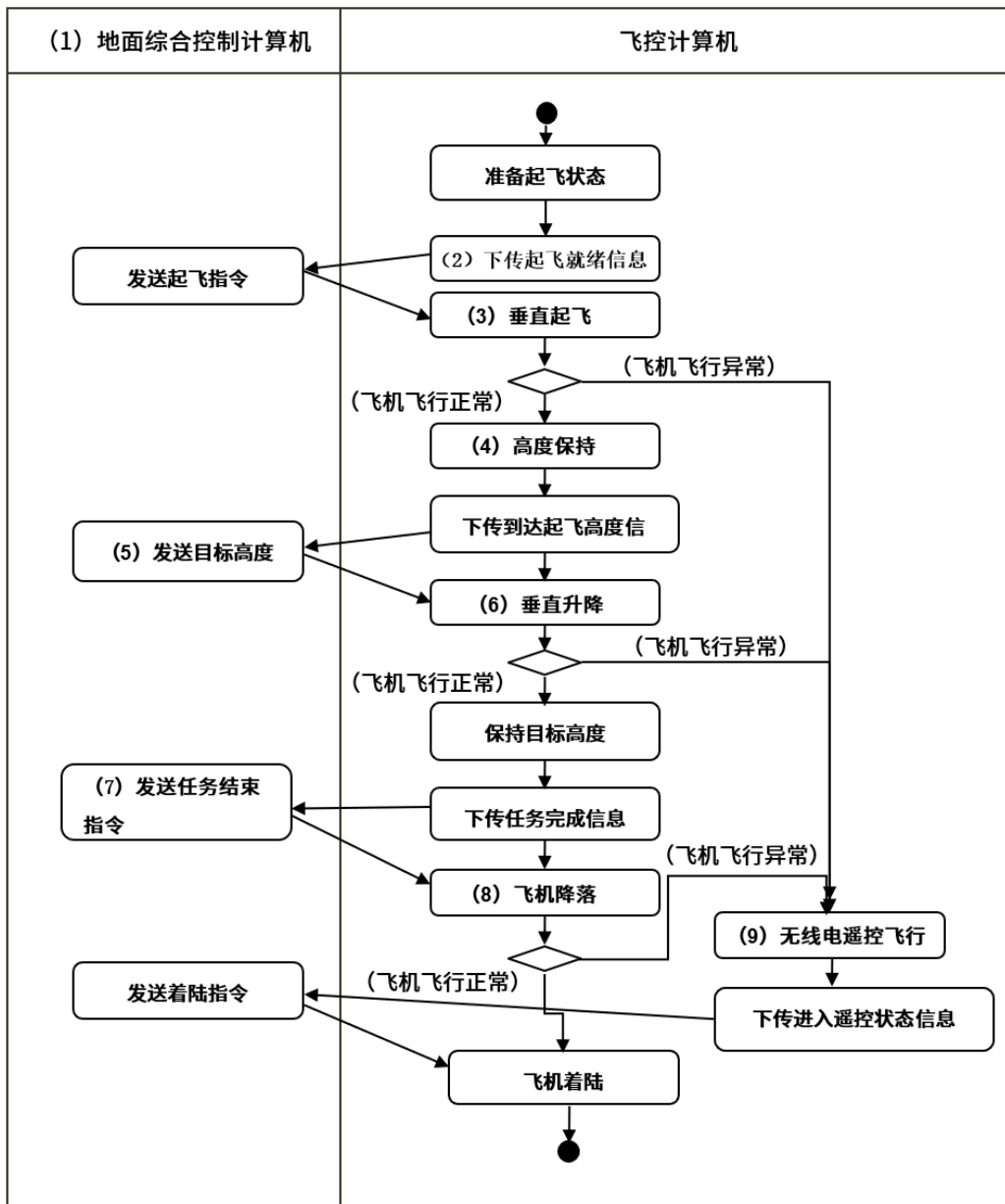
根据题目中描述的基本功能需求，架构师王工给出了无人直升机控制系统纵向控制的顶层活动图（图2-2）。请根据题干描述，完善图2-2活动图的（1）-（9），将答案填写在答题纸中



思路解析

顶层泳道活动图旨在刻画控制流程与责任主体。根据题干，主体至少包含GCC（地面综合控制计算机）与FCC/飞机两个泳道：前者负责指令生成与下发，后者负责飞行控制执行与状态反馈。据此：

- （1）应为地面综合控制计算机。在泳道图中，泳道=责任边界，用以划分谁发起或完成某个活动。
- 起始阶段（a）与（b）说明：FCC初始化完成并进入准备起飞，随后等待GCC的起飞指令。在流程上，常见的前置为地面完成联检/就绪确认，因此（2）填下传起飞就绪信息合理：它既可表示链路连通与状态同步，也为后续起飞指令提供业务前提。
- 飞机在收到起飞指令后执行（3）垂直起飞，到达预定起飞高度转入（4）高度保持，与（d）一致。
- 随后GCC会下发数值目标，即（5）发送目标高度；FCC据此进入（6）垂直升降逼近目标，与（e）吻合。
- 到达目标高度后再度高度保持并执行任务；任务完成由GCC触发（7）发送任务结束指令，FCC转入（8）飞机降落阶段，与（g）一致。
- 贯穿上述任何阶段，一旦检测到飞行异常，流程分叉至（9）无线电遥控飞行；在该泳道中，GCC可直接下发着陆指令（见（j）），流程以着陆收束。



答案-问题（1）

状态图主要用于描述一个对象在其生存期间的动态行为，表现一个对象所经历的状态序列，引起状态转移的事件（event），以及因状态转移而伴随的动作（action）。

活动图可以用于描述系统的工作流程和并发行为。活动图其实可看作状态图的特殊形式，活动图中一个活动结束后将立即进入下一个活动（在状态图中状态的转移可能需要事件的触发）。

两者最大的区别是：状态图侧重于描述行为的结果，而活动图侧重描述行为的动作。其次活动图可描述并发行为，而状态图不能。

答案-问题（2）

- (1) 垂直起飞状态
- (2) 飞机飞行异常
- (3) 高度保持状态
- (4) 垂直升降状态
- (5) 无线电遥控飞行状态

答案-问题（3）

- (1) 地面综合控制计算机

- (2) 下传起飞就绪信息
- (3) 垂直起飞
- (4) 高度保持
- (5) 发送目标高度
- (6) 垂直升降
- (7) 发送任务结束指令
- (8) 飞机降落
- (9) 无线电遥控飞行

第2章——区块链架构 (2025年5月系统架构设计师真题-第5题)

随着信息技术的发展，农产品溯源系统在保障食品安全、提升消费者信任方面发挥着越来越重要的作用。传统的农产品溯源系统在数据篡改防范、信任机制建立等方面存在一定的局限性。近年来，区块链技术因其去中心化、不可篡改、可追溯等特性，被广泛应用于农产品溯源领域。

某市农业信息化管理部门计划建设一个基于区块链的农产品全流程溯源平台，实现从农田到餐桌全过程的数据上链，提升农产品质量追溯的透明度和可信度。为此，项目团队需对区块链的技术架构和各个层次进行深入理解和设计。

问题（1）

请简要说明区块链技术的六个层次，并简要描述每一层的功能和作用。（12分）

思路解析

区块链技术体系通常分为六个层次，每一层都有独立又协同的功能。首先是**数据层**，它负责数据的结构定义与存储，是区块链的“地基”，包括区块、链式结构、时间戳、哈希函数、非对称加密、梅克尔树等技术，确保数据一旦写入就无法被篡改，具有高度可信度。

其次是**网络层**，这一层让区块链中的所有节点可以像“点对点通信网”一样自由交流。它采用P2P协议，实现数据广播、同步和节点发现，保证所有节点在没有中心服务器的情况下依然能高效协作。

接着是**共识层**，它定义了各个节点如何就交易达成一致，例如比特币的工作量证明（PoW）、以太坊的权益证明（PoS）、联盟链常用的PBFT等。这一层相当于“仲裁系统”，确保全网账本一致性，避免双重支付等问题。

激励层是吸引和管理参与者的关键，通过代币或交易费机制，奖励诚实节点、惩罚作恶行为。没有激励机制，很难维持一个长久运行的公有链系统。

合约层则是承载业务逻辑的地方，支持智能合约的编写和执行，让“合同自动履行”成为可能，从而提高效率，减少人为干预。它常见于以太坊等平台，推动了DeFi、NFT等生态的发展。

最后是**应用层**，这一层面面向终端用户，提供具体应用接口，比如数字钱包、供应链溯源系统、身份认证平台等。它将复杂的底层技术封装成易用的界面，是用户实际接触和感知区块链的入口。

问题（2）

该系统中涉及三类角色：信息填写人员、核对人员、审核人员。请简要说明三类人员在业务流程中的主要操作步骤与职责差异，字数控制在400字以内。

思路解析

在信息系统中，角色的划分不仅影响业务流程的清晰度和效率，也决定了数据质量与系统安全性。本题中出现的三类人员——信息填写人员、核对人员、审核人员，体现了“逐级验证、分层控制”的设计原则，是常见的多角色业务流程建模方式，广泛应用于政务平台、物流追踪、电子票据、农产品溯源等系统中。

首先，**信息填写人员**处于流程的起点，负责将原始业务数据录入系统，如农产品的品种、产地、采摘时间、运输批次等关键字段。他们面对的是第一手数据，其工作核心在于**完整性与时效性**，确保信息应填尽填、及时入库，是区块链“上链信息”的来源。

接下来，**核对人员**承担数据校验的任务，他们不再进行业务采集，而是复查信息填写人员提交的数据是否真实、合理、无误。例如是否存在时间冲突、是否缺项、字段值是否符合规范等。此环节的关键在于**准确性与一致性控制**，往往需要结合规则库或后台比对系统进行审核。

最后，**审核人员**作为数据流转前的“终审关口”，通常由有资质的监管角色或系统管理员担任，主要职责是对核对通过的数据做最终确认。一旦审核通过，数据即被写入区块链，变成不可篡改的信息。因此，该角色承担的是**权威性与责任确认**的任务。

问题（3）

介绍下什么是智能合约，并说明智能合约包含哪三方面

思路解析

智能合约（Smart Contract）是一种部署在区块链上的自动化协议程序，用于在特定条件被满足时自动执行预设的合约条款。这一概念最早由密码学家尼克·萨博提出，随着以太坊等平台的发展而广泛应用。与传统合同相比，智能合约不依赖中介或人工干预，一旦部署即不可更改，所有执行过程由区块链网络自动完成，确保了安全性与可信度。

智能合约的核心特点包括自动执行、去中心化、不可篡改和全网可验证。具体而言，只要合约设定的触发条件被满足，系统就会立即自动执行相关操作，无需等待或人为操作。同时，合约代码和执行记录对全体参与节点公开，增强了透明性和可追溯性。

为了实现这些功能，一个智能合约通常包含三方面的内容。第一是参与方的身份信息，也就是合同的发起人、接收方等身份标识，通常以区块链地址或公钥表示，用于明确操作权限。第二是合约执行规则，即程序逻辑部分，定义了“在什么条件下，执行什么操作”，这是合约自动运行的基础。第三是状态与结果记录，系统在每次合约执行后会更新状态数据并将结果写入区块链，确保信息无法被篡改且可供验证。

答案-问题（1）

区块链技术体系可分为以下六个层次，每一层分别承担特定的功能：

数据层 功能：负责区块链中数据的结构定义与存储，如区块结构、链式结构、时间戳、哈希函数、非对称加密和梅克尔树等。作用：确保数据的完整性、不可篡改性和可验证性。

网络层 功能：负责节点间的数据传播与通信，采用P2P网络协议。作用：构建一个去中心化、点对点的传输网络，支持节点发现、数据广播与同步等。

共识层 功能：定义各个节点之间如何就数据达成一致的机制，如PoW（工作量证明）、PoS（权益证明）、PBFT（拜占庭容错）等。作用：保障全网数据一致性，防止双重支付和分叉问题。

激励层 功能：设定经济激励机制，用于奖励参与维护区块链网络运行的节点。作用：吸引节点参与系统运行，并对不良行为起到制约作用。

合约层 功能：支持智能合约的定义与执行，允许用户设定自动执行的逻辑规则。作用：实现业务逻辑自动化执行，提高效率，减少人为干预。

应用层 功能：为用户提供各类应用接口和可视化工具，实现具体的业务功能。作用：承载具体的区块链应用，如农产品溯源、数字资产管理、身份认证等。

答案-问题（2）

在区块链溯源系统中，信息填写人员负责第一步的数据录入工作，主要采集并填写农产品在生产、加工、运输等环节的基础信息，确保数据完整、及时上传。核对人员在其后对录入的信息进行内容核实，包括检查数据是否准确、是否存在遗漏或格式错误，是系统的数据质量保障环节。审核人员作为最终把关者，负责对核对通过的数据进行审批确认，一旦审核通过，信息将被正式写入区块链

，不可篡改，确保数据的权威性与可追溯性。三类人员层层把关，构建了一个完整的数据采集与验证闭环。

答案-问题（3）

智能合约是一种运行在区块链上的自动化程序，用于在满足预设条件时自动执行合同条款。它不依赖第三方，具有去中心化、不可篡改和自动执行的特点，广泛应用于数字资产转账、供应链管理等场景。

智能合约主要包含以下三方面内容：1. **参与方身份信息**：包括合同各方的地址、公钥等。2. **合约执行规则**：明确合同执行的条件、逻辑和触发机制。3. **状态与结果记录**：记录合同执行状态、数据变更及最终结果，确保全网可验证与追溯。

第3章——架构评估方法 (2022年11月系统架构设计师真题-第1题)

阅读以下关于软件架构设计与评估的叙述，在答题纸上回答问题1和问题2。

【说明】

某电子商务公司拟升级其会员与促销管理系统，向用户提供个性化服务，提高用户的粘性。在项目立项之初，公司领导层一致认为本次升级的主要目标是提升会员管理方式的灵活性，由于当前用户规模不大，业务也相对简单，系统性能方面不做过多考虑。新系统除了保持现有的四级固定会员制度外，还需要根据用户的消费金额、偏好、重复性等相关特征动态调整商品的折扣力度，并支持在特定的活动周期内主动筛选与活动主题高度相关的用户集合，提供个性化的打折促销活动。

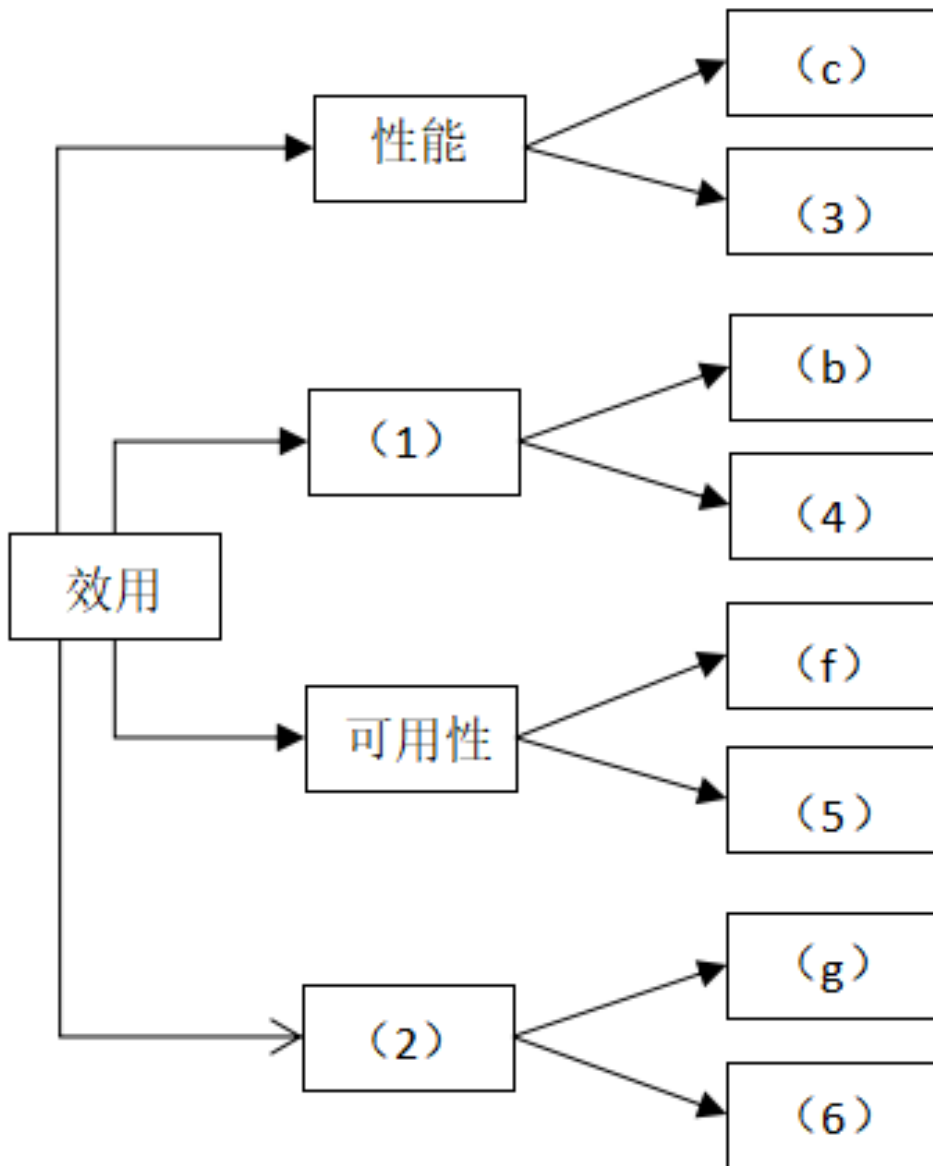
在需求分析与架构设计阶段，公司提出的需求和质量属性描述如下：

- (a) 管理员能够在页面上灵活设置折扣力度规则和促销活动逻辑，设置后即可生效；
- (b) 系统应该具备完整的安全防护措施，支持对恶意攻击行为进行检测与报警；
- (c) 在正常负载情况下，系统应在0.3秒内对用户的界面操作请求进行响应；
- (d) 用户名是系统唯一标识，要求以字母开头，由数字和字母组合而成，长度不少于6个字符；
- (e) 在正常负载情况下，用户支付商品费用后在3秒内确认订单支付信息；
- (f) 系统主站点电力中断后，应在5秒内将请求重定向到备用站点；
- (g) 系统支持横向存储扩展，要求在2人天内完成所有的扩展与测试工作；
- (h) 系统宕机后，需要在10秒内感知错误，并自动启动热备份系统；
- (i) 系统需要内置接口函数，支持开发团队进行功能调试与系统诊断；
- (j) 系统需要为所有的用户操作行为进行详细记录，便于后期查阅与审计；
- (k) 支持对系统的外观进行调整和配置，调整工作需要4人天内完成。

在对系统需求、质量属性描述和架构特性进行分析的基础上，系统架构师给出了两种候选的架构设计方案，公司目前正在组织相关专家对系统架构进行评估。

问题（1）

在架构评估过程中，质量属性效用树（utility tree）是对系统质量属性进行识别和优先级排序的重要工具。请将合适的质量属性名称填入图1-1中(1)、(2)空白处，并选择题干描述的(a)~(k)填入(3)~(6)空白处，完成该系统的效用树。



思路解析

质量属性效用树的设计关键在于根据系统需求，将用户关注的质量属性进行分解、映射，并结合优先级进行结构化展现。

(1) **安全性**：与系统的安全防护、审计相关。题干中 (b) 描述的是对恶意攻击的检测，(j) 描述的是用户行为日志，两者均体现对系统运行过程中的可追踪、可检测能力，属于安全性范畴。

(2) **可修改性**：指系统对变更（尤其是业务逻辑变更）的适应能力。(g) 描述的是系统支持扩展的能力、(k) 涉及界面调整。

而对应的需求项具体分析如下：

(a) 涉及规则配置，界面上操作，归属于易用性较为合适，说是一个功能性需求也可以。

(e) 为性能属性的体现，用户支付后3秒内完成确认，对系统响应的时效性有明确要求。

(j) 用户行为日志记录，归属安全性中的审计能力，保证系统运行的透明度与可追责。

(h) 系统宕机后的自动恢复体现了**可用性**属性中的故障恢复能力。

(k) 界面调整配置支持，明确限定了人力工时范围，体现了系统在界面层的可配置性，归属于**可修改性**。

问题 (2)

针对该系统的功能，李工建议采用面向对象的架构风格，将折扣力度计算和用户筛选分别封装为独立对象，通过对象调用实现对应的功能；王工则建议采用解释器 (interpreters) 架构风格，将折

扣力度计算和用户筛选条件封装为独立的规则，通过解释规则实现对应的功能。请针对系统的主要功能，从折扣规则的可修改性、个性化折扣定义灵活性和系统性能三个方面对这两种架构风格进行比较与分析，并指出该系统更适合采用哪种架构风格。

思路解析

本系统的设计目标是实现**促销规则的灵活配置与快速响应业务变更**，并不是以性能为首要目标。因此应优先考虑系统的灵活性与可维护性。

解释器架构风格通过将业务规则从代码逻辑中解耦，支持业务方独立制定与调整规则，匹配该系统对“灵活配置”“个性化促销”的核心需求。

尽管解释器在性能上略逊，但在用户量不大的初期阶段性能影响可控，而在后续可通过缓存、中间代码等技术手段优化性能。

综合评估，解释器架构风格更符合该系统的核心诉求。

答案-问题（1）

- （1）安全性
- （2）可修改性
- （3）（e）
- （4）（j）
- （5）（h）
- （6）（k）

答案-问题（2）

应该选择解释器架构风格。

折扣规则的可修改性：解释器更强。规则以外部化的DSL/规则表/表达式存储，运行时解析，**无需改动业务类与编译部署**，可快速上线与灰度；面向对象通常需要**修改类层级或策略集合**并重新发布，修改面相对代码、耦合更深。

个性化折扣定义灵活性：解释器更强。可**组合条件、权重、时效、冲突优先级**等形成千人千面的规则集；面向对象虽可用策略/装饰/责任链等模式组合，但**变体爆炸与组合复杂度**上升，长期维护成本高。

系统性能：面向对象通常优于解释器。解释器涉及**解析/解释执行/动态绑定**带来额外开销；面向对象的**静态分派与固定流程**更轻量。

（2021年11月系统架构设计师真题-第1题）

阅读以下关于软件架构设计与评估的叙述，在答题纸上回答问题1和问题2。

【说明】

某公司拟开发一套机器学习应用开发平台，支持用户使用浏览器在线进行基于机器学习的智能应用开发活动。

该平台的核心应用场景是用户通过拖拽算法组件灵活定义机器学习流程，采用自助方式进行智能应用设计、实现与部署，并可以开发新算法组件加入平台中。在需求分析与架构设计阶段，公司提出的需求和质量属性描述如下：

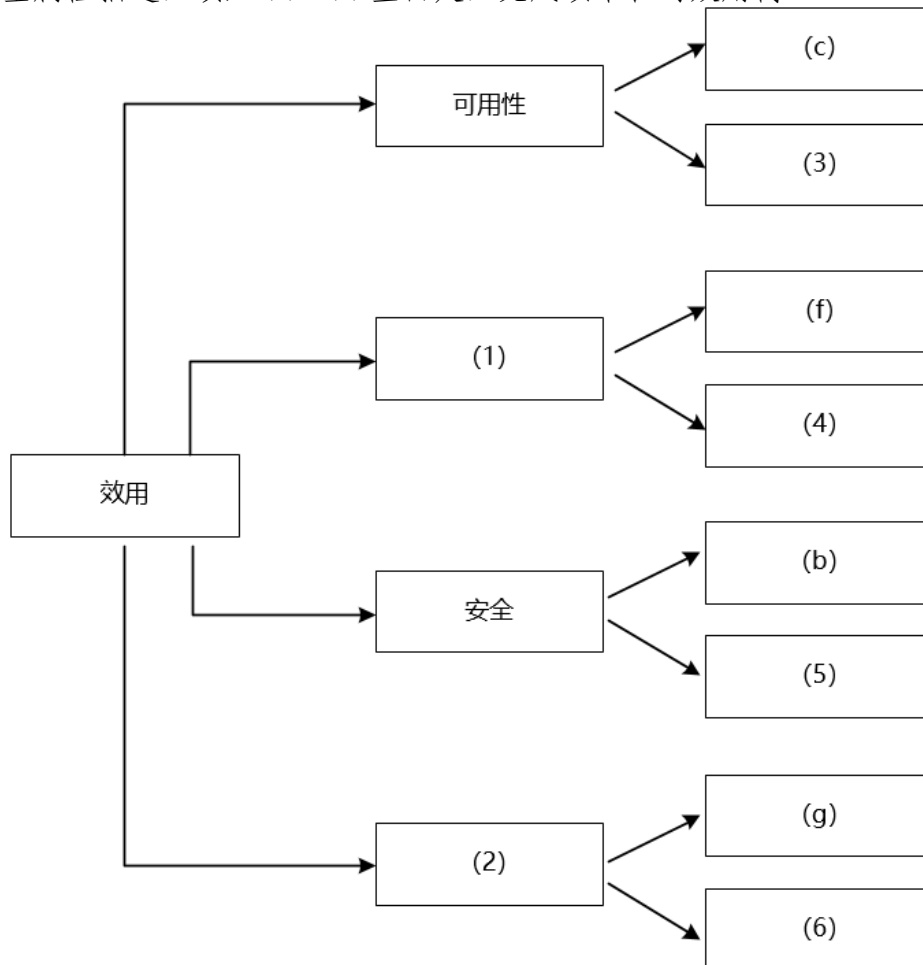
- （a）平台用户分为算法工程师、软件工程师和管理员等三种角色，不同角色的功能界面有所不同；
- （b）平台应该具备数据库保护措施，能够预防核心数据库被非授权用户访问；
- （c）平台支持分布式部署，当主站点断电后，应在20秒内将请求重定向到备用站点；
- （d）平台支持初学者和高级用户两种界面操作模式，用户可以根据自己的情况灵活选择合适的模式；

- (e) 平台主站点宕机后，需要在15秒内发现错误并启用备用系统；
- (f) 在正常负载情况下，机器学习流程从提交到开始执行，时间间隔不大于5秒；
- (g) 平台支持硬件扩容与升级，能够在3人天内完成所有部署与测试工作；
- (h) 平台需要对用户的所有操作过程进行详细记录，便于审计工作；
- (i) 平台部署后，针对界面风格的修改需要在3人天内完成；
- (j) 在正常负载情况下，平台应在0.5秒内对用户的界面操作请求进行响应；
- (k) 平台应该与目前国内外主流的机器学习应用开发平台的界面风格保持一致；
- (l) 平台提供机器学习算法的远程调试功能，支持算法工程师进行远程调试。

在对平台需求、质量属性描述和架构特性进行分析的基础上，公司的架构师给出了三种候选的架构设计方案，公司目前正在组织相关专家对平台架构进行评估。架构进行评估。

问题（1）

在架构评估过程中，质量属性效用树（utility tree）是对系统质量属性进行识别和优先级排序的重要工具。 请将合适的质量属性名称填入图1-1中(1)、(2)空白处，并从题干中的(a)–(l)中选择合适的质量属性描述，填入(3)–(6)空白处，完成该平台的效用树。



思路解析

质量效用树是一种在架构评估中广泛应用的工具，它的核心作用是将抽象的质量属性分解为具体的场景，并进一步细化为需求描述，以便于优先级排序和评估。

在软件体系结构中，常见的质量属性包括 **性能、可用性、安全性和可修改性**。

根据题干提供的需求：(e) 强调“主站点宕机后需要在 15 秒内发现错误并启用备用系统”，这是对 **系统故障恢复时间** 的要求，典型属于 **可用性**；(j) 强调“在正常负载情况下，平台应在 0.5 秒内对用户界面操作请求进行响应”，这是系统 **响应时间** 的要求，属于 **性能**；(h) 要求“对用户所有操作进行详细记录以便审计”，这是 **安全性** 的具体要求；(i) 要求“界面风格修改需要在 3 人·天内完成”，这是针对 **系统修改代价与效率** 的限定，属于 **可修改性**。

因此，质量效用树中（1）应为 **性能**，（2）应为 **可修改性**。在下层分支中，填入对应的需求描述即可。

问题（2）

针对该系统的功能，赵工建议采用解释器（interpreter）架构风格，李工建议采用管道过滤器（pipe-and-filter）的架构风格，王工则建议采用隐式调用（implicit invocation）架构风格。请针对平台的核心应用场景，从机器学习流程定义的灵活性和学习算法的可扩展性两个方面对三种架构风格进行对比与分析，并指出该平台更适合采用哪种架构风格。

思路解析

平台的核心应用场景是“用户通过拖拽算法组件，灵活定义机器学习流程”，并要求能够 **支持新算法组件的加入**。因此，架构风格的选择必须围绕“流程灵活性”和“算法扩展性”展开。

解释器风格：解释器风格通常通过定义一套规则语言与解释引擎，将业务逻辑以规则或脚本的方式表达，并由解释器运行。这意味着用户可以通过配置或脚本定义不同的流程，而无需更改系统核心代码。对于本平台，用户拖拽形成的机器学习流程可以直接映射为流程脚本，由解释器执行。这样既保证了流程定义的高度灵活性，又支持通过增加规则扩展新的算法组件。因此，解释器风格在本场景下具备最高的适配性。

管道过滤器风格：管道过滤器模型强调将数据处理分解为若干过滤器，并通过管道依次传递数据。这种架构在 **数据流固定、流程稳定** 的场景中效率很高，例如编译器的词法分析-语法分析-语义分析。但在本题场景下，用户需要灵活调整流程，例如“先做特征选择还是先做数据清洗”，这就意味着管道结构必须频繁重构。而且扩展新算法组件往往需要重新实现新的过滤器，并修改现有的管道。因此，该风格在本场景下的灵活性与扩展性不足。

隐式调用风格：隐式调用通过 **事件机制** 实现对象间解耦。用户操作触发事件，系统中已注册的算法组件或功能模块响应事件完成工作。这种模式确实能在一定程度上支持灵活性，但它更适合于“消息驱动”的场景，而非“全局流程定义”。在机器学习流程的建模中，用户希望以“整体流程图”的形式掌控执行逻辑，而隐式调用则缺乏这样的全局可控性。因此其灵活性与扩展性虽优于管道过滤器，但不及解释器。

综上所述，结合平台需求，**解释器风格能够更好地支持用户自助式流程定义和算法扩展**，因此是最佳的架构选择。

答案-问题（1）

- （1）性能
- （2）可修改性
- （3）（e）可用性
- （4）（j）性能
- （5）（h）安全性
- （6）（i）可修改

答案-问题（2）

解释器：机器学习流程定义的灵活性高，可扩展能力强，因为解释器风格可以通过自定义流程规则及配套流程解释引擎开发，做到用户层面的流程完全定义，而不需要修改代码，所以无论是修改已有的业务流程，还是要扩展不同的角色，创建新角色的流程都非常便利。

管道过滤器：机器学习流程定义的灵活性较低，可扩展能力较弱，因为管道过滤器是把数据处理职能做成过滤器，把数据传递做成管道，此时如果流程不发生变化，是可以通过这种方式实现的，但一旦流程变化，或是扩展功能，需要对过滤器进行修改调整，或是流程在程序层面重建，此时必须修改代码完成任务。

隐式调用：机器学习流程定义的灵活性一般，可扩展能力一般，隐式调用强调的是通过间接方式进

行调用，如采用事件机制，要完成某个动作时先触发事件，事件与相关动作关联，以提升灵活度，本题中可把角色执行业务的流程用事件触发。这种做法比管道过滤器强，但弱于完全自定义的解释器。

经过综合比较与分析，可以看出该系统更适合使用解释器风格。

(2020年11月系统架构设计师真题-第1题)

阅读以下关于软件架构设计与评估的叙述，在答题纸上回答问题1和问题2。

[说明]

某公司拟开发一套在线软件开发系统，支持用户通过浏览器在线进行软件开发活动。该系统的重要功能包括代码编辑、语法高亮显示、代码编译、系统调试、代码仓库管理等。在需求分析与架构设计阶段，公司提出的需求和质量属性描述如下：

- a) 根据用户的付费情况对用户进行分类，并根据类别提供相应的开发功能；
- b) 在正常负载情况下，系统应该在0.2s内对用户的界面操作请求进行响应；
- c) 系统应该具备完善的安全防护措施，能够对黑客的攻击行为进行检测和防御；
- d) 系统站点断电后，应在3s内将请求重定向到备用站点；
- e) 系统支持中文昵称，但用户名必须以字母开头，长度不少于8个字符；
- f) 系统宕机后，需要在15s内发现错误并启用备用系统；
- g) 在正常负载情况下，用户的代码提交请求应在0.5s内完成；
- h) 系统支持硬件设备灵活扩容，应保证在2人天内完成所有的部署与测试工作；
- i) 系统需要针对代码仓库的所有操作进行详细记录，便于后期查阅与审计；
- j) 更改系统web界面风格需要在4人天内完成；
- k) 系统本身需要提供远程调试接口，支持开发团队进行远程排错；

在对系统需求、质量属性和架构特性进行分析的基础上，该公司的系统架构师给出了两种候选的架构设计方案，公司目前正在组织相关专家对候选系统架构进行评估。

问题（1）

针对该系统的功能，李工建议采用管道过滤器 (pipe and filter) 的架构风格，而王工则建议采用仓库 (repository) 架构风格。请指出该系统更适合采用哪种架构风格，并针对系统的主要功能，从数据处理方式、系统的可扩展性和处理性能三个方面对这两种架构风格进行比较与分析，填写表1-1中的(1)~(4)空白处。

架构风格名称	数据处理方式	系统拓展方式	处理性能
管道-过滤器	数据驱动机制，处理流程事先确定，交互性差	(2)	劣势: 需要数据格式转换，性能降低 优势: 支持过滤器并发调用，性能提高
仓库	(1)	数据与处理解耦合，可动态添加和删除处理组件	劣势: (3) 优势: (4)

思路解析

从系统的核心功能看：代码编辑/语法高亮/编译/调试/代码仓库管理，本质是围绕中心代码与元数据开展的一系列交互式活动，强调“多工具围绕同一份项目/仓库数据的协同与一致性”。

仓库架构将系统关键数据（项目、版本、提交、构建产物、调试会话元数据等）集中在统一的数据仓库中，周边功能组件（编辑器服务、编译服务、调试服务、权限与审计服务等）通过标准接口访问仓库：这使得（1）数据处理以数据为中心、流程彼此独立，天然支持交互式与回溯操作；（2）可扩展性强：新增语法检查器、静态分析、SAST/DAST、安全审计、远程调试探针等，只需对接仓库接口与事件流即可，最小化对既有流程的侵扰；（3）一致性与可追溯：所有操作统一落库，便于实现 a) 差异视图、b) 多版本管理、c) 细粒度权限与全面审计（i），与题干的安全合规需求高度贴合。

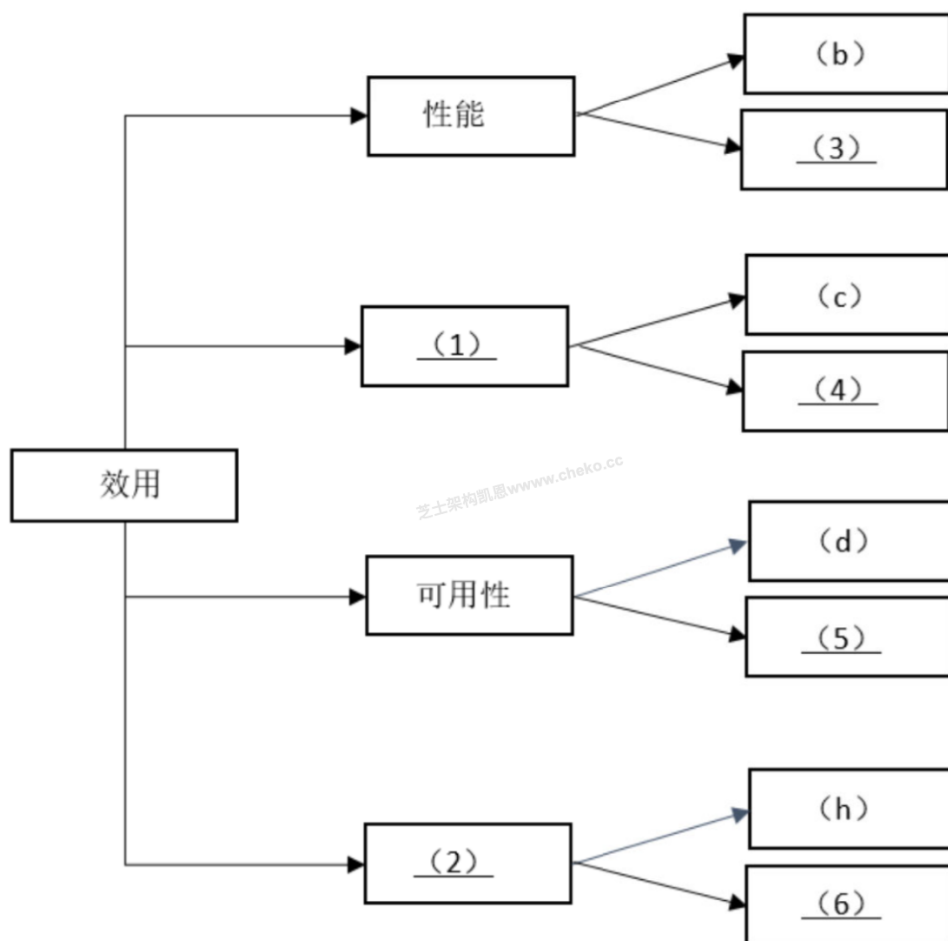
与之对比，管道过滤器强调固定顺序的数据流：输入沿若干过滤器经管道依次处理，适合数据加工型、批处理/流式场景（例如编译器前端流水线、日志清洗等）。

但本题场景多为人机交互、随机访问、频繁回退/跳转的模式，流程不是严格线性的；当要改变步骤或插拔新工具时，常需重排管道或重启（对应表中（2）），对已有会话与中间态处理不友好。性能层面，仓库风格因组件围绕共享数据工作，若需要跨组件计算则可能出现多次加载/反序列化，数据与处理分离，需要加载数据，性能降低（对应（3））；仓库架构无强依赖、可并发/并行，对吞吐友好（对应（4））。

综合“在线 IDE + 代码仓库管理 + 审计 + 远程调试（k）”的需求权重，仓库架构更契合系统形态，同时更容易与权限模型、审计日志、分支/标签/提交等领域概念对齐，后续叠加消息总线/事件溯源可进一步提升扩展性与可观测性。

问题（2）

在架构评估过程中，质量属性效用树(utilitytree)是对系统质量属性进行识别和优先级排序的重要工具。请将合适的质量属性名称填入图1-1中（1）、（2）空白处，并选择题干描述的（a）~（k）填入（3）~（6）空白处，完成该系统的效用树。



#443px #396px

思路解析

质量属性效用树用于把抽象的质量关注点分解为可度量的场景并排序。

在线开发系统的关键非功能关注通常围绕：性能、可用性、安全性、可修改性四大类。

题干中：（b）（g）（k）等体现性能/响应性与接口能力，（d）（f）体现可用性/恢复性（RTO、切换时间），（c）（i）体现安全性/审计（攻击检测、操作留痕），（h）（j）（e）体现可修改性/可演进性/可维护性（硬件扩容与部署时限、界面风格变更、用户名规则）。

据此，在效用树的顶层，我们将（1）放置为安全性，因为该系统承载代码与仓库等高价值资产，需优先考虑攻击检测/防御与全量审计（i）；（2）为可修改性，以支撑频繁的界面与工具链演进

(j、h)。

在叶子层，选择能代表典型质量场景且可度量的项：(3)填 g，它为“0.5s 提交”的性能场景，指导架构在存储、缓存、并发控制、事务边界上制定明确预算；(4)填 i，它对代码仓库的全操作审计提出要求，导出不可抵赖、可追踪的日志与合规模型；(5)填 f，规定15s 内故障检测与切换，明确健康探测、心跳、HA/故障转移与观测链路；(6)填 j，以4 人·天为上限，约束前端组件化、主题切换机制与样式隔离设计。

答案-问题(1)

应该采用仓库风格。

表(1) - (4)空的空白分别为：

- (1) 数据存储在中心仓库，处理流程独立，支持交互式处理。
- (2) 数据与处理紧密关联，调整处理流程需要系统重新启动。
- (3) 数据与处理分离，需要加载数据，性能降低。
- (4) 数据处理组件之间一般无依赖关系，可并发调用，提高性能。

答案-问题(2)

- (1) 安全性
- (2) 可修改性
- (3) g
- (4) i
- (5) f
- (6) j

(2019年11月系统架构设计师真题-第1题)

阅读以下关于软件架构设计与评估的叙述，在答题纸上回答问题1和问题2。

【说明】

某电子商务公司为了更好地管理用户，提升企业销售业绩，拟开发一套用户管理系统。该系统的基本功能是根据用户的消费级别、消费历史、信用情况等指标将用户划分为不同的等级，并针对不同等级的用户提供相应的折扣方案。在需求分析与架构设计阶段，电子商务公司提出的需求、质量属性描述和架构特性如下：

- (a) 用户目前分为普通用户、银卡用户、金卡用户和白金用户四个等级，后续需要能够根据消费情况进行动态调整；
- (b) 系统应该具备完善的安全防护措施，能够对黑客的攻击行为进行检测与防御；
- (c) 在正常负载情况下，系统应在0.5秒内对用户的商品查询请求进行响应；
- (d) 在各种节假日或公司活动中，针对所有级别用户，系统均能够根据用户实时的消费情况动态调整折扣力度；
- (e) 系统主站点断电后，应在5秒内将请求重定向到备用站点；
- (f) 系统支持中文昵称，但用户名要求必须以字母开头，长度不少于8个字符；
- (g) 当系统发生网络失效后，需要在15秒内发现错误并启用备用网络；
- (h) 系统在展示商品的实时视频时，需要保证视频画面具有1024×768像素的分辨率，40帧/秒的速率；
- (i) 系统要扩容时，应保证在10人月内完成所有的部署与测试工作；
- (j) 系统应对用户信息数据库的所有操作都进行完整记录；
- (k) 更改系统的Web界面接口必须在4人周内完成；
- (l) 系统必须提供远程调试接口，并支持远程调试。

在对系统需求、质量属性描述和架构特性进行分析的基础上，该系统架构师给出了两种候选的架构

设计方案，公司目前正在组织相关专家对系统架构进行评估。

问题（1）

针对用户级别与折扣规则管理功能的架构设计问题，李工建议采用面向对象的架构风格，而王工则建议采用基于规则的架构风格。请指出该系统更适合采用哪种架构风格，并从用户级别、折扣规则定义的灵活性、可扩展性和性能三个方面对这两种架构风格进行比较与分析，填写表1-1中的

（1）~（3）空白处。

表 1-1 两种架构风格的比较与分析

架构风格名称	灵活性	可扩展性	性能
面对对象	将用户级别、折扣规则等封装为对象，在系统启动时加载	(2)	(3)
基于规则	(1)	加入新的用户级别和折扣规则时只需要定义新的规则，解释规则即可进行扩展	需要对用户级别与折扣规则进行实时解释、性能较差

思路解析

该问题考察不同架构风格对**复杂业务规则管理**的适应性。面向对象（OO）架构将用户等级和折扣策略直接建模为类与对象，逻辑直观、性能较好，但存在缺陷：当新增或修改等级与规则时，往往需要扩展类层次、改动源代码，甚至重启系统重新部署，缺乏灵活性。基于规则的架构则将规则抽象为可配置的数据（如规则库、条件-动作对），通过规则引擎在运行时动态加载与匹配，能根据用户实时消费情况自动调整折扣，不需要频繁修改核心代码。结合题干中关于“等级动态调整”“节假日实时折扣调整”等场景，显然基于规则架构更契合。

从三个维度比较：**灵活性**上，规则架构能动态增删改规则，而 OO 架构受制于类定义；**可扩展性**上，规则架构可以轻松支持未来新增等级和优惠策略，而 OO 架构则需要重构；**性能**上，OO 架构因逻辑直接固化在代码中，运行效率高，而规则架构需额外解析规则、查找匹配，性能稍逊，但可通过规则优化、缓存策略缓解。综合来看，本系统属于“用户分级 + 折扣策略”高度动态的场景，**应优先采用基于规则的架构风格**，并在实现中结合 OO 管理基础实体，兼顾灵活性与性能。

问题（2）

在架构评估过程中，质量属性效用树（utility tree）是对系统质量属性进行识别和优先级排序的重要工具。请将合适的质量属性名称填入图1-1中（1）、（2）空白处，并选择题干描述的

（a）~（1）填入（3）~（6）空白处，完成该系统的效用树。

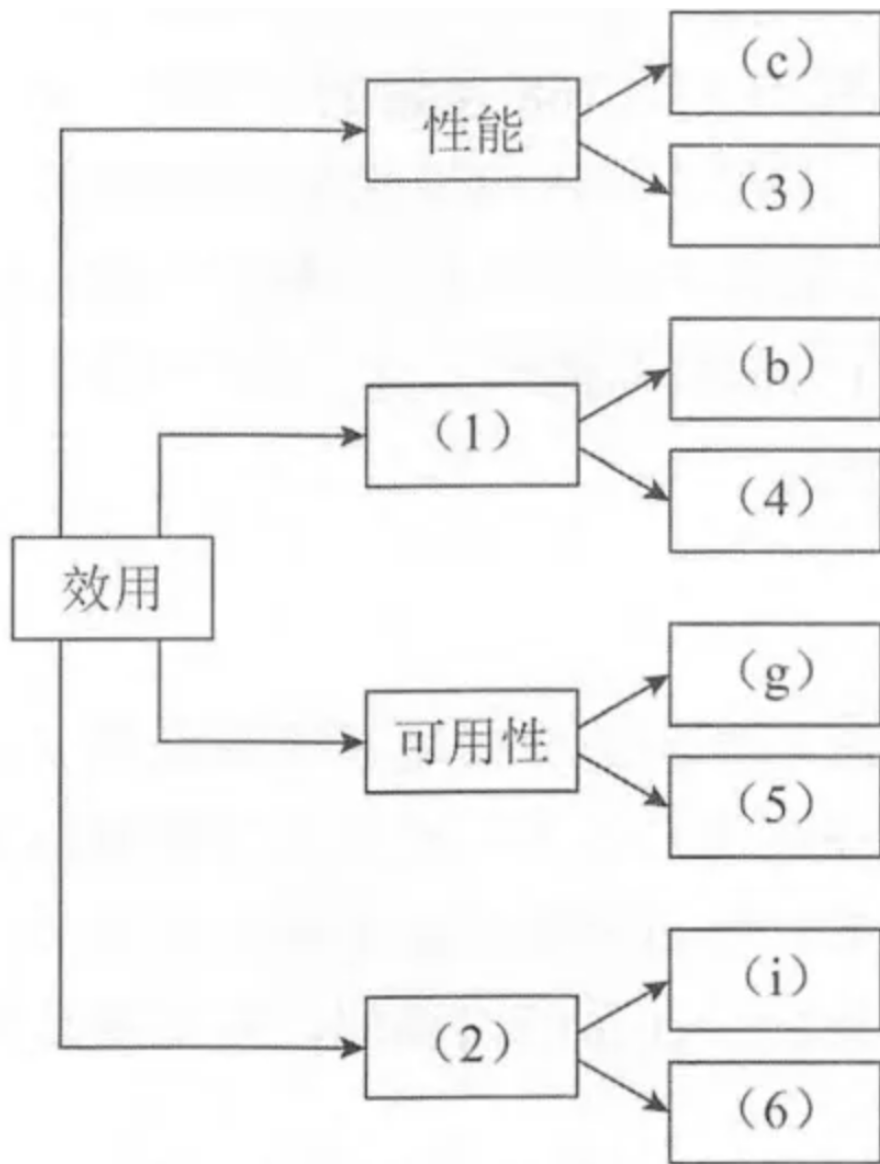


图 1-1 会员管理系统效用树

思路解析

题干中涉及的属性可归纳如下：

安全性包括 (b) 安全防护与 (j) 操作记录；

性能包括 (c) 查询响应、(h) 视频分辨率与帧率；

可用性包括 (e) 站点切换、(g) 备用网络启用；

可修改性包括 (i) 扩容时间、(k) 界面修改时限。

根据图 1-1 的空缺，填入 (1) 安全性、(2) 可修改性，并选择 (3) (h)、(4) (j)、(5) (e)、(6) (k) 即可。

答案-问题 (1)

用户级别与折扣规则管理功能更适合采用基于规则的架构风格。

(1) 将用户级别、折扣规则等描述为可动态改变的规则数据；

(2) 加入新的用户级别和折扣规则时需要重新定义新的对象，并需要重启系统；

(3) 用户级别和折扣规则已经在系统内编码，可直接运行，性能较好。

答案-问题 (2)

(1) 安全性

- (2) 可修改性
- (3) (h)
- (4) (j)
- (5) (e)
- (6) (k)

(2017年11月系统架构设计师真题-第1题)

阅读以下关于软件架构评估的叙述，在答题纸上回答问题1和问题2。

【说明】

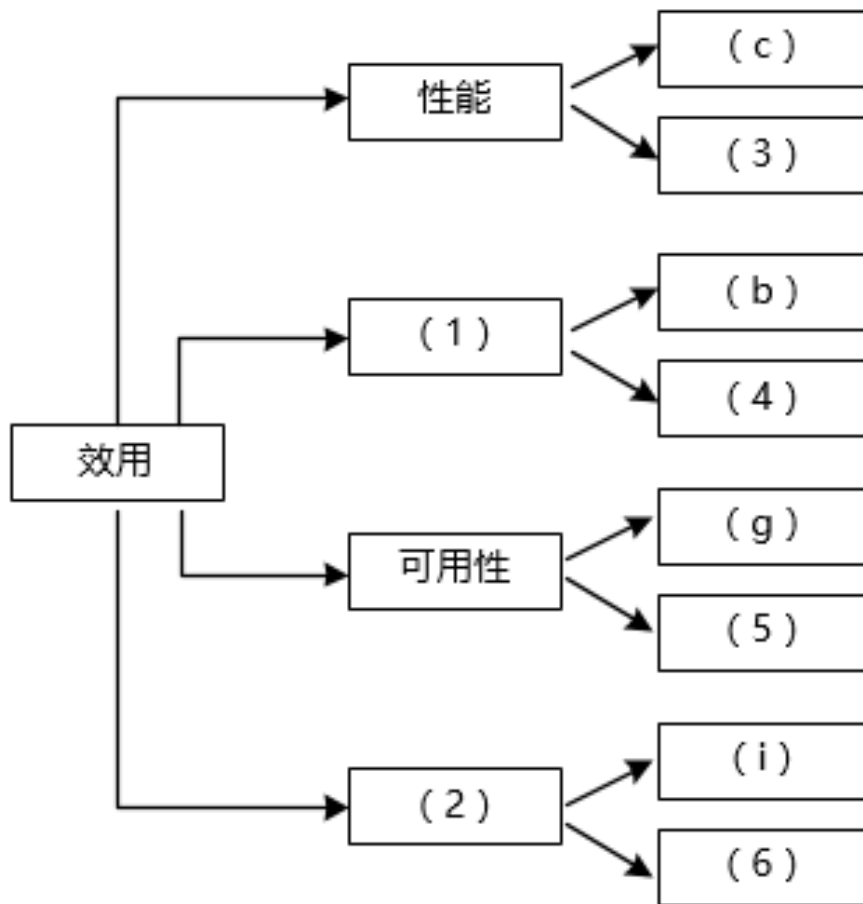
某单位为了建设健全的公路桥梁养护管理档案，拟开发一套公路桥梁在线管理系统。在系统的需求分析与架构设计阶段，用户提出的需求、质量属性描述和架构特性如下：

- (a) 系统用户分为高级管理员、数据管理员和数据维护员等三类；
- (b) 系统应该具备完善的安全防护措施，能够对黑客的攻击行为进行检测与防御；
- (c) 正常负载情况下，系统必须在0.5秒内对用户的查询请求进行响应；
- (d) 对查询请求处理时间的要求将影响系统的数据传输协议和处理过程的设计；
- (e) 系统的用户名不能为中文，要求必须以字母开头，长度不少于5个字符；
- (f) 更改系统加密的级别将对安全性和性能产生影响；
- (g) 网络失效后，系统需要在10秒内发现错误并启用备用系统；
- (h) 查询过程中涉及的桥梁与公路的实时状态视频传输必须保证画面具有1024×768的分辨率，40帧/秒的速率；
- (i) 在系统升级时，必须保证在10人月内可添加一个新的消息处理中间件；
- (j) 系统主站点断电后，必须在3秒内将访问请求重定向到备用站点；
- (k) 如果每秒钟用户查询请求的数量是10个，处理单个请求的时间为30毫秒，则系统应保证在1秒内完成用户的查询请求；
- (l) 对桥梁信息数据库的所有操作都必须进行完整记录；
- (m) 更改系统的Web界面接口必须在4人周内完成；
- (n) 如果“养护报告生成”业务逻辑的描述尚未达成共识，可能导致部分业务功能模块规则的矛盾，影响系统的可修改性；
- (o) 系统必须提供远程调试接口，并支持系统的远程调试。

在对系统需求、质量属性描述和架构特性进行分析的基础上，系统的架构师给出了三个候选的架构设计方案，公司目前正在组织系统开发的相关人员对系统架构进行评估。

问题(1)

在架构评估过程中，质量属性效用树 (utility tree) 是对系统质量属性进行识别和优先级排序的重要工具。请给出合适的质量属性，填入图 1-1 中 (1)、(2) 空白处；并选择题干描述的 (a)~(o)，填入 (3)~(6) 空白处，完成该系统的效用树。



思路解析

根据题干描述，**安全性、可修改性、性能和可用性**是最为重要的质量属性。安全性影响数据的保密性、完整性；可修改性影响系统未来功能扩展的能力；性能决定系统响应时间；可用性确保系统在出现故障时能够继续正常运作。

安全性：由于系统需要对黑客攻击进行防御，并对数据库操作进行完整记录，安全性是系统必须要重点考虑的质量属性。

可修改性：由于系统在未来可能会进行升级和更改（例如增加消息处理中间件或更改Web界面），需要具备较好的可修改性。

(h)：查询过程中涉及的桥梁与公路的实时状态视频传输必须保证画面具有1024×768的分辨率，40帧/秒的速率。这要求系统具有足够的处理能力和性能。

(1)：对桥梁信息数据库的所有操作进行完整记录，属于系统的安全性需求，确保数据的一致性和完整性。

(j)：系统主站点断电后必须在3秒内将请求重定向到备用站点，涉及系统的可用性，需要考虑冗余和故障转移机制。

(m)：系统Web界面接口的更改必须在规定时间内完成，涉及系统的可修改性，要求架构具备一定的灵活性和扩展性。

问题（2）

在架构评估过程中，需要正确识别系统的架构风险、敏感点和权衡点，并进行合理的架构决策。请用 300 字以内的文字给出系统架构风险、敏感点和权衡点的定义，并从题干（a）~（o）中分别选出1个对系统架构风险、敏感点和权衡点最为恰当的描述。

思路解析

架构风险：架构风险指架构设计中的潜在问题，它们可能导致系统在实施过程中遇到不可预见的挑战，从而影响系统的稳定性、性能和可扩展性。例如，架构师未充分考虑用户需求的变化或技术演进可能导致架构的固化。 **对应描述：**（n）"养护报告生成"业务逻辑的描述尚未达成共识，可能导致部分业务功能模块规则的矛盾，影响系统的可修改性。这个问题说明了系统可能面临的架构风险，未达成共识的业务逻辑会导致架构的不可修改性。

敏感点：敏感点指的是为了实现某一特定质量属性，系统中某个组件或模块的关键特性。例如，性能敏感点可能涉及数据库的查询处理速度或网络的传输速率。 **对应描述：**（d）查询请求处理时间的要求将影响系统的数据传输协议和处理过程的设计。这说明了查询请求处理时间的敏感性，设计时需要优先考虑该特性。

权衡点：权衡点是在多个质量属性之间需要做出的平衡。通常，设计时需要在性能、安全性、可用性等多个方面找到合适的折中点。 **对应描述：**（f）更改系统加密的级别将对安全性和性能产生影响。设计时需要在安全性和性能之间做出平衡，过高的加密级别可能导致系统性能的下降。

答案-问题（1）

（1）安全性 （2）可修改性 （3）（h）或（K） （4）（1） （5）（j） （6）（m）

答案-问题（2）

系统架构风险是指架构设计中潜在的、存在问题的架构决策所带来的隐患。

敏感点是指为了实现某种特定的质量属性，一个或多个构件所具有的特性。

权衡点是指影响多个质量属性的特性，是多个质量属性的敏感点。

风险点：（n）

敏感点：（d）

权衡点：（f）

（2015年11月系统架构设计师真题-第1题）

阅读以下关于软件架构评估的说明，在答题纸上回答问题1和问题2。

【说明】

某软件公司拟为某市级公安机关开发一套特种车辆管理与监控系统，以提高特种车辆管理的效率和准确性。在系统需求分析与架构设计阶段，用户提出的部分需求和关键质量属性场景如下：

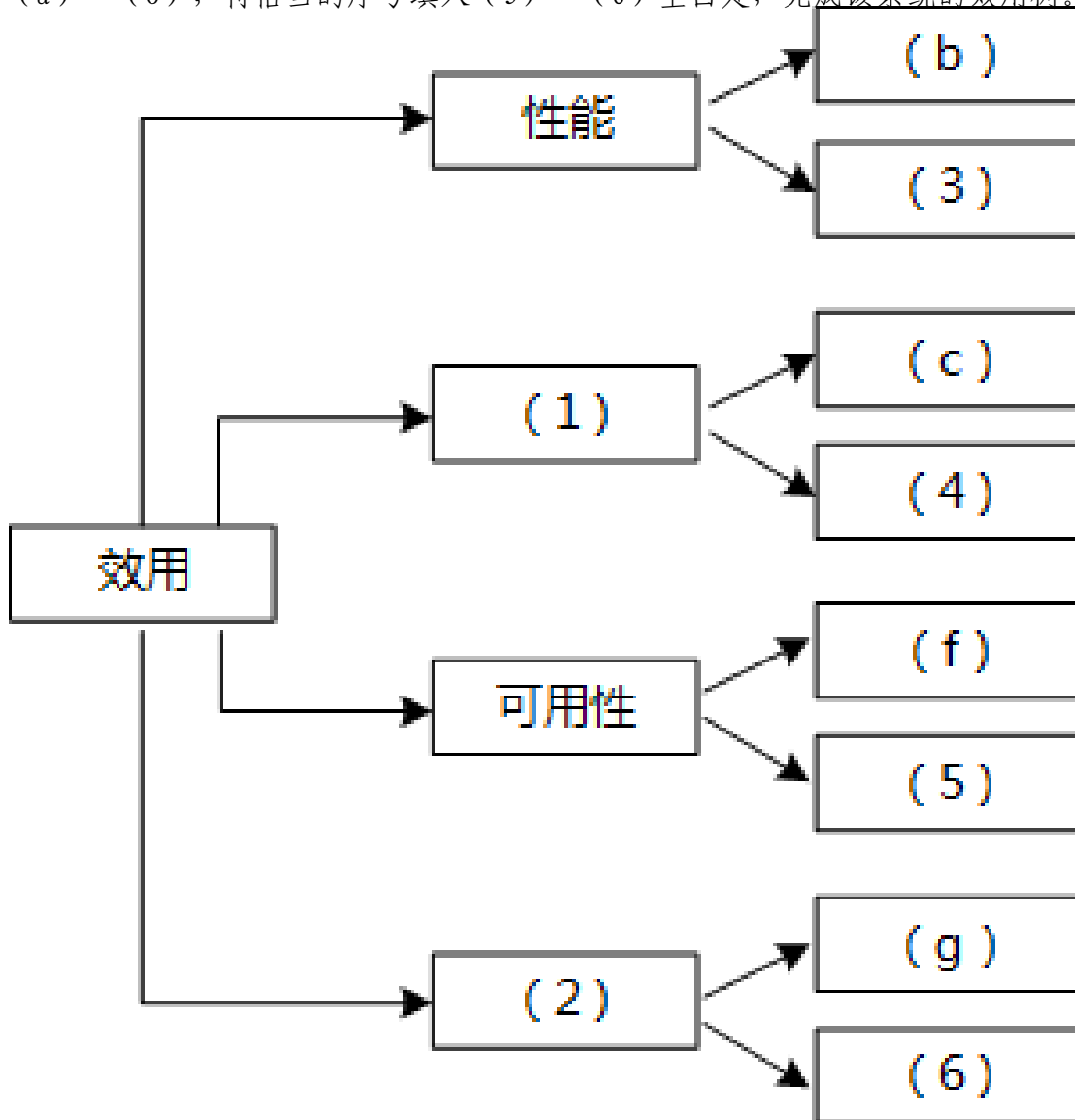
- （a）系统用户分为管理员、分管领导和普通民警等三类；
- （b）正常负载情况下，系统必须在0.5秒内对用户的车辆查询请求进行响应；
- （c）系统能够抵御99.999%的黑客攻击；
- （d）系统的用户名必须以字母开头，长度不少于5个字符；
- （e）对查询请求处理时间的要求将影响系统的数据传输协议和处理过程的设计；
- （f）网络失效后，系统需要在2分钟内发现并启用备用网络系统；
- （g）在系统升级时，需要保证在1个月内添加一个新的消息处理中间件；
- （h）查询过程中涉及的车辆实时视频传输必须保证20帧/秒的速率，且画面具有600×480的分辨率；
- （i）更改系统加密的级别将对安全性和性能产生影响；
- （j）系统主站点断电后，需要在3秒内将请求重定向到备用站点；
- （k）假设每秒钟用户查询请求的数量是10个，处理请求的时间为30毫秒，则"在1秒内完成用户的查询请求"这一要求是可以实现的；
- （l）对用户信息数据的授权访问必须保证99.999%的安全性；
- （m）目前对"车辆信息实时监控"业务逻辑的描述尚未达成共识，这可能导致部分业务功能模块的重复，影响系统的可修改性；
- （n）更改系统的Web界面接口必须在1周内完成；

(o) 系统需要提供远程调试接口，并支持系统的远程调试。

在对系统需求和质量属性场景进行分析的基础上，系统的架构师给出了三个候选的架构设计方案。公司目前正在组织系统开发的相关人员对系统架构进行评估。

问题（1）

在架构评估过程中，质量属性效用树（utility tree）是对系统质量属性进行识别和优先级排序的重要工具。请给出合适的质量属性，填入图1-1中（1）、（2）空白处；并选择题干描述中的（a）~（o），将恰当的序号填入（3）~（6）空白处，完成该系统的效用树。



思路解析

效用树（Utility Tree）是ATAM等架构评估方法中的核心工件，用于把“效用/质量”自顶向下分解为可度量、可验证的具体场景（QA Scenario），并在此基础上排序优先级与风险。顶层一般以Utility为根，下一层是质量属性（如性能、可用性、安全性、可修改性、可测试性等），再往下是场景。

本题给定的大量条目（b）（c）（f）（h）（j）（l）（n）等，天然对应不同质量关注点：如（b）（h）更偏性能（响应时间、视频帧率/分辨率），（j）（f）更偏可用性/容错（故障后的业务连续性），（c）（l）（i）与安全性紧密相关，（g）（n）（m）与可修改性/可扩展性直接相关。题图仅给出两条分支待填，结合内容密度与场景显著性，最合理的是**“安全性”和“可修改性”两大属性作为二级分支，即（1）=安全性、（2）=可修改性。原因在于：公安行业对数据与访问安全**、持续变更/集成新中间件的诉求突出，且题干（c）（l）（i）（g）（n）（m）均围绕这两类属性给出明确承诺或风险提示。

对四个场景占位（3）~（6），需做到**属性与场景语义强匹配**：

（3）挂在**性能/流媒体能力**相关的叶子下最合适，而题干中唯一明确规格化“视频实时性”的是（h）“20fps、600×480”，直接反映吞吐、时延、码率与负载的耦合，因此选（h）。

（4）与**安全性/数据保护**强相关；（1）“用户信息授权访问99.999%安全性”即是访问控制、最小权限、审计与加密的量化目标，故（4）=（1）。

（5）与**可用性/容错切换**强相关；（j）“主站断电3秒内重定向到备用站”是RTO（恢复时间目标）的明确指标，也体现冗余、健康探测、GSLB/Anycast/心跳等设计，故（5）=（j）。

（6）与**可修改性**最贴近；（n）“Web界面接口变更1周内完成”属于**修改成本与节拍**的量化约束，覆盖UI层解耦、前后端分离、适配层等策略，故（6）=（n）。

问题（2）

在架构评估过程中，需要正确识别系统的架构风险、敏感点和权衡点，并进行合理的架构决策。请用300字以内的文字给出系统架构风险、敏感点和权衡点的定义，并从题干描述中的（a）~（o）各选出1个属于系统架构风险、敏感点和权衡点的描述。

思路解析

架构风险强调后果与不确定性：当需求尚未收敛或关键决策缺乏验证时，系统可能在后期暴露出高代价问题。

本题（m）直接指出“实时监控业务逻辑尚未达成共识”，会带来**需求漂移、重复建设、接口不稳**等连锁反应，典型风险表现为：模块边界反复调整、代码分叉、版本兼容负担加剧、上线节奏被动——最终损害**可修改性与可维护性**。因此将（m）归类为**架构风险**最为贴切。

敏感点的关键是“某个设计取值对单一质量属性的显著影响”。（e）指出“查询时延目标会影响数据传输协议与处理流程的设计”，这正是**性能属性**下的核心敏感点：例如是否采用**二进制RPC/HTTP/3**、是否启用**服务端聚合/缓存/预计算**、是否引入**异步化与批处理**、是否在**内核/网络栈**上开启零拷贝/长连接复用等。任何一项的微小取值变化都会对性能产生显著影响，因此（e）是标准的**性能敏感点**。

权衡点强调“质量属性影响的一高一低”。（i）“更改加密级别影响安全与性能”就是经典的**Security vs. Performance**对立：更强算法/更长密钥/更高握手强度→更高CPU开销、更大时延；反之则可能降低抗攻击能力。

答案-问题（1）

- （1）安全性
- （2）可修改性
- （3）（h）
- （4）（1）
- （5）（j）
- （6）（n）

答案-问题（2）

架构风险：系统架构风险是指架构设计中潜在的、存在问题的架构决策所带来的隐患。

敏感点：是指为了实现某种特定的质量属性，一个或多个构件所具有的特性。

权衡点：是影响多个质量属性的特性，是多个质量属性的敏感点。

架构风险：（m）；**敏感点**：（e）；**权衡点**：（i）。

（2014年11月系统架构设计师真题-第4题）

请详细阅读有关软件架构评估方面的说明，在答题纸上回答问题1至问题2。

【说明】

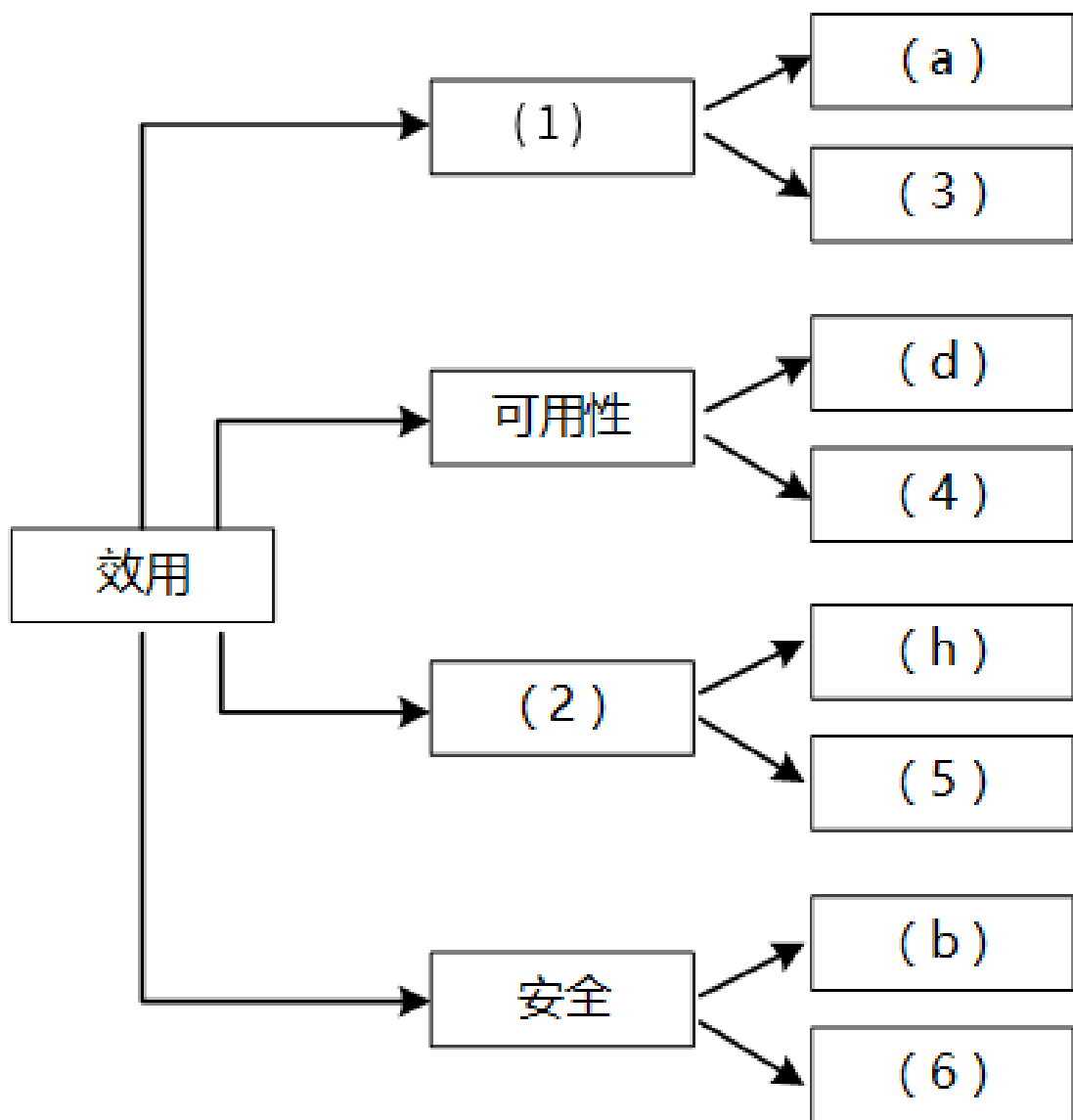
某电子商务公司拟升级目前正在使用的在线交易系统，以提高客户网上购物时在线支付环节的效率和安全性。公司研发部门在需求分析的基础上，给出了在线交易系统的架构设计。公司组织相关人员召开了针对架构设计的评估会议，会上用户提出的需求、架构师识别的关键质量属性场景和评估专家的意见等内容部分列举如下：

- (a) 在正常负载情况下，系统必须在0.5秒内响应用户的交易请求；
- (b) 用户的信用卡支付必须保证99.999%的安全性；
- (c) 系统升级后用户名要求至少包含8个字符；
- (d) 网络失效后，系统需要在2分钟内发现错误并启用备用系统；
- (e) 在高峰负载情况下，用户发起支付请求后系统必须在10秒内完成支付功能；
- (f) 系统拟采用新的加密算法，这会提高系统安全性，但同时会降低系统的性能；
- (g) 对交易请求处理时间的要求将影响系统数据传输协议和交易处理过程的设计；
- (h) 需要在30人月内为系统添加公司新购买的事务处理中间件；
- (i) 现有架构设计中的支付部分与第三方支付平台紧耦合，当系统需要支持新的支付平台时，这种设计会导致支付部分代码的修改，影响系统的可修改性；
- (j) 主站点断电后，需要在3秒内将访问请求重定向到备用站点；
- (k) 用户信息数据库授权必须保证99.999%可用；
- (l) 系统需要对Web界面风格进行修改，修改工作必须在4人月内完成；
- (m) 系统需要为后端工程师提供远程调试接口，并支持远程调试。

问题（1）

在架构评估过程中，质量属性效用树（utility tree）是对系统质量属性进行识别和优先级排序的重要工具。请给出合适的质量属性，填入图4-1中（1）、（2）空白处；并选择题干描述的

（a）~（m），填入（3）~（6）空白处，完成该系统的效用树。



思路解析

在软件架构评估过程中，**质量属性效用树（Utility Tree）** 是识别和组织质量需求的核心工具。它通过分层方式把系统的关键质量属性、场景和优先级逐步展开，帮助架构师和评估专家明确系统应重点关注哪些方面。结合题干中的描述，可以清楚地将部分需求映射到典型的质量属性。

首先，**性能** 是该系统的关键质量属性之一。题干（a）要求在正常负载下 0.5 秒内响应交易请求，（e）要求在高峰负载下 10 秒内完成支付，这些都直接涉及系统的响应时间和吞吐能力，典型地属于性能范畴，因此（1）应填“性能”，并将（e）填入（3）。

其次，**可修改性** 是指系统在功能扩展、界面变更时能否快速适应。题干（h）要求在 30 人月内完成中间件添加，（i）要求在 4 人月内修改界面风格，说明架构需要支持较低成本的修改，因此（2）填“可修改性”，并在（5）填（i）。

再看 **可用性**，它强调系统在出现故障时的持续服务能力。题干（d）要求在 2 分钟内切换备用系统，（j）要求主站点断电后 3 秒内重定向请求，显然属于典型的可用性要求，因此（4）应填（j）。

安全性 是关键属性。题干（b）明确提出支付安全性要求，而（k）则强调用户信息数据库授权必须保证极高可用性，本质上体现了对数据访问控制与保护的需求，因此（6）填（k）。

问题（2）

在架构评估过程中，需要正确识别系统的架构风险、敏感点和权衡点，并进行合理的架构决策。请用300字以内的文字给出系统架构风险、敏感点和权衡点的定义，并从题干（a）~（m）中各选出

1个对系统架构风险、敏感点和权衡点最为恰当的描述。

思路解析

在架构评估中，**风险、敏感点和权衡点** 是三类需要重点识别的关注对象。

架构风险是指潜在的隐患，通常源于架构设计中的某些不合理假设或紧耦合结构。风险一旦发生，往往会显著增加系统的维护成本，甚至导致项目失败。题干（i）描述“支付部分与第三方平台紧耦合”，意味着当业务需要接入新支付平台时，必须修改大量现有代码，显著影响系统的可修改性与扩展性，因此这是典型的架构风险。

敏感点是实现某种质量属性的核心依赖点。如果这些点的设计不合理，就可能无法达到目标质量。题干（g）指出“处理时间要求影响数据传输协议和交易处理过程设计”，说明系统的性能指标高度依赖于架构中某些协议与组件的选择，这是性能质量属性的敏感点。

权衡点则更加复杂，是指某个架构决策同时作用于多个质量属性，需要在不同属性之间取舍。题干（f）说明“新的加密算法提高安全性但降低性能”，正是安全与性能之间的冲突，这种情况往往需要评估权重并做出折中。

答案-问题（1）

- （1）性能
- （2）可修改性
- （3）（e）
- （4）（j）
- （5）（l）
- （6）（k）

答案-问题（2）

系统架构风险是指架构设计中潜在的、存在问题的架构决策所带来的隐患。敏感点是指为了实现某种特定的质量属性，一个或多个构件所具有的特性。权衡点是指影响多个质量属性的特性，是多个质量属性的敏感点。

风险点：（i）； 敏感点：（g）； 权衡点：（f）。

（2024年11月系统架构设计师真题-第1题）

阅读以下关于软件架构设计与评估的叙述，在答题纸上回答问题1和问题2。

【说明】

某软件公司拟为一所大型写字楼开发一套访客登记系统，以提升访客管理的效率和安全性。在系统需求分析与架构设计阶段，用户提出的部分需求和关键质量属性场景如下：

- （a）网络失效后，系统需要在 3 分钟内发现并启用备用网络系统；
- （b）对查询请求处理时间的要求将影响系统的数据传输协议和处理过程的设计；
- （c）系统能够抵御 99.999% 的黑客攻击；
- （d）能够连续运行的时间不小于240小时；
- （e）系统可以对数据进行导入导出，要求在1分钟内完成数据导出导出；
- （f）系统根据用户量访问量的增加拓展空间，要求在3天完成；
- （g）系统分为三个不同的国家语言，支持搜索自动填充补全等；

公司目前正在组织系统开发的相关人员对系统架构进行评估。

问题（1）

在架构评估过程中，质量属性效用树是对系统质量属性进行识别和优先级排序的重要工具。请选择题干描述的（a）~（m），填入（1）~（6）空白处。

序号	质量属性
(1)	可用性
c	(2)
d	(3)
e	(4)
f	(5)
g	(6)

思路解析

质量属性效用树是架构评估中常用的工具，其核心是识别、分类和排序系统的关键质量属性。题干中 (a) ~ (g) 的场景正对应了不同的质量属性。

- (a) 网络故障切换要求系统能在 3 分钟内恢复服务，体现了系统的**可用性**；
- (b) 查询请求处理时间与协议优化直接相关，这是典型的**性能**问题；
- (c) 系统需抵御黑客攻击，保障数据与服务安全，体现**安全性**；
- (d) 要求系统连续运行 240 小时，属于**可用性**的稳定性体现；
- (e) 数据导入导出在 1 分钟内完成，这是**性能**要求；
- (f) 用户访问量增加时，3 天内完成扩展，说明系统需具备较强的**可修改性（可扩展性）**；
- (g) 多语言与自动补全功能属于人机交互体验，体现**易用性**。

综上，正确匹配为 (1) a (2) 安全性 (3) 可用性 (4) 性能 (5) 可修改性 (6) 易用性。此题重点在于熟悉各类质量属性的定义与应用场景。

问题 (2)

针对质量属性可靠性可以使用ping/echo和心跳模式实现，请分别简述ping/echo和心跳的实现原理。张工认为从资源利用率的角度来看心跳比较适合，简述理由？

思路解析

Ping/Echo 模式是较为常见的节点健康检测方法。主控端通过定时发送 Ping 请求，若被监控节点正常工作，会返回 Echo 应答。该方法实现简单，适用于小规模或测试场景。但在大规模系统中，主控端需要同时管理大量探测请求，容易造成 **CPU、网络带宽和内存开销过大**，影响系统性能与可扩展性。

心跳机制的核心思想是“被监控节点自己证明自己存在”。节点会周期性地向主控端发送心跳包，通常为轻量级数据结构，主控端只需检查是否在指定时间窗口内收到心跳消息。如果心跳信号中断，便可认为节点出现故障。由于心跳包通常较小且由各节点独立发送，因此不会形成集中压力，能够更好地适应 **分布式系统和大规模集群场景**。

张工认为心跳机制更优，主要基于**资源利用率和可扩展性**的考虑。心跳避免了主控端高频、大量发起探测的资源开销，更加经济高效，符合大规模系统对可靠性和性能的双重要求。

答案-问题（1）

（1）a （2）安全性 （3）可用性 （4）性能 （5）可修改性 （6）易用性

答案-问题（2）

Ping/Echo 模式是主控端主动探测目标节点是否在线的一种机制，目标节点响应后即可判断其健康状况；而**心跳机制**是由被监控节点周期性向主控端发送心跳信号，主控端根据是否连续收到心跳来判断节点健康状况。

张工认为从**资源利用率**角度来看，**心跳机制**更适合大规模系统，因为它将压力从主控端分摊至各节点，避免了集中式的探测请求造成系统资源浪费，具备更好的扩展性与网络效率。

第4章——微服务与云原生架构 (2024年5月系统架构设计师真题-第1题)

某互联网企业正在对其已有的在线服务平台进行架构升级，以提升系统的可维护性、可扩展性和服务稳定性。原系统采用传统的单体架构，随着业务模块的不断增加，系统部署越来越困难，修改一处代码往往需要整个系统重新打包上线，导致维护成本高、故障影响范围广。

为解决上述问题，架构团队计划引入微服务架构，将系统按业务边界划分为多个服务单元，分别部署和独立扩展，并配套引入容器化、服务注册与发现、统一配置中心等能力。同时，团队也在开展质量属性建模分析，明确如可用性、性能、可维护性等非功能需求的权衡与保障机制。

请结合下列问题，完成系统架构设计分析任务。

问题（1）

简述微服务架构，并对比单体架构和微服务架构的优缺点。

思路解析

单体架构是将系统的所有功能模块**统一构建为一个整体应用**的架构模式。所有业务模块、逻辑处理、UI、数据库访问都在一个进程中运行，部署时打包为一个整体，统一上线。

微服务架构是将系统按照业务边界拆分为多个小型独立服务的架构模式。每个服务自治运行、独立部署，通过轻量级通信（如 HTTP/REST、消息队列）协同工作。微服务架构的特点是：

服务松耦合，独立扩展；

支持技术多样性；

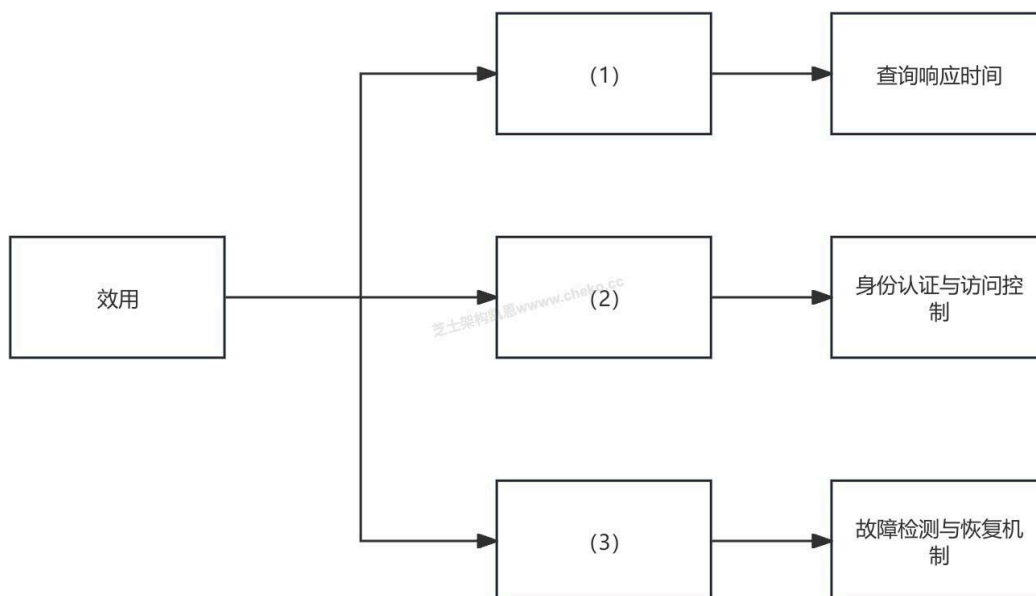
易于规模化协作和弹性部署；

运维与治理要求更高。

单体架构是一体化构建部署的整体系统，适合简单场景；微服务架构则将系统拆解成多个独立服务，提升灵活性和扩展性，适合大型复杂系统。

问题（2）

在架构评估过程中，质量属性效用树（Utility Tree）常用于识别和排序系统的重要质量属性。根据以下描述的需求片段，从选项中选择合适的质量属性，填写在图中空白处（1）~（3），完成简化的效用树结构。A. 性能 B. 安全性 C. 可用性



思路解析

(1) 查询响应时间：查询响应时间是系统对请求做出响应所需的时间，直接体现系统处理速度和响应效率。 **对应质量属性：A. 性能 (Performance)**

(2) 身份认证与访问控制：涉及对用户身份的验证和资源访问权限的限制，关系到系统的数据和功能不被非法使用。 **对应质量属性：B. 安全性 (Security)**

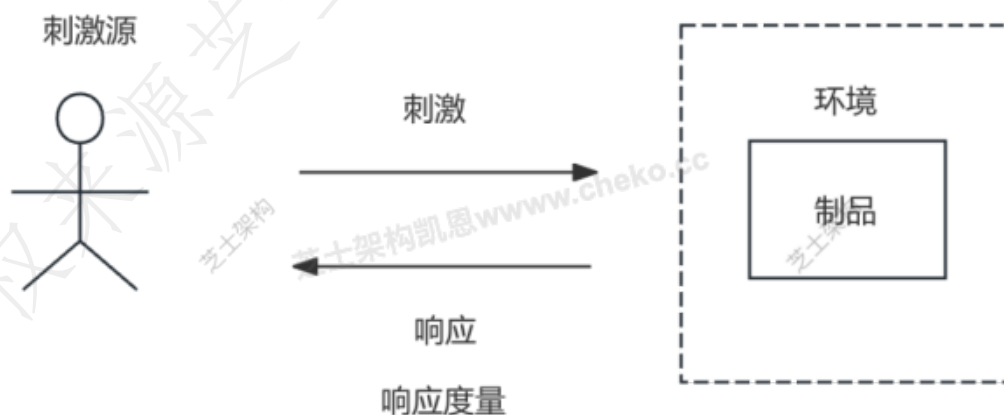
(3) 故障检测与恢复机制：系统在出现故障后，能否被及时发现并迅速恢复运行，体现了系统的持续运行能力。 **对应质量属性：C. 可用性 (Availability)**

问题 (3)

用质量属性6要素描述可用性质量属性场景描述。

思路解析

质量属性场景描述 = 刺激源+刺激+环境+制品+响应+响应度量，如下图所示



可用性、可修改性、性能、可测试性、易用性和安全性/ 的场景描述方法是书本重点，曾经考过一道大题，所以内容虽然多还是需要看和记。这部分不再展开，请参考红宝书相关章节。

答案-问题 (1)

微服务架构是一种将应用拆分为多个小型、独立部署的服务单元的架构模式。每个服务聚焦于一个

业务功能，运行在独立进程中，通过轻量级协议（如 HTTP/REST、消息队列等）与其他服务通信。服务间松耦合、自治开发、独立部署，常配合容器化和 DevOps 实现快速迭代和弹性伸缩。

微服务架构的优点在于服务独立、易于扩展部署，支持技术异构，有利于故障隔离和小团队并行开发，提高了系统的灵活性和开发效率；但缺点是系统整体复杂度高，带来通信开销、运维监控难度增加、跨服务数据一致性难以保障，同时对自动化运维和服务治理能力提出更高要求。

答案-问题（2）

A, B, C

答案-问题（3）

刺激源：系统内部、系统外部

刺激：疏忽、错误、崩溃、时间

环境：正常操作、降级模式

制品：系统处理器、通信信道、持久存储器、进程

系统响应：检测事件并进行以下一个或多个活动：将其记录下来。通知适当的各方，包括用户和其他系统。根据已定义的规则禁止导致错误或故障的事件源。在一段预先指定的时间间隔内不可用，其中，时间间隔取决于系统的关键程度。在正常或降级模式下运行。

响应度量：故障修复时间

第5章——领域驱动设计 (2025年11月系统架构设计师真题-第2题)

某连锁餐饮集团建设了“云端一体的点菜系统”。

终端侧（端）：部署在门店的收银机、服务员手持PAD、自助点餐机等，负责：1）桌台管理与点菜下单；2）本地订单暂存与支付；3）与云端同步菜品、价格等基础数据。

云端系统（云）：部署在集团数据中心或公有云，负责：1）菜品基础信息管理（菜品、原料、定价、上下架等）；2）订单管理（汇总各门店订单、对账、结算）；3）餐厅菜品与菜单配置（不同门店的可售菜品组合）；4）员工与权限管理；5）报表分析与运营决策。

企业计划采用领域驱动设计（DDD）方法，对复杂的业务域进行建模和拆分，同时需要考虑云端与终端之间的网络质量不稳定（弱网、间歇断网）的情况，保证点单业务“尽量可用 + 数据最终一致”。

问题（1）

（1）简要说明领域驱动设计（DDD）的基本思想。（2）给出“限界上下文”的严格定义，并说明其在复杂系统（如本点菜系统）中的作用和好处。

思路解析

此题出自北京交通大学一硕士论文《支持离线模式的易迭代餐饮系统设计与实现》。

领域驱动设计（DDD）的核心思想是：将软件设计的重心从“技术”转移到“业务领域”，通过与领域专家密切合作，形成一套贯穿需求、设计和实现的**统一语言（Ubiquitous Language）**，围绕领域模型组织代码结构（而不是围绕数据库或技术框架）。DDD 通常会划分领域层、应用层、基础设施层等，并强调聚合、实体、值对象、领域服务等概念，用清晰的模型来承载复杂业务规则。

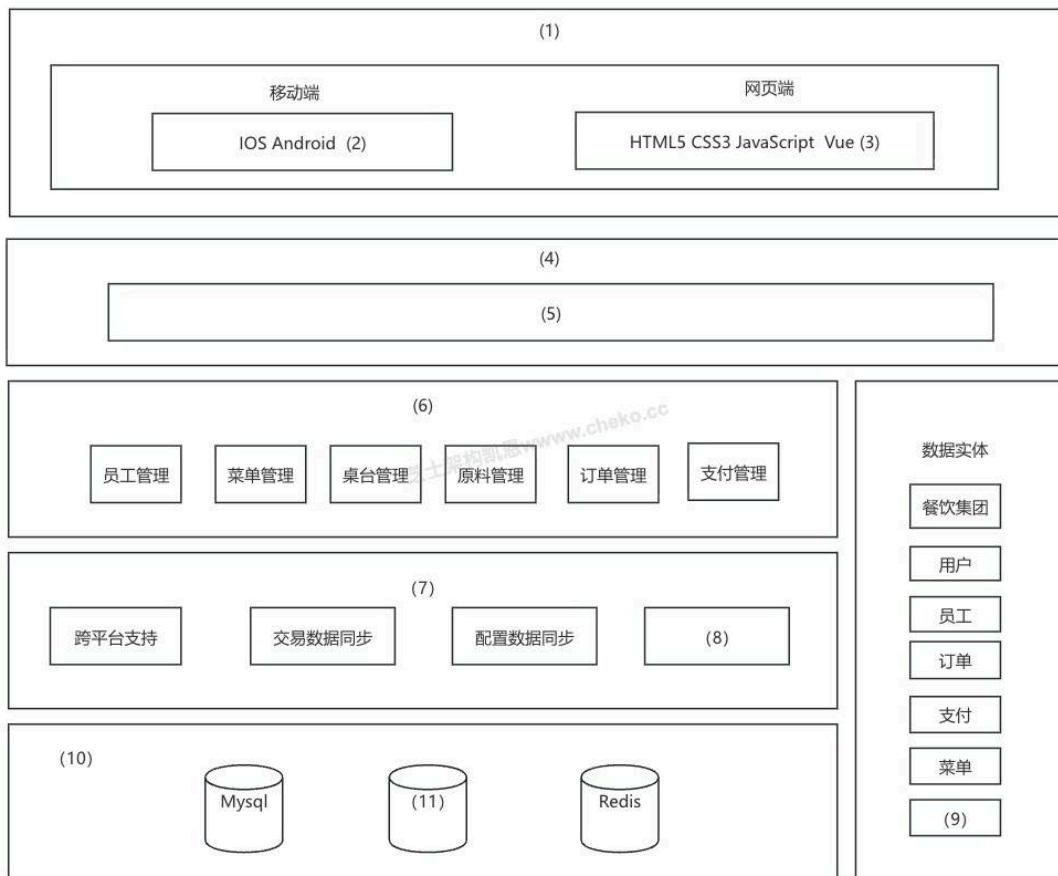
限界上下文（Bounded Context）是 DDD 中最关键的划分单元，可以理解为“统一语言适用的边界”。在一个限界上下文内部，诸如“订单”“菜品”“原料”等术语有清晰且唯一的语义，模型与规则高度一致；而在另一个上下文中，同名概念可以有不同含义，例如“订单”在“交易上下文”里重支付状态，在“结算上下文”里重对账金额。

通过为复杂系统划分多个限界上下文，可以：① 控制模型规模，降低复杂度；② 避免跨团队、跨系统的概念冲突；③ 支撑团队按上下文拆分为独立服务，形成**清晰的微服务边界**，独立演进与部署

。本案例中，将“订单交易、菜品基础信息、门店运营、员工权限”等拆分为不同上下文，有利于分别建模和优化，避免一个庞大的“上帝模型”。

问题（2）

请将下列要素归入合适的架构层：a. 基础能力层、b. 日志采集、c. 桌台、d. 包裹、e.Ktor、f.Realm、g.Ajax、h.Spring Security、i. 网络层、j.SQLite、k. 业务层、l. 服务层、m. 原料、n. 表示层、o. 设备层、p. 统一代理能力、q. 持久层、r. 领域上下文



思路解析

（1）表示层（n）

表示层负责用户界面展示及前端交互，是客户端访问系统的入口。本题中 Ktor 被用于移动端（iOS/Android）内部作为 HTTP Client，用来发起网络访问；而 Ajax 属于浏览器端常用的异步通信技术，用于页面局部刷新与后端交互。两者都属于前端与后端通信的手段，因此一并归入表示层，体现的是客户端的交互与呈现能力。

（2）统一代理能力层（p）

统一代理能力层承担前端访问业务系统前的入口治理职责，包括认证、授权、流量校验、请求转发等。本题中 Spring Security 提供完整的安全认证与授权机制，典型部署于网关或统一接入层，用于对所有进入系统的请求进行集中安全控制；而“统一代理能力”则是该层的功能定义。两者共同构成交互请求进入业务前的安全防护与接入代理层。

（3）业务层（k）

业务层负责实现系统的核心业务逻辑，承载如员工管理、菜单管理、桌台管理、原料管理、订单管理、支付管理等具体功能模块。该层基于领域对象组织业务流程，体现系统面向用户的实际业务能力，是应用服务的直接承载层。

（4）领域上下文层（r）

领域上下文层体现系统的知识结构，是 DDD 中用于划分业务边界、定义领域模型的重要层。桌台

(c)、原料(m)、包裹(d)均为业务的核心领域实体,代表系统业务本身具有的客观对象与规则。这些领域对象被划分到特定的领域上下文中,例如点餐上下文、库存上下文、配送上下文等,用以支撑业务层的功能实现。因此,它们与 r.领域上下文共同构成系统的领域模型层。

(5) 基础能力层(a)

基础能力层提供与业务无关、可复用的底层通用能力。本题中跨平台能力、交易数据同步、配置数据同步均属于跨业务的全局型能力,而“日志采集”(b)更属于典型的横切关注点,被所有业务模块共享,不参与业务流程本身,因此统一归入基础能力层。

(6) 持久层(q)

持久层用于处理数据访问、缓存管理、对象存储与本地持久化逻辑。本题中的 SQLite(j)和 Realm(f)均属于移动端常用的本地数据库与 ORM 技术,这里凯恩觉得 j、f 都是可以。而后端的 MySQL、Redis 同样属于持久化数据存储,实现业务数据的长期保存和缓存加速。因此它们均归属于持久层,体现系统的数据访问与保存能力。

(7) 设备层(o)与网络层(i)

设备层指系统运行所依赖的物理设备,如收银机、服务员 PAD、自助点单机等,用于承载前端应用的运行环境;网络层则提供设备与云端之间的通信能力,包括 TCP/IP、HTTP、WebSocket 等协议支持。虽然在本题展示架构图中未直接绘制,但从系统整体来看,它们构成了软件架构之下的基础设施层,是上层功能的运行与通信基础。

(8) 服务层(1)

题目参考图未单独绘制服务层,但从架构分层原则来看,服务层通常位于代理层与业务层之间,用于封装业务用例、提供统一的服务接口、协调领域对象执行业务流程。在本架构中,业务层中各功能模块对外暴露的应用接口即起到了服务层的作用,因此服务层可视为包含于业务层的应用服务部分。

问题(3)

在网络质量较差甚至间歇性断网的情况下,点单业务可能遇到哪些典型问题(至少列出 3 种),请结合本案例给出对应的解决思路。

思路解析

弱网环境下,最关键的挑战是如何在“网络经常出问题”的现实前提下,仍然保证业务“可用”,并在事后通过一定手段达成**数据最终一致性**。首先,点单操作不能依赖即时访问云端,因此在终端或边端需要使用**本地事务**将订单可靠写入本地数据库,即使云端不可达,也要保证“门店账本”是完整的。这些本地订单随后通过后台任务、消息队列等方式逐步同步到云端,一旦网络条件恢复,即可完成补传。

与此同时,需要设计**全局唯一订单号**(例如终端号 + 时间戳 + 序列)并在云端实现幂等消费,避免重试导致重复订单。其次,可设置**日终对账**流程:云端与各门店对比订单汇总金额与笔数,若不一致则回溯增量日志。

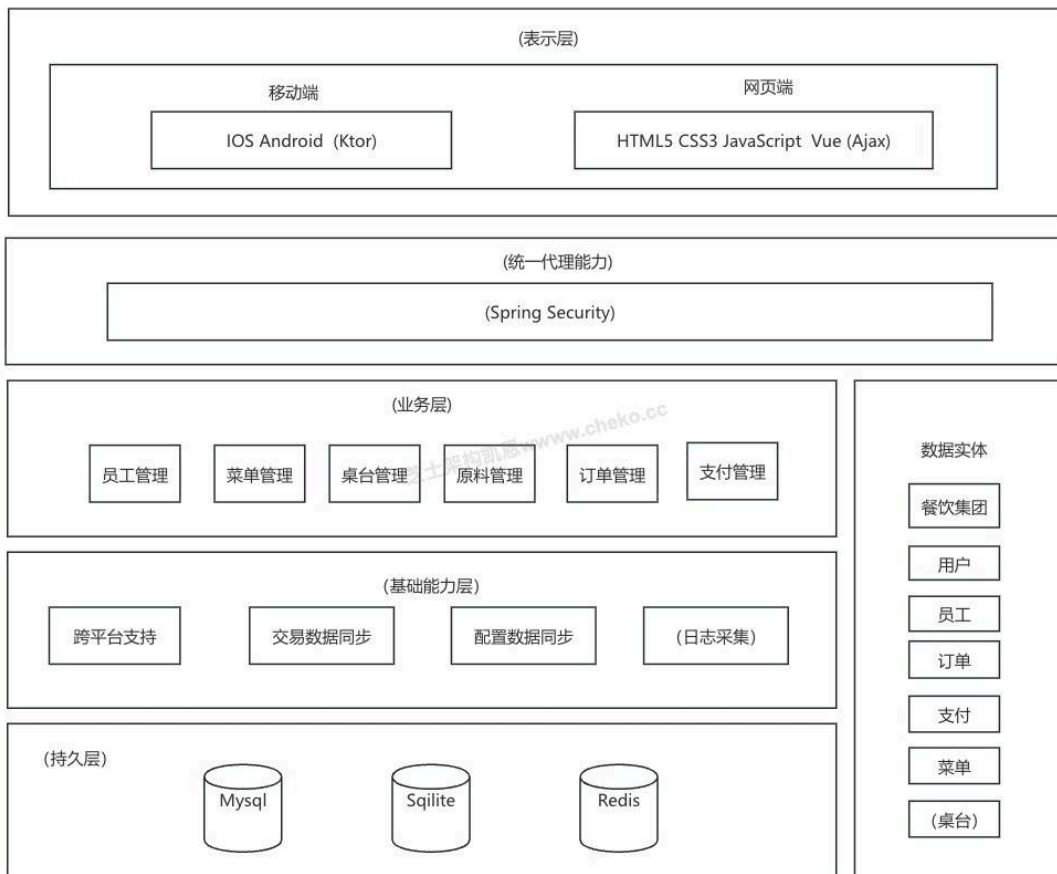
答案-问题(1)

(1) DDD 强调以业务领域为中心,通过领域专家与开发人员共同建立**统一语言**,根据业务边界划分领域、构建领域模型,用模型驱动设计实现。

(2) 限界上下文:在特定边界内,一个领域模型及其术语、规则保持**语义一致**的区域,边界外可以有不同含义。好处:消除概念冲突,限制复杂度,支持团队拆分与独立演进。

答案-问题(2)

(1) n (2) e (3) g (4) p (5) h (6) k (7) a (8) b (9) c (10) q (11) j、f(均可)



答案-问题（3）

下单时网络中断导致订单未能上传云端，出现云端与门店数据不一致。

重试机制处理不当，可能出现**订单重复提交**。

不同终端对同一桌台或订单并发操作，弱网情况下产生**冲突或覆盖**。

菜品配置在终端缓存过期或未及时同步，导致价格/可售状态不一致。

对应措施示例：

在终端/边端使用**本地事务 + 本地缓存**保存订单，网络恢复后再异步同步到云端，采用**最终一致性**策略。

所有订单采用**全局唯一 ID + 幂等接口**，云端对重复提交进行去重。

建立**对账机制**（日结、账单对比）。

第6章——类图与实体建模 (2022年11月系统架构设计师真题-第2题)

阅读以下关于软件系统设计与建模的叙述，在答题纸上回答问题1至问题3。

【说明】

煤炭生产是国民经济发展的主要领域之一，其煤矿的安全非常重要。某能源企业拟开发一套煤矿建设项目安全预警系统，以保护煤矿建设项目从业人员生命安全。本系统的主要功能包括如下

(a) ~ (h) 所述。

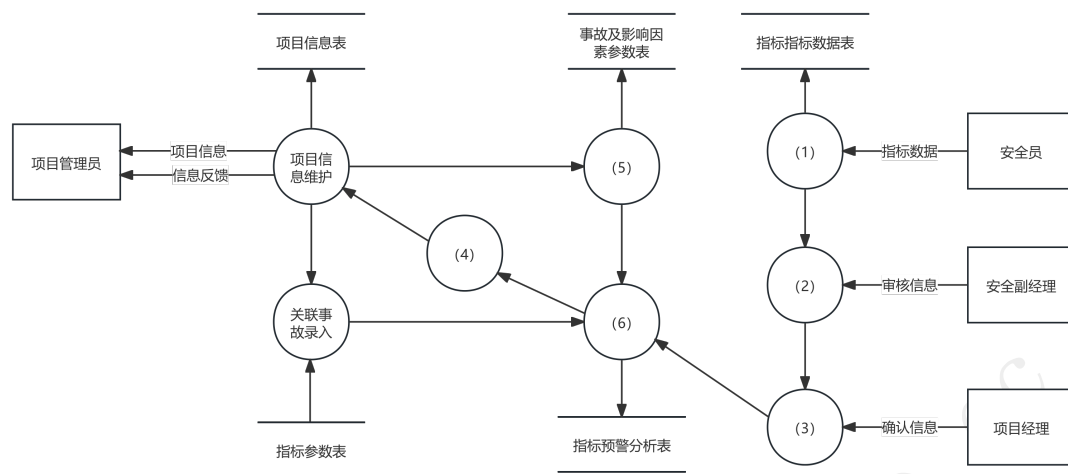
- (a) 项目信息维护
- (b) 影响因素录入
- (c) 关联事故录入
- (d) 安全评价得分
- (e) 项目指标预警分析
- (f) 项目指标填报

(g) 项目指标审核

(h) 项目指标确认

问题（1）

王工根据煤矿建设项目安全预警系统的功能要求，设计完成了系统的数据流图，如图2-1所示。请使用题干中描述的功能(a)～(h)，补充完善空(1)～(6)处的内容，并简要介绍数据流图在分层细化过程中遵循的数据平衡原则。



思路解析

从安全员→(1)→项目指标数据表，推断(1)是“项目指标填报”；

从安全副经理→(2)以及填报→审核，推断(2)是“项目指标审核”；

从项目经理→(3)以及审核→确认，推断(3)是“项目指标确认”；

图中(6)与“指标预警分析”相连，应为“项目指标预警分析”；

图中(5)与“事故及影响因素参数表”相连，应为“影响因素录入”；

剩余未出现的功能只有“安全评价得分”，因此(4)应为它。

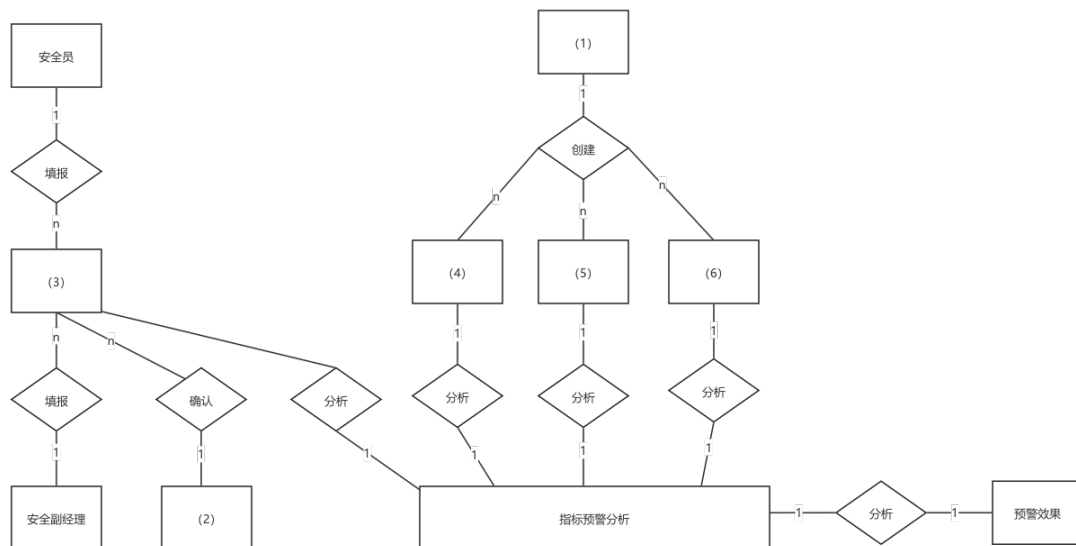
分层细化数据平衡原则：

父图与子图平衡：子图的输入输出必须与父图对应的数据流保持一致，若拆分为多条数据流，其数据项总和仍需与父图一致。

子图内部平衡：每个加工的输入和输出需成对出现，避免黑洞（只输入无输出）、奇迹（只输出无输入）、灰洞（输入输出不合理匹配）。

问题（2）

请根据【问题1】中数据流图表示的相关信息，补充完善煤矿建设项目安全预警系统总体E-R图（见图2-2）中实体(1)～(6)的具体内容，将正确答案填在答题纸上。



思路解析

由数据流图可知：安全员负责填报，副经理审核，经理确认，因此(2)对应“项目经理”；审核与确认作用的对象是“项目指标数据”，因此(3)应为该实体；数据流图中有“项目管理员”，因此(1)为“项目管理员”；其他三处为系统核心数据表，应分别是“指标参数、项目信息、事故及影响因素参数”，顺序对应(4)~(6)。

问题（3）

在结构化分析和设计过程中，数据流图和数据字典是常用的技术手段，请用200字以内的文字简要说明它们在软件需求分析和设计阶段的作用。

思路解析

在软件工程的结构化分析与设计过程中，**数据流图（DFD）和数据字典（DD）** 是两个相辅相成的重要工具。

首先，**数据流图**以图形化方式展示了系统中数据的流动与加工过程，它从功能角度刻画了系统如何接收输入、进行处理并生成输出。相比自然语言描述，数据流图能够清晰揭示业务活动之间的关系，避免需求表述的模糊和歧义。

在需求分析阶段，数据流图的作用主要体现在帮助分析人员与用户进行沟通，建立系统的功能模型，从而明确用户需求，形成一致的功能认知；在设计阶段，数据流图进一步细化为分层模型，既可以作为模块划分的依据，又能用于接口数据的定义，保证后续设计的合理性和可实现性。同时，在分层细化过程中，数据流图还必须遵循数据平衡原则，即父图与子图的输入输出保持一致，每个加工必须有对应的输入和输出，避免出现“黑洞”“奇迹”或“灰洞”等问题，这样才能确保功能模型的正确性和完整性。

其次，**数据字典**是对数据流图中涉及的所有数据流、数据存储、数据项等进行精确定义的工具，它类似于一份系统数据的“说明书”。在需求分析阶段，数据字典为每个数据元素提供统一的命名、格式和含义，从而消除了不同开发人员和用户之间对数据理解的偏差。在设计阶段，数据字典则作为系统实现的基础，保证数据在不同模块之间传递时的完整性与一致性。同时，它还能支持一致性检查和完整性检查，防止因数据定义不清或使用不当导致系统逻辑错误。

答案-问题（1）

- (1) f
- (2) g
- (3) h

(4) d

(5) b

(6) e

细化分层数据平衡原则：(1) 子图与父图之间的平衡：父图与子图之间的平衡要求子图边界上的输入/输出数据流必须与其父图对应的输入/输出数据流保持一致。如果父图中某个加工的数据流在子图中对应多条数据流，但这些数据流的数据项总量与父图中的数据流相匹配，则仍然保持平衡。

(2) 子图内部平衡：子图内部加工的输入和输出需保持平衡。

答案-问题(2)

(1) 项目管理员

(2) 项目经理

(3) 项目指标数据

(4)~(6) 指标参数、项目信息、事故及影响因素参数

答案-问题(3)

在软件需求分析阶段：数据流图主要用于建立软件的需求模型，以图形化方式呈现业务数据的流动和处理过程。数据字典则是关于数据的信息集合，用于对数据流图中每个组成部分加以定义和说明。

在软件设计阶段：数据流图主要用于经过一系列设计转换后，产生由模块图表示的软件设计模型；详细设计阶段数据流图可进行模块内部的数据流设计。数据字典用于描述系统中各类数据，为数据库概要设计、逻辑设计提供支持。

(2021年11月系统架构设计师真题-第2题)

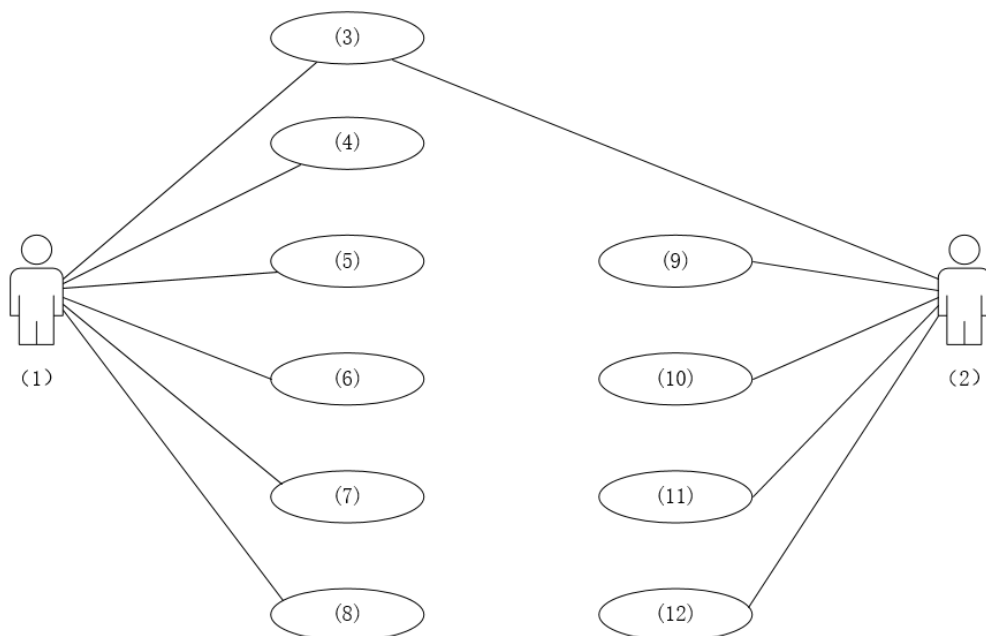
阅读以下关于软件系统设计与建模的叙述，在答题纸上回答问题1至问题3。

【说明】

某医院拟委托软件公司开发一套预约挂号管理系统，以便为患者提供更好的就医体验，为医院提供更加科学的预约管理。本系统的主要功能描述如下：(a) 注册登录，(b) 信息浏览，(c) 账号管理，(d) 预约挂号，(e) 查询与取消预约，(f) 号源管理，(g) 报告查询，(h) 预约管理，(i) 报表管理和(j) 信用管理等。

问题(1)

若采用面向对象方法对预约挂号管理系统进行分析，得到如图2-1所示的用例图。请将合适的参与者名称填入图2-1中的(1)和(2)处，使用题干给出的功能描述(a)~(j)，完善用例(3)~(12)的名称，将正确答案填在答题纸上。



思路解析

该问考查UML中的用例图填充，首先根据题意可以分析出患者这个参与者。而另一个参与者题目没有明示，然而从号源管理、预约管理等用例来看，定性为"医院管理员"较为合适，医院管理员是一个系统中比较常见的角色，起系统管理职能。然后通过用例的名称来分析判断哪些用例归属于患者哪些归属于医院管理员，按这个逻辑很容易分析出：

患者：（a）注册登录（b）信息浏览（c）账号管理（d）预约挂号（e）查询与取消预约（g）报告查询

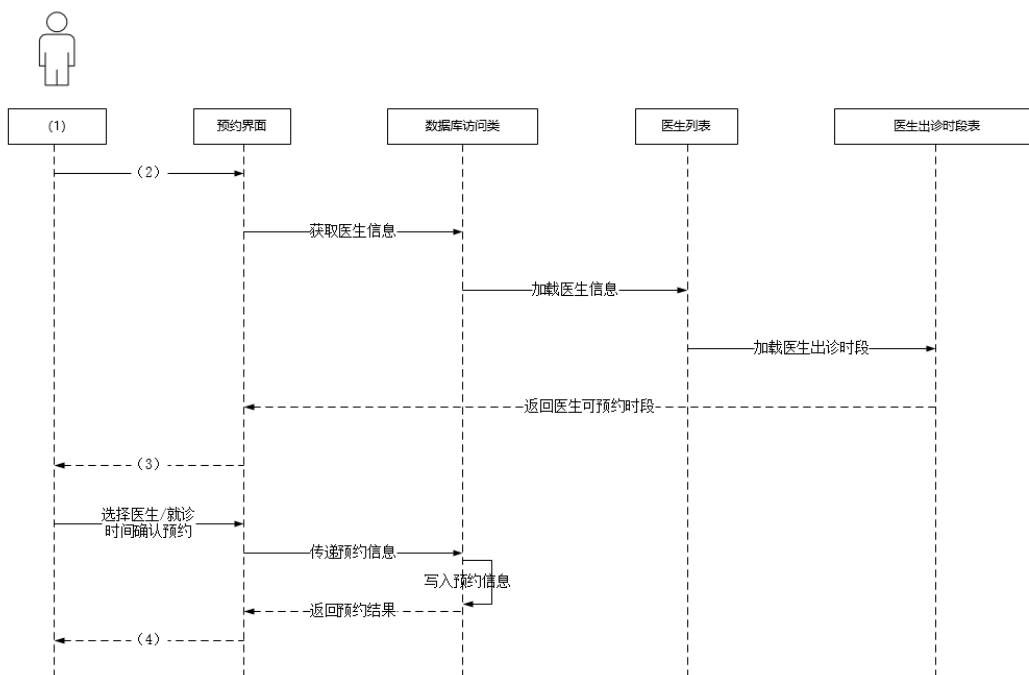
医院管理员：（a）注册登录（f）号源管理（h）预约管理（i）报表管理（j）信用管理

从而根据图中参与者对应的用例数给参与者和用例定位到具体的空中。

问题（2）

预约人员(患者)登录系统后发起预约挂号请求，进入预约界面。进行预约挂号时使用数据库访问类获取医生的相关信息，在数据库中调用医生列表，并调取医生出诊时段表，将医生出诊时段反馈到预约界面，并显示给预约人员；预约人员选择医生及就诊时间后确认预约，系统反馈预约结果，并向用户显示是否预约成功。

采用面向对象方法对预约挂号过程进行分析，得到如图2-2所示的顺序图，使用题干中给出的描述，完善图2-2中对象(1)，及消息(2)~(4)的名称，将正确答案填在答题纸上，请简要说明在描述对象之间的动态交互关系时，协作图与顺序图存在哪些区别。



思路解析

顺序图的核心是 **消息传递的时间先后关系**，重点表现从用户发出请求到系统反馈结果的过程。

根据题意，首先由患者发出预约挂号请求 (2)；系统调取医生时段并返回可预约时间 (3)；最终，系统确认预约并反馈结果 (4)。在 UML 中，**顺序图用于刻画“什么时候发生什么”**，强调交互的时间序列；而协作图（通信图）则更适合表现 **对象之间的静态关系和消息连接**，即“谁与谁通信”。两者都属于交互图，互为补充。

问题（3）

采用面向对象方法开发软件，通常需要建立对象模型、动态模型和功能模型，请分别介绍这3种模型，并详细说明它们之间的关联关系，针对上述模型，说明哪些模型可用于软件的需求分析？

思路解析

该题考查 OMT 方法的 **OOA 三模型**。对象模型关注系统的静态结构，帮助识别系统中的核心数据实体及关系；动态模型则描述对象随事件触发而发生的状态变化，强调系统的时间逻辑与控制流；功能模型则面向数据处理过程，反映用户需求对“功能”的直观要求。三者存在紧密关联：**对象模型提供系统实体，功能模型定义系统功能，动态模型决定何时触发何种功能**。三者结合起来，共同反映系统需求的全貌。由于需求分析既要明确数据、功能，又要体现控制逻辑，因此三者均可在需求分析阶段使用。

答案-问题（1）

- (1) 预约人员（患者）
- (2) 医院管理人员
- (3) (a) 注册登录
- (4) - (8) (b) 信息浏览 (c) 账号管理 (d) 预约挂号 (e) 查询与取消预约 (g) 报告查询
- (9) - (12) (f) 号源管理 (h) 预约管理 (i) 报表管理 (j) 信用管理

答案-问题（2）

- (1) 预约人员（患者）
- (2) 预约挂号请求
- (3) 显示医生可预约时段

(4) 显示预约是否成功

顺序图强调的是对象交互的时间次序。通信图强调的是对象之间的组织结构。

答案-问题（3）

对象模型用于描述系统数据结构：动态模型用于描述系统控制结构：功能模型用于描述系统功能。

这3种模型都涉及数据、控制和操作等共同的概念，但侧重点不同，从不同侧面反映了系统的实质性内容，综合起来全面地反映了对目标系统的需求。

功能模型指明了系统应该"做什么"：动态模型明确规定了什么时候做：对象模型则定义了做事的实体。

对象模型、动态模型和功能模型均可用于软件的需求分析。

(2020年11月系统架构设计师真题-第2题)

阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

国内快递包裹详情单（通知单联）

中国邮政 CHINA POST

KA 01905413323

接收局号码: 邮1106乙

收件人: 姓名: 电话: 单位名称: 详细地址: 是否保价: 是 ☐ 否 ☐ 寄件人名章:

寄件人: 姓名: 电话: 单位名称: 详细地址: 保价金额: 元 重量: 克 资费: 元 挂号费: 元 保价费: 元 回收费: 元 总计: 元

用户代码: 邮政编码: 签字: 检查人员名章:

KA 0190-5413 3 23 填写本单前, 请阅读本单背面“使用须知”! 您的签名意味着您理解并接受“使用须知”内容。

国家邮政局监制 10-10-013

图 2-1 包裹单示意图

CSDN @琅琊之榜PJ

[说明]

某企业委托软件公司开发一套包裹信息管理系统，以便于对该企业通过快递收发的包裹信息进行统一管理。在系统设计阶段，需要对不同快递信息的包裹单信息进行建模，其中，邮政包裹单如图 2-1 所示：

问题（1）

请说明关系型数据库开发中，逻辑数据模型设计过程包含哪些任务？该包裹单的逻辑数据模型中应该包含哪些实体？并指出每个关系模式的主键属性。

思路解析

逻辑数据模型设计是数据库开发的**核心步骤**，主要任务是将需求分析阶段得到的 E-R 模型转化为关系模型。

其过程通常包含：①**确定数据模型**：选择关系模型作为描述工具；②**将 E-R 转换为关系模式**：识别实体及其属性，明确主键与外键；③**确定完整性约束**：如实体完整性（主键唯一非空）、参照完整性（外键依赖）、用户自定义完整性（资费、费用范围约束等）；④**确定用户视图**：保证不同角色用户能够基于逻辑模型进行数据访问。

在本题所给的邮政包裹单中，至少应抽取三个实体：收件人、寄件人、包裹单。收件人与寄件人分别以电话为主键，因为电话在业务中往往是唯一标识用户的属性；包裹单则以编号为主键，唯一标识每一份快递记录。这种设计保证了实体之间的唯一性和完整性。

在关系模式层面，还应注意实体间的关联，例如包裹单与寄件人、收件人之间存在多对一的联系，可通过外键实现。通过这样的逻辑数据模型，可以支持系统的统一管理和高效查询。

问题（2）

请说明什么是超类实体？结合图中包裹单信息，试设计一种超类实体，给出完整的属性列表。

思路解析

在数据库设计中，**超类实体**用于抽取多个实体间的共同属性，以减少冗余并提高一致性。

子类实体则继承超类的公共属性，并在此基础上定义自身的特有属性。

在本题场景下，“收件人”和“寄件人”虽然是两个不同角色，但都包含 **姓名、电话、单位名称、详细地址** 等共性字段。如果不进行抽象，就会在数据库中形成两份重复的字段定义，既浪费存储，又可能导致数据不一致。

因此，可以抽象出一个名为“用户”的超类实体，统一保存公共信息；寄件人与收件人则作为用户的不同角色，通过额外字段（如角色类型）或关系表加以区分。这样设计能使数据库更简洁，维护成本更低，同时方便后续扩展（如未来增加“中转站联系人”等新的角色，也可复用该超类结构）。

问题（3）

请说明什么是派生属性？结合图2-1中包裹单信息说明哪个属性是派生属性。

思路解析

派生属性指不必单独存储，而可通过计算得出的属性。

它的值依赖于同一实体或相关实体的其他属性。例如，在工资管理系统中，“年收入”可以由“月薪×12”计算得出。在本题的包裹单中，“总计”费用显然不是独立输入的，而是由多个字段（资费、挂号费、保价费、回执费）相加计算得出，因此“总计”是典型的派生属性。

在逻辑建模中，对派生属性的处理一般有两种方式：①不存储，仅在需要时由系统计算（保证一致性，节省空间）；②冗余存储，用于提高查询效率，但需维护触发器或存储过程，确保在源字段更新时同步更新派生属性。对于本系统而言，“总计”常用于前端显示和业务核算，因此可采用存储计算并显示，或采用 SQL 视图动态生成。合理识别派生属性，有助于模型既保证**数据一致性**，又在**性能需求与存储代价**之间取得平衡。

答案-问题（1）

逻辑数据模型设计过程包含的任务：（1）确定数据模型（2）将E-R 图转换成为指定的数据模型（3）确定完整性约束和（4）确定用户视图。

包裹单的逻辑数据模型中包含的实体：（1）收件人（主键：电话）；（2）寄件人（主键：电话）；（3）包裹单（主键：编号）。

答案-问题（2）

超类实体是将多个实体中相同的属性组合起来构造出的新实体。用户（姓名、电话、单位名称、详细地址）

答案-问题（3）

派生属性是指某个实体的非主键属性由该实体其他非主键属性决定。包裹单中的总计是由资费、挂号费、保价费、回执费计算得出，所以是派生属性。

阅读以下关于软件系统建模的叙述，在答题纸上回答问题1至问题3。

【说明】

某公司欲建设一个房屋租赁服务系统，统一管理房主和租赁者的信息，提供快捷的租赁服务。本系统的主要功能描述如下：

- 登记房主信息。记录房主的姓名、住址、身份证号和联系电话等信息，并写入房主信息文件。
- 登记房屋信息。记录房屋的地址、房屋类型（如平房、带阳台的楼房、独立式住宅等）、楼层、租金及房屋状态（待租赁、已出租）等信息，并写入房屋信息文件。一名房主可以在系统中登记多套待租赁的房屋。
- 登记租赁者信息。记录租赁者的个人信息，包括：姓名、性别、住址、身份证号和电话号码等，并写入租赁者信息文件。
- 安排看房。已经登记在系统中的租赁者，可以从待租赁房屋列表中查询待租赁房屋信息。租赁者可以提出看房请求，系统安排租赁者看房。对于每次看房，系统会生成一条看房记录并将其写入看房记录文件中。
- 收取手续费。房主登记完房屋后，系统会生成一份费用单，房主根据费用单交纳相应的费用。
- 变更房屋状态。当租赁者与房主达成租房或退房协议后，房主向系统提交变更房屋状态请求。系统将根据房主的请求，修改房屋信息文件。

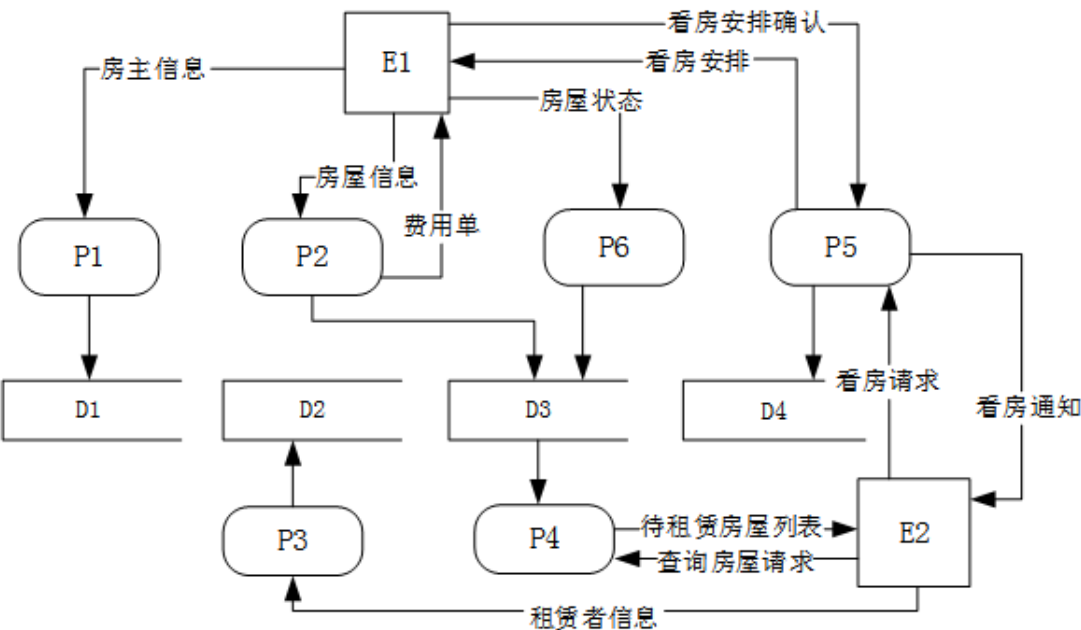


图 2-1 房屋租赁服务系统顶层 DFD

问题（1）

若采用结构化方法对房屋租赁服务系统进行分析，得到如图2-1所示的顶层DFD。使用题干中给出的词语，给出图2-1中外部实体E1~E2、加工P1~P6以及数据存储D1~D4的名称。

思路解析

外部实体可以是和系统交互的人或角色，以及和系统交互的外部系统或服务。根据题目中的描述，与本系统进行交互的角色是房主和租赁者。根据E1和P1之间的数据流“房主信息”，结合题目描述可知，E1表示的是房主，E2表示的是租赁者。

题目的描述中已经明确给出了系统的6个功能，需要将这些功能与加工P1~P6进行对应，这需要借助于各个加工的输入输出数据流进行分析，根据E1和P1之间的数据流“房主信息”可知，这条数据流符合“登记房主信息”功能的描述，因此可以确定P1是“登记房主信息”，同时可以确定D1是“房主信息文件”。E1和P2之间的数据流“房屋信息”“费用单”，这些都与房屋登记相关，因此P2是“登记房屋信息”。同时可以确定，D3对应的是“房屋信息文件”。同理，根据数据流及题干描述，可以推断出：P3对应“登记租赁者信息”、P4对应“查询租赁房屋信息”、P5对应“安排看房”以及P6对应“变更房屋状态”。

问题（2）

若采用信息工程（Information Engineering）方法对房屋租赁服务系统进行分析，得到如图2-2所示的ERD。请给出图2-2中实体（1）~（5）的名称。

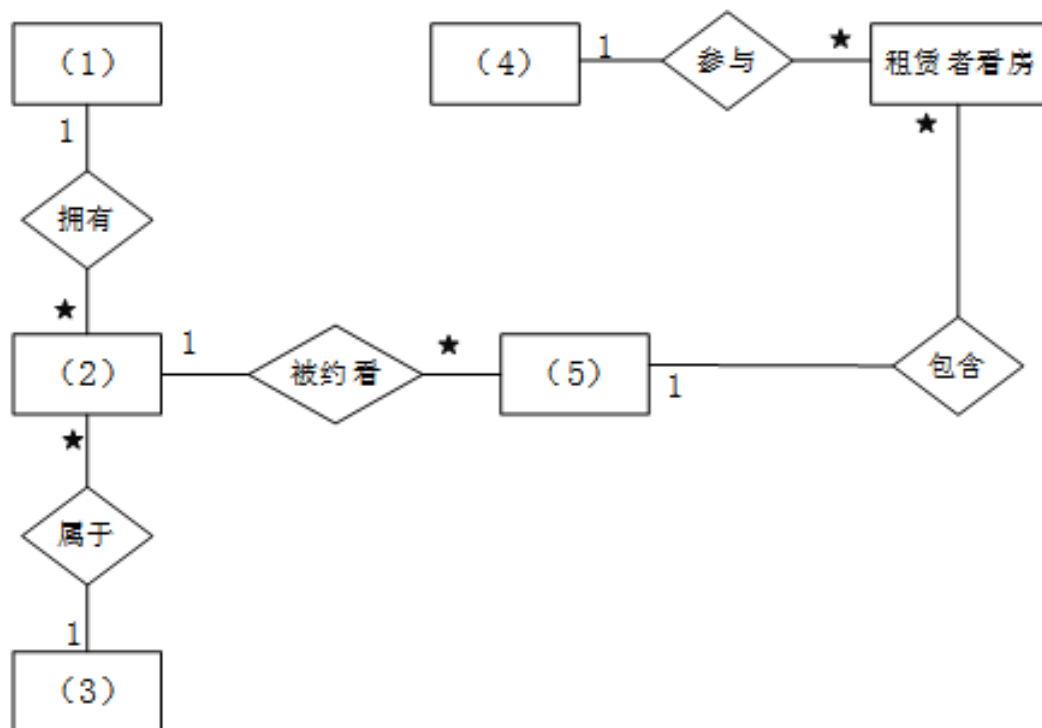


图 2-2 房屋租赁服务系统 ERD

思路解析

本问题考查信息工程方法中的模型ER图。ER图中包含两个主要元素：实体和联系。实体是现实世界中可以区别于其他对象的"事件"或"物体"，本题要求补充图2-2中的实体。根据题目描述和实体之间的联系可知，（1）和（2）分别对应房主和房屋，两者之间的联系为"房主拥有房屋"。同理可以推断出，（3）~（5）分别是实体"房屋类型" "租赁者"和"看房安排"。

这里比较容易错的是房屋类型有的同学写成房屋信息。他的依据是看到"登记房屋信息。记录房屋的地址、房屋类型（如平房、带阳台的楼房、独立式住宅等）、楼层、租金及房屋状态（待租赁、已出租）等信息，并写入房屋信息文件。"

这里你要看到（3）这个位置要和其他内容是 1 对多的关系，一个房屋信息只能对应一个房屋，所以这里比较合理的显然是房屋类型。

另外最难的是房屋安排。这个几乎做不对。因为题目里没有提到房屋安排和租客看房还有一对多的关系。这个留给大家讨论。

问题（3）

（1）信息工程方法中的"实体（entity）"与面向对象方法中的"类（class）"之间有哪些不同之处？

（2）在面向对象方法中通常采用用例（Use Case）来捕获系统的功能需求。用例可以按照不同的层次来进行划分，其中的Essential Use Cases和Real Use Cases有哪些区别？

请用100字以内文字解释说明上述两个问题。

思路解析

实体（Entity）是信息工程方法中的核心概念，强调数据存储结构，主要描述属性和关系；而类

(Class)是面向对象的核心单元,除属性外,还包含方法,强调封装和行为。两者虽然都用于建模,但出发点不同:前者面向数据,后者面向对象。

用例(Use Case)是面向对象需求分析的重要工具。**Essential Use Case**(抽象用例)着眼于用户目标和系统职责,不包含UI或技术实现,强调“做什么”;**Real Use Case**(真实用例)则面向设计与实现,描述具体的操作流程、界面、输入输出,强调“怎么做”。两者的关系类似于需求分析与系统设计的分工,前者抽象,后者具体。

答案-问题(1)

E1: 房主 E2: 租赁者

P1: 登记房主信息 P2: 登记房屋信息

P3: 登记租赁者信息 P4: 查询租赁房屋信息

P5: 安排看房 P6: 变更房屋状态

D1: 房主信息文件 D2: 租赁者信息文件

D3: 房屋信息文件 D4: 看房记录文件

答案-问题(2)

(1) 房主 (2) 房屋 (3) 房屋类型 (4) 租赁者 (5) 看房安排

答案-问题(3)

(1) 实体用于数据建模,而类用于面向对象建模。实体只有属性,而类有属性和操作。

(2) Essential Use Cases 可翻译为抽象用例,Real Use Cases 可翻译为真实用例。它们是区别在于: Essential Use Cases 用于分析阶段,Real Use Cases 用于设计阶段。Essential Use Cases 描述用例的本质属性,它与如何实现这个用例无关,独立于实现该用例的软硬件技术。Real Use Cases 描述的是用例的实现方式,表达了设计和实现该用例时所采用的方法和技术。

(2016年11月系统架构设计师真题-第2题)

阅读以下关于软件系统建模的叙述,在答题纸上回答问题1至问题3。

【说明】

某软件公司计划开发一套教学管理系统,用于为高校提供教学管理服务。该教学管理系统基本的需求包括:

- (1) 系统用户必须成功登录到系统后才能使用系统的各项功能服务;
- (2) 管理员(Registrar)使用该系统管理学校(University)、系(Department)、教师(Lecturer)、学生(Student)和课程(Course)等教学基础信息;
- (3) 学生使用系统选择并注册课程,必须通过所选课程的考试才能获得学分;如果考试不及格,必须参加补考,通过后才能获得课程学分;
- (4) 教师使用该系统选择所要教的课程,并从系统获得选择该课程的学生名单;
- (5) 管理员使用系统生成课程课表,维护系统所需的有关课程、学生和教师的信息;
- (6) 每个月到了月底系统会通过打印机打印学生的考勤信息。

项目组经过分析和讨论,决定采用面向对象开发技术对系统各项需求建模。

问题(1)

用例建模用来描述待开发系统的功能需求,主要元素是用例和参与者。请根据题目所述需求,说明教学服务系统中有哪些参与者。

思路解析

“参与者”是指系统外部与系统发生交互的实体,可能是用户、设备或其他系统。

本题中,显性参与者为“学生”“教师”“管理员”;隐性参与者为“时间”(定时触发打印考勤

) 和“打印机”(执行打印操作)。

问题(2)

用例是对系统行为的动态描述,用例获取是需求分析阶段的主要任务之一。请指出在面向对象系统建模中,用例之间的关系有哪几种类型?对题目所述教学服务系统的需求建模时,"登录系统"用例与"注册课程"用例之间、"参加考试"用例与"参加补考"用例之间的关系分别属于哪种类型?

思路解析

用例之间的关系主要有泛化(Generalization)、包含(Include)和扩展(Extend)。

(1) 当可以从两个或多个用例中提取公共行为时,可以使用包含关系来表示。

(2) 如果一个用例混合了两种或两种以上不同场景,即根据情况可能发生多种分支,则可以将这个用例分为一个基本用例和一个或多个扩展用例。

(3) 当多个用例共同拥有一个类似的结构和行为的时候,可以将它们的共性抽象成父用例,其他的用例作为泛化关系中的子用例。

在本题中,"注册课程"用例在执行前必须登录系统,因此"登录系统"是其**必要行为**,为包含关系;而"补考"只有在考试不及格的情况下才会触发,不是所有学生都必须补考,因此为**扩展关系**。

问题(3)

类图主要用来描述系统的静态结构,是组件图和配置图的基础。请指出在面向对象系统建模中,类之间的关系有哪几种类型?对题目所述教学服务系统的需求建模时,类University与类Student之间、类University和类Department之间、类Student和类Course之间的关系分别属于哪种类型?

思路解析

类University与类Student之间的关系是整体与部分关系。并且我要换成其他业务系统的时候,可以不设计类University,类Student可以单独设计。他们没有紧密关系,也就是我说的具有不同的生存周期,口诀**好聚好散**,所以是聚合关系。

类University和类Department之间的关系是整体与部分的关系,两者具有相同的生存周期。你不能说设计一个系统,其中有类Department,但是没有有类University,这个说不通。所以它们之间是组合关系。

同理,类Student和类Course我认为也是聚合关系,但是官方是关联关系,这个说不太通。因为关联包含了聚合和组合,粒度太粗了。

答案-问题(1)

学生、教师、管理员、时间、打印机。

答案-问题(2)

用例之间的关系包括:包含、扩展、泛化。

"登录系统"用例与"注册课程"用例之间的关系为:包含关系。

"参加考试"用例与"参加补考"用例之间的关系为:扩展关系。

答案-问题(3)

类之间的关系包括:关联、聚合、组合、依赖、泛化、实现(可写可不写,因为实现是接口与类之间的关系,而接口是一种特殊的类)。

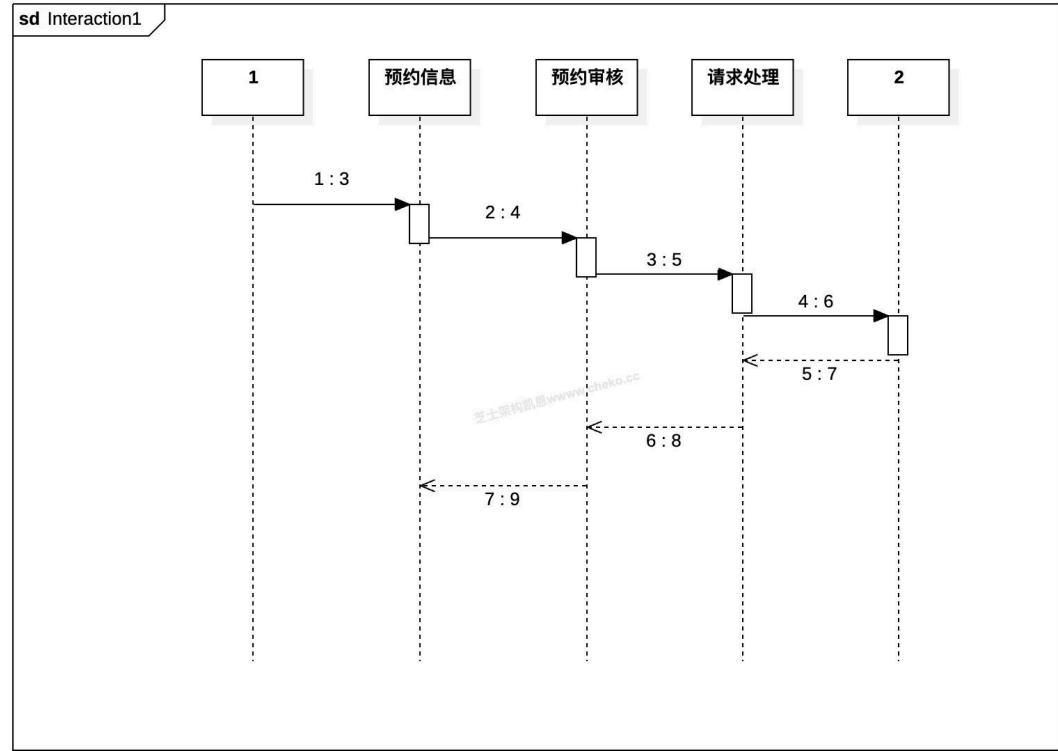
类University与类Student之间的关系是聚合关系。

类University与类Department之间的关系是组合关系。类Student与类Course之间的关系是关联关系。

第7章——顺序图与交互建模
(2024年5月系统架构设计师真题-第2题)

访问者通过微信公众号进入某单位的访客预约系统（以下简称“系统”），提交预约申请。系统将申请提交到预约审核模块进行审批。若审核通过，系统会将预约信息推送给被访人员。被访人员处理完成后，系统需将处理完毕的消息通知访问者本人及门禁管理人员。

该业务流程以UML序列图的形式表现如下图所示，请结合图示与业务场景，回答下列问题：



问题（1）
在序列图存在哪三种消息？

思路解析

UML 序列图的核心是展示对象之间的交互过程，其中“消息”是最关键的元素。常见的三类消息如下：

- 同步消息：**表示发送者发出请求后必须等待接收者完成处理才能继续执行。这类消息体现了严格的时序依赖，常见于函数调用或 RPC 调用。通常以实线加实心箭头表示。
- 异步消息：**发送者发出消息后无需等待接收者完成即可继续自身的行为。这种消息体现了并行与解耦，适用于事件驱动、消息队列等场景。图中用实线加空心箭头表示。
- 返回消息：**表示接收者处理完毕后返回的结果，通常用虚线箭头表示，体现了操作完成的反馈过程。

问题（2）
请根据这个业务场景，补充序列图中标数字部分。

思路解析

在该预约流程中，交互主体包括访问者、系统、预约审核模块和被访者等对象。结合业务场景，序列图中的数字应填充如下：

- 1 表示访问者，其通过微信公众号发起操作；

- 2 表示被访者，系统在审核通过后将预约信息推送给其处理；
- 3-5 描述访问者提交预约内容，系统将申请转发到审核模块；
- 6 表示审核模块对预约请求进行审批；
- 7-9 表示审核结果与最终预约信息返回给访问者及相关方。

问题（3）

系统分析设计过程中两种交互图的选取原则。

思路解析

UML 提供了两种常用的交互图：序列图与通信图。

序列图注重时间维度的交互过程，强调“先后顺序”。当我们需要描述业务流程的动态行为，特别是涉及多个对象之间复杂的消息传递、异步调用或并发操作时，序列图非常直观。例如访客预约系统中，提交、审核、通知的流程顺序至关重要，因此选用序列图建模。

通信图则突出对象之间的静态结构关系，强调“谁与谁交互”。当需要从系统结构的角度出发，展示对象间的连接关系和消息路由时，通信图更适合。它类似一张对象协作关系图，适合在系统结构设计阶段使用。

问题（4）

UML 序列图中表示条件分支的片段包括哪些？

思路解析

UML 序列图提供了多种片段（Combined Fragments）来表示复杂的控制逻辑，其中与条件分支或流程控制相关的主要有：

alt：用于表示互斥选择，类似 if-else 语句。例如在预约审核时，若审核通过则继续推送消息，否则结束流程。

opt：表示某一操作是可选的，类似 if 条件。例如系统是否通知门禁管理人员可以用 opt 表达。

loop：用于表示循环操作，如系统批量处理多个预约申请。

break：表示跳出当前片段，中断执行流程。

par：表示多个分支可以并行执行。

region：表示临界区，保证片段中的操作互斥执行。

答案-问题（1）

同步消息（Synchronous Message）

异步消息（Asynchronous Message）

返回消息（Return Message）

答案-问题（2）

1. 访问者
2. 被访者
3. 提交预约内容
4. 提交申请
5. 提交审核
6. 审核处理
7. 返回处理结果
8. 返回审核结果
9. 返回预约结果

答案-问题（3）

在系统分析设计过程中，我们常用两种交互图来描述系统的动态行为：序列图（顺序图）和通信图（或者说协作图）。

当你需要清晰地展示消息的时间顺序，以及对象之间交互的时序关系时，选择序列图。序列图强调的是时间顺序，它像电影胶片一样，按时间顺序展示了每个对象发送和接收消息的时刻。适用于描述复杂交互流程，特别是涉及多个对象之间消息传递顺序至关重要的场景，例如异步操作、并发行为等。

当你需要清晰地展示对象之间的静态关系，以及它们之间如何通过消息进行协作时，选择通信图。通信图强调的是对象之间的连接关系，它像地图一样，展示了对象之间的链接路径以及消息如何在这些路径上传递。适用于描述系统结构，以及对象之间协作关系，特别是需要清晰地展现对象之间静态连接关系的场景。

答案-问题（4）

UML序列图中表示条件分支的片段包括：

alt (Alternative)：表示“如果-否则”条件分支（if-else）

opt (Optional)：表示可选执行（if）

loop (Loop)：表示循环执行（如for、while）

break：表示中断流程的执行（类似break语句）

par (Parallel)：表示并发分支执行

region (Critical)：表示临界区，即互斥执行块

第8章——业务流程与活动建模 (2019年11月系统架构设计师真题-第2题)

阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

【说明】

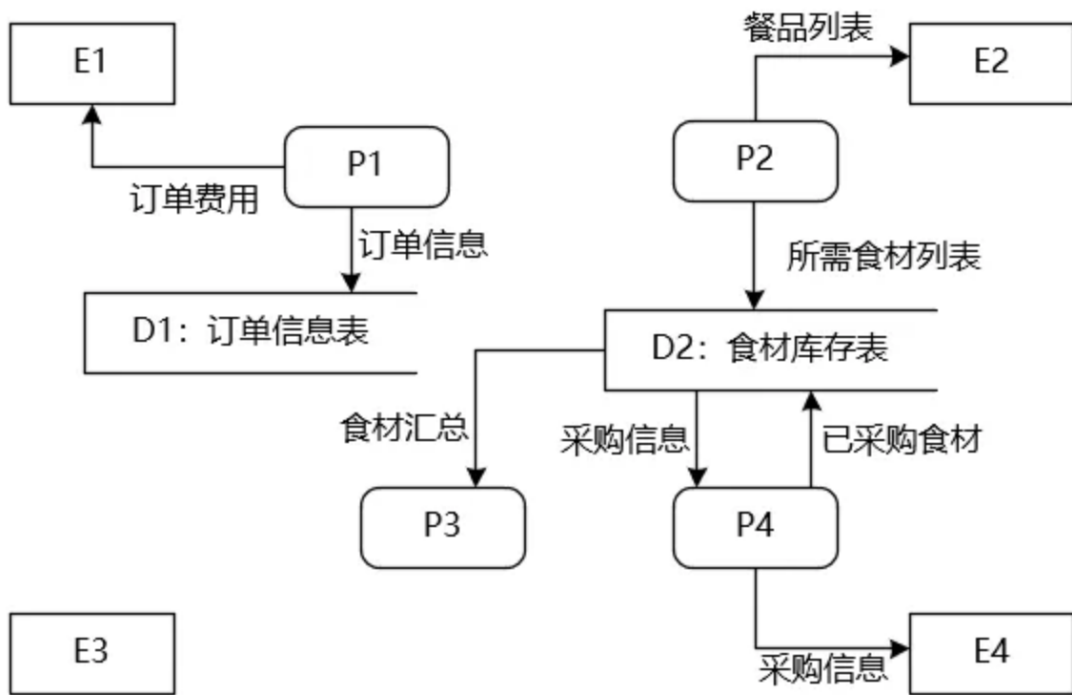
某软件企业为快餐店开发一套在线订餐管理系统，主要功能包括：

（1）在线订餐：已注册客户通过网络在线选择快餐店所提供的餐品种类和数量后提交订单，系统显示订单费用供客户确认，客户确认后支付订单所列各项费用。

（2）厨房备餐：厨房接收到客户已付款订单后按照订单餐品列表选择各类食材进行餐品加工。

（3）食材采购：当快餐店某类食材低于特定数量时自动向供应商发起采购信息，包括食材类型和数量。供应商接收到采购信息后按照要求将食材送至快餐店并提交已采购的食材信息。系统自动更新食材库存。

（4）生成报表：每个周末和月末，快餐店经理会自动收到系统生成的统计报表，报表中详细列出了本周或本月订单的统计信息以及库存食材的统计信息。现采用数据流图对上述订餐管理系统进行分析与设计，系统未完成的0层数据流图。如图2-1 所示。



问题（1）

（8分）根据订餐管理系统功能说明，请在图2-1所示数据流图中给出外部实体E1~E4和加工P1~P4的具体名称。

思路解析

判定外部实体与处理过程的关键，是紧扣题干对**业务边界与职责**的描述，并遵循数据流图（DFD）关于**实体（External Entity）、加工/处理（Process）、数据存储（Data Store）、**数据流（Data Flow）****的语义。

题干中的四类对外参与者明确为：**客户**（发起下单与支付、接收订单确认）、**厨房**（依据已付款订单进行备餐）、**供应商**（按系统自动发起的需求补货并上报采购明细）、**经理**（按周末/月末接收系统统计报表）。这四者都位于系统边界之外，承担输入或接收输出的角色，因此应标注为 E1~E4。四个核心业务活动恰对应四个**加工（P）**：**在线订餐**（订单创建、费用计算与确认支付）、**厨房备餐**（根据订单清单组织生产）、**食材采购**（库存阈值触发、发送采购信息、供应商回填与库存更新）、**生成报表**（定时汇总订单与库存信息并通知经理）。

总体来看就是，客户提交“餐品订单”→系统确认费用与支付→“已付款订单”触发厨房备餐；库存监测触发“采购信息”→供应商送货并反馈“已采购食材信息”→系统更新库存；到期定时任务触发“生成报表”→经理接收“统计报表”。

这些触发和输出与四个加工的**职责一一对应**，不存在跨层代办或职责重叠，符合**单一职责与层次清晰**的 DFD 规范。此外，注意不要把**仓储/库存、订单库**等数据持久化对象误标为外部实体，它们是系统**内部的数据存储（D）**，通常在 0 层 DFD 中以 D1、D2 等表示。综上，E/P 的命名与归属与题干描述严格吻合，既能支撑后续数据流补全，又能保证 0 层图的**语义一致性与可读性**。

问题（2）

（8分）根据数据流图规范和订餐管理系统功能说明，请说明在图2-1中需要补充哪些数据流可以构造出完整的0层数据流图。

思路解析

补流的逻辑依据主要有三点：（A）**黑洞/灰洞/无输入错误检查**，（B）**平衡性（Balancing）**，（C）**业务时序与触发机制**。

第一，自检“无输入/无输出”。若某处理只有输出无输入（或反之），就是**黑洞/白洞**错误。题干提示 P1“在线订餐”只有输出（例如生成订单确认或已付款订单），但未见客户输入数据，违反

“加工必须由输入驱动”的规则，因此需要补充 E1→P1 的“餐品订单”数据流，以表达“客户选择餐品并提交订单”的初始输入。同理，P2“厨房备餐”应由已付款订单触发。如果 0 层不细分支付环节，可将“已付款订单”抽象为“餐品订单”（含“已支付”状态）从 P1→P2 直接传递，体现事件驱动的实时性与可追溯性，这也优于“P2 主动去 D1 检索”的轮询式触发。

第二，校验 P3“生成报表”的输入与输出平衡。根据题干，报表至少包含订单统计与库存统计两部分。0 层可将订单侧的输入抽象为 D1→P3 的“订单汇总”，库存侧若已在图中体现为另一数据存储（如 D2 库存），还应并行输入“库存汇总”；若题面仅要求最小必要补流，至少需要“订单汇总”。另一方面，P3 不能是“黑洞”，应有输出流指向经理（E3），即 P3→E3 的“统计报表”。

第三，事件驱动优于轮询驱动。为何不是 D1→P2 而是 P1→P2？如果用 D1→P2 表达“厨房从数据库主动拉单”，则备餐触发依赖人工/定时轮询，实时性与业务一致性较弱；P1→P2 则在订单确认后即时推送，满足“支付即出单”的餐饮场景，更贴近真实业务。此处体现 DFD 设计中对触发条件与时效性的把握：尽量让上游加工的确定性输出成为下游加工的直接输入，减少不必要的数据库轮询。

问题（3）

根据数据流图的含义，请说明数据流图和系统流程图之间有哪些方面的区别。

思路解析

数据流图（DFD）与系统流程图（Flowchart）都是结构化分析/设计阶段的常用图法，但二者的关注点与抽象层次不同。

其一，在并行性与执行观念上不同。DFD 描述的是系统在逻辑层的“数据如何在系统内外与各处理之间流动”以及“被何种加工转换”，不等价于单线程的执行轨迹；不同处理可以由不同事件/定时/条件触发，天然支持并行/并发表达（例如“在线订餐”与“食材采购”的事件驱动可并行发生）。而系统流程图更多是沿着一条控制流路径，自上而下刻画“此刻先做什么，再做什么”，默认单条控制线推进，强调时序步骤与分支/循环。

其二，在描述对象上不同。DFD 的核心元素是外部实体、加工、数据存储、数据流，突出数据的产生、汇聚、分发与落库，回答“数据从哪里来、到哪里去、被怎样变换”。系统流程图的元素是处理框、判断框、连接/流程线等，突出控制条件与执行路径，回答“在什么条件下走哪条路、下一步是什么”。因此，DFD 更适合需求分析阶段的逻辑建模与跨域数据依赖阐明；流程图更适合设计/实现阶段的过程控制与算法/业务步骤表达。

其三，在“计时标准/触发节拍”的表达上不同。DFD 中的各处理可由异步事件、定时任务、外部输入等多源触发，不要求整体服从统一时钟；同一 0 层图内，不同加工可有不同的采样周期、批处理窗口或实时推送机制。而流程图常假定同一条控制流在统一“执行时序”下推进（尽管可含子流程/并发分支，但在表达上仍以控制先后为主）。

在本题场景中，用 DFD 搭建 0 层图能够清晰地把“客户下单”“厨房备餐”“供应商补货”“经理收报表”等数据关系与系统边界讲明白；若需要描述“下单后如何进行支付校验、失败重试、超时回滚”等控制细节，再用系统流程图或活动图/时序图表达，会更贴切开发实现需要。

答案-问题（1）

E1: 客户

E2: 厨房

E3: 经理

E4: 供应商

P1: 在线订餐

P2: 厨房备餐

P3: 生成报表

P4: 食材采购

答案-问题（2）

- （1）增加E1到P1数据流"餐品订单"；
- （2）增加P1到P2数据流"餐品订单"；
- （3）增加D1到P3数据流"订单汇总"；
- （4）增加P3到E3数据流"统计报表"。

答案-问题（3）

- （1）数据流图中的处理过程可并行；系统流程图在某个时间点只能处于一个处理过程。
- （2）数据流图展现系统的数据流；系统流程图展现系统的控制流。
- （3）数据流图展现全局的处理过程，过程之间遵循不同的计时标准；系统流程图中处理过程遵循一致的计时标准。

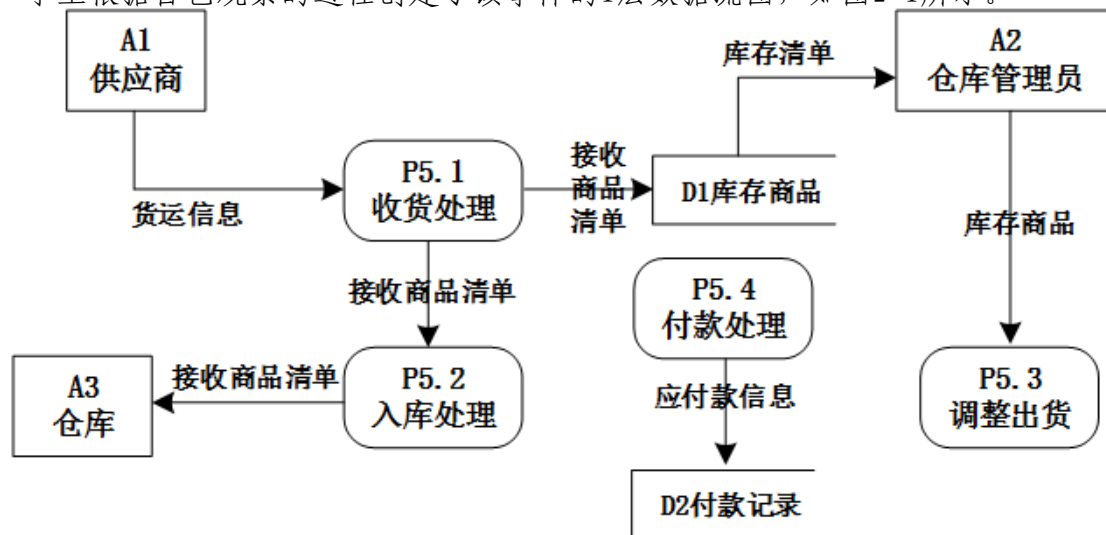
（2014年11月系统架构设计师真题-第2题）

请详细阅读以下关于系统过程建模的说明，在答题纸上回答问题1至问题3。

【说明】

某公司正在研发一套新的库存管理系统。系统中一个关键事件是接收供应商供货。项目组系统分析员小王花了大量时间在仓库观察了整个事件的处理过程，并开发出该过程所执行活动的列表：供应商发送货物和商品清单。公司收到商品后执行收货处理，包括卸载商品、确定收到了订单上的商品、处理与供应商的分歧等。对于已有商品，调整其库存信息，对于新采购的商品，在库存中添加新的商品记录。收货完成后，系统执行入库处理，将商品放到仓库对应的货架上。在付款处理活动中，自动生成应付账款信息，如果查询到该供应商有待付款记录，则进行合并付款，付款完成后消除应付账款记录。最后，仓库管理员根据最新的库存商品，调整出货信息。

小王根据自己观察的过程创建了该事件的1层数据流图，如图2-1所示。



问题（1）

请用300以内文字说明数据流图（Data Flow Diagram）的基本元素及其作用。

思路解析

数据流图（DFD）是结构化分析方法中用于表达逻辑数据运动与处理的核心模型，强调“做什么而非怎么做”。其基本元素与画法遵循一套统一约束，以保证图面的可读性、一致性和可验证性。首先，外部实体承担环境角色，是系统边界的锚点：例如“供应商”“银行支付网关”“仓库管理员”等。它们不会在系统内部实现任何处理逻辑，仅通过数据流与系统交互，从而明确系统的输入输出边界。其次，加工（Process）体现系统行为，是把输入数据转化为输出结果的功能节点。加工必

须既有输入又有输出，否则就会出现黑洞或奇迹等语义错误。加工的命名宜采用“动宾短语”（如“生成应付账款”“核对到货清单”），以突出处理动作和对象。第三，**数据存储**表示静态数据的安置之所，强调数据的持久化存在。它不能直接与外部实体或其他数据存储相连，**必须经过某个加工**来读写，这是 DFD 的重要语法约束之一。第四，**数据流**用带箭头的线段表示，明确数据**从哪里来、到哪里去、包含哪些关键数据项**（可在流名中体现，如“到货清单”“对账结果”），并且一条流的起点与终点必须是**加工、数据存储或外部实体**之一，不能指向自己，也不能悬空。

从建模价值看，DFD 的四要素共同完成了从业务观察到系统逻辑的抽象：**外部实体**规定了系统边界与交互角色；**数据流**承载业务语义的传递；**加工**定义了价值创造与数据变换；**数据存储**保证信息可追溯与可复用。配合**分层分解（Level-0、Level-1...）与平衡原则（Balancing）**，可将复杂业务在不同粒度层面清晰展开；同时，通过**命名一致性与接口对齐**，为后续的数据字典、实体关系、用例与流程协同提供依据。把握这些元素及其约束，有助于在库存管理场景中准确刻画“收货、入库、对账、付款、更新库存、调整出货信息”等活动的逻辑数据通道，避免实现阶段因理解偏差产生的返工。

问题（2）

数据流图在绘制过程中可能出现多种语法错误，请分析图2-1所示数据流图中哪些地方有错误，并分别说明错误的类型。

思路解析

数据流图的常见错误分为**逻辑错误**与**语法错误**两大类。**逻辑错误**主要指加工节点输入输出不平衡：

- ①**黑洞**——加工只有输入无输出，好比数据“被吞掉”；
- ②**奇迹**——加工无输入却能输出数据，仿佛“凭空变出”；
- ③**灰洞**——输入与输出在语义或数量级上不相称（如“输入一个订单号，输出整套综合统计报表”），违反了合理的因果与守恒。

语法错误聚焦于连线/对象的法定组合关系，例如**数据存储与外部实体、数据存储与数据存储之间禁止直连**，必须经由加工读写；不允许**悬空数据流**；同一层级图的**父子平衡**也必须成立，即上层输入/输出在下层应能追溯到具体加工与数据存储的组合。

回到图2-1：

- P5.3** 只有输入无输出，典型**黑洞**。这往往意味着该处理缺少对外可观察的结果（如状态更新、回执、日志入库等），会导致数据“无处安放”。
- P5.4** 只有输出无输入，是**奇迹**，说明缺少必要的触发或来源（例如应有的“应付账款信息”“对账结果”等输入）。
- D1→A2** 直接连线违反了**数据存储不可直接与外部实体相连**的规则，正确做法应当是：由某个**加工**从 D1 读取或写入，再与 A2 交互（如“生成对账单并发送给供应商”）。
- D2** 没有输出数据流，体现为数据只“写入不被读取”或“仅被动堆积”。就业务而言，库存管理场景应当存在从 D2（**库存/商品数据**）输出到“入库处理、出货调整、对账/对外查询”的流，如果缺失，说明**数据使用闭环不完整**。

问题（3）

系统建模过程中为了保证数据模型和过程模型的一致性，需要通过数据-过程-CRUD矩阵来实现数据模型和过程模型的同步，请在表2-1所示CRUD矩阵（1）~（5）中填入相关操作。

表 2-1 接收供应商供货的 CRUD 矩阵

	P5.1 收货处理	P5.2 入库处理	P5.3 调整出货	P5.4 付款处理
供应商	(1)			(2)
库存商品		(3)	(4)	
付款记录				(5)

思路解析

数据过程CRUD 矩阵用于把“过程模型（DFD 的加工/活动）”与“数据模型（实体/数据存储）”逐格映射，确保读写关系全覆盖、无越权、无遗漏。其中：**C (Create)** 创建新记录，**R (Read)** 读取，**U (Update)** 更新，**D (Delete)** 删除。构建矩阵的关键是基于业务语义逐一判断每个加工对每个数据存储的操作类型，并与DFD中的数据流方向、名称、含义保持一致，形成一致性与可追溯性证据链。

结合题干场景逐格解释：

(1) **R**：如“收货处理/核对到货清单”需要**读取**“供应商信息”之类的主数据以比对收货项，属于**Read**。

(2) **R**：付款处理需要**读取**供应商档案、历史付款记录或应付策略等主数据，确定付款方式与合并策略，故为**Read**。

(3) **CRU**：对**库存商品**而言，收货完成后会对“已有商品”**更新库存数量 (U)**，而**新采购商品**会**创建新商品记录 (C)**；同时在质检或上架确认后可能**再次更新 (U)** 状态/库位等，因此为 **CRU**。

(4) **RU**：调整出货时，系统需要 **读取 (R) 库存商品** 来确认当前库存数量；同时，出货操作可能导致库存商品的数量 **更新 (U)**。但是出货过程并不会新建库存，也不会删除库存。所以 (4) 是 **RU**。

(5) **CRUD**：付款处理和付款记录的关系最为复杂：**C (Create)**：生成新的应付账款时，需要创建付款记录。**R (Read)**：合并付款、对账或查询历史账款时，需要读取付款记录。**U (Update)**：付款完成后，要更新付款状态（未付款 → 已付款），金额可能也要调整。**D (Delete)**：部分场景下，付款完成后旧的应付账款记录可能被清除或归档（逻辑删除）。这个有争议，就是付款记录是否有必要删除。

答案-问题 (1)

数据流：数据流是数据在系统内传播的路径，因此由一组成分固定的数据组成。

外部实体：代表系统之外的实体，可以是人、物或其他软件系统。

加工（处理）：加工是对数据进行处理的单元，它接收一定的数据输入，对其进行处理，并产生输出。

数据存储：表示信息的静态存储，可以是文件、文件的一部分、数据库的元素等。

答案-问题 (2)

1、加工P5.3仅有输入流，缺少输出流，这属于黑洞。

2、加工P5.4仅有输出流，缺少输入流，这属于奇迹。

3、D1到A2：缺少处理数据流的加工。

4、D2：数据存储没有输出的数据流。

答案-问题 (3)

(1) **R**

(2) **R**

(3) **CRU**

(4) **RU**

(5) **CRUD**

第9章——架构质量属性场景

(2019年11月系统架构设计师真题-第5题)

阅读以下关于Web系统架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某公司拟开发一个物流车辆管理系统，该系统可支持各车辆实时位置监控、车辆历史轨迹管理、违规违章记录管理、车辆固定资产管理、随车备品及配件更换记录管理、车辆寿命管理等功能需求。其非功能性需求如下：

- (1) 系统应支持大于50个终端设备的并发请求；
- (2) 系统应能够实时识别车牌，识别时间应小于1s；
- (3) 系统应7×24小时工作；
- (4) 具有友好的用户界面；
- (5) 可抵御常见SQL注入攻击；
- (6) 独立事务操作响应时间应小于3s；
- (7) 系统在故障情况下，应在1小时内恢复；
- (8) 新用户学习使用系统的时间少于1小时。

面对系统需求，公司召开项目组讨论会议，制订系统设计方案，最终决定基于分布式架构设计实现该物流车辆管理系统，应用Kafka、Redis数据缓存等技术实现对物流车辆自身数据、业务数据进行快速、高效的处理。

问题(1)

请将上述非功能性需求(1)~(8)归类到性能、安全性、可用性、易用性这四类非功能性需求。

思路解析

非功能性需求是系统在“如何运行”层面的约束，主要分类包括性能、安全性、可用性、易用性等。

性能相关的是**系统吞吐、响应、实时性**，所以(1)支持50+并发、(2)实时识别车牌<1s、(6)独立事务响应<3s都是性能指标；

安全性关注**攻击防护与非法访问阻断**，因此(5)“抵御SQL注入”属于安全性；

可用性关注**系统稳定性与故障恢复能力**，(3)“7×24稳定运行”体现持续服务能力，(7)“故障后1小时恢复”体现恢复时间目标(RTO)；

易用性强调**界面友好与学习成本**，(4)界面友好、(8)新用户学习时间<1小时均属易用性。

问题(2)

经项目组讨论，完成了该系统的分布式架构设计，如图5-1所示。请从下面给出的(a)~(j)中进行选择，补充完善图5-1中(1)~(7)处空白的内容。

- (a) 数据存储层
- (b) Struct2
- (c) 负载均衡层
- (d) 表现层
- (e) HTTP协议
- (f) Redis数据缓存
- (g) Kafka分发消息
- (h) 分布式通信处理层
- (i) 逻辑处理层
- (j) CDN内容分发

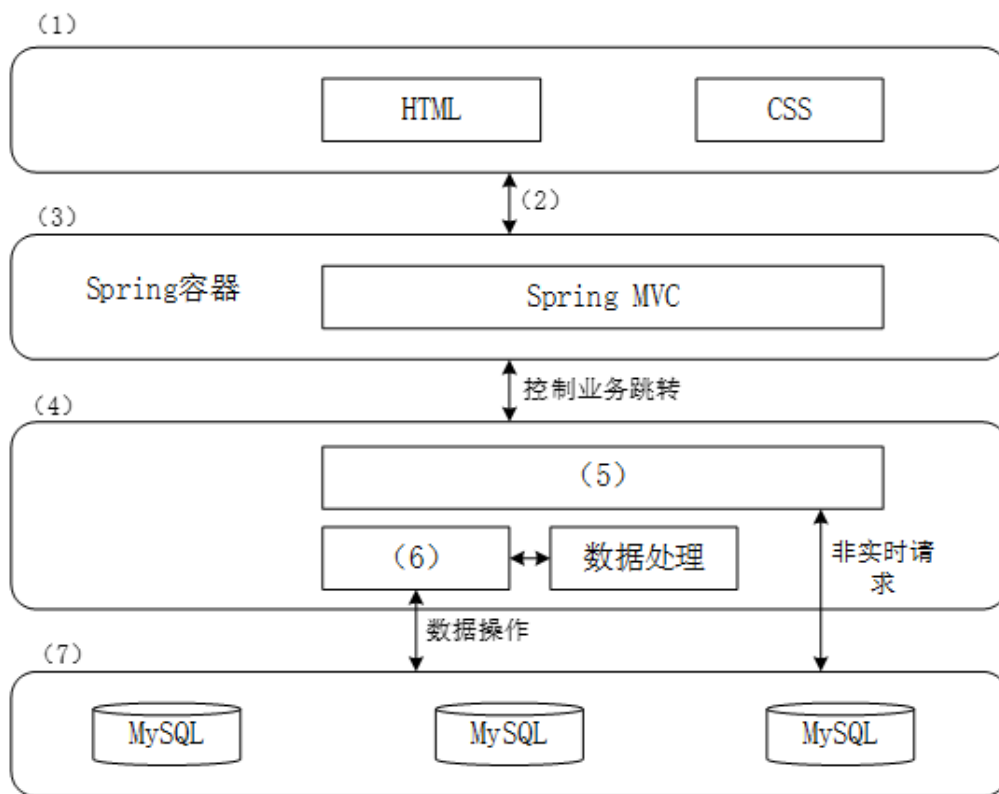


图 5-1 物流车辆管理系统架构设计图

思路解析

图 5-1 是典型的 分层分布式架构。从下至上依次为：数据存储层、缓存层、消息队列层、逻辑处理层、通信层、表现层。

首先，(7) 为最底层的数据存储层 (a)，负责保存车辆运行历史、轨迹、违规记录、资产与寿命信息；

其上 (6) 使用 Redis (f) 进行高速缓存，适合存放车辆实时状态与热点数据；

(5) 为 Kafka 消息分发 (g)，在分布式通信中充当高吞吐消息队列，缓冲采集与分析的峰值流量；

(4) 为 分布式通信处理层 (h)，负责协调消息与服务调用；

(3) 为 逻辑处理层 (i)，处理业务逻辑，如轨迹计算、寿命预测、违规记录写入；

(2) 为 HTTP 协议 (e)，作为表现层与逻辑层交互的标准协议；

(1) 为 表现层 (d)，负责用户界面与交互。

架构关键点在于 Kafka 与 Redis 的分工：Kafka 适合流式、异步解耦，支撑高并发数据流传输与日志分发；而 Redis 用于高频读写缓存，降低数据库压力。

这样设计的优势是：①性能上，缓存加速查询，消息队列消峰填谷；②可用性上，通信层具备冗余与高可用集群；③扩展性上，表现层和逻辑层可横向扩展；④安全性上，可在表现层部署防火墙、SQL 注入防护与用户认证。整体架构与题干中的分布式、快速处理、实时性要求高度一致。

问题 (3)

该物流车辆管理系统需抵御常见的SQL注入攻击，请用200字以内的文字说明什么是SQL注入攻击，并列举出两种抵御SQL注入攻击的方式。

思路解析

SQL 注入攻击是最常见的 Web 安全漏洞之一，攻击者在表单、URL 参数、HTTP 请求头中注入恶意 SQL 片段，绕过程序逻辑直接在数据库执行。例如，在登录输入框输入 ' OR ' 1' =' 1 可绕过身份验证，或构造 DROP TABLE 删除数据表。其危害包括：①信息泄露（车辆信息、资产数据）；②

数据篡改（违规记录、轨迹被恶意修改）；③**破坏完整性**（删除表、注入垃圾数据）。

防御 SQL 注入的根本是**不信任用户输入**。主要手段有两类：①**参数化查询/预编译**：在 JDBC/MyBatis 等框架中使用 PreparedStatement，系统自动将输入作为参数绑定，避免拼接到 SQL 字符串中，从源头阻断注入；②**输入验证与过滤**：采用白名单策略，验证输入是否符合业务规则，如车牌必须为特定格式，数值型字段不得含特殊字符。此外，还可辅以数据库安全审计、防火墙、最小化权限（限制应用账户仅能执行所需操作）、存储过程封装等措施，实现纵深防御。在本系统中，应结合 Redis 缓存、Kafka 消息队列做数据隔离，减少数据库直暴露面。

答案-问题（1）

性能：（1）、（2）、（6）

安全性：（5）

可用性：（3）、（7）

易用性：（4）、（8）

答案-问题（2）

（1）（d）

（2）（e）

（3）（i）

（4）（h）

（5）（g）

（6）（f）

（7）（a）

答案-问题（3）

SQL注入攻击是一种利用应用程序对用户输入数据的处理不当，通过在输入中插入恶意的SQL代码来实现对数据库的攻击方式。攻击者可以通过注入恶意SQL语句来执行未经授权的数据库操作，如删除表、泄露数据等。要抵御SQL注入攻击，可以采取以下两种方式：

使用参数化查询：通过预编译SQL语句并将用户输入的数据作为参数传递，而不是直接拼接到SQL语句中，可以有效防止SQL注入攻击。

输入验证和过滤：对用户输入的数据进行严格的验证和过滤，确保输入数据符合预期格式和范围，过滤掉可能包含恶意SQL代码的字符，从而防止注入攻击。

（2018年11月系统架构设计师真题-第1题）

阅读以下关于软件系统设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某文化产业集团委托软件公司开发一套文化用品商城系统，业务涉及文化用品销售、定制、竞拍和点评等板块，以提升商城的信息化建设水平。该软件公司组织项目组完成了需求调研，现已进入到系统架构设计阶段。考虑到系统需求对架构设计决策的影响，项目组先列出了可能影响系统架构设计的部分需求如下：

（a）用户界面支持用户的个性化定制；

（b）系统需要支持当前主流的标准和服务，特别是通信协议和平台接口；

（c）用户操作的响应时间应不大于3秒，竞拍板块不大于1秒；

（d）系统具有故障诊断和快速恢复能力；

（e）用户密码需要加密传输；

（f）系统需要支持不低于2GB的数据缓存；

（g）用户操作停滞时间超过一定时限需要重新登录验证；

(h) 系统支持用户选择汉语、英语或法语三种语言之一进行操作。

项目组提出了两种系统架构设计方案：瘦客户端C/S架构和胖客户端C/S架构，经过对上述需求逐条分析和讨论，最终决定采用瘦客户端C/S架构进行设计。

问题（1）

在系统架构设计中，决定系统架构设计的非功能性需求主要有四类：操作性需求、性能需求、安全性需求和文化需求。请简要说明四类需求的含义。

思路解析

在系统架构设计中，非功能性需求的四类含义如下：

操作性需求（Operational Requirements）：这类需求关注系统能够完成任务所需的操作环境要求，并确保系统能够适应未来可能的需求变化，涉及到系统的可移植性和可维护性。例如，系统能否在不同的平台上运行，如何进行系统的后续更新与维护等。

性能需求（Performance Requirements）：性能需求描述了系统必须满足的性能指标，通常包括响应时间、吞吐率等。例如，系统的响应时间不能超过3秒，竞拍板块的响应时间不能超过1秒等。

安全性需求（Security Requirements）：这类需求主要包括防止系统崩溃和保证数据安全所采取的保护措施。例如，要求用户密码加密传输，防止数据泄露或遭受攻击。

文化需求（Cultural Requirements）：文化需求是指系统必须考虑的，针对不同用户群体的特定要求。例如，系统支持不同语言的用户界面，或者支持个性化定制，符合当地文化习惯的需求。

问题（2）

根据表1-1的分类，将题干所给出的系统需求（a）~（h）分别填入（1）~（4）。

表 1-1 需求分类

需求类别	系统需求
操作性需求	(1)
性能需求	(2)
安全性需求	(3)
文化需求	(4)

思路解析

根据前面对四类非功能性需求的定义，我们可以将每条需求分类：

操作性需求关注的是系统的操作环境、维护与扩展能力。例如，需求（b）涉及到系统需要支持主流的标准和服务，特别是通信协议和平台接口，属于操作性需求；需求（d）要求系统具有故障诊断与快速恢复能力，属于操作性需求，因为它关心系统如何在发生故障时恢复正常运行。

性能需求关注的是系统的处理能力、响应速度等性能指标。例如，需求（c）涉及到响应时间要求，特别是竞拍板块不超过1秒，属于性能需求；需求（f）要求系统支持不低于2GB的数据缓存，属于性能需求，因为缓存大小会直接影响系统的性能。

安全性需求涉及到防止系统故障、数据安全等措施。例如，需求（e）涉及用户密码加密传输，属于安全性需求；需求（g）涉及用户操作停滞超过一定时限需要重新登录验证，属于安全性需求，因为它有助于防止未授权访问。

文化需求关注的是系统在文化和用户习惯方面的适应性。例如，需求（a）提到用户界面支持个性化定制，属于文化需求，因为它考虑了不同用户的偏好；需求（h）提到支持汉语、英语或法语三种语言之一进行操作，属于文化需求，因为它涉及到不同地区用户的语言需求。

问题（3）

请用100字以内文字说明瘦客户端C/S架构能够满足题干中给出的哪些系统需求。

思路解析

瘦客户端C/S架构将业务逻辑集中在服务器端，客户端负责展示和部分交互，因此在多个方面具有优势：

个性化定制（a）：瘦客户端的界面定制主要依赖于服务器端进行统一管理和更新，能够快速实现个性化定制，减少客户端更新负担。

标准和服务支持（b）：瘦客户端更容易适配新的标准和服务，特别是在不断发展的通信协议和平台接口方面，服务器端可迅速进行适配和升级。

故障诊断与恢复（d）：瘦客户端的业务逻辑集中在服务器端，因此一旦出现故障，只需要修复服务器端，而不需要更新多个客户端，故障恢复更加高效。

语言选择（h）：瘦客户端可以通过服务器端的配置轻松支持多语言界面，确保不同用户群体的需求得到满足。

答案-问题（1）

操作性需求（Operational Requirements）：指系统完成任务所需的操作环境要求及如何满足系统将来可能的需求变更的要求。

性能需求（Performance Requirements）：针对系统性能要求的指标。常见的包括：响应时间、吞吐率。

安全性需求（Security Requirements）：防止系统崩溃和保证数据安全所需要采取的保护措施或预防措施。

文化需求（Cultural Requirements）：使用本系统的不同用户群体对系统提出的特有要求。

答案-问题（2）

(1) (b) (d)

(2) (c) (f)

(3) (e) (g)

(4) (a) (h)

答案-问题（3）

瘦客户端C/S架构能更好满足：(a) (b) (d) (h)

(2020年11月系统架构设计师真题-第5题)

阅读以下关于Web系统架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

开发基于Web的基业设备检测系统，以实现对多种工业数据的分类采集，运行状态检测以及相关信息的管理该系统应具备以下功能：

现场设备状态采集功能，根据数据类型对设备检测指标状态信号进行分类采集，设备采集数据传输功能；9-11月可靠的传输技术，实现将设备数据从制造现场传输到系统后台设备检测显示功能；对设备的运行状态工作以及报警状态进行检测并提供相应的图形化界面设备信息管理功能；支持设备运行历史状态，报警记录参数信息的查询。

同时，该系统还需满足以下非功能性需求。

(a) 系统应支持大于100个工业设备的运行检测

(b) 设备数据以制造现场传输到系统后台传输时间小于1s

(c) 系统应在7*24小时工作

(d) 可抵御XSS攻击

(e) 系统在故障情况下，应在0.5小时内恢复

(f) 支持数据审计

面对系统需求，公司召开项目讨论会议，制定系统设计方案，最终决定使用三层拓补结构，即现场设备数据采集层、Web检测服务层和前端Web显示层。

问题（1）

请按照性能、安全性和可用性三种非功能需求分类将题干的（a）~（f）填入（1）~（3）空白处。非功能性需求归类表：

非功能性需求类别	非功能性需求
性能	(1)
安全性	(2)
可用性	(3)

思路解析

非功能需求的本质在于对系统“如何表现”的约束。

性能关注吞吐、时延与容量等指标：

(a) “支持 >100 台设备”体现**容量/吞吐**要求，属于性能的**规模化承载能力**；

(b) “传输时间 <1s”是典型的**端到端时延**指标，也归于性能。

安全性强调抵御攻击与可追溯：

(d) “抵御 XSS”是**输入与输出链路的前端安全控制**（输入校验、输出编码、CSP 等），

(f) “数据审计”要求对敏感操作与数据流进行**日志留痕与合规审计**，二者均属安全范畴。

可用性关注服务可达与故障恢复：

(c) “7×24 连续运行”直指**持续可服务能力**（MTBF/维护窗口设计），

(e) “0.5 小时内恢复”是**RTO（恢复时间目标）的明确承诺**，属于故障后的恢复/切换能力。

问题（2）

该系统Web检测服务层拟采用SSM框架进行系统研发，SSM工作流程图如下图5-1所示，请从下面给出的（a）~（k）中进行选择，补充完善图5-1中（1）~（7）处空白的内容：

(a) Connection pool

(b) Struts2

(c) Persistent Layer

(d) Mybatis

(e) HTTP

(f) MVC

(g) Kafka

(h) ViewLayer

(i) Jsp

(j) Controller Layer

(k) Spring

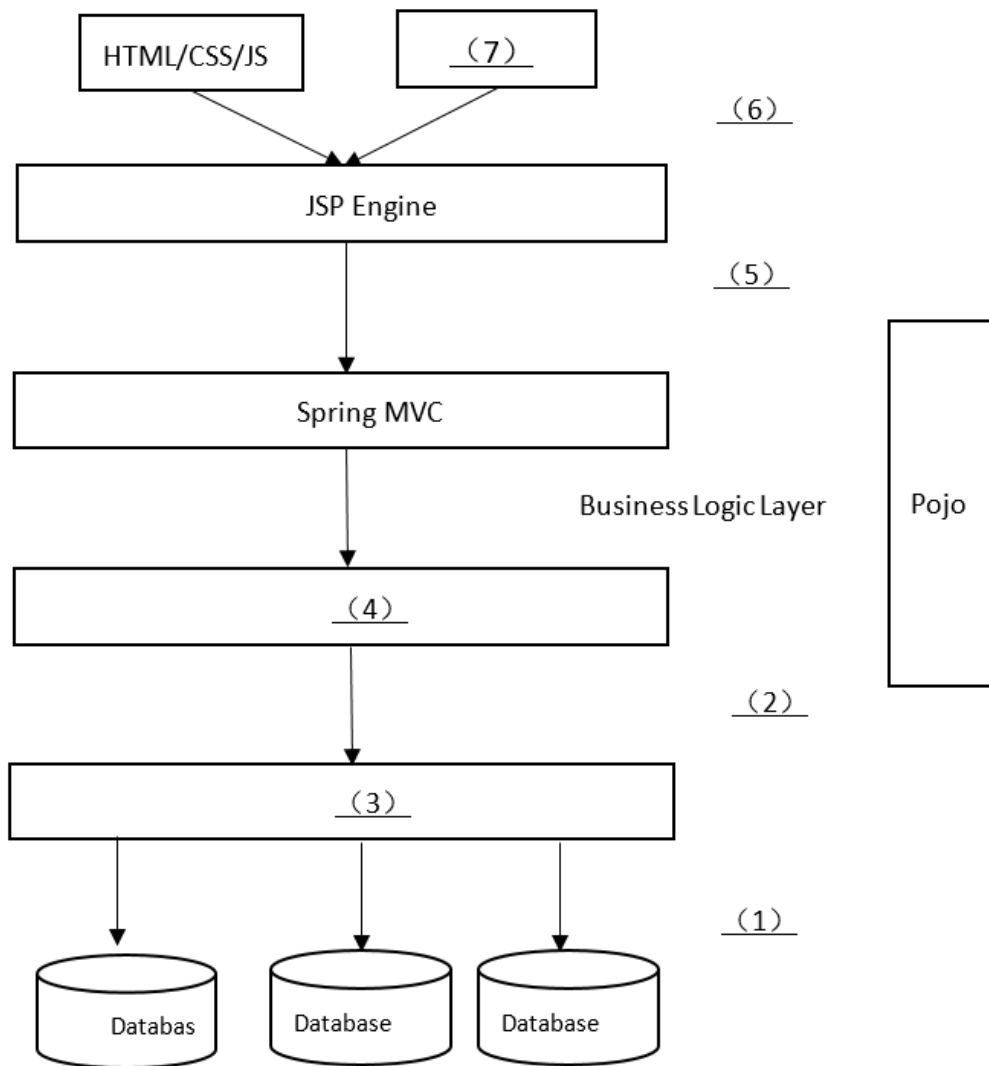


图 5-1 SSM 框架工作流程图

思路解析

SSM 由 **Spring + Spring MVC + MyBatis** 组成，面向 Web 的典型调用链为：

浏览器经 HTTP（图中已给）将请求交由前端控制器（DispatcherServlet），路由至**控制器层（Controller Layer）（5）**，控制器调用**服务/业务**并进一步访问**持久层（Persistent Layer）（2）**。

持久层借助 **MyBatis（3）** 组织 SQL、映射结果，底层通过**连接池（Connection pool）（1）** 复用数据库连接以降低创建/销毁开销与时延抖动。

全局由 **Spring（4）** 负责 IOC/DI、AOP 与事务管理（声明式事务对读写分离、重试、回滚尤为关键），向上返回模型数据交由**视图层（View Layer）（6）** 渲染，

最终通过 **JSP（7）** 生成动态页面响应。

以上映射与 SSM 的职责边界一致：**Controller 聚合与编排、Service 承载领域逻辑、DAO/MyBatis 负责数据访问、连接池提供资源高效复用。**

为何不是干扰项？****Struts2（b）**** 属另一套 MVC 框架，非 SSM 组合；****Kafka（g）**** 是分布式日志/消息流平台，用于异步解耦与高吞吐日志/指标收集，不是持久层或视图层组件；****MVC（f）**** 是架构模式而非具体落位构件。

结合本题背景，SSM 在本系统中的优势包括：**清晰的分层解耦**（便于将采集层推送的数据经服务层校验后入库）、**易测试/易维护**（IOC 促进可替换实现、AOP 统一横切关注点如审计与鉴权）、**性能与可靠性可控**（连接池 + MyBatis 缓存 + 事务配置 + 拦截器限流）。在满足 Q1 的三类非功能需求时：性能可通过**连接池参数、SQL 调优、缓存与异步批处理**保障；安全可通过 **Spring MVC 拦截**

器 + XSS 过滤 + 审计 AOP 实现；可用性可配合健康检查、熔断/降级与只读模式，并在部署层面（反向代理/多实例）支持滚动升级与快速恢复。

问题（3）

该工业设备检测系统拟采用工业控制领域中统一的数据访问机制，实现与各种不同设备的数据交互，请用100以内的文字说明采用标准的数据访问机制的原因。

思路解析

工业现场设备厂商、协议与数据语义高度异构（如采集频率、编码、时间戳精度、质量码、告警语义各异）。若平台为每类设备定制适配，系统将陷入“接口爆炸”与“语义碎片化”，导致测试矩阵膨胀、版本演进困难。

标准数据访问机制（可理解为统一的采集 SDK/Agent、抽象的设备驱动层、统一传输与数据模型，如**统一点表/标签模型、时间序列与事件模型、质量位与告警码表**）可在设备侧封装差异，在平台侧只面向统一接口与模型：

- 其一，**互操作性提升**——新设备接入只需实现对标准接口/模型的映射；
- 其二，**可扩展性增强**——设备量级增长时，接入流程复用，便于横向扩容与灰度上线；
- 其三，**低耦合与可维护**——业务服务只依赖标准语义，不感知设备细节，便于规则/报表/监控复用；
- 其四，**质量与合规**——统一数据字典、审计字段与校验规则（如来源、时间、校验和、签名）让**追踪与审计**可落地；
- 其五，**性能与稳定性**——标准机制可集中优化（批量传输、压缩、边缘缓冲、断点续传、幂等重放），减少端到端时延与抖动。

答案-问题（1）

- （1）a、b
- （2）d、f
- （3）c、e

答案-问题（2）

- （1）a
- （2）c
- （3）d
- （4）k
- （5）j
- （6）h
- （7）i

答案-问题（3）

通过采用标准的数据访问机制，可以屏蔽不同设备之间数据交互的差异，解决了系统使用数据不一致性，又在一定程度上减低了数据结构与应用系统的耦合度，减少了应用系统的维护的工作量。

（2025年5月系统架构设计师真题-第1题）

某科技公司开发了一个面向开发者的在线大模型训练平台系统，该平台主要面向具备一定编程基础的用户，支持基于 Python 的模型编写、训练和部署流程。系统旨在简化模型开发流程，用户无需了解底层计算硬件配置，平台将自动解析代码并智能匹配训练资源进行运行。

该系统设计初期就充分考虑了系统质量属性对平台稳定性、易用性和用户体验的影响，具体需求如

下:

1. 用户提交模型训练任务时, 系统应在1 分钟内分配资源并开始任务运行;
2. 数据库发生故障时, 能够在 20 分钟内切换至备用数据库, 保证平台继续运行;
3. 服务器发生故障时, 能够自动切换至备用服务器, 保障系统业务连续;
4. 系统出现故障时, 平台能够继续正常运行并通知管理员, 同时应提供相关的操作日志、系统日志、访问日志和调试日志等信息;
5. 用户界面需自动适配不同设备的分辨率和屏幕比例;
6. 提供常用快捷键操作, 提升平台易用性;
7. 系统支持切换界面语言, 方便不同语言背景用户使用;
8. 支持远程用户进行测试和操作, 但需限定为注册用户;
9. 系统应支持来自不同终端设备(如浏览器、命令行工具、移动设备等)和操作系统(如 Windows、Linux、macOS)的注册用户同时远程访问与操作平台, 系统需能正确解析并响应各类客户端的指令, 保证功能一致性与协同操作能力;
10. 当系统功能需要调整时, 应在 3 天内完成功能修改和部署上线;
11. 系统需具备良好的故障恢复能力, 在发生系统级故障时, 15 分钟内需完成修复;
12. 系统需具备良好的异常容错机制, 部分模块出错不应影响平台整体运行。

问题(1)

根据上述系统描述, 依次写出每条需求对应的系统质量属性。

1. 用户提交模型训练任务时, 系统应在 1 分钟内分配资源并开始任务运行(1);
2. 数据库发生故障时, 能够在 20 分钟内切换至备用数据库, 保证平台继续运行(2);
3. 服务器发生故障时, 能够自动切换至备用服务器, 保障系统业务连续(3);
4. 系统出现故障时, 平台能够继续正常运行并通知管理员, 同时应提供相关的操作日志、系统日志、访问日志和调试日志等信息(4);
5. 用户界面需自动适配不同设备的分辨率和屏幕比例(5);
6. 提供常用快捷键操作, 提升平台易用性(6);
7. 系统支持切换界面语言, 方便不同语言背景用户使用(7);
8. 支持远程用户进行测试和操作, 但需限定为注册用户(8);
9. 系统应支持来自不同终端设备(如浏览器、命令行工具、移动设备等)和操作系统(如 Windows、Linux、macOS)的注册用户同时远程访问与操作平台, 系统需能正确解析并响应各类客户端的指令, 保证功能一致性与协同操作能力(9);
10. 当系统功能需要调整时, 应在 3 天内完成功能修改和部署上线(10);
11. 系统需具备良好的故障恢复能力, 在发生系统级故障时, 15 分钟内需完成修复(11);
12. 系统需具备良好的异常容错机制, 部分模块出错不应影响平台整体运行(12)。

思路解析

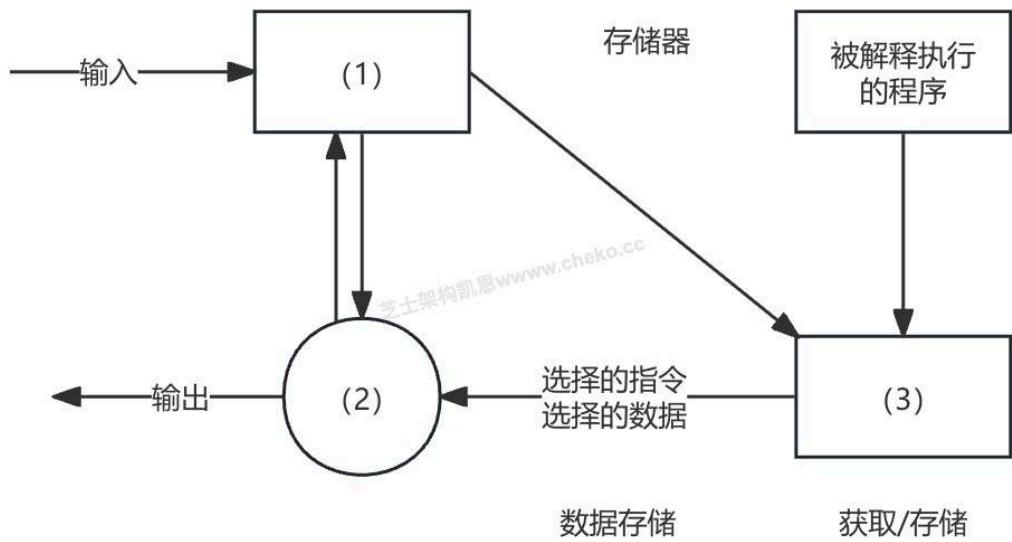
本题要求把自然语言需求映射到系统质量属性。第1条强调“1分钟内分配资源并启动”, 典型关注**响应时间与吞吐**, 归属**性能**。第2、3条提出“数据库/服务器故障后切换并持续运行”, 其核心是**服务不中断与高可用**, 因此归为**可用性**。第4条要求“故障仍可运行并上报且提供各类日志”, 体现**容错、降级与运行可观测性**, 其度量对象是**业务是否可继续**, 仍归**可用性**。第5、6、7条围绕**适配不同分辨率、多语言、快捷键**等交互体验改良, 显然指向**易用性**。第8条“仅注册用户可远程测试与操作”, 强调**可控测试、可观测与可隔离性**, 面向测试组织与执行的便利, 归**可测试性**。第9条要求“多终端多OS并发、统一解析与一致语义”, 本质是跨平台、跨协议、跨形态的**协议与语义兼容**, 归**互操作性**。第10条“3天内改动上线”, 直指**变更成本与发布效率**, 归**可修改性**。第11条“系统级故障15分钟内恢复”, 度量的是**恢复时间目标(RTO)与业务连续性**, 归**可用性**。第12条“模块出错不影响整体”, 强调**故障隔离、降级与容错**, 仍归**可用性**。

需要注意的是, 实务中“可用性/可靠性/可恢复性”常被混用, 但在质量属性分类里更推荐以**是否保持对外可用服务**来判定归类: 只要指标落在**不中断、可切换、可恢复、可继续**等范畴, 优先归入**可用性**; 若主要关注**故障率、平均无故障时间**等统计性指标, 再偏向**可靠性**。本题第2/3/4/11/12条

都以“业务是否继续提供”为判据，因而**可用性**更贴切。此外，第8条若从“认证授权、安全性”理解也可成立，但题干将其与“测试和操作”捆绑，意在**可控测试与受限访问**，以**可测试性**为佳；第9条若从“跨端适配”理解可落在可移植性，但其强调“同时远程访问、正确解析指令、保证一致性与协同”，更符合**互操作性**对**接口/协议/语义**一致与协作的要求。该映射方案与标准答案一致，亦与常见质量属性术语库（如SEI、ISO/IEC 25010）相容。

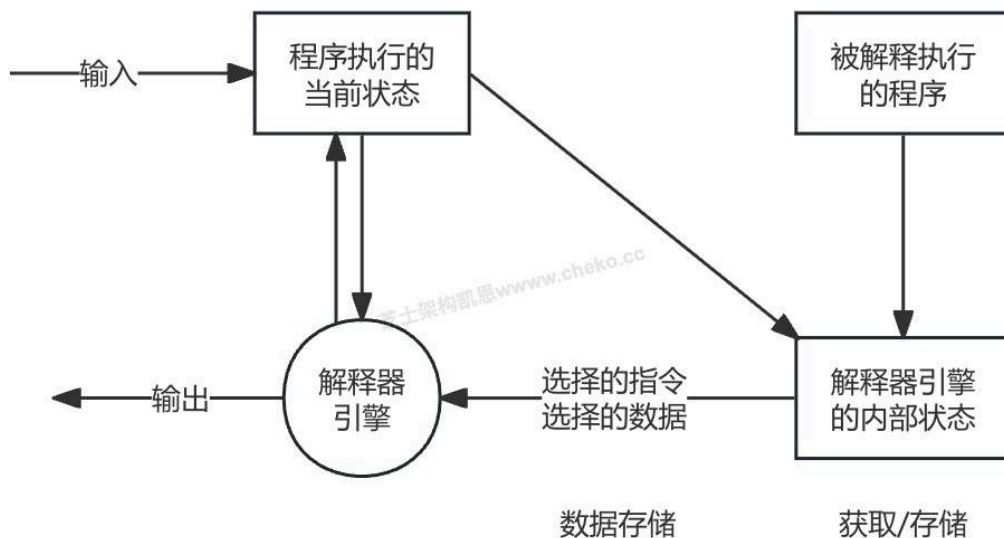
问题（2）

根据题干描述，将下图空白处补充完整，并说明该平台为什么适合用解释器风格



思路解析

解释器风格由“程序（待解释的指令序列/AST）”“解释器引擎”“引擎内部状态”“程序执行的当前状态”等构件协作完成。**程序执行的当前状态**刻画运行时上下文（如指令计数器、作用域栈、变量与张量缓存、随机种子、会话与资源句柄等），**解释器引擎**负责将用户代码解析为中间表示并驱动逐条执行，**引擎内部状态**保存解释器自身的控制信息（如调度策略、优化开关、后端选择、算子注册表、插件与安全沙箱策略等）。对本平台而言，用户“提交代码即提交任务”，平台需在不暴露硬件复杂性的前提下对脚本进行**静态/动态分析、依赖解析、算子/设备匹配、数据与日志通道编排**，随后按策略在GPU/CPU/分布式集群上执行——这些都与解释器在**运行时做决策**的特性天然契合。



采用解释器风格的收益体现在四方面：（1）动态性与灵活性）解释器可在运行期装载扩展、热插新算子、新优化pass，便于快速适配不同模型与框架版本；（2）可控性与可观测性）解释执行过程中可以细粒度插桩：操作日志、系统/访问/调试日志统一采集；出错时可回溯到当前程序状态以便断点续训、自动重试与幂等提交；（3）资源与安全治理）解释器在语义层能拦截敏感操作，执行配额与隔离（限显存、限并发、沙箱/容器化），实现“注册用户远程操作但受控”；（4）可维护性与可演进性）当“3天内功能修改上线”时，多数变更可通过新增/调整内置指令或规则完成，无需改动用户脚本；多端接入（浏览器/CLI/移动端）只需统一向解释器投递标准化指令，解释器保证指令语义一致，支撑题干第9条的互操作性目标。综上，解释器风格既满足“代码即任务、自动匹配资源”的业务本质，又在可用性、可修改性、互操作性与可测试性等质量属性上提供了工程化落点，因而是该平台的契合架构选择。

答案-问题（1）

（1）性能（2）可用性（3）可用性（4）可用性（5）易用性（6）易用性（7）易用性（8）可测试性（9）互操作性（10）可修改性（11）可用性（12）可用性

答案-问题（2）

（1）程序执行的当前状态（2）解释器引擎（3）解释器引擎的内部状态

该平台适合采用解释器风格架构，是因为其核心功能是对用户提交的 Python 模型代码进行解析和执行，而不需要用户关注底层硬件实现。

解释器风格能够将代码解析为抽象语法树，并逐条解释执行，天然适用于这种“代码即任务”的场景。它不仅支持灵活地扩展新语法和命令，还便于在执行过程中嵌入日志记录、调试信息和资源调度逻辑，增强系统的可控性与可维护性。因此，解释器风格非常契合该平台对动态性、灵活性和任务控制能力的需求。

（2025年11月系统架构设计师真题-第1题）

某市计划建设一套电动车充电管理系统，为城市中的集中式充电站、路侧分布式充电桩以及用户手机 APP 提供统一的接入与管理服务。系统需满足如下质量需求场景（a~n）：

a) 当用户通过手机 APP 或现场终端发起“开始充电”操作后，系统必须在 3 秒内给出响应结果（成功 / 失败及原因），否则视为超时。

b) 在充电站方圆 200 米范围内，即使同时有超过 300 个设备在充电或等待充电口分配，系统 CPU 使用率仍需控制在 40% 以下，保证系统性能稳定。

- c) 当运营方提出集成新的第三方支付或计费 API 时，自提出需求之日起，须在 10 个工作日内完成设计、开发、联调和上线，不影响原有业务功能。
- d) 当系统发生软件故障或节点宕机时，从故障发生到恢复正常对外服务的时间不得超过 5 分钟。
- e) 系统需支持模拟雷电、暴雨、冰雹等恶劣天气场景下的充电过程，包括电压波动、网络抖动等，用于环境仿真测试。
- f) 系统需支持远程调用测试接口（如开放 RESTful 或 gRPC 的测试端点），便于第三方测试平台进行自动化回归测试。
- g) 对于用户敏感信息（如用户账号、密码、位置信息等），系统需采用 AES-256 加密算法 进行存储与传输加密，防止数据泄露。
- h) 系统支持多语言界面、多种交互模式（触屏、键鼠等），并满足针对视力、听力或肢体障碍用户的无障碍使用标准（如放大字体、语音播报等），以提高易用性与可访问性。
- i) 系统整体采用模块化设计，要求运维人员在不依赖开发人员参与的前提下，能在 2 小时内通过手动配置快速替换故障模块（如更换计费模块、风控模块），保证业务连续性。
- j) 针对普通新用户，从首次接触系统到能独立完成一次充电操作的时间，不应超过 60 秒，体现系统的易学性和操作简单。
- k) 系统前端界面需在不同分辨率与不同类型终端设备（手机、平板、PC、大屏）上自适应显示，保证布局清晰、控件可用。
- l) 系统需支持计划内的滚动升级：在升级某个业务子系统时，已建立的充电会话不能中断，正在充电的用户不受影响，仅新建会话可能短暂切换到备用节点。
- m) 系统需支持通过配置文件方式新增或调整充电策略（如分时电价、阶梯计费策略），无需重新编译和发布应用程序。
- n) 当用户手机暂时没有网络时，APP 自动进入安全模式，从本地安全缓存中展示用户最近一次充电记录和账户关键信息，保证基本信息可见，待网络恢复后自动同步最新数据。

问题（1）

请将上述场景（a~n）与填入下表对应位置

可用性	
安全性	
性能	
可修改性	
可测试性	
易用性	

思路解析

本题考察考生对**常见质量属性及其典型场景特征**的理解和识别能力。

首先，可用性关注系统在发生故障或特殊情况下能否继续提供服务及恢复能力。场景 d 明确描述“故障恢复时间不超过 5 分钟”；l 要求在系统升级时不影响既有充电会话；n 描述手机离线时提供安全模式和历史记录显示，这三条都体现了系统在异常情况下仍能对外提供服务，因此归为**可用性**。可用性和可靠性真题中基本不会同时出现，因为可用性是可靠性的子属性。这道真题又一次证明了这个结论。

性能主要体现为**响应时间**、**吞吐量**、**资源利用率**等。a 中的“3 秒内响应”典型属于响应时间要求；b 中的“CPU 使用率不超过 40% 且同时支持大量设备”体现了资源利用率与并发能力，因此归入**性能属性**。

安全性关注信息保护和访问控制，g 使用 AES-256 加密用户敏感信息，属于典型的信息安全场景。可修改性强调系统在需求变化、功能扩展时，修改成本和风险。c 要求 10 个工作日内集成新 API，i 要求运维可在 2 小时内替换模块，m 通过配置文件新增计费策略，这三条都体现“改动快、影响面可控”，因此属于**可修改性**。

可测试性在于系统是否易于被构造测试场景与执行自动化测试，e 的恶劣天气仿真与 f 的远程测试调用接口都明显便于测试设计，体现**可测试性**。

易用性关注用户学习成本和使用体验，h 的多语言与无障碍支持、j 的新用户 60 秒上手、k 的多分辨率自适应均指向用户交互体验，因此归为**易用性**。

问题（2）

质量属性场景一般包含哪 6 个组成部分？并指出场景中“集成新 API”以及“故障恢复时间不超过 5 分钟”分别对应哪两个组成部分。

思路解析

质量属性场景是 ATAM、QAW 等架构方法中非常重要的概念，一般由六个部分构成，用于将质量需求写成“可讨论、可评估、可验证”的形式。这六部分分别是：**刺激源**、**刺激**、**环境**、**制品**、**响应**、**响应度量**。

其中，**刺激源**描述是谁或什么引发了场景（例如：用户、管理员、外部系统、硬件故障等）；**刺激**则是具体发生的事件或请求（例如：提交一个新功能需求、系统某模块发生宕机、用户发起一次支付操作等）；**环境**说明场景发生时的系统状态，如正常运行、高负载、维护模式、网络不稳定等；**制品**指该场景作用到哪个系统元素上，如整个系统、某服务、某模块或数据库；**响应**是系统在该刺激下采取的行为，如切换到备机、重新启动服务、记录日志、返回错误提示等；**响应度量**则将响应变为可度量的指标，如在 5 分钟内恢复、99.99% 正常运行时间、响应小于 3 秒等。

在本题中，“可在 10 个工作日内集成新 API”这一描述，从质量属性场景角度来看，“集成新 API 的需求被提出”就是对系统的一个**刺激**，通常由运营或业务部门提出，因此归类为**刺激**。而“故障恢复时间不超过五分钟”对应的是系统对“发生故障”这一刺激所做出的恢复动作所需时间，是一个典型的**响应度量**，用于衡量可用性场景是否达标。

看图片上的总结你会对可修改性有一个更加全面的认识。

可修改性质量属性场景描述	
刺激源	最终用户、开发人员、系统管理员
刺激	改变功能、扩容等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	修改且不会影响其他功能
响应度量	成本、努力、资金；该修改对其他功能的影响

串起来形成一个故事就是，开发人员或系统管理员（刺激源）在运行时需要对系统进行功能修改或容量扩展（刺激），在系统处于正常运行环境下（环境），用户界面、平台或相关交互系统（制品）应当能够被修改且不影响其他功能（响应），并以修改所需的成本、时间和对其他功能的影响程度为指标进行衡量（响应度量）。

问题（3）

用户个人信息、位置信息可能长度较长，而 AES-256 为定长128bit（16 字节）块加密算法。对于超长数据，系统应如何处理才能正确进行加密？请从处理流程上进行说明。

思路解析

本题重点考察对 **分组对称加密算法工作机制** 的理解。AES-256 是一种分组加密算法，其**密钥长度为 256bit**，但**分组长度固定为 128bit（16 字节）**。这意味着：算法一次只能对 16 字节的明文块进行加密，不能直接对任意长度的长数据一次性处理。因此，对于用户账号、密码、位置信息等长度可能超过 16 字节的数据，需要采用“**分块 + 填充 + 模式**”的方式来完成加密。

首先，根据 AES 的分组大小，将原始明文数据按 16 字节划分成若干块，中间的完整块可以直接参与后续加密流程。对于最后一个不足 16 字节的块，必须采用标准的**填充算法**（如 PKCS#7），在明文末尾补足若干字节，使其恰好达到 16 字节长度。

完成分块与填充后，需要结合合适的**分组工作模式**，例如 CBC、CFB、OFB 或 GCM 等。在 CBC 模式中，每个明文块在加密前会与前一个密文块（或初始向量 IV）进行异或，提高安全性；GCM 模式还同时提供机密性与完整性校验，适合网络传输场景。

系统在实现时，对每个 16 字节块依次调用 AES-256 原语进行加密，得到对应的密文块，然后将所有密文块**按原有顺序拼接**，形成完整的密文数据。在解密时，按相同模式逐块解密，并在最后一块移除填充字节，即可恢复原始超长明文。通过这种分块、填充和组合的方式，可以让定长分组算法正确、安全地处理任意长度的用户敏感数据。

答案-问题（1）

- （1）可用性 d l n
- （2）安全性 g
- （3）性能 a b
- （4）可修改性 c i m
- （5）可测试性 e f
- （6）易用性 h j k

答案-问题（2）

质量属性场景的六个组成部分为：刺激源 刺激 环境 制品 响应 响应度量

“可在 10 个工作日内集成新 API” 属于刺激

“故障恢复时间不超过五分钟” ——属于响应度量

答案-问题（3）

对于长度超过 16 字节的用户敏感数据，应采用：

对原始数据进行**分块处理**，按 16 字节划分多个数据块；

对最后一个不足 16 字节的数据块进行**填充（Padding）**（如 PKCS#7）；

结合合适的**分组工作模式**（如 CBC、CFB、GCM 等），逐块加密；

最终将多个密文块**按顺序组合**形成完整密文数据。

第10章——软件架构风格与模式 (2026年5月系统架构设计师真题-第1题)

阅读以下关于数据处理与入侵检测系统架构的叙述，在答题纸上回答问题1-3。

【说明】

某大型园区计划建设一套安全态势感知与入侵检测系统，对园区出口流量、服务器日志、终端安全事件、应用访问日志和设备运行状态进行统一采集、处理、分析和告警。系统需要持续接收多源异构数据，完成格式转换、噪声过滤、特征提取、规则匹配、异常检测和告警通知等处理，并将高风险事件推送给安全运营人员。

在需求分析阶段，用户提出了如下质量属性场景：

（a）当网络采集节点发生故障时，系统应在30秒内切换到备用采集节点，并继续接收关键安全事件；

（b）在日常高峰期，系统应支持每秒5万条日志的接入与处理，普通告警的端到端延迟不超过3秒；

（c）系统需要对安全管理员进行身份认证和权限控制，并记录所有规则配置和处置操作；

（d）当新增一种日志来源或攻击检测规则时，开发人员应在2天内完成接入、测试和上线；

（e）当告警通知服务暂时不可用时，系统应将告警事件暂存，服务恢复后自动补发，避免告警丢失；

（f）系统应能够识别异常登录、暴力破解、端口扫描等攻击行为，并生成处置建议。

架构师提出两种候选架构风格。方案一采用批处理风格，定期汇总日志文件后集中分析，适合离线统计和报表生成；方案二采用管道-过滤器风格，将采集、清洗、转换、特征提取、检测和告警拆分为多个过滤器，并通过数据流管道连接，支持各处理环节独立扩展和替换。项目组需要结合入侵检测系统实时性、可扩展性和可维护性的要求完成架构分析。

问题（1）

根据题干中的质量属性描述，将场景归入可用性、安全性、性能、可修改性等质量属性，并说明a所对应的质量属性场景的基本描述要素。

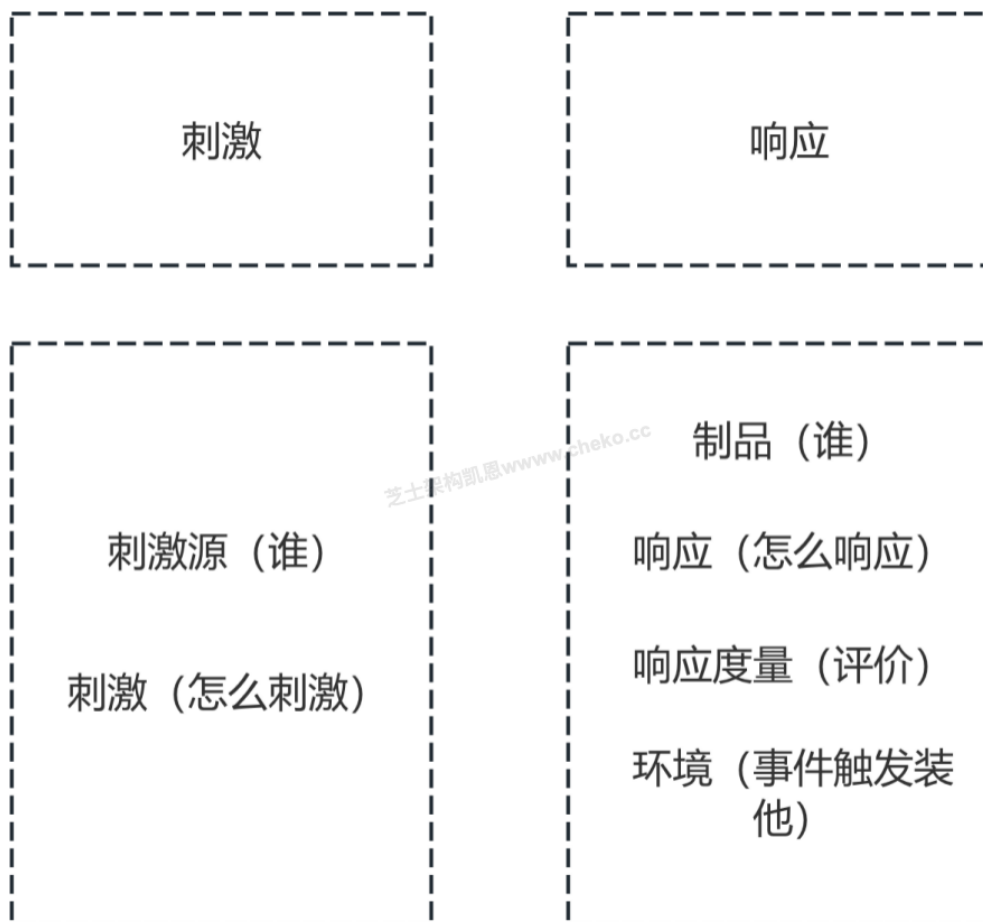
质量属性	需求描述
可用性	(1)
性能	(2)
安全性	(3)
可修改性	(4)

思路解析

本问考查质量属性场景的识别和表达。解题时不能只凭质量属性名称判断，而要看题干中的刺激、响应和响应度量。场景（a）中“采集节点故障”“30秒内切换备用节点”“继续接收关键安全事件”说明系统在故障发生后仍能恢复并维持服务，属于可用性。场景（e）中告警通知服务暂时不可用时先暂存、恢复后补发，本质也是通过缓冲和补偿避免服务中断或告警丢失，因此也应归入可用性。这两处有时会被写成可靠性，考试中如果备选质量属性给的是可用性、安全性、性能、可修改性，通常按“故障恢复、不中断、恢复时间”归到可用性。

场景（b）给出了每秒5万条日志和端到端延迟不超过3秒，前者是吞吐量，后者是响应时间，都是性能属性的典型度量。场景（c）包含身份认证、权限控制 and 操作审计，场景（f）包含异常登录、暴力破解、端口扫描识别和处置建议，均围绕防护、检测、授权和追踪展开，属于安全性。场景（d）要求新增日志来源或攻击检测规则在2天内完成接入、测试和上线，强调修改成本、影响范围和交付效率，属于可修改性。

质量属性场景的六要素要完整记忆：刺激源、刺激、环境、制品、响应、响应度量。以（a）为例，刺激源可以是故障节点或运行环境中的故障事件，刺激是采集节点失效，环境是系统正常运行或高峰运行期间，制品是数据采集链路，响应是切换备用采集节点并继续接收关键事件，响应度量是30秒内完成。答题时如果能把分类和六要素结合起来，说明你不是机械背定义，而是能把质量属性场景落到具体架构需求上。



书本上有书本上的记法，我个人比较喜欢按照上图这种方法来记：首先把它分成左右两边：左侧是刺激，右侧是响应。

刺激：分为谁来刺激，以及怎么刺激。

响应：分为谁来响应、怎么响应，以及怎么去评价响应。

环境：整个事件触发的一个环境。

那么在这个基础上，你再把用户的场景描述套到我们的六要素里面，就比较容易了。

那么另外几个场景描述6要素的识别，大家可以自行去比对，试用上面的方法，看看和这里的答案对不对。

性能场景：刺激源是业务系统、安全设备、日志采集节点等日志来源，刺激是持续向系统发送大量日志并触发普通告警处理，环境是日常高峰期，制品是日志接入模块、日志处理模块和告警处理链路，响应是系统接收、解析、处理日志并生成普通告警，响应度量是系统每秒支持 5 万条日志的接入与处理，普通告警端到端延迟不超过 3 秒。

安全性场景：刺激源是安全管理员，刺激是登录系统并进行规则配置、告警处置等操作，环境是系统正常运行期间，制品是身份认证模块、权限控制模块和审计日志模块，响应是系统验证管理员身份、检查操作权限并记录规则配置和处置操作，响应度量是未授权用户不能越权操作，所有规则配置和处置操作都被完整记录、可追溯。

可修改性 / 可扩展性场景：刺激源是开发人员或提出新增需求的安全运营人员，刺激是新增一种日志来源或新增一种攻击检测规则，环境是系统需要扩展日志接入能力或攻击检测能力时，制品是日志接入模块、规则引擎、检测规则配置模块和测试发布机制，响应是系统支持开发人员完成新日志来源或新检测规则的接入、配置、测试和上线，响应度量是开发人员应在 2 人天内完成接入、测试和上线。

可靠性 / 可用性场景：刺激源是告警通知服务或外部通知通道，刺激是告警通知服务暂时不可用导致告警无法立即发送，环境是系统运行过程中通知服务发生临时故障，制品是告警通知模块、消息队列和告警事件暂存模块，响应是系统将告警事件暂存，等通知服务恢复后自动补发，响应度量是告警事件不丢失，服务恢复后能够自动补发。

功能性场景：刺激源是攻击者、异常登录行为或异常网络访问行为，刺激是出现异常登录、暴力破解、端口扫描等攻击行为，环境是系统正常进行安全监测期间，制品是攻击检测模块、规则引擎、告警分析模块和处置建议模块，响应是系统识别攻击行为、生成安全告警并给出处置建议，响应度量是能够识别异常登录、暴力破解、端口扫描等指定攻击行为，并生成对应处置建议。

问题（2）

在批处理架构与管道-过滤器架构之间，为入侵检测和告警处理选择更合适的架构风格并说明理由。

思路解析

本问考查架构风格选择。入侵检测和安全态势感知系统面对的是持续到达的日志、流量、终端事件和设备状态，业务目标是边接入、边清洗、边分析、边告警，因此更适合管道-过滤器风格。该风格把系统拆成一串过滤器，每个过滤器完成相对独立的数据转换或分析任务，过滤器之间通过管道传递数据。对应到本题，可以形成采集、协议解析、格式归一化、噪声过滤、特征提取、规则匹配、异常检测、事件关联、风险评分、告警生成和告警通知等流水线。这种拆分与入侵检测的处理路径高度一致，也便于对某个过滤器进行扩容、替换或灰度上线。

与批处理相比，管道-过滤器的优势主要有三点。第一，实时性更好，数据不必等到日志文件批量汇总后再处理，可以在管道中逐步流转，满足普通告警3秒内送达的要求。第二，可修改性更好，新增一种日志格式时主要影响解析或归一化过滤器，新增攻击规则时主要影响规则匹配或异常检测过滤器，不需要重写整条链路。第三，可扩展性更好，高吞吐场景下可以对热点过滤器做并行实例、消息缓冲和背压控制。

批处理风格并非没有价值，它适合离线报表、历史攻击趋势分析、样本回放和模型训练数据准备，但它天然要等待批次形成，难以承担实时告警主链路。所以本题最合理的答案是：实时入侵检测主链路采用管道-过滤器，离线统计或报表子系统可保留批处理。这样既给出选择，也体现了架构权衡。

进一步展开时，可以把管道-过滤器和题干质量属性对应起来。实时性来自数据在管道中的连续流动，高吞吐来自过滤器可并行和可水平扩展，可修改性来自过滤器职责单一、替换边界清楚，安全运营可追溯性来自每个过滤器输出中间事件和处理日志。同时也要看到这种风格的代价：如果多个过滤器都需要共享上下文，状态管理会复杂；如果某个下游过滤器处理慢，管道可能积压，需要背压、限流和降级；如果告警生成后再回溯原始日志，需要链路追踪和事件 ID 串联。把优点和代价都写出来，能体现架构权衡，而不是简单背“管道-过滤器适合数据流处理”。

问题（3）

基于所选架构风格，补充系统中的主要构件。

思路解析

本问考查把架构风格落成构件的能力。管道-过滤器不是简单把模块名排一排，而是要求每个过滤器有清晰输入、处理职责和输出，并且过滤器之间通过连接件传递数据。在本题中，最基本的数据流可以写成：数据采集过滤器接入网络流量、服务器日志、终端事件和应用访问日志；协议解析和格式转换过滤器把不同来源的数据转换为统一事件格式；数据清洗过滤器负责去重、补全字段、过滤噪声和异常格式；特征提取过滤器提取 IP、端口、用户、时间窗口、访问频率、行为序列等特征；规则匹配过滤器按照攻击规则、黑白名单和阈值规则进行匹配；异常检测过滤器结合统计模型或机器学习模型发现未知异常；事件关联过滤器把多个低级事件关联为高风险安全事件；风险评分过滤器给出等级；告警生成和告警通知过滤器负责写入告警库并推送给安全运营人员。

除主链路过滤器外，还应补充支撑构件。规则库保存攻击规则和检测策略，特征库保存可复用特征定义，攻击样本库用于模型训练和回放验证，告警事件库存储告警结果，审计日志库记录管理员配置和处置行为，配置中心用于规则和过滤器参数下发，监控模块用于观察吞吐量、延迟、积压和失败率。过滤器之间的连接件可以是消息队列、流处理通道、内存缓冲或事件总线。若题目要求画图

或填空，顺序通常按“采集-转换-清洗-分析-告警-展示”展开；若是简答，最好同时写出主链路构件和运行支撑构件，答案会更完整。

作答时还可以按职责层次补充连接件和数据存储。输入侧需要采集代理、日志接收器、流量镜像接入和终端事件接入；处理侧需要解析、清洗、特征、规则、模型和关联过滤器；输出侧需要告警库、通知服务、安全运营工作台和可视化大屏；支撑侧需要规则管理、模型管理、配置中心、监报告警和审计。如果题干强调每秒5万条日志和3秒延迟，应在答案中体现消息队列、流式处理和并行过滤器；如果强调新增规则2人天上线，应体现插件化规则或独立规则匹配过滤器。也就是说，构件不是凭空列举，而是从题干质量属性和数据处理链路推导出来。

答案-问题（1）

可用性：场景（a）和（e）。其中（a）强调采集节点故障后30秒内切换备用节点并继续接收事件，（e）强调告警通知服务不可用时暂存并恢复后补发，都体现故障情况下继续提供服务或恢复服务的能力。

性能：场景（b）。每秒5万条日志、端到端延迟不超过3秒，属于吞吐量和响应时间要求。

安全性：场景（c）和（f）。身份认证、权限控制、审计记录，以及异常登录、暴力破解、端口扫描识别都属于安全防护和攻击检测。

可修改性：场景（d）。新增日志来源或攻击检测规则要求2人天内完成接入、测试和上线，关注的是修改成本、影响范围和上线效率。

质量属性场景通常由六个要素描述：刺激源、刺激、环境、制品、响应、响应度量。

以（a）为例，刺激源是采集节点或故障事件，刺激是采集节点故障，环境是系统运行期间，制品是数据采集链路，响应是切换备用节点并继续接收关键事件，响应度量是30秒内完成切换。

答案-问题（2）

更适合选择管道-过滤器架构风格。

理由如下：第一，入侵检测系统面对的是连续到达的日志、流量和终端事件，处理链路天然可以拆分为采集、格式转换、清洗、特征提取、规则匹配、异常检测、告警生成等步骤，符合管道-过滤器的流水线处理特征。第二，各过滤器职责单一，便于独立替换和扩展，例如新增一种攻击规则时只需要扩展规则匹配或检测过滤器，不必改动整个系统。第三，各过滤器之间通过数据流连接，便于并行处理、缓冲削峰和分布式部署，有利于满足实时告警和高吞吐要求。第四，管道-过滤器风格便于复用已有过滤器，例如清洗、归一化、告警通知等组件可在不同数据源之间复用。

批处理架构更适合离线统计、日报月报、历史安全分析等周期性任务，但它需要等待数据批次形成后再统一处理，实时性较弱，不适合作为关键告警链路的主架构。

答案-问题（3）

可按数据流向补充如下构件：数据采集过滤器、协议解析/格式转换过滤器、数据清洗过滤器、特征提取过滤器、规则匹配过滤器、异常检测过滤器、事件关联过滤器、风险评分过滤器、告警生成过滤器、告警通知过滤器和可视化展示过滤器。各过滤器之间通过消息队列、流式管道或内存缓冲通道连接。

若需要体现运行支撑，还可以补充规则库、特征库、攻击样本库、告警事件库、审计日志库、配置中心、监报告警模块等辅助构件。一个合理的数据处理链路可以写成：采集原始日志和流量数据 → 解析与归一化 → 去重和噪声过滤 → 提取 IP、端口、用户、行为序列等特征 → 匹配规则或模型检测 → 事件关联与风险评分 → 生成告警 → 推送给安全运营平台。

第11章——SOA 与企业应用集成 (2018年11月系统架构设计师真题-第5题)

阅读以下关于Web系统设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某银行拟将以分行为主体的银行信息系统，全面整合为由总行统一管理维护的银行信息系统，实现统一的账户管理、转账汇款、自助缴费、理财投资、贷款管理、网上支付、财务报表分析等业务功能。但是，由于原有以分行为主体的银行信息系统中，多个业务系统采用异构平台、数据库和中间件，使用的报文交换标准和通信协议也不尽相同，使用传统的EAI解决方案根本无法实现新的业务模式下异构系统间灵活的交互和集成。因此，为了以最小的系统改进整合现有的基于不同技术实现的银行业务系统，该银行拟采用基于ESB的面向服务架构（SOA）集成方案实现业务整合。

问题（1）

请分别用200字以内的文字说明什么是面向服务架构（SOA）以及ESB在SOA中的作用与特点。

思路解析

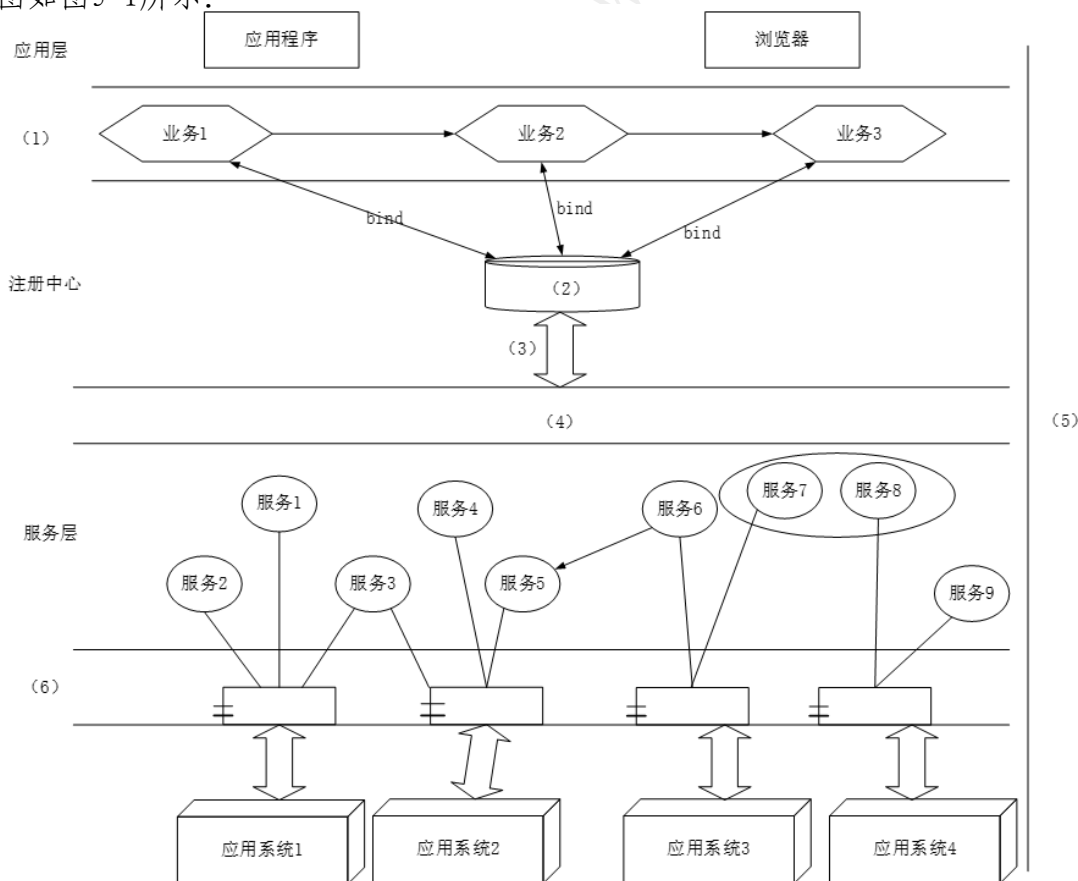
SOA 的本质在于将企业内部各类异构系统统一抽象为“服务”，通过标准化接口（如 WSDL、REST API）和消息交互（如 XML、JSON）进行组合调用，从而避免过去点对点集成的复杂依赖。

SOA 的优势在于**提高系统的可重用性与扩展性**：例如，银行“用户认证”“账户查询”“支付处理”都可以独立封装为服务，在不同业务场景下复用或编排。对银行而言，这意味着不同分行的异构系统不必全部重写，而是通过服务封装与组合实现统一化管理。

ESB 在 SOA 中的角色是“胶水”与“高速公路”。传统 EAI 往往依赖定制适配器实现点对点连接，扩展困难。而 ESB 提供了统一的总线式通信模式，具备**协议转换、数据格式转换、路由、异步队列、事务管理与安全控制**等功能，避免了各系统间的直接依赖。例如，分行 A 使用 SOAP，分行 B 使用 JMS，通过 ESB 可统一转化为 XML 消息并传递到核心系统。这样，银行能在保持原有异构系统的同时，实现集中管理与灵活扩展，满足业务快速迭代需求。

问题（2）

基于该信息系统整合的实际需求，项目组完成了基于SOA的银行信息系统架构设计方案。该系统架构图如图5-1所示：



请从（a）~（j）中选择相应内容填入图5-1的（1）~（6），补充完善架构设计图。

（a）数据层

- (b) 界面层
- (c) 业务层
- (d) bind
- (e) 企业服务总线ESB
- (f) XML
- (g) 安全验证和质量管理
- (h) publish
- (i) UDDI
- (j) 组件层
- (k) BPEL

思路解析

该架构图体现了 **SOA 的分层设计**。最上层通常是**界面层**，其下是**业务层（1）**，用于封装银行的核心业务逻辑，如账户管理、支付、理财等。

业务层服务通过 **UDDI（2）** 注册与发现机制提供查询能力，结合 **publish（3）** 操作实现服务发布。底层的**服务交互**通过 **ESB（4）** 实现消息传递与中介功能，是整个 SOA 的枢纽，屏蔽不同系统的协议差异。ESB 周围附加 **安全验证与质量管理（5）**，确保服务调用符合安全与 SLA 要求。最底层则是 **组件层（6）**，对应银行原有的系统或第三方模块，如支付网关、核心账务、报表分析模块等。

通过这样的设计，银行可以把原本分散在各分行的异构系统统一在一个**面向服务的平台**下。用户请求从界面层进入，经业务层调用服务，再通过 ESB 转发到不同组件。这样既实现了**业务解耦与灵活编排**，又为后续扩展提供了清晰的架构支撑。尤其在跨分行业务场景下，ESB 提供的**统一通信与安全管控**能显著降低集成复杂度，并避免传统 EAI 的“接口爆炸”。

问题（3）

针对银行信息系统的数据交互安全性需求，列举3种可实现信息系统安全保障的措施。

思路解析

银行信息系统对安全性要求极高，尤其在数据交互层面必须保证**机密性、完整性、可用性**。

常见措施包括：①**通信加密**：通过 HTTPS、SSL/TLS 或专用加密通道，防止交易数据在传输过程中被窃取或篡改；②**完整性校验**：利用信息摘要（MD5、SHA-256）和数字签名机制，确保交易指令和报文在传输中未被篡改，且来源可信；③**访问控制与边界防护**：部署防火墙、入侵检测/防御系统，结合网络扫描与漏洞修复，防止黑客攻击和非法访问。

除此之外，还可结合**数据库审计、双因素认证、行为监测**等措施，构建纵深防御体系。对银行而言，采用这些多层安全手段，才能确保在 ESB 架构下的分布式服务交互环境中，业务数据不被泄露或破坏。

答案-问题（1）

面向服务架构（SOA）是一种软件架构设计理念，将应用程序的功能模块抽象为可重用的服务，通过服务之间的松耦合和相互通信来实现系统的灵活性、可扩展性和互操作性。SOA强调服务的独立性、标准化接口和服务的组合，使得系统更易于维护、扩展和集成。

企业服务总线（ESB）是SOA架构中的关键组件，用于实现不同服务之间的通信、消息路由、转换和协调。ESB提供了统一的消息传递机制和中介服务，可以实现服务之间的解耦和集成，同时具备消息传递、安全性、事务管理等功能。其特点包括灵活性、可扩展性、异步通信、消息路由和转换、安全性和监控等，为构建复杂的分布式系统提供了支持。ESB在SOA中扮演着关键的角色，促进了服务之间的互联和协作，提高了系统的可靠性和可维护性。

答案-问题（2）

（1）（c）业务层 （2）（i）UDDI （3）（h）publish

（4）（e）企业服务总线ESB （5）（g）安全验证和质量管理 （6）（j）组件层

答案-问题（3）

采用 HTTPS 协议或传输加密，确保数据机密性。

使用 信息摘要/数字签名，保证数据完整性。

部署 防火墙/入侵检测/网络扫描，提升系统防御能力。

（2016年11月系统架构设计师真题-第1题）

阅读以下关于软件架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某软件公司为某品牌手机厂商开发一套手机应用程序集成开发环境，以提高开发手机应用程序的质量和效率。在项目之初，公司的系统分析师对该集成开发环境的需求进行了调研和分析，具体描述如下：

a. 需要同时支持该厂商自行定义的应用编程语言的编辑、界面可视化设计、编译、调试等模块，这些模块产生的模型或数据格式差异较大，集成环境应提供数据集成能力。集成开发环境还要支持以适配方式集成公司现有的应用模拟器工具。

b. 经过调研，手机应用开发人员更倾向于使用Windows系统，因此集成开发环境的界面需要与Windows平台上的主流开发工具的界面风格保持一致。

c. 支持相关开发数据在云端存储，需要保证在云端存储数据的机密性和完整性。

d. 支持用户通过配置界面依据自己的喜好修改界面风格，包括颜色、布局、代码高亮方式等，配置完成后无需重启环境。

e. 支持不同模型的自动转换。在初始需求中定义的机器性能条件下，对于一个包含50个对象的设计模型，将其转换为相应代码框架时所消耗时间不超过5秒。

f. 能够连续运行的时间不小于240小时，意外退出后能够在10秒之内自动重启。

g. 集成开发环境具有模块化结构，支持以模块为单位进行调试、测试与发布。

h. 支持应用开发过程中的代码调试功能：开发人员可以设置断点，启动调试，编辑器可以自动卷屏并命中断点，能通过变量监视器查看当前变量取值。

在对需求进行分析后，公司的架构师小张查阅了相关的资料，认为该集成开发环境应该采用管道-过滤器（Pipe-Filter）的架构风格，公司的资深架构师王工在仔细分析后，认为应该采用数据仓储（Data Repository）的架构风格。公司经过评审，最终采用了王工的方案。

问题（1）

识别软件架构质量属性是进行架构设计的重要步骤。请分析题干中的需求描述，填写表1-1中

（1）~（5）处的空白。

质量属性名称	需求描述编号
可用性	(1)
(2)	e
可修改性	(3)
可测试性	(4)
安全性	c
易用性	(5)

思路解析

在软件架构设计中，识别质量属性是将需求转化为架构决策的重要环节。通过分析题干需求，可以发现不同需求对应不同的质量属性。

需求 f：要求系统连续运行时间 ≥ 240 小时，且异常退出后能在 10 秒内恢复。这显然属于 **可用性** 的要求，因此表中 (1) 应填 f。可用性主要衡量系统在规定条件下维持运行和快速恢复的能力。

需求 e：要求模型转换 50 个对象时不超过 5 秒，体现了系统处理任务的效率，即 **性能**。因此 (2) 填性能。性能是衡量系统响应时间、处理速度的重要指标。

需求 d：强调界面风格可修改且无需重启，说明系统需要具备良好的 **可修改性**。因此 (3) 填 d。可修改性保障用户在不破坏系统整体稳定性的前提下自由调整界面配置。

需求 g：系统具备模块化结构，支持单独调试、测试与发布，明显属于 **可测试性**。因此 (4) 填 g。可测试性关注系统能否被验证和定位问题。

需求 b：界面需要与主流工具风格一致，属于提升用户操作便利性的 **易用性**。因此 (5) 填 b。易用性直接影响用户的学习成本和开发效率。

而像 **需求 a 和 c**，一个涉及功能实现（数据集成与工具适配），另一个涉及 **安全性**（数据的机密性与完整性），并未出现在本表对应的考查点中，因此不在本题答案范围。

综上，正确填法为：

(1) f → 可用性

(2) 性能 → 响应时间限制

(3) d → 可修改性

(4) g → 可测试性

(5) b → 易用性

问题 (2)

请在阅读题干需求描述的基础上，从交互方式、数据结构、控制结构和扩展方法4个方面对两种架构风格进行比较，填写表1-2中 (1) ~ (4) 处的空白。

比较因素	管道—过滤器风格	数据仓储风格
交互方式	顺序结构或有限的循环结构	(1)
数据结构	(2)	文件或模型
控制结构	(3)	业务功能驱动
扩展方法	接口适配	(4)

思路解析

架构风格的选择决定了系统运行时的交互方式、数据存储组织、控制逻辑以及未来的可扩展性。

交互方式 (1)： 数据仓储风格中，所有模块通过一个中心数据仓库（或黑板系统）进行交互，这种模式类似 **星型结构**，即中心仓库是“星心”，其他工具是“星角”，通过仓库共享数据。相比之下，管道-过滤器风格是线性或链式交互，数据在模块之间依次流动。

数据结构 (2)： 管道-过滤器风格的本质是数据流（流式数据），每个过滤器对输入流进行处理并输出新的数据流。而仓库风格则依赖一个持久化的共享数据存储（如数据库、语法树等），各模块围绕该中心数据展开操作。

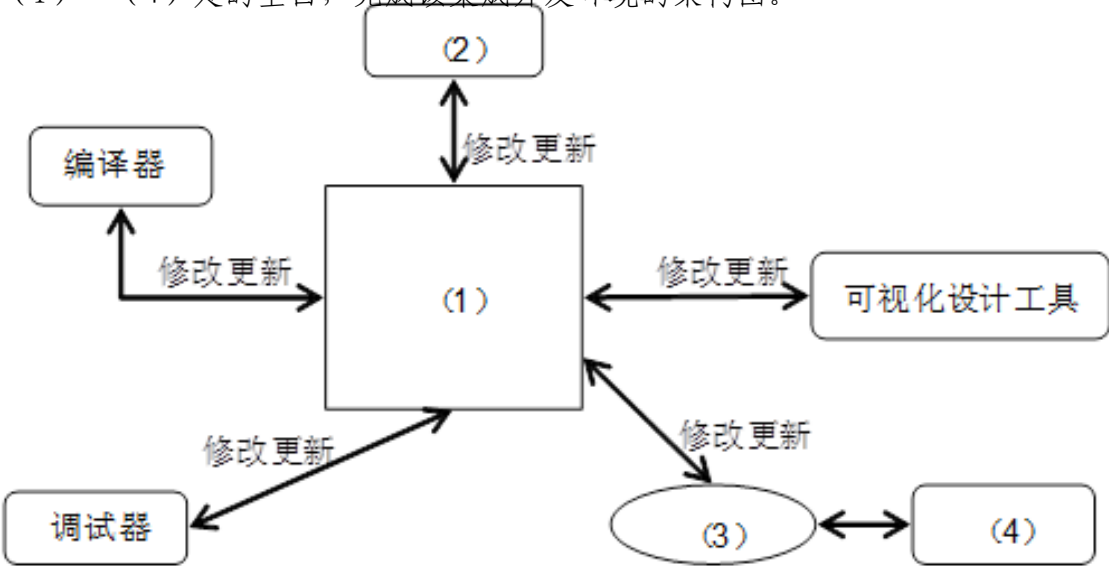
控制结构 (3)： 仓库风格通常采用 **数据驱动** 控制，即当仓库数据发生变化时，触发相应模块执行。与之相对，管道-过滤器是 **数据流驱动**，即控制由输入流推动。

扩展方法 (4)： 在仓库风格下，新增功能通常需要围绕数据模型进行 **模型适配**，以保证与中央仓库的数据一致性。而在管道-过滤器中，扩展通常通过新增过滤器模块，并将其插入数据流管道中完成。

综合来看，仓库风格更适合题干描述的 IDE 项目，因为其需求核心是 **不同工具和数据的集成共享**，而非简单的数据流处理。

问题 (3)

在确定采用数据仓库架构风格后，王工给出了集成开发环境的架构图。请填写图1-1中 (1) ~ (4) 处的空白，完成该集成开发环境的架构图。



思路解析

数据仓储风格的核心在于 **中央数据存储** 和 **围绕数据进行操作的功能模块**。结合题干需求，可以逐一对应：

(1) 模型/数据库/语法结构树： 仓库的中心必须是数据实体。在 IDE 场景中，最合理的中心数据是 **语法结构树 (AST)** 或类似的模型/数据库，因为编译、调试、可视化设计等都基于语法树或统一的中间模型进行处理。

(2) 编辑器： 编辑器是 IDE 的核心模块之一，负责源代码输入、语法高亮和界面操作，必须直接与仓库交互，以保证用户修改的代码能即时反映到仓库数据中。

(3) **适配器**: 题干中明确提到需要通过适配方式集成公司现有的模拟器工具, 因此在仓库结构中必须增加一个 **适配器** 作为桥梁。

(4) **应用模拟器工具**: 通过适配器对接的现有工具就是 **模拟器**, 它可以运行编译后的程序, 支持调试和测试。

因此, 架构图应围绕“数据仓库 + 模块”的模式进行, 仓库位于中央, 其他功能(如编辑器、编译器、适配器等)分布在周围, 通过仓库交互。

答案-问题(1)

(1) f (2) 性能 (3) d (4) g (5) b

答案-问题(2)

(1) 星型(工具之间通过中心结点进行交互) (2) 数据流(或流式数据)

(3) 数据驱动 (4) 模型适配(与数据仓储进行数据适配)

答案-问题(3)

(1) 模型/数据库/语法结构树 (2) 编辑器 (3) 适配器 (4) 应用模拟器工具

(2013年11月系统架构设计师真题-第1题)

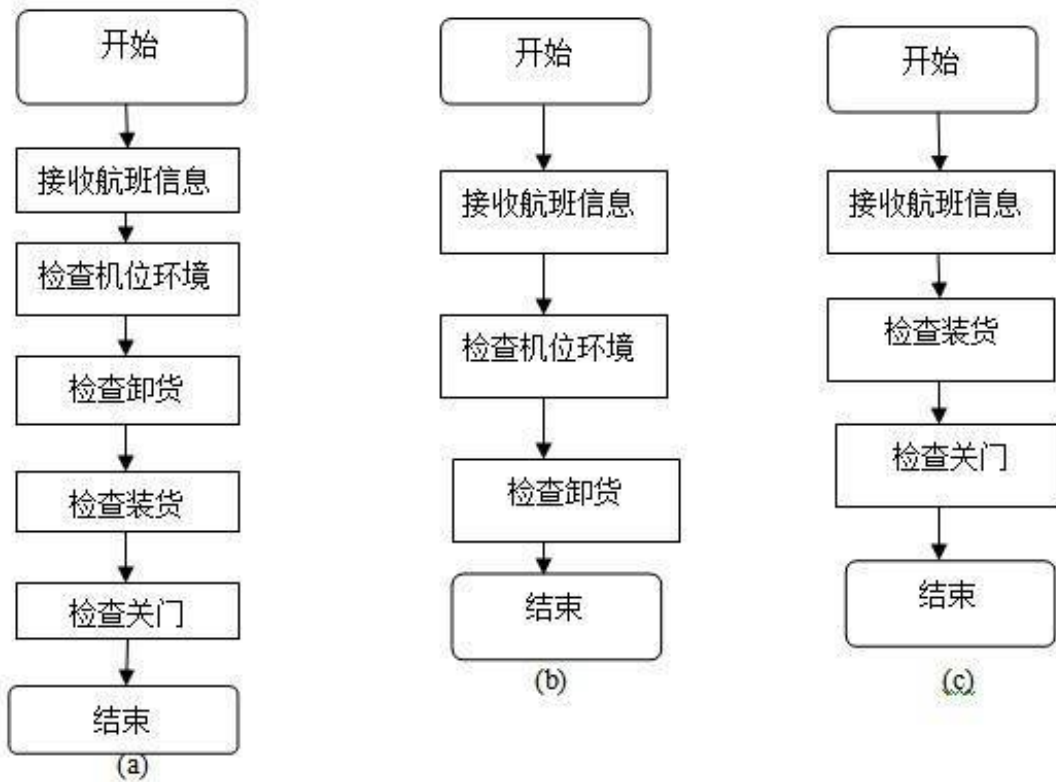
阅读以下关于企业应用系统集成架构设计的说明, 在答题纸上回答问题1和问题2。

【说明】

某航空公司希望对构建于上世纪七、八十年代的主要业务系统进行改造与集成, 提高企业的竞争力。由于集成过程非常复杂, 公司决定首先以Ramp Coordination系统为例进行集成过程的探索与验证。

在航空业中, Ramp Coordination是指飞机从降落到起飞过程中所需要进行的各种业务活动的协调过程。通常每个航班都有一位员工负责Ramp Coordination, 称之为Ramp Coordinator。由Ramp Coordinator协调的业务活动包括检查机位环境、卸货和装货等。

由于航班类型、机型的不同, Ramp Coordination的流程有很大差异。图1-1 (a) 所示的流程主要针对短期中转航班, 这类航班在机场稍作停留后就起飞; 图1-1 (b) 所示的流程主要针对到达航班, 通常在机场过夜后第二天起飞; 图1-1 (c) 所示的流程主要针对离港航班, 这类航班是每天的第一班飞机。这三种类型的航班根据长途/短途、国内/国外等因素还可以进一步细分, 每种细分航班类型的Ramp Coordination的流程也略有不同。



为了完成上述业务，Ramp Coordination信息系统需要从乘务人员管理系统中提取航班乘务员的信息、从订票系统中提取乘客信息、从机务人员管理系统中提取机务人员信息、接收来自航班调度系统的航班到达事件。其中乘务人员管理系统和航班调度系统运行在大型主机系统之上，机务人员管理系统运行在Unix操作系统之上，订票系统基于Java语言，具有Web界面，运行在Linux操作系统之上。

目前Ramp Coordination信息系统主要由人工完成所有协调工作，效率低且容易出错。公司领导要求集成后的Ramp Coordination信息系统能够针对不同需求迅速开展业务流程，灵活、高效地完成协调任务。

针对上述要求，公司IT部门的架构师经过分析与讨论，最终采用面向服务的架构，以服务为中心进行Ramp Coordination信息系统的集成工作。

问题（1）

服务建模是对Ramp Coordination信息系统进行集成的首要工作，公司的架构师首先对Ramp Coordination信息系统进行服务建模，识别出系统中的两个主要业务服务组件：

- （1）Ramp Control：负责Ramp Coordination信息系统中各种相关业务活动的组件；
- （2）Flight Management：负责航班相关信息的管理，包括航班日程、乘客信息等。针对上述服务模型，结合题干描述，请为每个业务服务组件提供的服务进行分析与整理，完成表1-1中的空白部分

业务服务组件	提供服务名称
Ramp Control	
Flight Management	

思路解析

对Ramp Coordination信息系统进行**服务建模**的起点，是把可复用、可编排的业务能力抽象为“服务”，并与后端异构系统的资源解耦。题干明确：Ramp Coordinator要协调的业务活动包含**检查机位**

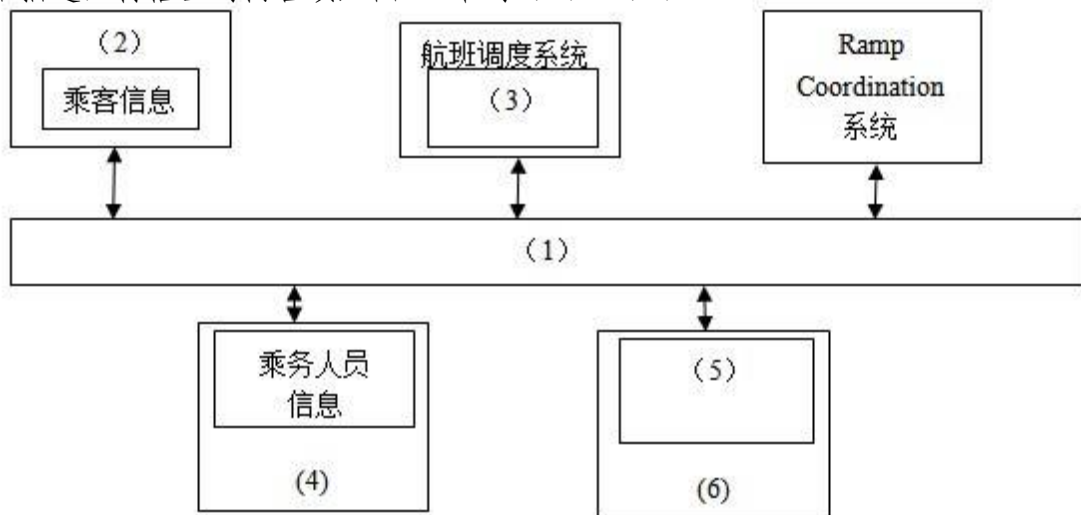
环境、卸货、装货等；结合图示三类典型航班流程（中转、到达过夜、离港首班），我们可进一步识别在放行前的**关门检查**也是常见的关键节点。因此，在“以服务为中心”的SOA方法下，把这些与地面保障现场**直接耦合**的可操作性步骤，统一归入 **Ramp Control** 业务服务组件最为合理。它们的共同特征是：面向运行现场、与航班在站位的一次保障周期强相关、对外呈现为**可编排的原子或复合服务**（如“检查卸货”可能细分为货舱门状态检测、ULD核对、卸载清单校验等）。将这些能力打包为Ramp Control的服务，可以被前端 workflow 或规则引擎按不同航班类型灵活编排，满足“流程差异大”的诉求。

与此相对，**Flight Management** 的职责是**航班信息管理**：题干指出系统需“从乘务人员管理系统提取乘务员信息、从订票系统提取乘客信息、从机务系统提取机务信息、接收航班调度系统的到达事件”。这些都属于“**航班主数据与运行态数据的汇聚、维护与发布**”。因此在服务边界上，Flight Management对外暴露的最核心能力应是**接收与管理航班信息**（包括日程、乘客、事件与关联系统人员信息），为Ramp Control等下游流程提供“事实源”。这类服务偏向**信息型（Information Services）**，强调**一致性、时效性与订阅/推送**（例如，航班到达事件被接收后触发Ramp流程实例的启动或变更）。

拓展来说，这种划分具备三点好处：第一，**高内聚低耦合**——现场操作类能力与信息管理类能力边界清晰；第二，**可编排性**——Ramp Control的原子服务可以被BPMN/规则驱动按航班类型动态组合，契合“流程差异大”的业务现实；第三，**可演进性**——当后端异构来源或接入方式变化时，只需在Flight Management侧适配，不影响Ramp Control的服务契约。

问题（2）

对Ramp Coordination信息系统的集成涉及对乘务人员管理系统、航班调度系统、机务人员管理系统和订票系统的组织与协调，公司架构师决定采用企业服务总线（Enterprise Service Bus, ESB）技术进行系统集成，请用200字以内的文字对ESB的定义进行描述，给出ESB的五个主要功能，并针对题干描述，将恰当的内容填入图1-2中的（1）~（6）。



思路解析

在本案例中，企业需要把分散在**大型主机、Unix、Linux/Java Web**等平台上的多个关键系统——乘务人员管理、航班调度、机务人员管理、订票系统——与新的Ramp Coordination信息系统进行统一编排与协同。典型挑战是**异构、差异与演进**：平台不同（Mainframe/Unix/Linux）、语言不同（COBOL/Unix C/Java）、接口风格不同（文件、消息、API、Web界面背后服务），以及事件驱动（航班到达/离港）与数据拉取（人员/乘客清单）并存。为同时达成**松耦合互联、服务化封装、可路由可监控**，采用ESB是业界成熟做法。

为什么是ESB？

首先，ESB通过**服务注册/发现**屏蔽“调用者—提供者”的位置耦合，实现**服务位置透明**；当航班调度系统从主机迁移或扩容时，Ramp侧既有流程无须改代码，只需更新注册信息。

其次，ESB内置协议桥接（如JMS/HTTP/SOAP/REST/MQ/File/主机接入适配器），解决“主机事务网关—消息中间件—HTTP API”之间的互通。

第三，消息格式转换与数据映射能力，解决主机EBCDIC报文、机务系统专有格式与Java Web系统XML/JSON之间的语义对齐；配合基于内容的智能路由与过滤，能按航班类型、机场规则、航站运行态等把消息投递到相应的Ramp Control服务编排。

第四，消息增强在生产环境极其关键：可统一施加**安全（鉴权、签名、加解密）、可靠性（重试、补偿、幂等）、事务边界、审计日志、追踪ID（Correlation ID）**等横切关注点，避免把非功能逻辑散落在各业务系统中。第五，监控与管理（可观测性）让运维能从总线上实时观察服务可用性、吞吐与延迟，进行告警与容量规划，支撑“灵活、高效”的业务目标。

图1-2的填写思路：外部四个分支节点分别代表四套源系统/应用（乘务、航调、机务、订票），每个节点的内层标注其提供的关键信息（乘务人员信息、航班到达事件、机务人员信息、乘客信息），中心为ESB；另一个分支为Ramp Coordination信息系统，它既是消费者（订阅航班到达事件、拉取乘客/人员清单），也是提供者（向下游或门户暴露“协调进度/状态”等服务）。这样的布局突出了“系统在外、服务在中（ESB）、流程在内（Ramp）”的集成拓扑：上游系统变化不影响Ramp流程，只需在ESB层完成适配；Ramp侧可依据航班类型在BPM/规则引擎中快速编排调用Ramp Control与Flight Management等服务，满足“针对不同需求迅速开展业务流程”的诉求。

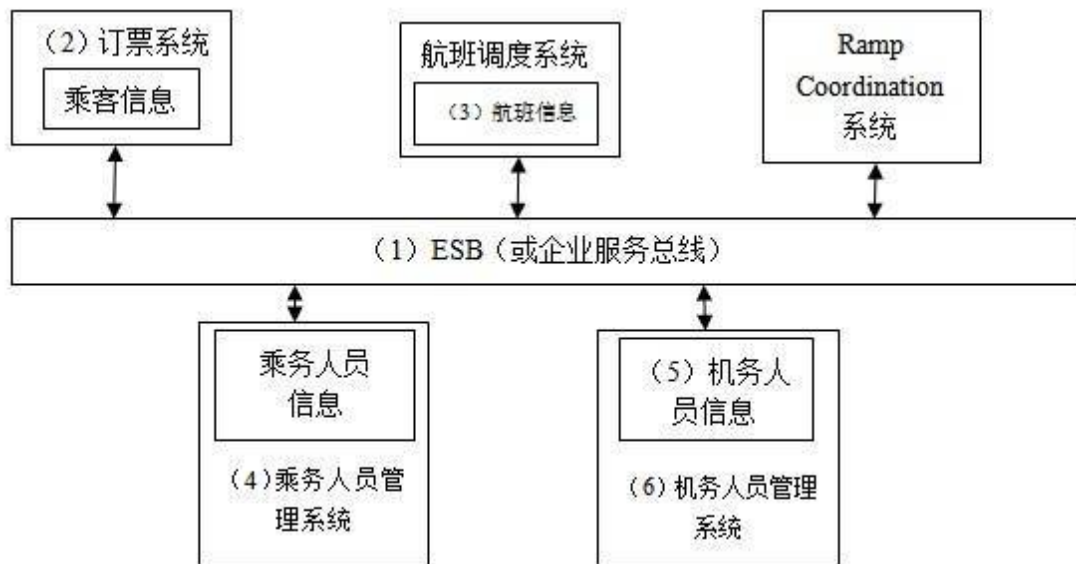
答案-问题（1）

- （1）检查机位环境、检查卸货、检查装货、检查关门
- （2）接收航班信息

答案-问题（2）

ESB是传统中间件技术与XML、Web服务等技术结合的产物，主要支持异构系统集成。ESB基于内容的路由和过滤，具备复杂数据的传输能力，并可以提供一系列的标准接口。ESB的主要功能：

- （1）服务位置透明性；（2）传输协议转换；（3）消息格式转换；（4）消息路由；
- （5）消息增强；（6）安全性；（7）监控与管理。



第12章——分布式架构与协调机制 (2024年5月系统架构设计师真题-第4题)

在分布式系统中，由于多个节点同时操作共享资源，容易产生竞争条件和资源冲突问题。为了保证数据一致性和系统的稳定性，需要一种有效的分布式锁机制来协调各节点对共享资源的访问。为了简化分布式锁的实现并提高性能，张工提出了一种利用数据库主键索引的方法来实现分布式锁。该

方法的核心思想是利用数据库的唯一性约束和事务特性，通过对特定记录的插入和删除操作来实现锁的获取和释放。

问题（1）

基于数据库实现分布式锁的缺点。

思路解析

基于数据库实现分布式锁常见的实现方式主要有以下两种：

1. 利用唯一索引插入

基本原理是 **唯一性约束充当“互斥机制”**，确保同一时间只有一个客户端能获得锁。

创建一张锁表，如 distributed_locks (lock_key VARCHAR PRIMARY KEY, owner_id VARCHAR, expire_time DATETIME)

某客户端尝试通过插入一条唯一的 lock_key 记录来加锁：

```
INSERT INTO distributed_locks (lock_key, owner_id, expire_time)
VALUES ('my-lock', 'instance-1', NOW() + INTERVAL 10 SECOND);
```

如果插入成功，则加锁成功；如果因主键冲突插入失败，说明锁已被持有。

由 owner_id 判断是否是当前客户端持有锁，只有持有者才可删除：

```
DELETE FROM distributed_locks WHERE lock_key = 'my-lock' AND owner_id = 'instance-1';
```

2. 利用数据库的 SELECT ... FOR UPDATE 加悲观锁

基本原理是通过数据库行锁实现互斥。

在锁表中先插入一条标志性记录。

使用事务和 SELECT ... FOR UPDATE 对该记录加锁。

```
BEGIN;
```

```
SELECT * FROM distributed_locks WHERE lock_key = 'my-lock' FOR UPDATE;
```

```
-- 执行临界区代码
```

```
COMMIT;
```

这种方法的主要的问题是

竞争激烈时插入失败频繁。在高并发场景中，大量节点同时尝试 INSERT 锁记录会频繁触发主键冲突，导致性能下降，数据库压力剧增。

锁重入难实现。默认情况下无法支持可重入（同一线程/服务可多次加锁），需要额外存储 owner_id 和 lock_count 等字段，并设计相应逻辑，增加实现难度。

数据库成为瓶颈。分布式锁操作本质是对数据库的高频写入和查询操作，容易将数据库打满，拖慢主业务性能。

问题（2）

举一个产生Redis分布式锁死锁的场景。

思路解析

死锁的本质是

锁未及时释放 + 锁还未自动过期

所有其他节点都被“锁”住了，无法进入临界区 死锁

系统使用 Redis 实现分布式锁，每个节点在执行关键业务逻辑前通过 SET key value NX PX 加锁。

加锁成功后执行临界区操作，最后显式调用 DEL key 解锁。

服务 A 加锁成功，执行如下命令： bash复制编辑SET lock:order:123 "nodeA-uuid" NX PX 30000表示锁住订单 123，锁有效期为 30 秒。

服务 A 开始执行下单逻辑。

执行过程中服务 A 崩溃（进程退出、机器重启、OOM 等）。

因为服务 A 没来得及执行 DEL 解锁命令，Redis 上的锁 lock:order:123 仍然存在。
此时锁还没过期（30 秒未到），其他服务节点（如服务 B）尝试加锁失败，始终得不到锁，业务阻塞。

问题（3）

根据描述，在括号中填入对应的 zset 指令，存入秒杀的分数命令（1），获取分数范围的命令（2），获取分数（3）。

思路解析

（1）ZADD — 添加元素到有序集合（ZSet）

将一个或多个成员及其对应的“分数”添加到指定的有序集合中。如果成员已存在，则更新其分数。

```
ZADD seckill 1624698000 user123
```

表示将 user123 加入名为 seckill 的有序集合，分数为 1624698000（通常是时间戳，可用于记录秒杀时间）。

（2）ZRANGEBYSCORE — 按分数范围获取成员

返回指定有序集合中 分数在给定范围内的所有成员，按分数从小到大排序。

```
ZRANGEBYSCORE seckill 1624698000 1624699000
```

表示查询在指定时间段（秒杀时间段）内参与的用户列表。

（3）ZSCORE — 查询成员对应的分数

有序集合中的分数（即排序权重）。

```
ZSCORE seckill user123
```

表示查询 user123 在 seckill 集合中的分数（如其参与时间戳）。

答案-问题（1）

基于数据库实现分布式锁虽然实现简单、依赖少，但存在明显缺点：在高并发场景下性能较差，容易成为系统瓶颈；若服务异常未能释放锁，易导致死锁；可重入性和公平性支持弱；扩展性差，难以支撑大规模分布式系统的稳定运行。因此，在对性能和可靠性要求较高的系统中，通常不推荐使用数据库作为分布式锁的实现方式。

答案-问题（2）

当用 Redis 做分布式锁时，当 A 用户竞争锁成功，A 用户所在的主机挂了，这时候还没有来得及释放锁，且设置的过期时间较长，还未到期，那么其他用户去用 setnx 指令去竞争锁时发现 Redis 有这个锁的 key，所以就导致其他用户永远都竞争不成功。

答案-问题（3）

（1）ZADD

（2）ZRANGEBYSCORE

（3）ZSCORE

第13章——云边端与智能架构 (2022年11月系统架构设计师真题-第5题)

阅读以下关于 Web 系统架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某公司拟开发一套基于边缘计算的智能门禁系统，用于如园区、新零售、工业现场等存在来访、被访业务的场景。来访者在来访前，可以通过线上提前预约的方式将自己的个人信息记录在后台，被

访者在系统中通过此请求后，来访者在到访时可以直接通过"刷脸"的方式通过门禁，无需做其他验证。此外，系统的管理员可对正在运行的门禁设备进行管理。

基于项目需求，该公司组建项目组，召开了项目讨论会。会上，张工根据业务需求并结合边缘计算的思想，提出本系统可由访客注册模块、模型训练模块、端侧识别模块与设备调度平台模块等四项功能组成。李工从技术层面提出该系统可使用 Flask 框架与 SSM 框架为基础来开发后台服务器，将开发好的系统通过 Docker 进行部署，并使用 MQTT 协议对 Docker 进行管理。

问题（1）

MQTT 协议在工业物联网中得到广泛的应用，请用300字以内的文字简要说明 MQTT 协议。

思路解析

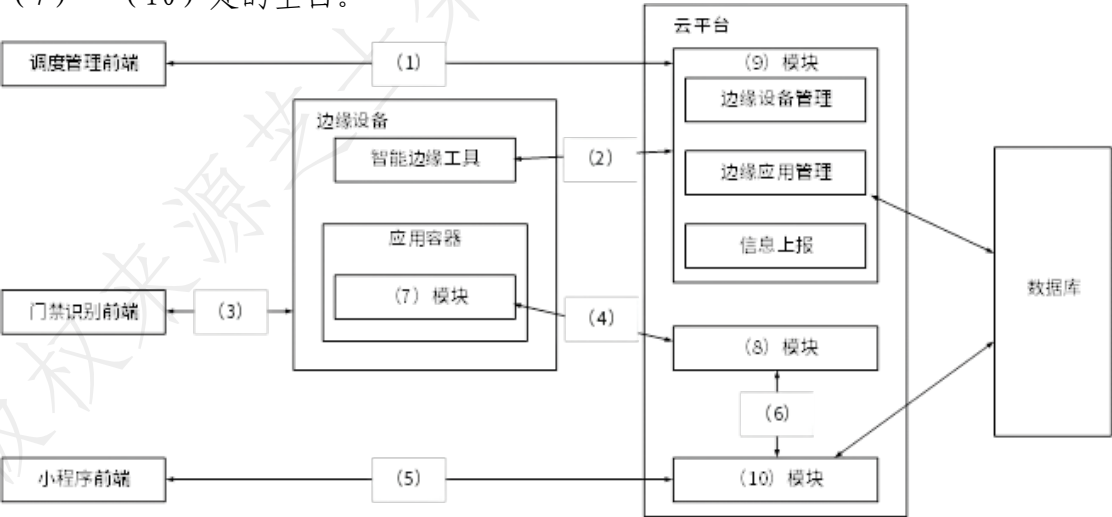
MQTT 的核心优势在于 **轻量化与高效传输**。其工作流程是：终端设备（客户端）将消息发布到 Broker，Broker 根据订阅关系分发消息。与 HTTP 的请求/响应模式不同，MQTT 的 **发布/订阅机制** 可以实现一对多通信，极大提升消息分发效率。协议层面仅需极少数据报文，典型报文长度只有几字节，非常适合嵌入式设备和低功耗传感器。

在工业物联网中，设备数量庞大且网络复杂，MQTT 的 **QoS（消息服务质量等级）机制** 可以保证消息至少一次、至多一次或保证一次送达，提升系统可靠性。同时，MQTT 还支持持久会话、遗嘱消息等特性，为工业场景下的设备管理与状态同步提供保障。因此，MQTT 成为边缘计算与物联网中事实上的标准协议。

问题（2）

在会议上，张工对功能模块进行了更进一步的说明：访客注册模块用于来访者提交申请与被访者确认申请，主要处理提交来访申请、来访申请审核业务，同时保存访客数据，为训练模块准备训练数据集；模型训练模块用于使用访客数据进行模型训练，为端侧设备的识别业务提供模型基础；端侧识别模块在边缘门禁设备上运行，使用训练好的模型来识别来访人员，与云端服务协作完成访客来访的完整业务；设备调度平台模块用于对边缘门禁设备进行管理，管理人员能够使用平台对边缘设备进行调度管理与状态监控，实现云端协同。

图5-1给出了基于边缘计算的智能门禁系统架构图，请结合 HTTP 协议和 MQTT 协议的特点，为图 5-1中（1）～（6）处选择合适的协议；并结合张工关于功能模块的描述，补充完善图5-1中（7）～（10）处的空白。



思路解析

在该系统架构中，不同通信环节需选择合适的协议：

- （1）、（5）、（6）选 HTTP：访问者与访问者通过 Web/移动端发起请求，适合使用 HTTP 协议，其通用性强，兼容浏览器与服务器的交互；管理员访问管理平台同样基于 HTTP。
- （2）、（3）、（4）选 MQTT：边缘设备与云端交互时，要求轻量、实时、低带宽消耗，MQTT 协

议更适合设备状态上报、指令下发等场景。

模块补充部分：

- (7) **端侧识别**：部署在门禁前端，完成刷脸识别；
- (8) **模型训练**：基于采集的访客数据生成识别模型；
- (9) **设备调度**：用于边缘设备的统一调度与监控；
- (10) **访客注册**：负责来访者预约与数据录入。

问题（3）

请用300字以内的文字，从数据通信、数据安全和系统性能等方面简要分析在传统云计算模型中引入边缘计算模型的优势。

思路解析

传统云计算模式下，所有数据需集中上传云端处理，存在 **延迟高、带宽消耗大、安全风险集中** 等问题。引入边缘计算后，数据可以在靠近数据源的边缘节点处理，极大减少长距离传输，提升通信效率与实时响应能力。

在数据安全方面，边缘计算通过 **本地化存储与计算**，减少了原始敏感数据在网络中传输的机会，黑客即使入侵单个节点也难以获取全局数据，降低系统整体风险。

在系统性能上，边缘节点承担部分 AI 推理或数据预处理工作，云端只需负责复杂计算与集中管理，形成“边-云协同”的模式。这样不仅缓解云端压力，也提升了用户侧的交互体验，特别适合智能门禁、无人零售、工业物联网等 **对实时性和安全性要求高的场景**。

答案-问题（1）

MQTT（消息队列遥测传输）是一个基于发布/订阅模式的消息协议。它工作在 TCP/IP协议族上，是为硬件性能低下的远程设备以及网络状况糟糕的情况下而设计的发布/订阅型消息协议。MQTT协议是轻量、简单、开放和易于实现的。

答案-问题（2）

- (1) HTTP
- (2) MQTT
- (3) MQTT
- (4) MQTT
- (5) HTTP
- (6) HTTP
- (7) 端侧识别
- (8) 模型训练
- (9) 设备调度
- (10) 访客注册

答案-问题（3）

数据通信：通信更快捷，数据量更少。因为数据处理对比在边缘设备上完成，通信更多时候只传输匹配与结果的指令。

数据安全：数据以加密方式存储在需要用到的边缘设备上，本地化处理比对，减少原始信息在网上的传递带来的安全隐患。黑客也无法通过攻破一个结点使整个系统瘫痪。

系统性能：性能更高，以人脸识别为例，在进行识别时，只在本地进行比对不用把人脸数据传递到远程服务器对比。

(2021年11月系统架构设计师真题-第5题)

阅读以下关于Web系统架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某公司拟开发一个智能家居管理系统，该系统的主要功能需求如下：1) 用户可使用该系统客户端实现对家居设备的控制，且家居设备可向客户端反馈实时状态；2) 支持家居设备数据的实时存储和查询；3) 基于用户数据，挖掘用户生活习惯，向用户提供家居设备智能化使用建议。

基于上述需求，该公司组建了项目组，在项目会议上，张工给出了基于家庭网关的传统智能家居管理系统的设计思路，李工给出了基于云平台的智能家居系统的设计思路。经过深入讨论，公司决定采用李工的设计思路。

问题（1）

请用400字以内的文字简要描述基于家庭网关的传统智能家居管理系统和基于云平台的智能家居管理系统在网关管理、数据处理和系统性能等方面的特点，以说明项目组选择李工设计思路的原因。

思路解析

本题考查对比分析两种智能家居架构设计方案的能力，重点在于理解**家庭网关**与**云平台**在架构上的差异，及其对系统性能、扩展性和未来发展的影响。

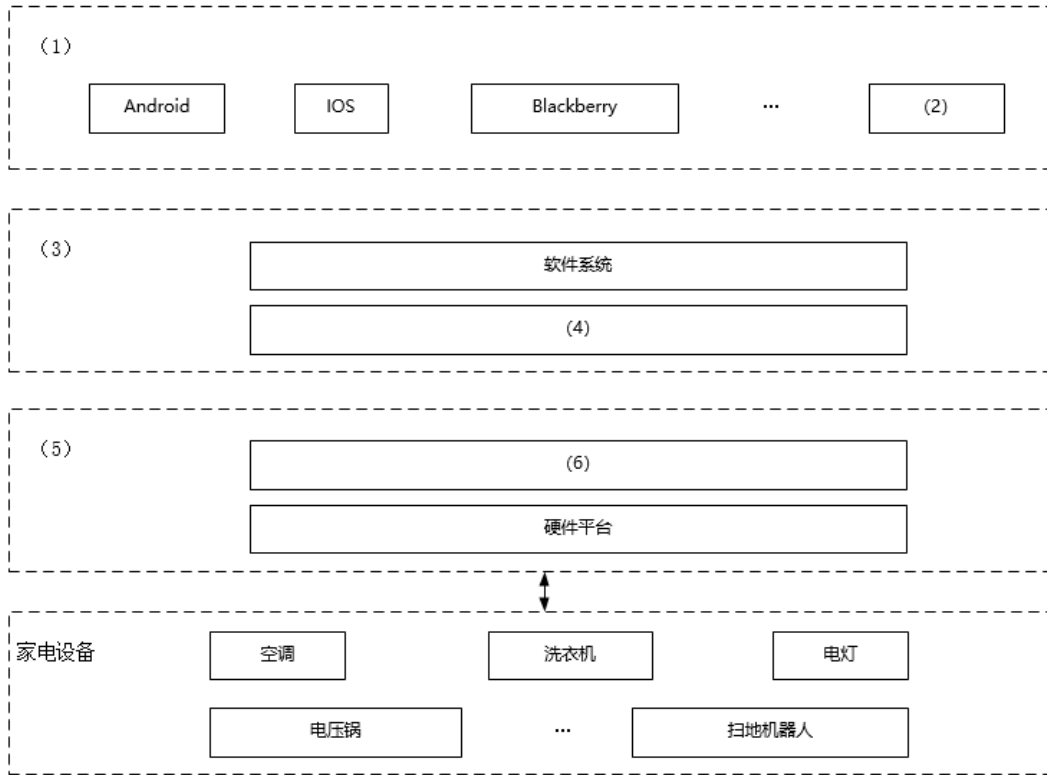
从**网关管理**角度来看，**基于家庭网关**的系统采用局部中心式架构，家庭网关作为智能家居的“中枢”，管理本地设备通信和协议转换。但这种方式对网关硬件的性能要求高，系统升级和设备兼容性受限，而且每个家庭独立维护一个网关，缺乏统一的远程管理手段。而**基于云平台**的系统则以云端服务为核心，家庭网关变成了简单的接入节点，设备控制和管理集中在云端，具备远程配置、集中管理的能力，更易实现规模化部署与运维。

从**数据处理能力**看，传统家庭网关架构将数据处理限制在本地，资源有限，仅适用于简单的状态记录和控制逻辑。无法有效支持数据挖掘、行为分析和机器学习等高级功能。而云平台拥有强大的计算资源和灵活的数据调度能力，可以对来自不同家庭的大量数据进行实时分析，实现复杂的用户画像、场景自动化以及智能推荐等功能，显著提升系统的“智能化”水平。

从**系统性能和扩展性**角度出发，传统网关方案受限于硬件设备的处理能力和网络条件，当设备数量增长或功能复杂度增加时，容易出现性能瓶颈。而基于云平台的系统具有天然的可扩展性，借助负载均衡、微服务架构等方式，能够动态扩展服务节点，保持系统稳定运行，支持高并发、高可靠的服务需求。

问题（2）

请从下面给出的(a) ~ (j) 中进行选择，补充完善图5-1中空(1) ~ (6)处的内容，协助李工完成该系统的架构设计方案。



- (a) Wi-Fi
- (b) 蓝牙
- (c) 驱动程序
- (d) 数据库
- (e) 家庭网关
- (f) 云平台
- (g) 微服务
- (h) 用户终端
- (i) 鸿蒙
- (j) TCP/IP

思路解析

架构图从用户使用端到设备控制端分为应用层、系统层、平台层与设备层：

用户终端代表用户操作的入口；

鸿蒙是用于智能终端的国产操作系统；

云平台承担系统主控与数据处理功能；

数据库用于存储用户行为数据与设备状态；

家庭网关作为设备接入节点；

驱动程序直接控制硬件设备的运行。

问题（3）

该系统需实现用户终端与服务端的双向可靠通信，请用300字以内的文字从数据传输可靠性的角度对比分析TCP和UDP通信协议的不同，并说明该系统应采用哪种通信协议。

思路解析

本题主要考查对 TCP 和 UDP 两种传输层协议的理解。

在智能家居管理系统中，用户终端需要通过网络控制家电设备，如开关、状态查询、模式调节等。这些操作属于“**控制类通信**”，特点是需要确保命令的正确传达与设备的准确反馈，一旦指令丢失

或失序，可能导致严重的执行偏差甚至安全隐患。

TCP（传输控制协议）是面向连接的协议，采用“三次握手”建立连接，具有**确认应答机制、重传机制和顺序控制机制**，能保证数据包的完整、准确、有序到达。尤其适用于要求可靠性较高的数据传输场景，如智能设备状态同步、配置下发、报警反馈等。

****UDP（用户数据报协议）****虽然具有较低的开销和更快的传输速度，但属于无连接协议，不保证数据的完整性与顺序性，不具备重传与拥塞控制机制，存在丢包和乱序的风险，适用于如视频流、语音通话等对实时性要求高但可容忍部分数据丢失的场景。

结合本系统的需求，涉及到**命令控制与状态反馈的可靠性**，显然需要选择 TCP 协议以保障通信的准确性和稳定性。尽管 TCP 的传输效率稍低，但智能家居控制操作频率相对较低，不存在大规模数据并发传输，因此可以接受其带来的开销。

答案-问题（1）

基于家庭网关的传统智能家居管理系统主要特点包括：**网关管理**：家庭网关作为中心控制设备，负责连接和管理各种智能设备，但功能受限于硬件性能和扩展性。**数据处理**：数据处理主要在本地进行，对于大规模数据处理和存储能力有限，难以实现复杂智能算法和数据分析。**系统性能**：受限于网关硬件性能，系统响应速度和处理能力有限，难以支持大规模智能家居设备的管理和控制。

基于云平台的智能家居管理系统特点包括：**网关管理**：云平台作为中心控制设备，可实现远程管理和监控，支持多设备连接和管理。**数据处理**：数据处理和存储在云端进行，具有强大的计算和存储能力，支持复杂智能算法和大规模数据处理。**系统性能**：云平台具有高性能和可扩展性，能够提供快速响应和稳定的服务，支持大规模智能家居设备的管理和控制。

项目组选择李工设计思路的原因可能是基于云平台的智能家居管理系统具有更强大的数据处理和存储能力，能够支持复杂智能算法和大规模数据处理，同时具备远程管理和监控功能，提供更灵活、稳定和高效的智能家居管理解决方案，满足未来智能家居发展的需求和趋势。

答案-问题（2）

- (1) (h) 用户终端
- (2) (i) 鸿蒙
- (3) (f) 云平台
- (4) (d) 数据库
- (5) (e) 家庭网关
- (6) (c) 驱动程序

答案-问题（3）

TCP（传输控制协议）和UDP（用户数据报协议）是两种常用的传输层协议，它们在数据传输可靠性方面有所不同。TCP提供可靠的、面向连接的数据传输服务，具有数据完整性校验、流量控制和拥塞控制等机制，确保数据的可靠传输，但因为这些机制会增加通信开销，导致传输速度相对较慢。

UDP则是一种无连接的传输协议，不提供数据完整性校验、流量控制和拥塞控制，传输速度快，但数据传输可靠性较差，可能会丢失数据包或乱序传输。

在实现用户终端与服务端的双向可靠通信的系统中，应该选择TCP通信协议。由于系统需要保证数据传输的可靠性，确保数据完整性和顺序性，采用TCP协议能够提供可靠的数据传输服务，保证数据的准确传输，同时TCP的流量控制和拥塞控制机制也有助于维持通信的稳定性。尽管TCP会带来一定的通信开销，但在要求数据传输可靠性较高的场景下，选择TCP通信协议是更为合适的选择。

（2017年11月系统架构设计师真题-第3题）

阅读以下关于机器人操作系统架构的描述，回答问题1至问题3。

【说明】

随着人工智能技术的发展，工业机器人已成为当前工业界的热点研究对象。某宇航设备公司为了扩大业务范围，决策层研究决定准备开展工业机器人研制新业务。公司将论证工作交给了软件架构师王工，王工经过分析和调研，从机器人市场现状、领域需求、组成及关键技术和风险分析等方面开展了综合论证。论证报告指出：首先，为了保障本公司机器人研制的持续性，应根据领域需求选择一种适应的设计架构；其次，为了规避风险，公司的研制工作不能从零开始，应该采用国际开源社区所提供机器人操作系统（Robot Operating System, ROS）作为机器人开发的基本平台。

在讨论会上，架构师李工提出不同意见，他认为公司针对宇航领域已开发了某款嵌入式实时操作系统，且被多种宇航装备使用，可靠性较高。因此应该采用现有架构体系作为机器人的开发平台。会上王工说明了机器人操作系统与该款操作系统的差别，要沿用需要进行改造，投入较大。经过激烈讨论，公司领导同意了王工采用ROS的意见。

问题（1）

王工拟采用的ROS具有分布式进程框架，以点对点设计以及服务和节点管理器方式，使得执行程序可以各自独立地设计，松散地、实时地组合起来。这些进程可以按照功能包和功能包集的方式分组，因而可以容易地分享和发布。请用400字以内文字说明ROS与嵌入式实时操作系统的共同点，以及在实时性和任务通信方式两个方面的差异。

思路解析

共同点：

系统微型化：ROS和嵌入式实时操作系统（RTOS）都属于微型化系统，设计上要求高效利用资源。它们都用于对资源受限的设备进行控制，且通常运行在嵌入式硬件上，因此必须在有限的资源下提供稳定的性能。

系统专用性强：这两种操作系统通常面向特定的应用场景。ROS主要用于机器人系统，而嵌入式实时操作系统多用于嵌入式设备和实时控制任务。

软硬件依赖性强：无论是ROS还是RTOS，都高度依赖于硬件平台，通常需要根据特定硬件来进行定制开发。两者的操作效率和稳定性与底层硬件的兼容性和性能密切相关。

系统资源受限：由于它们主要应用于嵌入式环境，这意味着它们的计算资源（如存储、内存、处理能力）都相对有限，要求系统在资源管理上做出优化。

差异：

实时性：ROS的实时性相对较弱，它主要注重任务的灵活性和扩展性，适用于机器人系统的复杂任务调度。由于其通信机制基于消息传递，因此在处理时间敏感的任务时可能存在延迟。而嵌入式实时操作系统通常设计为确定性系统，能够确保在严格的时间约束下完成任务，具有严格的实时性能保证。它们能够满足对时间的高精度要求，确保任务在规定时间内完成。

通信方式：ROS提供了丰富的通信机制，包括发布/订阅模式、服务调用、动作等多种通信方式。这种灵活性使得ROS可以处理复杂的分布式系统和多任务协调。然而，嵌入式实时操作系统的通信机制通常较为简单，更多采用信号量、消息队列等较基础的同步/异步机制。这些机制虽然简单，但对于一些实时任务来说更为高效。

问题（2）

ROS为应用程序间通信提供了主题（Topic）、服务（Service）和动作（Action）三种消息通信方式，每种通信方式都有其特点。请将以下给出的三类通信的主要特点填入表3-1中（1）-（5）的空白处，将答案写在答题纸上。

- （a） 适合用于传输传感器信息（数据流）
- （b） 能够知道是否调用成功
- （c） 一对多模式
- （d） 有握手信号
- （e） 服务执行完会有反馈
- （f） 可以监控长时间执行的进程

- (g) 较复杂
- (h) 可能让系统过载（数据太多）
- (i) 服务执行完之前，程序会等待
- (j) 建立通信较慢
- (k) 可能丢失数据

类型	特点
主题 (Topic)	(a) 适合用于传输传感器信息（数据流）
	(1)
	(2)
	(h) 可能让系统过载（数据太多）
服务 (Service)	(b) 能够知道是否调用成功
	(3)
	(e) 服务执行完会有反馈
	(4)
动作 (Action)	(5)
	(g) 较复杂
	(d) 有握手信号

思路解析

ROS提供了三种主要的通信方式，每种方式适用于不同的应用场景：

主题 (Topic)： 主题采用发布/订阅模式，适用于数据流（如传感器信息）。它支持一对多模式，即一个发布者可以向多个订阅者发送信息。然而，由于数据量可能很大，它也可能导致系统过载。

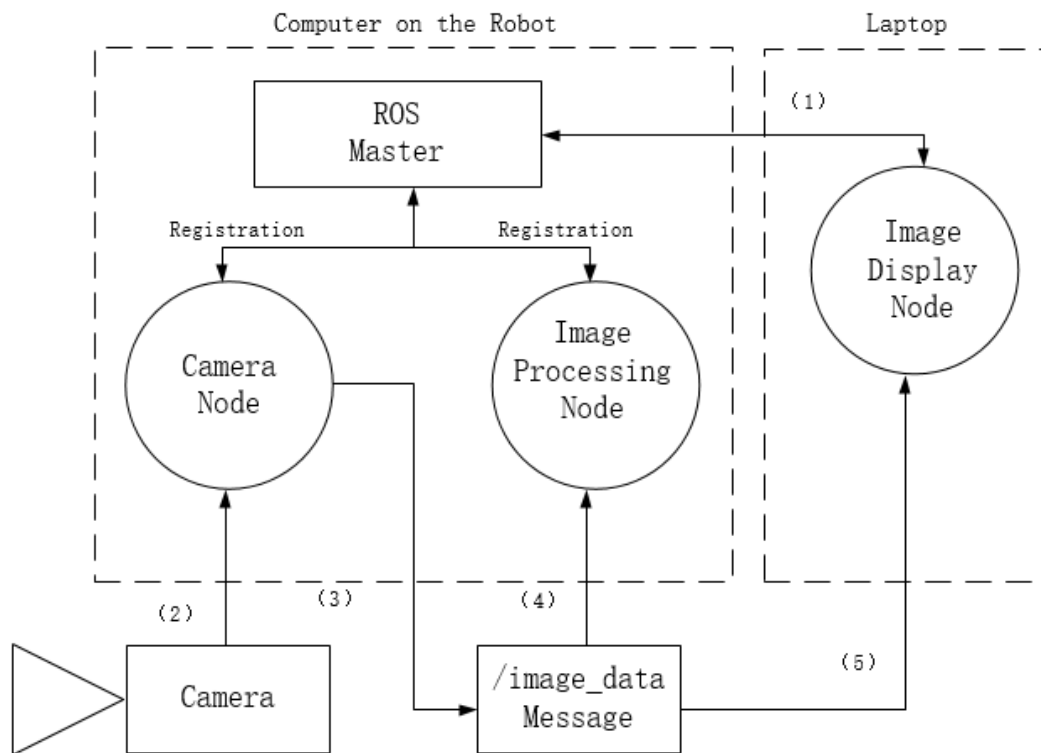
服务 (Service)： 服务采用请求/响应模式，适用于需要确认结果的任务。它能够确保服务执行完后返回反馈，且在服务执行前，客户端会等待响应。然而，建立服务的通信会比较慢。

动作 (Action)： 动作适用于监控长时间执行的任务。它提供了更多的交互和反馈功能，支持任务的实时监控和控制，且具有握手信号机制，能够在任务执行时提供状态反馈。

问题 (3)

ROS的架构定义了ROS系统由多个各自独立的节点（组件）组成，并且各个节点之间可以通过发布/订阅（Publish /Subscribe）消息模型进行通信。图3-1给出一个简单机器人结构实例，请根据以下文字描述，补充图3-1中（1）~（5）处空白，将答案写在答题纸上。

"机器人开始阶段，所有节点都要注册（Registration）到Master上，注册后，摄像头节点声明它要发布（Publish）一个叫作/image-data的消息。另外两个节点（图像处理节点和图像显示节点）声明它们需要订阅（Subscribe）这个/image-data消息。因此，一旦摄像头节点收到相机发送的数据（Data），就立即将数据/image-data直接发送到另外两个节点。"



思路解析

在ROS系统中，所有的节点（组件）在启动时都需要先注册到Master节点上，Master负责协调节点之间的通信。之后，摄像头节点发布图像数据，图像处理节点和图像显示节点订阅这个数据流并进行处理和显示。每个节点根据其功能定义其操作步骤，如发布（Publish）或订阅（Subscribe）消息。ROS的架构通过这种松散耦合的设计，促进了模块化和可扩展性，使得各个组件可以独立开发和部署。

答案-问题（1）

ROS与嵌入式实时操作系统的共同点：

- （1）系统微型化
- （2）系统专用性强
- （3）软硬件依赖性强
- （4）系统资源受限

ROS与嵌入式实时操作系统的差异：

实时性：ROS弱于嵌入式实时操作系统。

通信方式：ROS的通信方式较为丰富，嵌入式实时操作系统通信方式单一。

答案-问题（2）

- （1）（2）（c）（k）
- （3）（4）（i）（j）
- （5）（f）

答案-问题（3）

- （1）Registration （2）Data （3）Publish （4）Subscribe （5）Subscribe

（2025年5月系统架构设计师真题-第4题）

随着人工智能技术的发展，AI计算能力的部署方式呈现多样化趋势，主要包括云侧AI与端侧AI两种模式。两种部署模式各具优势，适用于不同的应用场景。

问题（1）

请简要说明这两种AI的基本定义，并结合实际应用场景，说明端侧AI相较于云端AI的三项优势。

思路解析

云端AI是指将人工智能模型部署在远程云服务器上，借助强大的算力和海量数据资源完成模型的训练和推理，适用于对计算能力要求高、需集中管理和大规模数据分析的场景，如金融风控、搜索引擎和智能客服等。而端侧AI则是将模型部署在本地终端设备（如手机、摄像头、无人机等）上运行，强调对实时响应、数据隐私和网络独立性的支持。

相较于云端AI，端侧AI在以下三个方面具有显著优势：

- 一是**低延迟，响应更快**，因为数据无需上传云端处理，模型在本地即可推理，适用于自动驾驶、智能安防等需要毫秒级响应的场景；
- 二是**数据本地处理，隐私性更强**，用户敏感信息（如语音、图像）不经云端传输，有效降低隐私泄露风险，如智能手机中的本地语音识别与人脸解锁功能；
- 三是**离线可用，网络依赖小**，即使在无网或弱网环境下也能正常运行，广泛应用于边远地区的农业无人机、巡检机器人等任务中。

问题（2）

资源池的核心架构设计需要考虑以下三个方面：（a）资源抽象与异构计算 （b）动态调度与能效优化 （c）安全机制与隔离

下表列出了一些资源池设计方法，请将相应的类别填入下标空格处。

序号	设计方法	类别
1	基于任务优先级、能耗阈值和硬件亲和性（如NPU加速）动态分配资源	(1)
2	通过虚拟化或容器技术隔离多用户任务，防止资源争抢	(2)
3	将端侧设备的CPU、GPU、NPU等异构算力统一抽象为虚拟资源池，屏蔽硬件差异，实现灵活调度	(3)
4	本地数据处理避免敏感信息上传云端，结合硬件级安全模块实现端到端加密	(4)
5	根据负载自动调整算力分配	(5)
6	近存计算优化	(6)
7	通过模型蒸馏、量化和稀疏编码压缩模型体积，适配端侧资源限制	(7)

思路解析

- （1）“**基于任务优先级、能耗阈值和硬件亲和性（如NPU加速）动态分配资源**” 此设计方法强调任务调度、硬件亲和性与能耗门限，属于典型的**动态调度与能效优化（b）**。
- （2）“**通过虚拟化或容器技术隔离多用户任务，防止资源争抢**” 该方法体现了提供隔离能力，典型属于**安全机制与隔离（c）**的范畴。
- （3）“**将端侧设备的CPU、GPU、NPU等异构算力统一抽象为虚拟资源池，屏蔽硬件差异，实现灵活调度**” 明确提到对异构算力的统一抽象，正是**资源抽象与异构计算（a）**的核心思想。
- （4）“**本地数据处理避免敏感信息上传云端，结合硬件级安全模块实现端到端加密**” 这是为了实现数据安全和隐私保护，典型属于**安全机制与隔离（c）**的范畴。
- （5）“**根据负载自动调整算力分配**” 动态调整资源以匹配当前计算需求，属于**动态调度与能效优化（b）**。
- （6）“**近存计算优化**” 优化计算路径、降低能耗、提升效率，也归类为**动态调度与能效优化（b）**。
- （7）“**通过模型蒸馏、量化和稀疏编码压缩模型体积，适配端侧资源限制**” 本质上是为了让模型适应不同硬件能力，也是一种**资源抽象与异构计算（a）**的技术手段。

问题（3）

请根据下列指标对比三种资源池架构，前3空填写“高”“中”或“低”，后3空填写相应资源池的

典型适用场景。

对比维度	集中式资源池	分布式资源池	混合式资源池
扩容能力	(1)	(2)	中
管理复杂性	(3)	高	(4)
资源调度规模	(5)	低	(6)
适用场景	(7)	(8)	(9)

李工在当前项目中基于系统资源实时调度需求高，且系统规模相对固定，决定采用集中式资源池架构。但王工指出该方案存在以下三个潜在缺陷。请结合集中式资源池架构特点，指出这三项可能的缺陷。

思路解析

在资源池架构设计中，三种主流模式——集中式资源池、分布式资源池、混合式资源池，适用于不同业务场景，其性能特点、部署策略和适用范围存在明显差异。

集中式资源池适合系统规模中等、部署区域集中的场景，特别是在对资源调度的实时性、管理统一性和数据安全性有较高要求时表现优越。其采用单一控制面统一调度所有计算和存储资源，可实现快速响应与调度优化，便于实施安全策略与统一运维。因此，常见于金融、电信、数据中心等需要高吞吐、集中管理的行业系统。

分布式资源池适用于业务广泛分布、系统跨地域部署或边缘计算需求较强的环境。它通过多个节点组成资源网络，各自独立、协同运行，具备良好的扩展性与故障隔离能力，即使部分节点宕机也不会影响整个系统。其典型应用包括IoT平台、车联网、CDN、跨省政务服务等场景，能满足低延迟、地域就近计算等需求。

混合式资源池结合集中式与分布式的优势，既能通过中心节点统一管控，又能利用边缘节点进行分布式扩展和优化处理，适合业务复杂、层级多样、需要灵活调度与快速扩展的企业架构。例如在大型互联网企业或政企单位中，云端处理核心任务，边缘负责本地计算，是数字化转型背景下常见选择。

答案-问题（1）

云端AI是指将AI模型部署在云服务器上进行训练与推理，适合处理高复杂度、需要大数据支撑的任务；而端侧AI则是在本地设备上运行模型，更注重实时性、隐私性和对网络的独立性。相比之下，端侧AI具有三大优势：一是响应速度快，适用于自动驾驶等需毫秒级决策的场景；二是数据无需上传，能保护用户隐私，如本地人脸识别；三是在离线或弱网环境下仍能正常运行，适合智能农机等应用。

答案-问题（2）

(1) b (2) c (3) a (4) c (5) b (6) b (7) a

答案-问题（3）

(1) 低 (2) 高 (3) 低 (4) 中 (5) 高 (6) 中
(7) 集中式资源池适用于系统规模中等、部署集中、资源调度实时性要求高、运维集中化、安全性要求高的场景
(8) 分布式资源池适用于多业务、多地域部署或边缘计算等资源分散场景
(9) 混合式资源池则适用于需同时支持云端与本地、具备弹性扩展需求的复杂业务系统。
集中式资源池的缺陷主要包括：扩展能力弱，难以灵活应对业务增长；存在单点故障风险，影响系统稳定性；不适用于用户分布广泛的场景，难以满足边缘计算对低延迟的需求。

某科技公司希望在其嵌入式产品线上全面提升智能化能力，计划研发一套“嵌入式智能融合操作系统”。项目目标强调AI 与操作系统深度融合，以实现统一的智能调度、更高效的数据流动、边云协同以及更智能的用户交互体验。

但在项目需求讨论阶段，老王与老李对 “AI for OS (AI 支撑操作系统)” 与 “OS for AI (操作系统支撑 AI)” 的关系理解不清；同时对智能化操作系统的内涵、智能功能、轻量化内核等关键概念也较为模糊。为推进项目立项，公司要求项目组完成以下问题的分析。

问题（1）

给出“智能化操作系统”的定义，并列举 5 个以上典型的智能化功能。

思路解析

智能化操作系统本质上是传统操作系统与人工智能技术的深度融合，它不再只是“资源管理与任务调度”的基础系统，而是通过 **系统级 AI 模型与智能策略引擎** 让 OS 具有“感知—分析—决策—执行”的能力链条。其内部通常包含智能调度器、智能策略中心、AI 推理模块、智能感知组件等。与普通 OS 最大的差异是：传统 OS 以静态规则为主，而智能化 OS 以**动态学习、自适应优化**为核心，从而在复杂不确定环境中保持高性能与高稳定性。

同时，智能化 OS 需要在系统级支持多模态感知、智能安全、预测性运维、数据流动优化等，这些都依赖系统本身具备 AI 模型的部署能力与调度能力。随着边缘计算、IoT、智能机器人等场景兴起，智能化 OS 成为嵌入式系统发展的关键趋势。

问题（2）

根据公司“嵌入式智能融合 OS”建设目标，老王老李需要判断系统能力属于：AI for OS 还是 OS for AI，现给出 11 项系统能力，请分类并说明理由：

a、智能化调度、b、系统动态调优、c、获得无限算力（扩容）、d、获得推理能力、e、更好的可靠性、f、更好的安全性、g、运维简单化、h、云端边端协同、i、更好的虚拟隔离环境、j、人机交互 AI 化、k、智能化数据流动

编号	项目	判断
a	智能化调度	
b	系统动态调优	
c	获得无限算力（扩容）	
d	获得推理能力（AI 加速框架）	
e	更好的可靠性	
f	更好的安全性	
g	运维简单化（智能化运维）	
h	云端 - 边端协同	
i	虚拟隔离环境	
j	人机交互 AI 化	
k	智能化数据流动	

思路解析

“AI for OS”与“OS for AI”是智能化操作系统中的两个核心概念，它们的定位不同：前者强调“AI 让操作系统变得更聪明”，后者强调“操作系统让 AI 能够运行得更好”。判断时关键看 **作用方向**：是 AI 主动改变 OS 工作方式，还是 OS 主动为 AI 提供基础设施。

从 AI for OS 角度看：智能化调度（a）与系统动态调优（b）本质上是 AI 在分析系统运行状态后，对调度器、资源管理策略进行自适应调整；智能运维（g）依赖机器学习完成故障预测、配置自愈，因此属于 AI 驱动型能力；人机交互 AI 化（j）明显依赖语音、多模态、NLP，因此是 AI 赋予 OS 新能力；智能数据流动（k）则由 AI 根据流量模式、任务类型优化数据路径，所以属于 AI 驱动优化。此类能力共同特点是：**AI 作为智能大脑，改善 OS 的决策逻辑，使 OS 行为从“静态规则”变为“智能自治”**。

而 OS for AI 则更多强调基础支撑，如无限算力（c）依靠虚拟化、分布式资源池、弹性伸缩等技术，是 OS 为 AI 提供资源；推理能力（d）需要 OS 提供 AI 加速器驱动、推理框架调度等；可靠性（e）和安全性（f）是 OS 的基础属性，直接决定 AI 运行的环境稳定性；虚拟隔离（i）也属于 OS 的资源抽象能力，为 AI 服务隔离安全；云边协同（h）则是 OS 通过分布式管理框架承载 AI 模型下沉与任务分发，因此属于 OS 为 AI 建设运行平台。总体来说，OS for AI 特点是：**操作系统提供算力、环境、调度、隔离等基础设施，使 AI 模型能稳定、安全、高效地运行**。

问题（3）

说明“轻量化内核”的定义与主要特点。

思路解析

轻量化内核是面向嵌入式系统的重要内核形态，其核心理念是“只做最必要的事”。由于嵌入式设备常常受限于 CPU 性能、存储空间、功耗预算，因此不能使用大型通用操作系统的完整内核，而需要将内核裁剪为最小可用集。其典型技术包括：微内核化、模块化驱动架构、按需加载、中断与任务调度精简化、同步原语减少等。此外，随着 AI 模型下沉到边缘设备，轻量化内核开始具备对 AI 推理任务的专门优化，例如轻量级算子管理、异构加速器调度接口、DMA 数据通道优化等，使资源有限的设备也能进行小型推理任务。

与传统 OS 内核相比，轻量化内核强调高可控性、可验证性、低延迟以及对关键任务的实时保障。一些 RTOS、IoT OS（如 LiteOS、Zephyr、FreeRTOS）都是典型代表。由于裁剪充分，轻量化内核往往可以以 KB 级体积在 MCU 上运行，同时保持 determinism（确定性）行为，这是嵌入式 AI 场景的重要基础。

答案-问题（1）

智能化操作系统是指在传统操作系统的基础上，深度集成人工智能（AI）能力，使系统能够具备自感知、自决策、自优化的能力，并能够根据任务场景变化自动调节资源、实现高效推理与更加自然的人机交互。

典型智能化功能包括：1. 智能调度（AI 驱动的任务/资源调度）2. 智能化 UI 交互（自然语言、人脸、手势等多模态）3. 智能运维（自动检测、预测、自治性优化）4. 智能安全防护（行为识别、智能风险分析）5. 数据流动优化（数据路径自适应与强化学习优化）6. 云边协同智能决策（任务下沉、动态分割）7. 智能功耗管理（AI 预测式能耗控制）

答案-问题（2）

编号	项目	判断
a	智能化调度	AI for OS
b	系统动态调优	AI for OS
c	获得无限算力（扩容）	OS for AI
d	获得推理能力（AI 加速框架）	OS for AI
e	更好的可靠性	OS for AI
f	更好的安全性	OS for AI
g	运维简单化（智能化运维）	AI for OS
h	云端 - 边端协同	OS for AI
i	虚拟隔离环境	OS for AI
j	人机交互 AI 化	AI for OS
k	智能化数据流动	AI for OS

答案-问题（3）

轻量化内核是指在保证关键实时性、安全性与基础资源管理能力的前提下，对传统内核进行裁剪、模块化设计，以降低资源占用（CPU、内存、功耗），适用于嵌入式设备、IoT 芯片、边缘节点等资源受限场景的操作系统内核。

主要特点：1. 核心功能高度裁剪（仅保留必需机制）2. 模块化、组件化，可按需加载 3. 高实时性与低延迟 4. 占用资源少（KB 级或低 MB 级）5. 低功耗运行 6. 对 AI 推理可进行算子微内核化或调度优化 7. 更适合边缘设备、传感器节点等场景

（2026年5月系统架构设计师真题-第2题）

阅读以下关于居家养老智能系统架构设计的叙述，在答题纸上回答问题1-4。

【说明】

某养老服务机构计划建设一套面向独居老人的居家养老智能系统。系统需要在老人住所内部署温湿度传感器、光照传感器、水电表、烟雾报警器、跌倒检测设备、浴室滞留检测设备、一键呼叫按钮、智能药盒和陪护机器人等终端设备，实时采集环境、健康、用电用水和紧急求助信息。系统还需要支持机器人送药、送餐提醒和紧急救援联动，在老人发生跌倒、长时间未活动或燃气异常时及时通知家属、社区医生和养老服务中心。

项目组拟采用云边端协同架构，将系统划分为感知层、边缘层、平台层和应用层。感知层负责终端设备的数据采集和执行控制；边缘层部署在家庭网关或社区网关中，负责协议转换、本地缓存、离线告警和低时延联动；平台层负责设备管理、消息接入、流式处理、业务数据存储和统一调度；应用层为养老机构、家属和医生提供 Web 看板、移动 App、告警处置和服务派单功能。

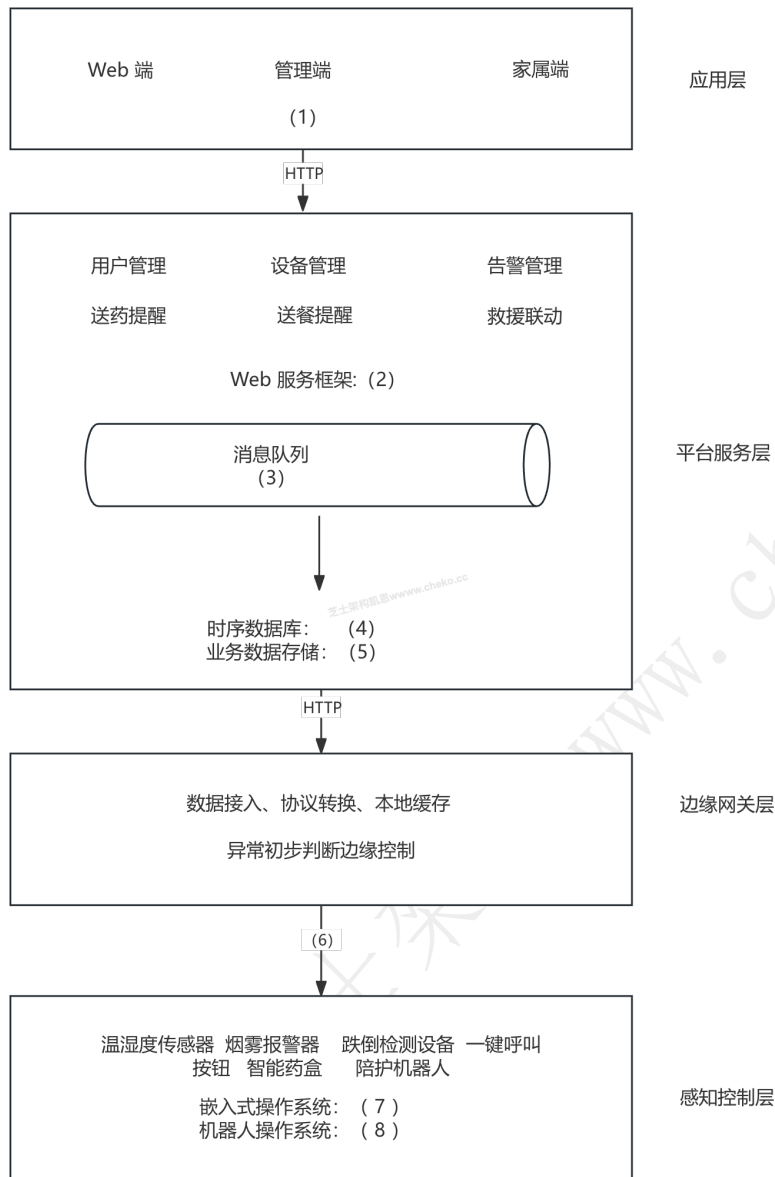
由于老人住所网络可能出现弱网或短时断网，边缘网关还需要保存连续采样数据，并根据消息重要程度选择不同的 MQTT QoS 等级。

问题（1）

结合备选技术，补充系统分层中的关键技术。

A. Vue.js

- B. Flask
- C. TDengine
- D. MySQL
- E. MQTT
- F. Kafka
- G. FreeRTOS
- H. ROS



思路解析

最上层是应用层，Web 端通常使用 Vue.js 这类前端框架实现；平台服务层负责业务接口、设备管理、告警管理等，可使用 Flask 提供 Web 服务；传感器数据具有明显的时间序列特征，适合使用 TDengine 存储；用户、设备、告警规则等结构化业务数据适合使用 MySQL；边缘网关与设备之间常用 MQTT 进行轻量级物联网通信；平台内部异步消息传递可使用 Kafka；普通嵌入式设备适合运行 FreeRTOS；陪护机器人涉及感知、运动控制和任务调度，适合使用 ROS。

问题 (2)

说明时序数据库的含义，以及边缘网关为什么适合使用时序数据库。

思路解析

本问考查时序数据库的适用场景。时序数据库是面向时间序列数据优化的数据库，数据通常由时间戳、指标值和标签组成，例如设备编号、房间编号、老人编号、采集类型、采集时间和采样值。它不是普通关系表里多一个时间字段，而是在高频写入、按时间范围查询、窗口聚合、降采样、压缩存储、数据保留策略和趋势分析方面做了专门优化。典型场景包括物联网传感器、设备监控、能源表计、车联网轨迹和系统性能指标。

居家养老系统非常适合使用时序数据库。温湿度、光照、水电表、药盒状态、设备心跳和行为检测结果都具有连续采样特征，数据量大且天然按时间查询。边缘网关使用时序数据库有几个实际价值：第一，弱网或短时断网时可以本地保存连续采样数据，网络恢复后再同步到云端，避免数据缺口；第二，边缘规则可以直接查询最近5分钟、30分钟或24小时的变化趋势，例如判断长时间未活动、夜间用水异常、浴室滞留或室内温度异常；第三，本地聚合和降采样可以减少上云流量，只上传摘要、异常片段或必要明细；第四，连续历史数据能帮助护理人员观察老人状态变化，而不是只看到单点告警。

因此，本题不能只答“用来存传感器数据”，应进一步写出高频写入、时间窗口查询、断网缓存、本地告警和云端减压这些关键词。

从工程实现看，边缘侧使用时序数据库还要考虑容量和同步策略。边缘网关不可能长期保存全部历史数据，因此可设置保留周期、压缩策略和降采样策略，例如保留最近7天明细、最近30天分钟级聚合，并在网络恢复后按时间戳和设备编号增量同步到云端。对告警判断而言，时序数据库能支持“连续多次异常”“一段时间内没有活动”“夜间用水突增”等窗口规则，这比只看某一次上报更可靠。答题时如果能写出“时间戳、标签、窗口聚合、断网缓存、恢复同步”，就能明显区别于普通数据库概念题。

问题（3）

先把QoS 0、QoS 1、QoS 2填入下方表格合适的位置，然后解释 MQTT QoS 0、QoS 1、QoS 2 的含义，并说明关键告警为什么可使用 QoS 2。

业务场景	描述	建议 QoS 级别
天气、温度、光线监控	周期性采集环境数据，用于居家环境展示和趋势分析，数据上报频率较高，偶尔丢失一两条影响不大	(1)
日常用电、用水监控	周期性采集水电表数据，用于用量统计、异常用水用电趋势分析，要求数据尽量可靠，但允许后续补传或校正	(2)
老人跌倒告警	检测到老人跌倒后，需要立即通知养老服务中心、家属和社区医生，属于救援类关键告警，不能丢失，也不应重复触发救援流程	(3)
一键求助	老人在房间内主动按下一键求助按钮，表示身体不适或遇到紧急情况，需要可靠送达并触发救援处置	(4)
机器人默认正常状态	机器人定期上报位置、在线状态、电量、充电情况等运行状态，用于看板展示和日常运维，数据频繁且允许短暂丢失	(5)
机器人紧急状态	机器人发现异常情况，或进入紧急救援、异常停机、无法返回、老人呼救联动等状态，需要可靠通知家属和医护人员	(6)

思路解析

本问考查 MQTT QoS 机制和业务分级。QoS 0 表示最多一次，发布端只发送一次，不等待确认，消息可能丢失，但交互最少、开销最低，适合温湿度、光照、普通心跳等高频且可容忍少量丢失的数

据。QoS 1 表示至少一次，发送方通过确认机制保证消息至少到达一次，但在网络抖动或重传时可能重复，适合设备状态、一般告警、药盒开关状态等需要到达但可通过消息 ID 去重的数据。QoS 2 表示恰好一次，通过更完整的握手流程保证消息既不丢失也不重复，可靠性最高，但通信轮次和状态维护成本也最高。

养老场景不能简单把所有消息都设为 QoS 2。普通环境数据几秒钟就会上报一次，丢一条通常不会影响判断；若全部使用 QoS 2，会增加网关和平台压力，反而影响关键告警。但跌倒、一键呼叫、燃气异常、浴室长时间滞留这类事件具有强时效和强业务后果，一旦丢失可能延误救援，重复触发又可能造成重复派单、家属反复通知或应急资源浪费，因此可以使用 QoS 2，并配合告警编号、幂等处理和处置状态机。答题时要体现“按消息重要性选择 QoS”的思想，而不是把 QoS 2 描述成通用最优方案。

还要注意 QoS 与业务幂等并不矛盾。即使使用 QoS 2 保证 MQTT 层恰好一次，也不能完全替代应用层幂等，因为业务系统还可能因重试、网关补发、消费者失败或人工补录产生重复事件。所以关键告警最好同时携带告警 ID、设备 ID、时间戳和事件类型，在平台侧建立告警状态机，避免同一次跌倒事件重复生成多个处置工单。如果题目问“为什么关键告警使用 QoS 2”，不能只写“更可靠”，应补充关键告警的业务后果：丢失会延误救援，重复会浪费应急资源并干扰家属和医生判断。

答案-问题（1）

- (1) Vue.js
- (2) Flask
- (3) TDengine
- (4) MySQL
- (5) MQTT
- (6) Kafka
- (7) FreeRTOS
- (8) ROS

答案-问题（2）

时序数据库是面向时间序列数据设计的数据库，数据通常由时间戳、指标值和标签组成，适合保存传感器、设备状态、能耗、监控指标等连续采样数据。它通常支持高频写入、按时间范围查询、滑动窗口聚合、降采样、压缩存储、数据保留策略和快速趋势分析。

边缘网关适合使用时序数据库的原因包括：第一，居家养老设备会持续产生温湿度、电表、设备心跳、跌倒检测等时间序列数据；第二，边缘侧就近存储可减少上传云端的压力；第三，弱网或断网时可先本地缓存，网络恢复后再同步；第四，边缘规则可直接基于最近一段时间的数据做本地告警，例如长时间未活动、水电异常或浴室滞留；第五，时间窗口查询和趋势分析有利于护理人员判断老人状态变化。

答案-问题（3）

业务场景	描述	建议 QoS 级别
天气、温度、光线监控	周期性采集环境数据，用于居家环境展示和趋势分析，数据上报频率较高，偶尔丢失一两条影响不大	(1) QoS0
日常用电、用水监控	周期性采集水电表数据，用于用量统计、异常用水用电趋势分析，要求数据尽量可靠，但允许后续补传或校正	(2) QoS1
老人跌倒告警	检测到老人跌倒后，需要立即通知养老服务中心、家属和社区医生，属于救援类关键告警，不能丢失，也不应重复触发救援流程	(3) QoS2
一键求助	老人在房间内主动按下一键求助按钮，表示身体不适或遇到紧急情况，需要可靠送达并触发救援处置	(4) QoS2
机器人默认正常状态	机器人定期上报位置、在线状态、电量、充电情况等运行状态，用于看板展示和日常运维，数据频繁且允许短暂丢失	(5) QoS0
机器人紧急状态	机器人发现异常情况，或进入紧急救援、异常停机、无法返回、老人呼救联动等状态，需要可靠通知家属和医护人员	(6) QoS2

MQTT QoS 0 表示最多一次发送，发送方只发一次，不确认送达，消息可能丢失，但开销最低，适合温湿度等普通高频数据。

QoS 1 表示至少一次送达，通过确认机制保证消息到达，但可能重复，适合一般状态上报、设备心跳或普通告警。

QoS 2 表示恰好一次送达，通过更完整的握手机制避免丢失和重复，可靠性最高，但通信开销也最大。

跌倒、一键呼叫、燃气异常、浴室长时间滞留等关键告警可使用 QoS 2，因为这类消息一旦丢失会影响救援时效，重复触发又可能造成误派单或重复处置。

(2026年5月系统架构设计师真题-第3题)

阅读以下关于 AI 决策的智能安防管理架构的叙述，在答题纸上回答问题1-3。

【说明】

某产业园区计划建设一套智能安防管理系统，对人员通行、车辆出入、重点区域周界、访客身份和异常事件进行统一管理。园区内已经部署摄像头、智能门禁、闸机、生物识别终端、红外周界探测器、巡检机器人和边缘网关等设备。系统需要支持人脸识别、身份核验、异常闯入识别、黑名单告警、门禁联动、事件派单、大屏展示以及与物业、公安、消防等第三方系统集成。

架构师提出采用 AIoT 分层架构，将系统划分为感知层、边缘层、AI 决策层和应用层。感知层负责采集视频、门禁、身份和周界等原始信息；边缘层负责设备接入、协议适配、本地缓存、边缘推理和低时延联动；AI 决策层负责模型推理、视觉分析、规则分析、事件关联和风险判断；应用层负责事件中心、处置流程、可视化展示、移动端通知和第三方系统集成。

系统设计图中需要放置的组件包括：智能门禁、生物识别设备、身份核验服务、周界防护设备、边缘网关、边缘控制器、AI 模型、视觉分析引擎、规则/认知引擎、事件中心、APP、大屏展示和第三方系统集成模块。项目组还需要分析该四层架构在设备异构、边缘资源、模型准确性、数据安全和运维管理方面可能存在的不足。

问题（1）

说明感知层、边缘层、AI 决策层和应用层的主要职责。

思路解析

本问考查 AIoT 分层架构的职责边界。感知层的关键词是“采集和执行”，它负责把园区物理世界

中的视频、门禁刷卡、人脸或指纹、车辆出入、周界报警和巡检状态转换为系统可处理的数据，同时也可能执行开门、关闸、声光报警等基础动作。边缘层的关键词是“就近接入和低时延处理”，它负责把摄像头、门禁、闸机和传感器接入系统，完成协议适配、数据预处理、本地缓存、轻量推理和本地联动。例如网络断开时，边缘控制器仍应能根据本地白名单完成门禁控制，或在周界异常时触发现场声光告警。

AI 决策层的关键词是“识别、关联和判断”。它不只是运行一个 AI 模型，而是综合视觉分析、身份核验、规则引擎、事件关联和风险评分，把分散的摄像头画面、门禁记录和周界事件组合成可解释的安全判断。应用层的关键词是“展示、流转和处置”，包括事件中心、告警确认、工单派发、移动端通知、大屏展示、统计报表以及物业、公安、消防等第三方系统集成。

容易混淆的是身份核验和门禁联动。身份采集设备属于感知层，身份比对算法或核验服务属于 AI 决策层，低时延开门控制可能落在边缘层，最终处置流程和展示则在应用层。能把一个跨层功能拆成多个职责，是架构题得分的关键。

答题时可以用“数据从哪里来、在哪里快速处理、在哪里做智能判断、最后由谁处置”来组织。摄像头画面和门禁刷卡记录来自感知层；边缘层负责把多设备数据整理成统一事件，并在低时延场景下先做本地联动；AI 决策层把单点数据变成判断结果，例如黑名单人员进入、异常闯入或尾随通行；应用层把判断结果转为可执行流程，例如通知保安、生成工单、联动公安或消防。如果把 AI 决策层和应用层混在一起，答案会显得像功能清单；如果把边缘层只写成“传输数据”，又会漏掉边缘推理和本地联动。

问题（2）

将智能门禁、生物识别、身份核验、周界防护、边缘网关、AI 模型、视觉分析引擎、规则/认知引擎、事件中心、第三方系统集成、APP 或大屏等组件放入合适层次。

思路解析

本问考查组件归层。归层时不要只看组件名称，要看它在系统中承担的主要职责。智能门禁、闸机、摄像头、生物识别采集设备、周界防护设备、红外探测器和巡检机器人采集端直接面对现场，负责采集原始信息或执行现场动作，应放在感知层。边缘网关、边缘控制器、本地缓存、本地联动模块和边缘推理节点承担设备接入、协议转换、预处理、断网缓存和低时延控制，应放在边缘层。

AI 模型、视觉分析引擎、规则/认知引擎、身份核验服务、事件关联分析和风险评分模块的职责是识别、推理、关联和决策，应放在 AI 决策层。其中视觉分析引擎可以做人脸识别、车辆识别、入侵检测、轨迹分析；规则/认知引擎可以把“非工作时间进入重点区域”“黑名单人员靠近门禁”“周界报警同时摄像头出现人员”这类条件组合成风险事件。事件中心、APP、大屏展示、告警处置、工单流转、报表统计和第三方系统集成模块面向业务用户和外部系统，应放在应用层。

如果题目给出身份核验这样的跨层组件，可以拆开回答：生物特征采集终端在感知层，身份核验算法或服务在 AI 决策层，核验结果触发的开门联动可能在边缘层，核验记录展示和审计在应用层。这种拆分比机械地把组件放到单一层更符合实际架构。

如果题目是图中填空，可以优先根据箭头和数据流判断。离设备最近、输入原始视频/刷卡/探测信号的空一般填感知组件；连接多种设备并向上转发的一般填边缘网关；接收预处理数据并输出识别结果或风险判断的一般填 AI 模型、视觉分析引擎或规则引擎；面向用户展示或外部联动的一般填事件中心、APP、大屏和第三方集成。同时要注意“第三方系统集成”不是感知层设备，它通常属于应用层的对外接口或集成适配模块，用来把处置结果同步给物业、公安、消防等外部系统。

问题（3）

列举该四层 AIoT 架构可能存在的五类不足或风险。

思路解析

AIoT 智能安防的风险要从 IoT、AI 和安防业务三条线同时分析。IoT 侧首先是设备异构和协议适

配风险，不同厂家摄像头、门禁、闸机、生物识别终端和周界设备接口不统一，如果缺少统一设备模型和适配层，后续联动、监控和运维会非常困难。其次是边缘资源受限，边缘节点算力、存储和网络条件有限，高清视频长期保存、大模型推理和多路并发分析都可能超出能力边界，需要做本地预处理、轻量模型、事件片段上传和云边协同。

AI 侧风险主要是模型准确性和可运营性。光照变化、遮挡、逆光、雨雪、口罩、摄像头角度、人员拥挤都会影响人脸识别和入侵检测，带来误报和漏报。误报会增加保安派单和运营成本，漏报则可能导致真实安全事件未处置。模型还涉及版本管理、灰度发布、效果评估和回滚机制，如果没有统一管理，园区不同区域可能运行不同模型版本，难以解释结果差异。

安防业务侧还要关注隐私安全、合规和处置闭环。人脸、轨迹、门禁记录、访客身份都属于敏感数据，需要加密、脱敏、权限控制和审计；事件中心如果没有状态机、责任人、超时升级和闭环反馈，即使识别到了异常，也可能停留在告警列表里无人处理。其他可写风险包括网络故障影响联动、第三方系统集成成本高、设备远程升级失败、规则配置不一致等。答题时每类风险最好带一个业务后果。

展开风险时，可以补充治理措施来增强答案深度。设备异构可通过统一设备接入规范、适配器和设备模型解决；边缘资源不足可通过边缘预处理、轻量模型和云端复核缓解；模型误报漏报可通过样本持续标注、阈值调优、人工复核和灰度发布降低；隐私风险可通过最小权限、加密存储、脱敏展示和审计日志控制；云边协同可通过配置版本号、模型版本号、心跳、状态同步和回滚机制保障。考试问“问题”时主要列问题即可，但附带一两句解决方向，可以让解析更像完整架构分析。

答案-问题（1）

感知层负责采集园区物理世界的原始信息，包括视频图像、门禁刷卡记录、人脸或指纹等生物特征、周界报警、车辆出入和环境传感数据。

边缘层负责设备接入、协议适配、视频或图片预处理、本地缓存、边缘推理和低时延联动，例如在网络不稳定时仍能完成本地开门、拦截或声光告警。

AI 决策层负责模型推理、视觉分析、身份核验、规则分析、事件关联、风险评分和策略生成，把单点告警转化为可处置的安全事件。

应用层负责事件中心、处置流程、工单派发、可视化大屏、移动端通知、报表统计以及与物业、公安、消防等第三方系统集成。

答案-问题（2）

感知层：智能门禁、闸机、生物识别采集设备、摄像头、周界防护设备、红外探测器、巡检机器人采集端等，主要负责采集和执行。

边缘层：边缘网关、边缘控制器、本地缓存、本地联动模块、边缘推理节点等，主要负责接入、协议转换和低时延控制。

AI 决策层：AI 模型、视觉分析引擎、规则/认知引擎、身份核验服务、事件关联分析、风险评分模块等，主要负责识别、推理和决策。

应用层：事件中心、APP、大屏展示、告警处置、工单流转、报表统计和第三方系统集成模块等，主要面向用户和业务流程。

若图中要求放置“身份核验”，可将采集设备放在感知层，将身份核验算法或服务放在 AI 决策层；第三方系统集成一般放在应用层或集成适配层。

答案-问题（3）

可能存在的不足或风险包括：

设备兼容和协议适配复杂，不同厂商门禁、摄像头和传感器接口不统一。

边缘算力和存储有限，难以长期保存高清视频或运行大模型。

AI 模型存在误报、漏报和场景泛化不足问题，例如光照变化、遮挡、口罩、拥挤场景都会影响识别准确率。

隐私与数据安全风险高，人脸、轨迹和门禁记录需要加密、脱敏、访问控制和审计。

云边协同复杂，模型版本、规则配置、告警状态和处置结果需要保持一致。

还可以补充：网络故障影响联动、模型更新灰度困难、第三方系统集成成本高、误告警带来运营压力、设备远程运维复杂等。

第14章——可靠性与容错架构 (2015年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式系统可靠性设计方面的描述，回答问题1至问题3。

【说明】

某宇航公司长期从事宇航装备的研制工作，嵌入式系统的可靠性分析与设计已成为该公司产品研制中的核心工作，随着宇航装备的综合化技术发展，嵌入式软件规模发生了巨大变化，代码规模已从原来的几十万扩展到上百万，从而带来了由于软件失效而引起系统可靠性降低的隐患。公司领导非常重视软件可靠性工作，决定抽调王工程师等5人组建可靠性研究团队，专门研究提高本公司宇航装备的系统可靠性和软件可靠性问题，并要求在三个月内，给出本公司在系统和软件设计方面如何考虑可靠性设计的方法和规范。可靠性研究团队很快拿出了系统及硬件的可靠性提高方案，但对于软件可靠性问题始终没有研究出一种普遍认同的方法。

问题（1）

请用200字以内文字说明系统可靠性的定义及包含的4个子特性，并简要指出提高系统可靠性一般采用哪些技术？

思路解析

系统可靠性定义：系统可靠性强调的是 **在规定环境、规定时限内执行预期功能的能力**。它不仅仅是“不会出错”，更重要的是即便出现异常，也要能够恢复并维持可接受的运行状态。**可靠性子特性**如下：

成熟性：系统具有稳定运行的能力，错误发生概率低。

容错性：在部分组件失效或运行错误时，系统能通过冗余、回滚或其他机制继续运行。

易恢复性：一旦系统出现故障，能够快速恢复正常功能，减少停机时间。

可靠性依从性：系统符合相关标准和规范，可靠性可验证和可度量。

提高系统可靠性的常用技术：

N 版本程序设计：通过多个功能等价但独立开发的程序版本，运行时采用表决机制选择正确结果，防止单点错误导致系统崩溃。

恢复块方法：在程序运行时设置“主块”和“备用块”，当主块结果经验证失败时，回退到备用块执行，确保功能实现。

防卫式编程：在代码设计时预先考虑潜在错误输入和运行环境异常，进行边界检查和异常处理。

双机热备/集群：通过多台设备冗余备份，提高系统可用性和可靠性。

冗余设计：包括硬件冗余、信息冗余、时间冗余，确保部分失效时系统仍可运行。

问题（2）

王工带领的可靠性研究团队之所以没能快速取得软件可靠性问题的技术突破，其核心原因是他们没有搞懂高可靠性软件应具备的特点。软件可靠性一般致力于系统性地减少和消除对软件程序性能有不利影响的系统故障。除非被修改，否则软件系统不会随着时间的推移而发生退化。请根据你对软件可靠性的理解，给出表3-1所列出的硬件可靠性特征对应的软件可靠性特征之间的差异或相似之处，将答案写在答题纸上。

表3-1 硬件和软件可靠性对比

序号	硬件可靠性	软件可靠性
1	失效率服从浴缸曲线。老化状态类似于软件调试状态	(1)
2	即使不适用，材料劣化也会导致失效	(2)
3	硬件维修会恢复原始状态	(3)
4	硬件失效之前会有报警	(4)

思路解析

硬件和软件在可靠性特征上有显著差异：

失效率随时间变化：：硬件的失效率通常呈“浴盆曲线”：初期（磨合期）高、稳定期低、老化期高。但软件不发生物理老化，在 **不考虑演化时**，其失效率统计上是非增的，只要代码不修改，错误模式不会随时间增加。

使用与失效关系：硬件即便闲置也会因老化（如元件氧化）导致失效；而 **软件如果不被运行**，则永远不会发生失效，因为其错误是逻辑性的，而非物理性的。

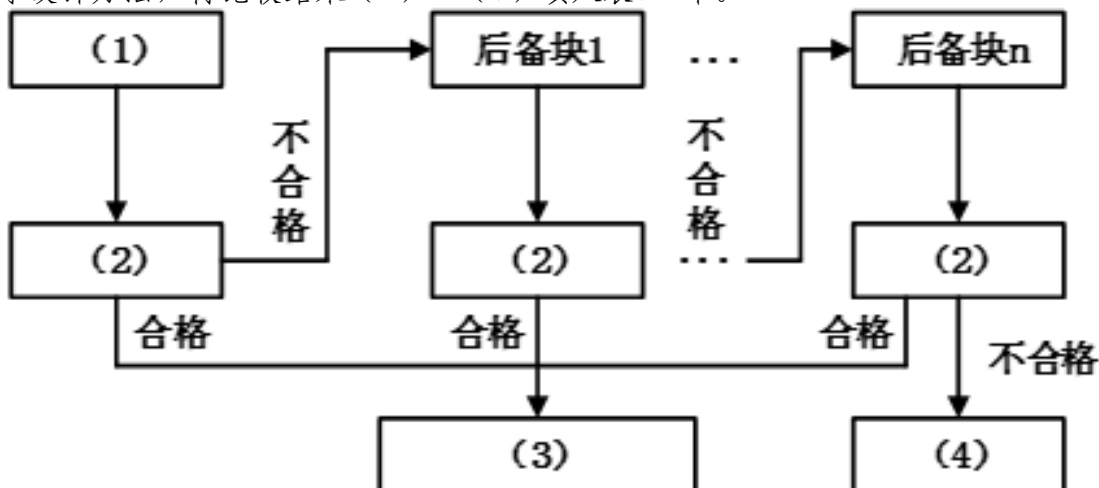
维护影响：硬件维护通常是替换损坏部件，不会引入新故障；但 **软件维护需要修改或增加代码**，这既可能修复缺陷，也可能引入新的错误。

故障报警特征：硬件往往在失效前会出现物理征兆（如温度升高、性能下降），而 **软件失效之前通常没有明显报警**，错误多在运行中直接暴露（如异常终止）。

因此，软件可靠性工作应从 **开发过程管理、预防性设计和维护控制** 出发，而非照搬硬件的可靠性分析模式。

问题（3）

王工带领的可靠性研究团队在分析了大量相关资料基础上，提出软件的质量和可靠性必须在开发过程构建到软件中，也就是说，为了提高软件的可靠性，必须在需求分析、设计阶段开展软件可靠性筹划和设计。研究团队针对本公司承担的飞行控制系统制定出了一套飞控软件的可靠性设计要求。飞行控制系统是一种双余度同构型系统，输入采用了独立的两路数据通道，在系统内完成输入数据的交叉对比、表决制导率计算，输出数据的交叉对比、表决、输出等功能，系统的监控模块实现对系统失效或失步的检测与定位。其软件的可靠性设计包括恢复块方法和N版本程序设计方法。请根据恢复块方法工作原理完成图3-1，在（1）～（4）中填入恰当的内容。并比较恢复块方法与N版本程序设计方法，将比较结果（5）～（8）填入表3-2中。



	恢复块方法	N版本程序设计
硬件运行环境	单机	多机
错误检测方法	验证测试程序	(5)
恢复策略	(6)	向前恢复
实时性	(7)	(8)

思路解析

恢复块方法是一种 **后向恢复机制**。其基本流程是：

运行 **主块 (1)**，即默认实现的功能模块；

执行结果必须通过 **验证测试 (2)**，确保输出正确；

如果验证通过，则 **输出正确结果 (3)**；

若验证失败，则进入 **异常处理 (4)**，调用备用实现（备用块），直到获得正确结果。

N 版本程序设计 (NVP) 与恢复块方法比较：

判别方式 (5)：NVP 使用 **表决** 机制比较多个版本的输出，选择多数一致结果；恢复块方法依赖单一版本的验证测试。

恢复机制 (6)：恢复块属于 **后向恢复**，即失败后回退并执行备用块；NVP 属于 **前向恢复**，直接通过表决选择正确输出。

开销 (7)：恢复块实现相对简单，通常需要较少的冗余资源；NVP 需要多个独立版本并行执行，代价 **高**。因此恢复块方法的代价相对 **差**。

可靠性 (8)：NVP 不依赖单一验证测试，而是依靠多版本独立实现的差异化提高正确率，因此可靠性 **更好**。

答案-问题 (1)

可靠性是指产品在规定的条件下和规定的时间内完成规定功能的能力。子特性：成熟性，容错性，易恢复性，可靠性的依从性。

提高可靠性的技术：(1) N版本程序设计 (2) 恢复块方法 (3) 防卫式程序设计 (4) 双机热备或集群系统 (5) 冗余设计

答案-问题 (2)

(1) 不考虑软件演化的情况下，失效率在统计上是非增的

(2) 如果不使用该软件，永远不会发生失效

(3) 软件维护会创建新的软件代码

(4) 软件失效之前很少会有报警

答案-问题 (3)

(1) 主块

(2) 验证测试

(3) 输出正确结果

(4) 异常处理

(5) 表决

(6) 后向恢复

(7) 差

(8) 好

第15章——安全架构与威胁防护 (2013年11月系统架构设计师真题-第5题)

阅读以下有关软件与信息安全方面的说明，在答题纸上回答问题1至问题3。

【说明】

某软件公司拟开发一套信息安全支撑平台，为客户的局域网业务环境提供信息安全保护。该支撑平台的主要需求如下：

- (1) 为局域网业务环境提供用户身份鉴别与资源访问授权功能；
- (2) 为局域网环境中交换的网络数据提供加密保护；
- (3) 为服务器和终端机存储的敏感持久数据提供加密保护；
- (4) 保护的主要实体对象包括局域网内交换的网络数据包、文件服务器中的敏感数据文件、数据库服务器中的敏感关系数据和终端机用户存储的敏感数据文件；
- (5) 服务器中存储的敏感数据按安全管理员配置的权限访问；
- (6) 业务系统生成的单个敏感数据文件可能会达到数百兆的规模；
- (7) 终端机用户存储的敏感数据为用户私有；
- (8) 局域网业务环境的总用户数在100人以内。

问题(1)

在确定该支撑平台所采用的用户身份鉴别机制时，王工提出采用基于口令的简单认证机制，而李工则提出采用基于公钥体系的认证机制。项目组经过讨论，确定采用基于公钥体系的机制，请结合上述需求具体分析采用李工方案的原因。

思路解析

在用户身份鉴别机制的选择上，**安全性与效率的平衡**是关键。**口令认证**的优势在于实现简单、成本低，但缺点十分突出：一方面，用户口令可能过于简单、重复使用或未及时更新，极易遭受**暴力破解、字典攻击、社工攻击**；另一方面，口令在传输或存储过程中若缺乏完善的加密保护，容易被窃取或伪造，从而导致身份冒充。在本案例中，用户数量虽不大，但涉及的安全对象涵盖**网络数据、文件、数据库敏感字段**等，一旦口令被攻破，系统将失去防护意义。

相比之下，**基于公钥体系的认证机制(PKI)**依托非对称加密技术，客户端通过持有的私钥和公钥对进行认证，私钥仅保存在用户本地，从未在网络中传输，因此可以避免口令传输的脆弱性。同时，系统能够利用用户的私钥为其生成独有的签名，从而绑定用户身份与私有数据，实现**身份认证与数据保护一体化**。虽然其计算代价较大，但考虑到业务环境为局域网、用户规模仅100人以内，性能开销完全可接受。因此，该系统选择基于公钥体系的认证机制，能够兼顾**高安全性与可行性**，是更加合理的架构决策。

问题(2)

针对需求(7)，项目组经过讨论，确定了基于数字信封的加密方式，其加密后的文件结构如图5-1所示。请结合需求说明对文件数据进行加密时，应采用对称加密的块加密方式还是流加密方式，为什么？并对该机制中的数据加密与解密过程进行描述。

<其他文件头信息>

以数字信封形式封装的对称密钥

用对称密钥加密的文件数据

思路解析

在数据加密方式的选择上，核心考虑因素是**性能与安全性**。对称加密通常有两类：**块加密（Block Cipher）**和**流加密（Stream Cipher）**。块加密将数据分成固定大小的块逐一处理，适合中小规模数据的高安全性保护；流加密则将明文与伪随机密钥流逐位或逐字节异或运算，能够快速处理大规模数据文件，效率优势明显。本案例中的文件可能达到**数百兆**，若采用块加密，不仅需要复杂的分组和填充，还可能因文件过大造成**性能下降与存储开销增加**。因此，应采用流加密方式。

数字信封机制结合了非对称加密与对称加密的优点。非对称加密（公钥/私钥）安全性高但效率低，适合用于密钥保护；对称加密效率高但密钥分发困难，适合用于大数据加密。在加密过程中，系统先生成一个临时对称密钥，用用户的公钥加密该密钥并放在文件头部，之后利用该对称密钥对文件数据执行流加密，从而实现**高效的数据保护**。在解密过程中，用户使用私钥解密文件头的对称密钥，再用该密钥解密数据正文，恢复原文。此机制确保了**密钥传输的安全性**与**大数据处理的效率**，完全契合本案例中“文件大、数据敏感”的需求。

问题（3）

对数据库服务器中的敏感关系数据进行加密保护时，客户业务系统中的敏感关系数据主要是特定数据库表中的敏感字段值，客户要求对不同程度的敏感字段采用不同强度的密钥进行防护，且加密方式应尽可能减少安全管理与应用程序的负担。目前数据库管理系统提供的基本数据加密方式主要包括加解密API和透明加密两种，请用300字以内的文字对这两种方式进行解释，并结合需求说明应采用哪种加密方式。

思路解析

在数据库加密方案设计中，选择合适的方式关系到**性能、开发成本和安全保障**。**加解密API**是数据库提供的编程接口，允许应用系统在读写敏感数据时主动调用API完成加密和解密。其优势在于灵活

，开发者可以根据业务需求为不同字段定义加密方式和密钥强度。但它的劣势同样明显：需要修改应用程序代码，增加开发与维护成本；并且加密逻辑嵌入在应用层，可能导致**安全职责分散**，不利于集中管理。

透明加密 (Transparent Data Encryption, TDE) 由数据库管理系统自动完成加解密工作，应用程序看到的依然是明文数据，所有加密逻辑都在数据库内部完成。安全管理员可通过策略配置为不同字段、表或库设置不同强度的加密方式与密钥，而应用层无需做任何改动。这种方式大幅降低了**开发人员负担和安全管理复杂性**。结合本案例的需求：客户要求对不同敏感字段采用不同强度的密钥，并且“尽可能减少安全管理与应用程序负担”。显然，透明加密能够很好满足要求，在提供分级加密能力的同时，也避免了应用系统的大规模改造。因此，本系统应采用透明加密方案，确保在**安全性、易用性和可维护性**之间取得最佳平衡。

答案-问题 (1)

(1) 基于口令的认证方式实现简单，但由于口令复杂度及管理方面的原因，易受到认证攻击；而在基于公钥体系的认证方式中，由于其密钥机制的复杂性，同时在认证过程中私钥不在网络上传输，因此可以有效防止认证攻击，与基于口令的认证方式相比更为安全。

(2) 按照需求描述，在完成用户身份鉴别后，需依据用户身份进一步对业务数据进行安全保护，且受保护数据中包含用户私有的终端机数据文件，在基于口令的认证方式中，用户口令为用户和认证服务器共享，没有用户独有的直接秘密信息，而在基于公钥的认证方式中，可基于用户私钥对私有数据进行加密保护，实现更加简便。

(3) 基于公钥体系的认证方式协议和计算更加复杂，因此其计算复杂度要高于基于口令的认证方式，但业务环境的总用户数在100人以内，用户规模不大，运行环境又为局域网环境，因此基于公钥体系的认证方式可以满足平台效率要求。

答案-问题 (2)

应采用流加密方式。因为需求中提及“单个敏感数据文件可能会达到数百兆的规模”，文件数据量较大，使用流加密方式可以获得更高的加解密效率。

数据加密与解密过程如下：其加密过程为：首先生成一个对称密钥，使用用户公钥加密这个对称密钥后存储在文件头，然后用生成的对称密钥加密文件数据存储。其解密过程为：用户首先使用自己的私钥解密被加密的对称密钥，再用该对称密钥解密出数据原文。

答案-问题 (3)

数据库管理系统提供的基本数据加密支持主要有两种方式：一是加解密API，允许应用程序通过调用API自行构建加密基础架构；二是透明加密，由安全管理员选择加密方式和密钥强度，数据库系统自动处理数据的加密和解密。加解密API方式灵活但复杂，透明加密方式简单但灵活性较差。考虑到用户要求减少安全管理与应用程序负担，建议选择透明加密方式。

(2024年11月系统架构设计师真题-第5题)

阅读以下关于安全分析的叙述，在答题纸上回答问题1-3。

【说明】

随着科技的进步与医疗需求的增长，某医疗科技公司计划开发一套智能胰岛素注射系统，旨在为糖尿病患者提供更精准、便捷且安全的胰岛素注射方案。

系统的核心流程如下：患者需先使用配套的血糖仪进行血糖测量，血糖仪会将实时测量得到的血糖数据，通过蓝牙等无线通信技术，快速且准确地传输至胰岛素泵系统。胰岛素泵系统内置了先进的算法模型，该模型会结合患者的个体信息（如年龄、体重、病史等）以及实时测量的血糖数据，自动、精准地计算出本次需要注射的胰岛素剂量。

计算完成后，胰岛素泵系统会依据计算所得的剂量，自动控制泵设备，将适量的胰岛素准确无误地

注射到患者体内。

问题（1）

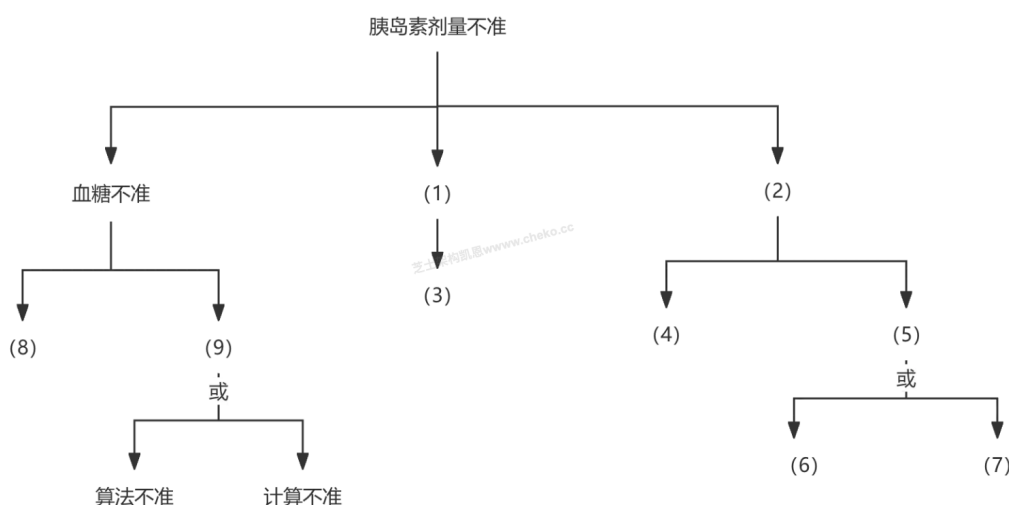
请用150字简要说明风险驱动的安全分析4个步骤的内容。

思路解析

风险驱动的安全分析的概念书本没有。经过凯恩调研，风险驱动的安全测试机制通过结合风险分析和威胁建模，识别系统中的高风险状态。该方法通过风险可能性、风险影响和风险阈值等指标，识别系统中的风险。

问题（2）

胰岛素剂量传输不准会带来一些安全问题，请将下列内容填入下方的空格处，完善可能导致胰岛素剂量不准的原因。



备选选项：（a）血糖传感器错误（b）传输系统异常（c）血糖计算不准（d）定时器失效（e）泵信号失效（f）错误时间推送预定的量（g）算法错误（h）计算错误（i）胰岛素计算错误

思路解析

这是典型的故障分析树（FTA），书本没有介绍过这个知识点。故障分析树是一种系统性、演绎性的分析方法，用于识别和评估导致系统故障或不良事件的潜在原因。它通过构建树状结构，将顶事件（即不希望的事件）分解为一系列基本事件，并通过逻辑门表示这些事件之间的因果关系。

根据题干给出的信息，你要分析导致胰岛素剂量不准的原因。备选选项传达出了一些信息。顶层问题是“胰岛素剂量不准”，它被细分为三个主要原因：1. **血糖不准** 2. **传输系统异常** 3. **胰岛素计算错误**，我们逐一分析各个编号对应的选项。

(1) → f: **错误时间推送预定的量**

胰岛素在不该输送的时候输送了“正确量”，也会导致剂量不准。

(3) → d: **定时器失效**

与“错误时间推送”相关的一个直接原因是定时器没有正常工作。

(2) → b: **传输系统异常**

胰岛素输送系统故障，会造成剂量不准。

(4) → e: **泵信号失效**

输送系统依赖于泵的信号驱动，信号失败自然会导致输送问题。

(5) → i: **胰岛素计算错误**

即便系统正常，若计算错误也会导致剂量不准。

(6) → g: **算法错误**

可能的计算错误之一。

(7) → h: 计算错误

泛指代码或实现层面的计算逻辑错误。

血糖不准 的两种来源:

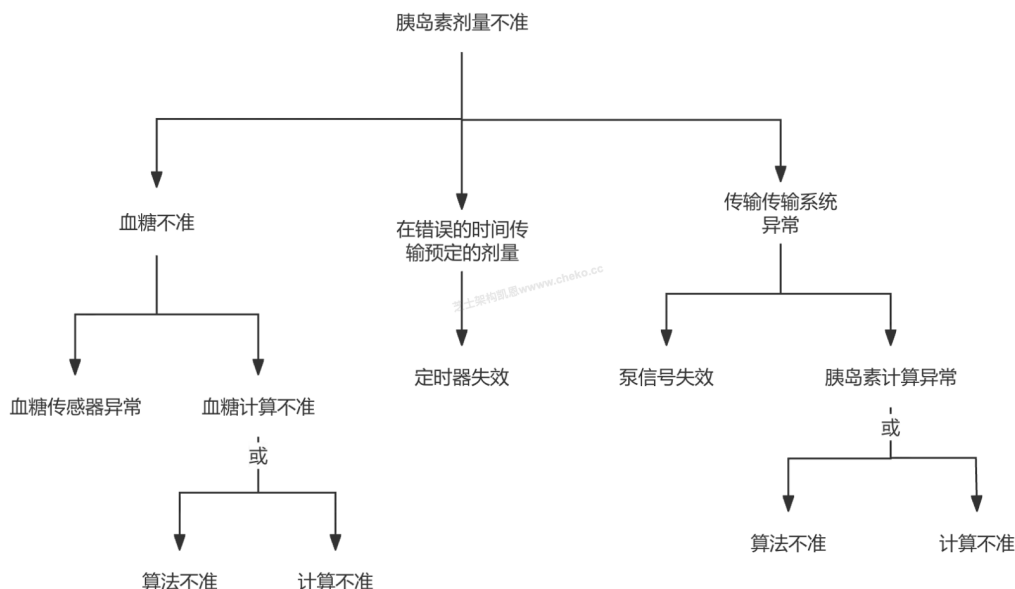
(8) → a: 血糖传感器错误

采集原始数据出错, 导致血糖测不准。

(9) → c: 血糖计算不准

即便传感器读数正确, 后续处理 (如算法换算) 出错, 也会导致血糖不准。

参考答案如下图所示。



问题 (3)

请用300字论述形式化开发和软件测试技术各自的特点。

思路解析

纯概念题, 形式化开发再讲软件工程的时候提到过, 主要是通过数学模型和逻辑推理确保软件的正确性, 纯理论的方法, 工程上很少用。

答案-问题 (1)

风险识别: 全面查找系统中可能存在的各类危险, 如物理风险、软件故障等。

风险评估: 对识别出的风险进行评估, 考量其发生的可能性及潜在后果的严重程度。

安全需求定义: 基于风险评估结果, 明确系统应具备的安全需求和功能。

安全设计与验证: 依据安全需求进行系统设计, 并验证设计是否满足需求, 降低风险。

答案-问题 (2)

(1) f (2) b (3) d (4) e (5) i (6) g (7) h (8) a (9) c

答案-问题 (3)

形式化开发和软件测试技术是保障软件质量的两种重要方法, 它们在理念、实施方式和适用场景上各具特点。

形式化开发基于数学逻辑和形式化语言对系统需求、设计与实现进行建模和验证, 强调在开发早期消除缺陷。其主要优点是严谨性强、适用于高安全性系统, 如航空、核能或医疗系统。形式化方法

能发现传统测试难以覆盖的边界条件或逻辑错误，但其缺点是学习成本高、建模复杂、推广受限。相比之下，软件测试技术是一种动态验证方法，主要通过实际运行软件并观察输出结果来发现问题，覆盖面广、实施灵活，适合各类开发场景。测试包括单元测试、集成测试、系统测试等，支持自动化。其缺点是无法穷尽所有路径、不能完全保证程序无错。

第16章——构件化与插件架构 (2017年11月系统架构设计师真题-第2题)

阅读以下关于软件系统设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某软件企业受该省教育部门委托建设高校数字化教育教学资源共享平台，实现以众筹众创的方式组织省内普通高校联合开展教育教学资源内容建设，实现全省优质教学资源整合和共享。该资源共享平台的主要功能模块包括：

(1) 统一身份认证模块：提供统一的认证入口，为平台其他核心业务模块提供用户管理、身份认证、权限分级和单点登录等功能；

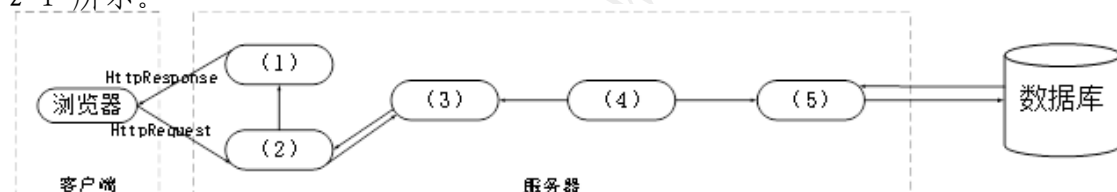
(2) 共享资源管理模块：提供教学资源申报流程服务，包括了资源申报、分类定制、资料上传、资源审核和资源发布等功能；

(3) 共享资源展示模块：提供教育教学共享资源的展示服务，包括资源导航、视频点播、资源检索、分类展示、资源评价和推荐等功能；

(4) 资源元模型管理模块：依据资源类型提供共享资源的描述属性、内容属性和展示属性，包括共享资源统一标准和规范、资源加工和在线编辑工具、数字水印和模板定制等功能；

(5) 系统综合管理模块：提供系统管理和维护服务，包括系统配置、数据备份恢复、资源导入导出和统计分析等功能。

项目组经过分析和讨论，决定采用基于 Java EE 的 MVC 模式设计资源共享平台的软件架构，如图 2-1 所示。



问题 (1)

MVC 架构中包含哪三种元素，它们的作用分别是什么？请根据图 2-1 所示架构将 JavaEE 中 JSP、Servlet、Service、JavaBean、DAO 五种构件分别填入空 (1)~(5) 所示位置。

思路解析

在 MVC 架构中，三种主要元素及其作用是：

视图 (View)：负责显示数据并向用户展示交互界面。它接收用户的输入，但不处理业务逻辑。典型的实现是 JSP。

控制器 (Controller)：负责接收用户输入、处理请求并决定如何更新模型和视图。Servlet 是 MVC 中典型的控制器。

模型 (Model)：表示业务逻辑或数据，并与视图和控制器交互。它处理数据，计算或其他业务操作。典型的 JavaBean 和 Service 类属于模型部分，DAO 负责数据访问，封装了对数据库的操作。

根据图 2-1 中的架构，JavaEE 中的构件如下所示：

(1) JSP → 视图 (View)：负责展示数据给用户。

(2) Servlet → 控制器 (Controller)：接收用户请求并调度处理。

(3) JavaBean → 模型 (Model)：包含业务逻辑或数据。

(4) Service → 模型 (Model)：处理具体的业务逻辑，服务层。

(5) DAO → 模型 (Model)：负责数据访问，处理与数据库的交互。

问题（2）

项目组架构师王工提出在图2-1所示架构设计中加入EJB构件，采用企业级JavaEE架构开发资源共享平台。请说明EJB构件中的Bean（构件）分为哪三种类型，每种类型Bean的职责是什么。

思路解析

EJB（Enterprise JavaBeans）是一种服务器端组件模型，用于开发企业级应用程序。EJB中的Bean分为三种类型：

会话Bean（Session Bean）：职责：负责执行业务逻辑并处理客户端的请求。会话Bean有两种类型：无状态会话Bean（Stateless）和有状态会话Bean（Stateful）。有状态会话Bean会在会话期间维护客户端的状态，而无状态会话Bean不会保持客户端的状态。

实体Bean（Entity Bean）：职责：表示持久化数据对象，通常与数据库中的表格行对应。

Entity Bean用于保存应用程序的业务数据，并可以通过JPA（Java Persistence API）进行数据库的交互。

消息驱动Bean（Message-Driven Bean）：职责：用于异步处理消息。它从消息队列中接收消息并响应处理。通常与消息传递系统（如JMS）配合使用，处理异步消息。

问题（3）

如果采用王工提出的企业级Java EE架构，请说明下列（a）-（e）所给出的业务功能构件中，有状态和无状态构件分别包括哪些。

- （a）Identification Bean（身份认证构件）
- （b）ResPublish Bean（资源发布构件）
- （c）ResRetrieval Bean（资源检索构件）
- （d）OnlineEdit Bean（在线编辑构件）
- （e）Statistics Bean（统计分析构件）

思路解析

根据EJB中的有状态和无状态构件的特点：

有状态构件（Stateful Session Beans）：

（a）Identification Bean（身份认证构件）：身份认证需要维护用户的会话状态，确保认证过程中的状态信息得到保留，因此需要设计为有状态构件。

（d）OnlineEdit Bean（在线编辑构件）：在线编辑构件通常需要记录用户的编辑状态、前一次操作的结果，因此它也是有状态构件。

无状态构件（Stateless Session Beans）：

（b）ResPublish Bean（资源发布构件）：资源发布操作通常独立于其他操作，且每个请求都是独立的，因而它是无状态构件。

（c）ResRetrieval Bean（资源检索构件）：资源检索是一个独立的、无须维护状态的操作，因此它是无状态构件。

（e）Statistics Bean（统计分析构件）：统计分析通常处理的是独立请求的结果，不需要保留客户端的交互状态，因此它也是无状态构件。

答案-问题（1）

MVC架构包含：视图、控制器、模型

视图（View）：视图是用户看到并与之交互的界面。视图向用户显示相关的数据，并能接收用户的输入数据，但是它并不进行任何实际的业务处理。

控制器（Controller）：控制器接受用户的输入并调用模型和视图去完成用户的需求。该部分是用户界面与Model的接口。一方面它解释来自于视图的输入，将其解释成为系统能够理解的对象，同时

它也识别用户动作，并将其解释为对模型特定方法的调用；另一方面，它处理来自于模型的事件和模型逻辑执行的结果，调用适当的视图为用户提供反馈。

模型（Model）：模型是应用程序的主体部分。模型表示业务数据和业务逻辑。一个模型能为多个视图提供数据。

(1) JSP (2) Servlet (3) JavaBean (4) Service (5) DAO

答案-问题（2）

EJB中的Bean分三种类型：Session Bean、Entity Bean 和 Message-Driven Bean。Session Bean的职责是：维护一个短暂的会话。Entity Bean 的职责是：维护一行持久稳固的数据。Message-Driven Bean的职责是：异步接受消息。

答案-问题（3）

有状态：（a）、（d）

无状态：（b）、（c）、（e）

（2014年11月系统架构设计师真题-第1题）

请详细阅读以下关于网络设备管理系统架构设计的说明，在答题纸上回答问题1和问题2。

【说明】

某软件公司欲开发一个网络设备管理系统，对管理区域内的网络设备（如路由器和交换机等）进行远程监视和控制。公司的系统分析师首先对系统进行了需求分析，识别出如下3项核心需求：

（a）目前需要管理的网络设备确定为10类20种，未来还将有新类别的网络设备纳入到该设备管理系统中；

（b）不同类别的网络设备，监视和控制的内容差异较大；同一类别的网络设备，监视和控制的内容相似，但不同厂商的实现方式（包括控制接口格式、编程语言等）差异较大；

（c）网络管理员能够在一个统一的终端之上实现对这些网络设备的可视化呈现和管理操作。

针对上述需求，公司研发部门的架构师对网络设备管理系统的架构进行了分析与设计，架构师王工认为该系统可以采用MVC架构风格实现，即对每种网络设备设计一个监控组件，组件通过调用网络设备厂商内置的编程接口对监控指令进行接收和处理；系统管理员通过管理模块向监控组件发送监控指令，对网络设备进行远程管理；网络状态、监控结果等信息会在控制终端上进行展示。针对不同网络设备的差异，王工认为可以对当前的20种网络设备接口进行调研与梳理，然后通过定义统一操作接口屏蔽设备差异。李工同意王工提出的MVC架构风格和定义统一操作接口的思路，但考虑到未来还会有新类别的网络设备接入，认为还需要采用扩展接口的方式支持系统开发人员扩展或修改现有操作接口。公司组织专家进行架构评审，最终同意了王工的方案和李工的改进意见。

问题（1）

请用300字以内的文字解释什么是MVC架构风格以及其中的组件交互关系，并根据题干描述，指出该系统中的M、V、C分别对应什么。

思路解析

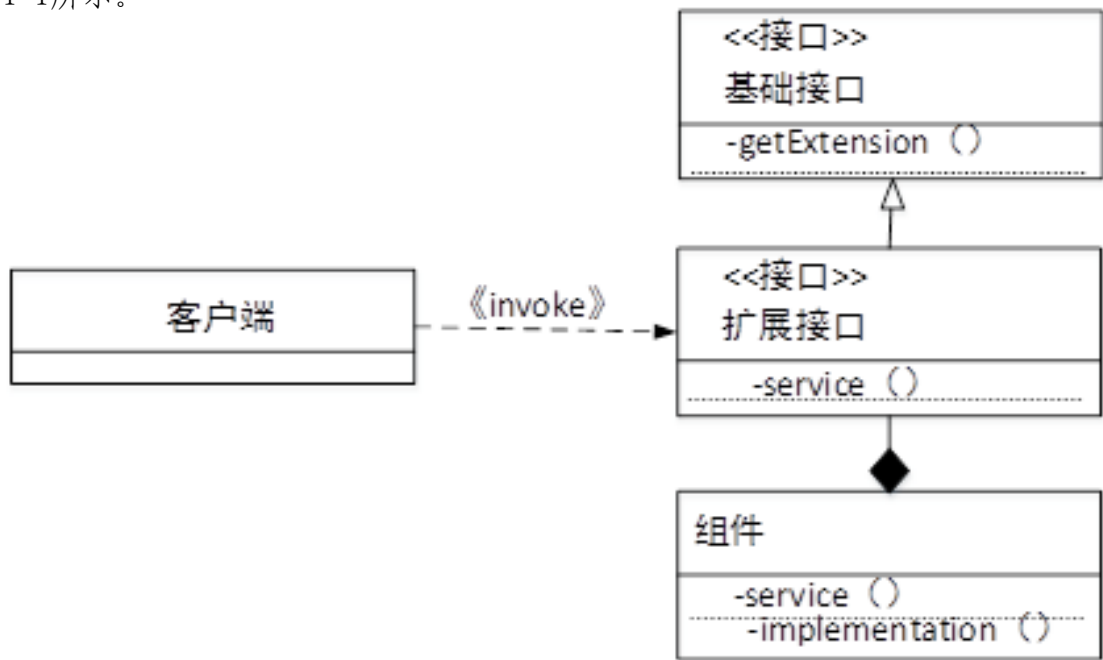
MVC（Model-View-Controller）架构风格最早起源于 Smalltalk-80，现已广泛应用于软件系统的架构设计，尤其是用户界面与后台逻辑分离的系统。它的优势在于将**关注点分离**：模型聚焦于业务逻辑与数据一致性，视图聚焦于信息呈现和用户交互，控制器则充当中间层协调两者。这种分工有效降低了代码耦合，使系统在**扩展性、可维护性、可移植性**方面表现良好。例如，在更换显示界面时，无需修改核心的业务处理模块；当业务逻辑调整时，也不会直接影响视图的实现。

在网络设备管理系统的具体应用中，**监控组件**是模型（M），它直接与厂商提供的接口交互，完成数据采集与指令下发；**控制终端**是视图（V），为管理员提供统一的操作界面，负责数据展示与结果反馈；**管理模块**是控制器（C），它把用户发出的命令转化为对监控组件的调用，从而实现对设备的远

程控制。这样的分层使得当未来有新类别的网络设备接入时，仅需在模型层（监控组件）新增实现，不会影响已有的终端界面和管理模块逻辑，从而实现系统的扩展性与解耦。

问题（2）

扩展接口模式结构通常包含四个角色：基础接口、组件、扩展接口和客户端，它们之间的关系如图1-1所示。



其中每个扩展接口需要通过扩展基础接口获得基本操作能力，然后加入自己特有的操作接口，并通过设置全局唯一接口ID对自身接口进行标识；每个具体的组件需要实现扩展接口完成实际操作；客户端不与组件直接交互，而需要通过扩展接口进行交互从而提出调用请求，扩展接口根据请求查找并选择合适的实现组件响应客户端请求。请根据上图所示和题干描述，指出扩展接口模式结构中的四个角色分别对应网络设备管理系统的哪些部分；并以客户端发起调用操作这一场景为例，填写表1-1中的（1）～（5）。

表 1-1 客户端发起调用操作过程描述

序号	操作
1	客户端调用某个 (1) A 上的操作接口，该操作接口可能是基础接口，也可能是扩展接口
2	若实现 A 的 (2) 存在被执行请求的操作接口，则调用该操作接口向用户返回结果
3	如果所有组件均没有实现 (3)，则客户端调用 A 上的 getExtension 方法，传入需要的 (4)，通过查找与定位，找到实现该操作接口的 (5) B，并将 B 的引用传回客户端
4	客户端调用 B 上的操作接口，通过相应的实现组件返回结果

备选答案：基础接口、扩展接口、操作接口、接口 ID、客户端、组件。

思路解析

扩展接口模式（Extension Interface Pattern）是为了解决系统在支持未来新功能或新设备时，能够以较低代价进行扩展的一种架构设计模式。其基本思想是：定义一个**统一的基础接口**，所有扩展接口都要继承该接口以保证基本操作一致性；同时，每个扩展接口可以在继承的基础上增加自己的特有方法，以满足新类别设备或功能的需求。组件则是对这些扩展接口的具体实现，而客户端则通过扩展接口与组件间接交互，避免直接依赖具体实现类，从而实现**高内聚低耦合**和**可扩展性**。在本题的网络设备管理系统中：**基础接口**是统一操作接口，它屏蔽了不同厂商设备实现的差异；**组件**是监控组件，负责调用设备厂商的底层接口并实现具体操作；**扩展接口**则是面向未来新设备所定义的特定接口，例如针对新型交换机的扩展接口；**客户端**是控制终端，管理员通过终端发出操作请求，但并不直接与监控组件交互，而是通过扩展接口来完成。这样，客户端只需依赖扩展接口，就

能在后续新增设备时无需改动，真正实现了“对扩展开放，对修改关闭”的设计原则。

结合表1-1的调用场景：客户端首先通过（1）扩展接口发起调用请求；扩展接口会定位并选择合适的（2）组件来完成具体操作；组件实现具体的（3）操作接口逻辑；为了区分不同扩展接口，系统利用全局唯一的（4）接口ID进行标识和调用路由；最后，执行结果通过（5）扩展接口返回给客户端。整个流程保证了调用统一、实现分散、扩展灵活。

答案-问题（1）

MVC架构风格：用一种业务逻辑、数据、界面显示分离的方法组织代码，将业务逻辑聚集到一个部件里面，在改进和个性化定制界面及用户交互的同时，不需要重新编写业务逻辑。

MVC架构将整个软件系统划分为模型、视图和控制器3个部分。模型负责维护并保存具有持久性的业务数据，实现业务处理功能，并将业务数据的变化情况及时通知视图；视图负责呈现模型中包含的业务数据，响应模型变化通知，更新呈现形式，并向控制器传递用户的界面动作；控制器负责将用户的界面动作映射为模型中的业务处理功能并实际调用之，然后根据模型返回的业务处理结果选择新的视图。

在本题中：M：监控组件；V：控制终端；C：管理模块。

答案-问题（2）

基础接口：统一操作接口，组件：监控组件，扩展接口：新网络设备的操作接口，客户端：控制终端。

（1）扩展接口（2）组件（3）操作接口（4）接口ID（5）扩展接口

（2014年11月系统架构设计师真题-第3题）

请详细阅读有关嵌入式构件设计方面的说明，在答题纸上回答问题1至问题3。

【说明】

构件（component）也称为组件，是一个功能相对独立的具有可复用价值的软硬件单元。近年来，构件技术正在逐步应用于大型嵌入式系统的软件设计。某公司长期从事飞行器电子设备研制工作，已积累了大量成熟软件。但是，由于当初管理和设计等原因，公司的大量软件不能被复用，严重影响了公司后续发展。公司领导层高度重视软件复用问题，明确提出了要将本公司的成熟软件进行改造，建立公司可复用的软件构件库，以提升开发效率、降低成本。公司领导层决定将此任务交给技术部门的王工程师负责组织实施。两个月后，王工程师经过调研、梳理和实验，提交了一份实施方案。此方案得到了公司领导层的肯定，但在实施过程中遇到了许多困难，主要表现在公司软件架构的变更和构件抽取的界面等方面。

问题（1）

请用200字以内文字说明获取构件的方法有哪几种？开发构件通常采用哪几种策略？并列举出两种主流构件标准。

思路解析

获取构件的方式多种多样，主要分为四类：**直接复用现有构件**、**遗留工程提取**、**购买COTS构件**以及**重新开发构件**。直接复用或经过小幅度适应性修改的构件能够大幅节省研发成本；遗留工程则强调对已有系统进行分析、分解，从中抽取潜在的可复用构件，适合像题干中这类有大量积累软件的公司；购买COTS构件是一种工业界常见手段，例如现成的通信库、加密库；最后，在没有可用资源时，就需要从零开始开发新的构件。

在构件开发策略上，**分区**、**抽象**、**分割**是核心手段。分区强调将复杂问题划分为相对独立的部分，以便构件化设计；抽象强调将软件/硬件功能提炼为统一接口，使其能够独立于具体实现而复用；分割则通过在构件中引入结构，支持形式化推理和组合验证。关于主流标准，业界常见的有**CORBA**（通用对象请求代理架构）、**COM/DCOM/COM+**（微软组件对象模型）、**EJB**（企业级Java

Bean），它们为构件化设计提供了规范和工具。

问题（2）

由于该公司已具备大量的成熟软件，王工程师此次的主要工作就是采用遗留工程（Legacy Engineering）方法，将具有潜在复用价值的软件提取出来，得到可复用的构件。因此，在设计软件时与原开发技术人员产生了重大意见分歧，主要分歧焦点在于大家对构件概念理解上的差异。请根据你对构件的理解，判断表3-1给出的有关构件的说法是否正确，将答案写在答题纸上。

表 3-1 有关构件的 6 种说法

序号	关于构件的说明	正确：√ 不正确：×
1	构件是系统中的一个封装了设计与实现，而只披露接口的可更换的部分	(1)
2	构件是解决软件复用的基础，复用的形式可分为垂直式复用和水平式复用。而水平式复用的主要关键在于领域分析，具有领域特征和相似性，受到广泛关注	(2)
3	构件构建在平台之上，平台提供核心平台服务，是构件实现与构件组装的基础。构件组装通常采用基于功能的组装技术、基于数据的组装技术和基于配置的组装技术等三种技术	(3)
4	软件架构为软件系统提供了一个结构、行为和属性的高级抽象，由构件的描述、构件的相互作用（连接件）、指导构件集成的模式以及这些模式的约束组成	(4)
5	构件可分为硬件构件、软件构件、系统构件和应用构件。RTL（运行时库）属于软件构件，由于 RTL 与应用领域相关，所以 RTL 应属于垂直式复用构件	(5)
6	硬件构件的功能被给定的硬件结构如 ASIC 预先确定，是不能修改的。同样，软件构件的功能由在 FPGA 或者 CPU 上的软件确定的，也是不能修改的	(6)

思路解析

（1）构件是系统中的一个封装了设计与实现、而只披露接口的可更换部分。正确。这是构件的核心特征：封装实现、暴露接口、可独立替换。符合 Szyperski《Component Software》中构件的经典定义。

（2）构件是解决软件复用的基础，复用可分为垂直复用和水平复用，水平复用的关键在于领域分析。不正确。虽然“水平复用/垂直复用”是复用的重要区分，但它们针对的是复用技术分类，而不是构件概念本身。构件复用关注接口标准化与独立交付，而非简单地等同于水平/垂直复用。此处表述偏离了构件定义。

（3）构件构建在平台之上，平台提供核心平台服务，是构件实现与构件组装的基础。构件组装通常采用基于功能的组装技术、基于数据的组装技术和面向对象的组装技术等三种技术。题干中配置只是一种构件功能组合动态方法，而不是构件组装的技术。

（4）软件架构为软件系统提供结构、行为和属性的高级抽象，由构件及其交互组成。正确。软件架构就是由构件 + 连接件构成的整体抽象，并描述了它们的交互关系。构件是架构的核心组成单元，这与 IEEE 1471 和 ISO/IEC 42010 的架构定义相符。

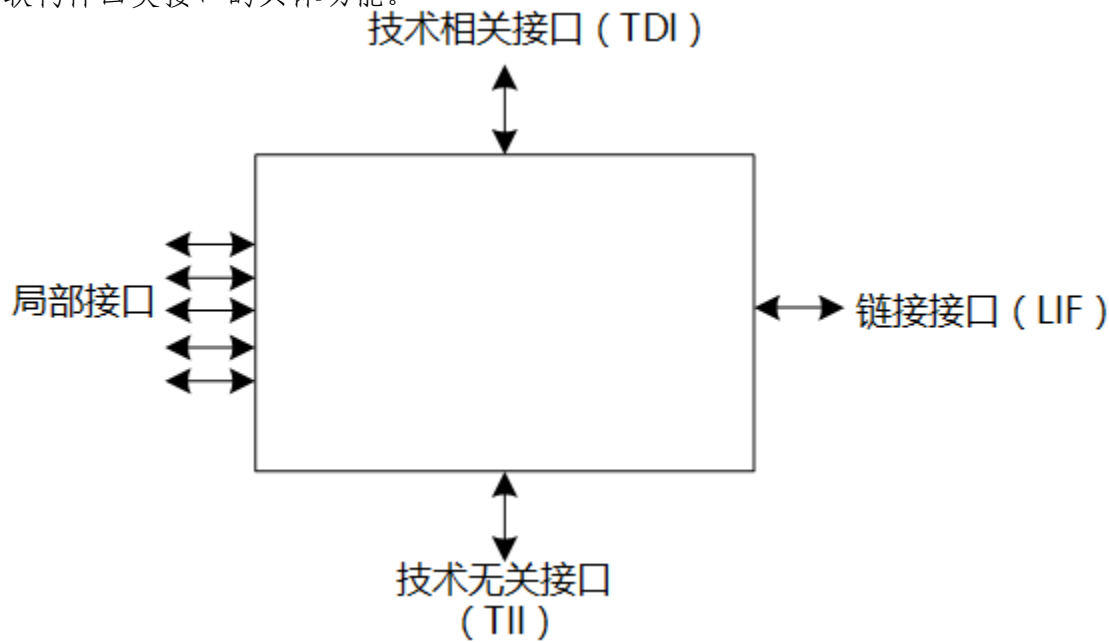
（5）构件可分为硬件构件、软件构件、系统构件和应用构件，RTL 属于软件构件，应属于垂直复用构件。不正确。RTL，由于RTL可适应多种应用领域，所以RTL属于水平式复用构件”。不属于垂直式复用构件。

（6）硬件构件功能固定不可修改，软件构件在 FPGA 或 CPU 上确定后也不可修改。不正确。构件强调的是可独立部署、可复用，而非“固定不可修改”。软件构件恰恰是可扩展、可替换的，而不

是一旦放在某平台上就不能改。这种说法明显背离了构件复用的初衷。

问题（3）

王工程师的实施方案指出：本公司的大部分产品是为用户提供标准计算平台的，而此平台中的主要开发工作是为嵌入式操作系统研制板级支持软件（BSP）。为了提高BSP软件的复用，应首先开展BSP构件的开发，且构件架构应符合国外GENESYS规范定义的嵌入式系统架构风格。图3-1给出了架构风格定义的构件通用接口，其中：链接接口（LIF）是构件对外提供的功能服务接口；局部接口建立了构件和它的局部环境的连接，如传感器、作动器或人机接口；技术相关接口（TDI）提供了查看构件内部、观察构件的内部变量的手段，如诊断等；技术无关接口（TII）用来在运行时配置、复位、重启构件的接口。现需要针对BSP中常用的RS-232串行驱动程序设计一个可复用的软构件，请说明该软构件四类接口的具体功能。



思路解析

根据 GENESYS 规范，嵌入式系统构件的通用接口主要分为 LIF、局部接口、TDI、TII 四类。它们共同构成了嵌入式构件在系统中的完整交互机制。在 RS-232 驱动构件的设计中，这四类接口的功能应当清晰划分。

链接接口（LIF） 是该构件对外暴露的核心功能接口。对于 RS-232 串行驱动而言，必须包括初始化功能（设置波特率、数据位、停止位、校验位等），以及数据收发接口（send/receive）。这部分接口直接面向上层应用，是最常用的调用点。**局部接口** 则是驱动与底层硬件环境的连接，如将外部中断信号连接到 CPU、访问芯片寄存器，这使得驱动能够感知硬件状态并控制硬件运行。**技术相关接口（TDI）** 主要用于诊断与调试，典型功能如检测传输过程中的奇偶校验错误、帧错误、溢出错误等，便于开发和维护人员定位故障。**技术无关接口（TII）** 则属于管理类接口，例如复位驱动构件、在运行时重新配置传输速率，这些功能与具体硬件无关，但关系到运行时的灵活性和容错性。

通过这种划分，RS-232 驱动构件既能作为独立的可复用单元存在，又能通过标准接口在不同系统中灵活组合。这种设计方式大大提高了 **BSP 软件的可复用性和可维护性**，符合现代嵌入式构件化开发的趋势。

答案-问题（1）

获取构件的方法包括：（1）从现有构件中获得符合要求的构件，直接使用或做适应性（flexibility）修改，得到可重用的构件；（2）通过遗留工程（Legacy engineering），将具有潜在复用价值的构件提取出来，得到可重用的构件；（3）从市场上购买现成的商业构件，即

COTS (Commercial Off-The-Shelf) 构件; (4) 开发新的符合要求的构件。

开发构件的策略: 分区: 指的是将问题情景的空间分割成几乎可以独立研究的部分; 抽象: 是对给定时间内执行指定计算的软/硬件单元的一种抽象; 分割: 是将结构引入构件的行为, 支持对行为性质进行时序推理。主流构件标准包括: CORBA、COM/DCOM/COM+、EJB (答出其中两种即可)。

答案-问题 (2)

(1) ✓ (2) × (3) × (4) ✓ (5) × (6) ×

答案-问题 (3)

局部接口: 将RS-232芯片的外部中断向量连接到CPU, 从RS-232芯片中读取寄存器的内容, 将控制信息定稿RS-232芯片寄存器。

技术相关接口 (TDI): 诊断数据传输校验的错误。

链接接口 (LIF): 对RS-232芯片初始化, 并向其发送数据与从芯片接收数据。

技术无关接口 (TII): RS-232芯片或驱动构件复位和重启, 重新配置传输速率。

第17章——数据架构与数据治理 (2021年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式数据架构设计的相关描述, 在答题纸上回答问题1至问题3。

【说明】

数据架构(Data architecture)是系统架构设计的主要工作之一。它主要用于描述业务数据以及数据间的关系。数据架构着重考虑"数据需求", 关注的是持久化数据的组织。数据架构的设计过程主要包括: 数据定义、数据分布与数据管理。某公司为了适应宇航装备的持续发展, 提升本公司的核心竞争力, 改变原来事件驱动的架构设计模式。公司领导将新产品架构规划工作交给张工。张工经过分析、调研给出了本企业宇航产品的未来架构规划方案。

问题 (1)

张工在规划方案中指出: 宇航装备要实现以数据为中心的架构设计模式, 就应改变传统的各个子系统独 的设计方式, 打破原宇航装备的生产关系。为了达到这个目标, 我们首先要解决装备数据的共享、管理和存储等问题, 做好顶层的数据架构规划工作。请用300字以内的文字说明数据定义、数据分布与数据管理的具体内涵。

思路解析

在顶层数据架构设计中, 首先要解决数据定义问题。数据定义决定了数据元素的 **语义、格式、约束和关系**, 通过统一的数据模型可实现跨系统的数据理解与复用。其次是数据分布, 它关系到 **数据存储的物理位置和访问策略**, 涉及集中存储与分布式存储的权衡, 保证不同子系统的数据流转与共享。最后是数据管理, 即对数据生命周期全过程的 **制度化管控**, 如数据采集、清洗、存储、备份、安全与权限控制等, 确保数据完整性和一致性。通过这三方面的统筹, 能够实现宇航装备的数据驱动设计与生产, 促进协同开发和高效决策。

问题 (2)

张工在规划方案中提出公司未来产品设计要遵从一种开放式的架构体系, 并在此基础上完善数据架构的设计工作, 形成一套规格化的数据模型语言。张工给出了基于FACE (Future Airborne Capability Environment)架构的新产品架构, 其中, 图3-1说明了数据模型语言在架构模型中的作用。

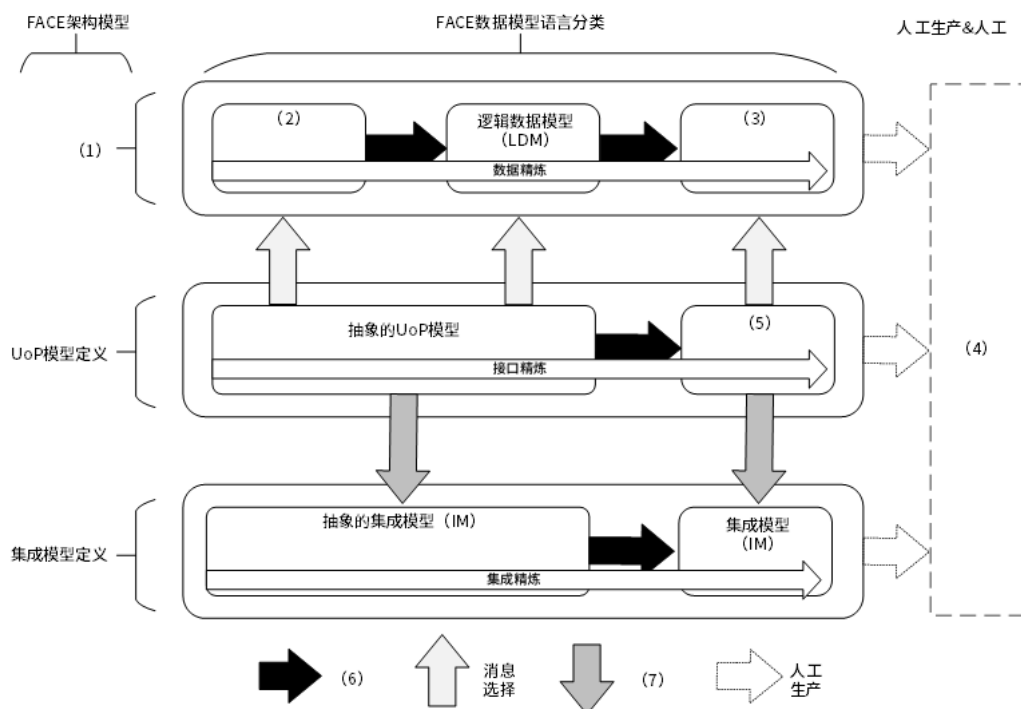


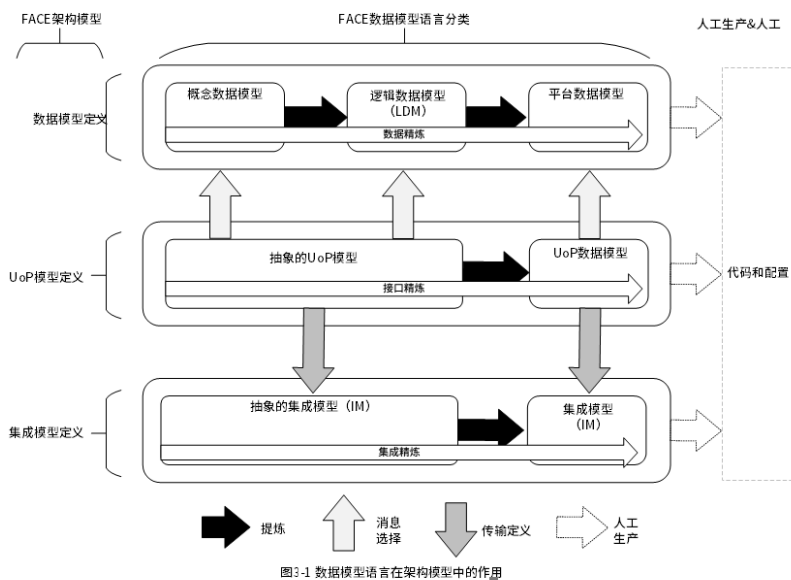
图3-1 数据模型语言在架构模型中的作用

请根据你所掌握的数据架构的相关知识，从以下a~g中进行选择，填充完善图3-1中的(1)~(7)空格。

- a. 数据模型定义
- b. 平台数据模型 (PDM)
- c. UoP (Unit of Portability) 数据模型 (UM)
- d. 提炼
- e. 传输定义
- f. 代码和配置
- g. 概念数据模型 (CDM)

思路解析

在 FACE 架构中，数据模型语言承担了从 **概念建模到实现落地** 的桥梁作用。首先进行 **数据模型定义** (a)，再通过 **概念数据模型** (g) 表达业务语义。随后形成面向平台的 **PDM** (b)，最终生成可运行的 **代码与配置** (f)。同时，还需映射至 **UoP 数据模型** (c)，以保证可移植性和复用性。数据模型的演进过程中要经过 **提炼** (d) 的过程，以去除冗余，提取核心信息，并通过 **传输定义** (e) 来确保跨模块、跨系统间的数据交换一致性。该流程展示了 FACE 架构下数据建模的分层思想。



问题（3）

"数据需求"是数据架构设计中需要着重考虑的问题。在张工给出的基于FACE架构的新产品架构中，分别就架构中的各个部分逐条给出了需求项。请判断表3-1给出的9项需求是否属于数据需求。

表 3-1 张工给出的需求项

序号	需求项	属于数据需求 (是/否)
1	操作系统段的一个可移植单元应支持分区、进程、线程和存储管理功能	(1)
2	在安全传输时，模块支持单元(USM) 应该为可移植单元提供所有可用的数据元素	(2)
3	I/O 服务应该为对应的每条 I/O 总线的体系结构提供 I/O 服务管理能力	(3)
4	USM 访问传输段接口时应包括可移植单元需要发送的所有数据元素	(4)
5	特定域数据模型 (DSDM) 应该遵循 FACE 共享数据模型管理	(5)
6	当安全传输作为可移植组件段内可移植单元执行时，访问安全传输的所有数据的界限应与 FACE 数据架构保持一致	(6)
7	DSDM 应该是遵守 EMOF 2.0 约束的 XML 文件	(7)
8	可移植单元应该为每个 FACE 接口提供一个注入实例	(8)
9	在可移植单元包中的所有可移植单元应该设计在相同分区内操作	(9)

思路解析

数据需求是指对 **数据内容、结构、质量、安全与共享方式** 的要求，而非对功能、性能或硬件的要求。表 3-1 中第 (2) (4) (5) (7) 涉及数据一致性、共享、存储与安全等，符合数据需求的范畴；而第 (1) (3) (6) (8) (9) 则涉及业务逻辑、接口性能、系统实现或运行特性，不属于数据需求。明确区分数据需求与其他需求，有助于架构设计中各层次的责任划分，避免混淆。特别注意这里的 (6)，是指我们的“安全传输模块”在访问数据时，必须遵守 FACE 数据架构规定的边界与访问规范，不可以随意访问或操作架构外的数据。所以他强调的是数据安全规范的一致性，而不是数据本身的内容，了解即可，不用深究。

表 3-1 张工给出的需求项。

序号。	需求项。	属于数据需求 (是/否)。
1。	操作系统段的一个可移植单元应支持分区、进程、线程和存储管理功能。	功能需求。
2。	在安全传输时，模块支持单元(USM) 应该为可移植单元提供所有可用的数据元素。	数据需求。
3。	I/O 服务应该为对应的每条 I/O 总线的体系结构提供 I/O 服务管理能力。	功能需求。
4。	USM 访问传输段接口时应包括可移植单元需要发送的所有数据元素。	数据需求。
5。	特定域数据模型 (DSDM) 应该遵循 FACE 共享数据模型管理。	数据需求。
6。	当安全传输作为可移植组件段内可移植单元执行时，访问安全传输的所有数据的界限应与 FACE 数据架构保持一致。	功能需求。
7。	DSDM 应该是遵守 EMOF 2.0 约束的 XML 文件。	数据需求。
8。	可移植单元应该为每个 FACE 接口提供一个注入实例。	功能需求。
9。	在可移植单元包中的所有可移植单元应该设计在相同分区内操作。	功能需求。

答案-问题（1）

数据定义：数据定义要反映业务模式的本质，确保数据架构为业务需求提供全面、一致、完整的高质量数据。数据定义要划分应用系统边界，明确数据引用关系，定义应用系统间的集成接口。定义数据模型主要包括：数据概念模型、数据逻辑模型、数据物理模型和数据标准。

数据分布：数据分布是数据系统分布的基础。包含数据业务、数据分析和数据存储。数据业务是分析数据在业务各环节的创建、引用、修改或删除的关系。数据分析是在单一应用系统中的分析数据结构与应用系统各功能间的引用关系，分析数据在多个系统间的引用关系。数据存储包含分析数据集中存储和数据分布存储两种模式，要根据需求选择数据分布策略。

数据管理：数据管理是要制定贯穿数据生命周期的各项管理制度，包括：数据模型与数据标准管理，数据分布管理，数据质量管理和数据安全管理制度。确定数据管理组织或岗位。

答案-问题（2）

- (1) a
- (2) g
- (3) b
- (4) f
- (5) c
- (6) d
- (7) e

答案-问题（3）

- (1) 否
- (2) 是
- (3) 否
- (4) 是
- (5) 是
- (6) 否
- (7) 是
- (8) 否
- (9) 否

(2015年11月系统架构设计师真题-第4题)

阅读以下关于应用系统数据架构的说明，在答题纸上回答问题1至问题3。

【说明】

某软件公司拟开发一套贸易综合管理系统，包括客户关系管理子系统和商品信息管理子系统两部分。客户关系管理子系统主要管理客户信息，并根据贸易业务需要频繁向客户发送相关的电子邮件、短信等提醒信息。商品信息管理子系统主要为客户提供商品信息在线查询功能，包括商品基本信息、实时库存与价格等。

在对系统进行数据架构设计时，公司项目组的架构师王工主张采用文件系统进行数据管理，原因是目前公司客户和商品数量不大，且系统功能较为简单，采用文件系统进行数据管理简单直观，开发周期短。架构师李工则建议采用关系数据库进行数据管理，原因在于公司目前正处在高速扩张期，虽然目前的客户和商品数量不大，但随着公司快速发展，需要管理的数据必然飞速膨胀，采用关系数据库作为数据存储层，系统的扩展性更强，并能够对未来可能增加的复杂业务提供有效支持。经过讨论，项目组初步采纳了李工的意见，决定采用关系数据库存储客户数据，并针对业务特征对系统性能进行优化。

问题（1）

请从设计难度、数据冗余程度、数据架构、应用扩展性等4个方面对关系型数据库管理系统和文件系统两种数据存储方式进行比较，填写表4-1中（1）~（4）。

	设计难度	数据冗余程度	数据架构	应用扩展性
关系型数据库	（1）	遵守数据库范式，数据冗余较少	以数据库为中心组织、管理数据	（4）
文件系统	针对特定应用系统设计，难度较小	（2）	（3）	符合特定应用系统要求的文件数据很难在不同的应用系统之间共享

思路解析

在信息系统架构设计中，选择文件系统还是关系数据库管理系统（RDBMS），取决于系统复杂性、数据增长趋势、扩展需求等。

设计难度：文件系统的数据组织简单，开发者仅需编写 I/O 操作即可，但缺乏数据模型约束，容易造成混乱。关系数据库必须遵循关系模式，需要设计数据表、主外键、范式约束等，设计难度更大，但数据结构更规范，利于长期维护。

数据冗余程度：文件系统往往是应用驱动的数据存储，各应用独立管理数据，容易出现相同数据被重复存储在多个文件中，冗余度大，数据一致性难以保证。关系数据库通过集中式数据管理和规范化设计，能够有效减少冗余，提高一致性。

数据架构：文件系统是“以应用为中心”的数据架构，即应用直接驱动数据存取，数据组织依赖具体应用，缺乏统一的全局管理。关系数据库是“以数据为中心”的架构，数据库系统作为独立组件运行，应用通过标准接口访问数据，更具规范性。

应用扩展性：文件系统扩展性差，新应用往往无法直接重用已有数据，需要单独开发数据接口。关系数据库具有标准化接口（如 JDBC/ODBC），实现了数据与应用的分离，使得数据更易共享和扩展。

综上，文件系统适用于数据量小、业务简单、开发周期短的场景，而关系数据库更适合数据量大、业务复杂、扩展性要求高的企业应用。

问题（2）

对系统的核心业务需求进行认真分析后，公司的资深架构师张工提出一种内存数据库和关系数据库的混合存储架构，其核心思想是将需要频繁读写的数据存入内存数据库，而将相对固定不变的数据

存入关系数据库。请首先分析比较内存数据库和关系数据库在数据模型、读写性能、存储容量、可靠性等方面的差异，填写表4-2中（1）～（4）的空白，并根据张工的思路指定各种业务数据的存储方式，填写表4-3中（5）～（9）的空白。

	主要数据模型	读写性能	存储容量	可靠性
内存数据库	(1)	内存直接读写，性能相对较高	(3)	(4)
关系数据库	关系模式	(2)	基于磁盘存储，存储容量大	内建恢复机制，可靠性较高
业务数据		存储方式		
客户基本信息		关系数据库		
客户电子邮件		(5)		
客户联系电话		(6)		
商品基本信息		(7)		
商品库存信息		(8)		
商品价格信息		(9)		

思路解析

1. 内存数据库与关系数据库的差异:

- 数据模型:** 关系数据库采用表-行-列的关系模型，支持 SQL 查询和复杂事务。而内存数据库多采用 Key-Value 结构（如 Redis），结构简单，适合高频快速访问。
- 读写性能:** 关系数据库以磁盘存储为主，需经过磁盘 I/O，性能相对较低。而内存数据库将数据存放在内存中，访问延迟极低，吞吐量可提升数十倍。
- 存储容量:** 关系数据库容量大，可管理海量数据；内存数据库容量受物理内存限制，适合存储热点数据或频繁访问数据。
- 可靠性:** 关系数据库具有 日志、备份、恢复机制，可靠性高。内存数据库虽然有快照和持久化机制，但在掉电、宕机等情况下，部分数据可能丢失，可靠性相对较低。

2. 业务数据的存储方式:

- (5) 客户电子邮件: 需要频繁调用和发送，存入 内存数据库。
- (6) 客户联系电话: 同样高频使用，存入 内存数据库。
- (7) 客户基本信息: 变化不频繁，适合存入 关系数据库。
- (8) 商品库存信息: 需实时更新，存入 内存数据库。
- (9) 商品价格信息: 经常变动，客户频繁查询，存入 内存数据库。

这种混合架构充分利用了关系数据库的 稳定性 与内存数据库的 高性能，实现了性能与可靠性的平衡，特别适用于电商、金融、贸易类系统。

问题（3）

系统开发完成进行压力测试时，发现在大数据量的情况下，部分业务查询响应时间过长，经过分析发现其主要原因是部分SQL查询语句效率低下。请判断表4-4中的SQL语句设计策略哪些可能会提升查询效率，哪些可能会降低查询效率，在（1）～（4）中填入"提升"或"降低"。

SQL语句设计策略	查询性能影响情况
查询时不要返回不需要的行、列	(1)
需要进行多表连接查询时，尽量采用子查询结构	(2)
尽量采用NOT IN、NOT EXIST、LIKE等操作	(3)
尽量避免使用DISTINCT关键字	(4)

思路解析

SQL 查询效率与 语句结构、索引利用、返回数据量 密切相关。

(1) 只返回需要的列：减少数据传输量，降低网络与内存开销，显然 **提升** 查询效率。

(2) 使用子查询替代连接查询：子查询往往会导致数据库先生成临时表，再执行外层查询，增加开销，相比 JOIN 性能更差，因此 **降低** 效率。

(3) 使用 NOT IN 或 NOT EXISTS：此类语句通常触发全表扫描，尤其在大数据量下效率低下，因此 **降低**。

(4) 避免使用DISTINCT：DISTINCT 本质是“后处理去重”，往往增加一次**全局操作**，降低查询效率。如果能通过 **索引设计、SQL 改写、逻辑优化** 来避免产生重复数据，就能显著提升性能。

答案-问题(1)

- (1) 数据结构需要符合关系模式，设计难度较大
- (2) 可能在多个文件中复制相同的数据属性，数据冗余较大
- (3) 以应用系统为中心管理数据
- (4) 数据独立于应用系统，数据库系统接口标准化，易于在不同应用之间共享数据

答案-问题(2)

- (1) Key-Value模式
- (2) 外存读写，性能相对较低
- (3) 运行时整个数据库基本全调入内存，数据库容量受内存容量限制，容量较小
- (4) 虽然也有恢复机制，但并不是所有故障都能恢复，可靠性较低
- (5) 内存数据库
- (6) 内存数据库
- (7) 关系数据库
- (8) 内存数据库
- (9) 内存数据库

答案-问题(3)

- (1) 提升
- (2) 降低
- (3) 降低
- (4) 提升

第18章——数据库缓存策略 (2022年11月系统架构设计师真题-第4题)

阅读以下关于数据库缓存的叙述，在答题纸上回答问题1至问题3。

【说明】

某大型电商平台建立了一个在线 B2B 商店系统，并在全国多地建设了货物仓储中心，通过提前备货的方式来提高货物的运送效率。但是在运营过程中，发现会出现很多跨仓储中心调货从而延误货物运送的情况。为此，该企业计划新建立一个全国仓储货物管理系统，在实现仓储中心常规管理功能之外，通过对在线 B2B 商店系统中订单信息进行及时的分析和挖掘，并通过大数据分析预测各地仓储中心中各类货物的配置数量，从而提高运送效率，降低成本。

当用户通过在线 B2B 商店系统选购货物时，全国仓储货物管理系统会通过该用户所在地址、商品类别以及仓储中心的货物信息和地址，实时为用户订单反馈货物起运地（某仓储中心）并预测送达时间。反馈送达时间的响应时间应小于1秒。

为满足反馈送达时间功能的性能要求，设计团队建议在全国仓储货物管理系统中采用数据缓存集群的方式，将仓储中心基本信息、商品类别以及库存数量放置在内存的缓存中，而仓储中心的其它商品信息则存储在数据库系统。

问题（1）

设计团队在讨论缓存和数据库的数据一致性问题时，李工建议采取数据实时同步更新方案，而张工则建议采用数据异步准实时更新方案。请用200字以内的文字，简要介绍两种方案的基本思路，说明全国仓储货物管理系统应该采用哪种方案，并说明采取该方案的原因。

思路解析

题干中强调用户操作需“实时反馈起运地并预测送达时间”，其关键是**快速响应**，而非缓存数据绝对实时。异步准实时方案在绝大多数场景下可实现秒级或更快的数据同步，能有效减少系统阻塞与性能开销，是大型系统缓存设计的主流做法。

问题（2）

随着业务的发展，仓储中心以及商品的数量日益增加，需要对集群部署多个缓存节点，提高缓存的处理能力。李工建议采用缓存分片方法，把缓存的数据拆分到多个节点分别存储，减轻单个缓存节点的访问压力，达到分流效果。缓存分片方法常用的有哈希算法和一致性哈希算法，李工建议采用一致性哈希算法来进行分片。请用200字以内的文字简要说明两种算法的基本原理，并说明李工采用一致性哈希算法的原因。

思路解析

一致性哈希其实是为了解决「分布式系统中节点变化时数据迁移太多」的问题。

最初的负载均衡办法是轮询，比如 3 个节点就轮流分配请求，这样适合每个节点存的都是一样的数据。但一旦进入分布式场景，每个节点保存的数据不同，就需要一种“定点”的办法来找到某个 key 在哪个节点上。

于是大家想到哈希取模： $\text{hash}(\text{key}) \% \text{节点数}$ 。这种方式简单高效，但有致命缺陷：只要节点数一变（扩容或缩容），几乎所有 key 的映射都会失效，最坏情况需要迁移全部数据，成本太高。举个例子，假设一开始有 3 个节点 A、B、C，key-01 落在 A，key-02 落在 B，key-03 落在 C；当我们新增一个节点 D 时，取模公式从 $\%3$ 变成 $\%4$ ，结果 key-01 被映射到 C、key-02 被映射到 D、key-03 又被映射到 A，几乎所有数据都要迁移，非常麻烦。

一致性哈希的思路是：不对节点数取模，而是固定对一个大数（比如 2^{32} ）取模，把结果组织成一个环，叫哈希环。节点和数据都经过哈希后落到环上。数据的归属规则是：顺时针找到的第一个节点就是它的宿主。这样一来，新增或减少一个节点，只会影响它在环上邻近的一小部分数据，不再需要全局迁移。比如刚才新增节点 D 的场景下，只有 key-02 会被迁移到 D，key-01 和 key-03 依然留在原来的节点，迁移量大大减少。

不过，一致性哈希有个问题：节点可能在环上分布不均，导致有的节点负担过重。为了解决这个问题，引入了“虚拟节点”。一个真实节点可以对应多个虚拟节点，这些虚拟节点分散映射到环上，再统一指向真实节点。这样一来，分布会更均匀，负载也能根据权重灵活分配。工程里常常一个真实节点就会挂上百个虚拟节点。比如节点 A 不再只出现在环上的一个位置，而是映射成 A-01、

A-02、A-03 等多个虚拟节点，均匀散落在环上，这样数据就不会都挤到某个节点上了。

问题（3）

全国仓储货物管理系统开发完成，在运营一段时间后，系统维护人员发现大量黑客故意发起非法的商品送达时间查询请求，造成了缓存穿透。张工建议尽快采用布隆过滤器方法解决。请用200字以内的文字解释布隆过滤器的工作原理和优缺点。

思路解析

布隆过滤器（Bloom Filter）是处理**海量数据存在性判断**的利器。

简单来说，它回答的问题是：“某个元素可能在集合中吗？”——如果它说“不在”，那一定不在；如果它说“在”，则有极小的概率是误判。

它的原理并不复杂。布隆过滤器由一个**位数组（bit array）**和若干个**哈希函数**组成。初始时，位数组全是 0。当我们往布隆过滤器里添加一个元素时，会经过多个哈希函数映射出几个下标，并把对应下标的位设置为 1。以后要判断某个元素是否存在时，再用相同的哈希函数去计算，如果这些下标对应的位都为 1，就认为元素存在；只要有一位是 0，就可以确定元素一定不存在。这就是它的节省空间的秘密：每个元素并不存储实际内容，而只是“打了几个标记”。

布隆过滤器的优缺点非常鲜明。**优点**是空间效率极高，比如要存 100 万个元素，只需要大约 122KB 的内存；查询速度也很快，适合高并发场景。**缺点**是会有误判，尤其是元素越多，误判率越高；另外删除不方便，因为多个元素可能共用同一位置。

在实际应用中，布隆过滤器可以做到：

缓存穿透防护：用户请求一个数据库中根本不存在的 key，如果没有布隆过滤器，可能每次都打到数据库，拖垮后端。有了布隆过滤器，可以快速判断请求是否无效，直接拦截。

爬虫去重：在抓取网页时，已经爬过的 URL 通过布隆过滤器标记，下次遇到就可以直接跳过。

黑名单拦截：比如邮箱垃圾过滤、手机号黑名单判断，快速筛查某个元素是否在海量名单中。

在工程上，我们既可以手写一个布隆过滤器（用 Java 的 BitSet 和自定义哈希函数实现），也可以直接用 **Google Guava** 提供的布隆过滤器工具类，设置容量和误判率即可开箱即用。如果想在分布式场景下使用，还可以借助 **RedisBloom** 模块，把布隆过滤器部署在 Redis 中，天然支持扩展和多语言调用。

答案-问题（1）

实时同步方案：数据库数据一旦更新，立即同步更新缓存中的对应数据，保证数据强一致性。

异步准实时方案：数据库更新时不立即更新缓存，而是将变更记录入日志或消息队列中，再由后台任务异步刷新缓存，最终实现数据一致性。

全国仓储货物管理系统应采用**异步准实时方案**。原因是该系统对响应性能要求极高（小于1秒），而实时同步在高并发场景下存在性能瓶颈和风险。异步更新可以减小响应延迟，满足业务对高性能的需求。

答案-问题（2）

普通哈希算法通过哈希函数将键值映射到固定缓存节点上，节点变动会导致大量数据迁移。

一致性哈希算法将节点和数据通过哈希函数映射到一个环形空间，节点增减只影响局部数据，迁移量小。

李工建议使用一致性哈希算法，是因为它具备良好的节点扩展性与容错性，适应系统动态变化，同时保持数据分布均衡，保障缓存访问效率。

答案-问题（3）

布隆过滤器通过多个哈希函数将元素映射到位数组上，用于判断某元素是否存在于集合中。查询时检查对应位是否全为1，若有0则必然不存在。

优点：占用内存小，查询速度快，适合大规模数据过滤。缺点：存在误判率（可能将不存在的数据判断为存在），且不支持删除操作。

布隆过滤器可有效拦截大量无效请求，防止缓存穿透，提升系统稳定性。

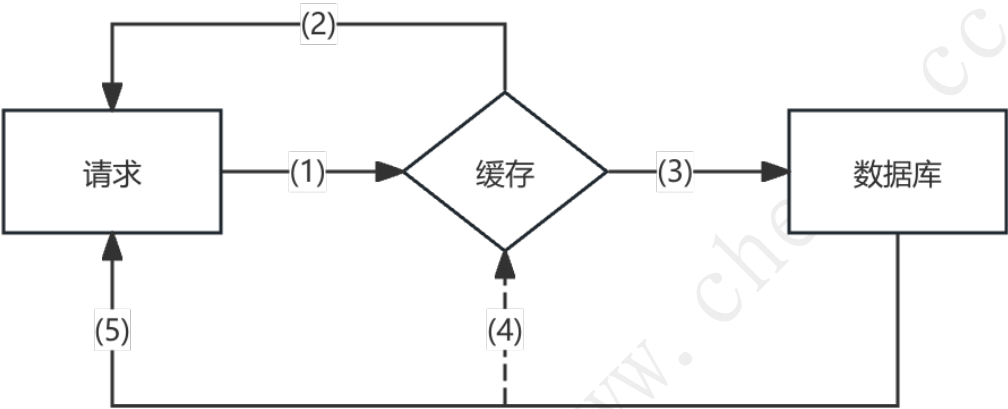
(2024年11月系统架构设计师真题-第2题)

阅读以下关于数据库缓存架构的叙述，在答题纸上回答问题1-3。

某电商平台使用缓存来存储热点商品数据。旁路缓存模式是常见的一种缓存模式，其核心是将缓存模块设置为I/O路径的一个非强制性组成部分。这意味着缓存模块可以根据需要灵活地加入或移除，而不会影响原有的I/O路径的完整性。在这种模式下，缓存是一个附加组件，提供了一个可选的数据访问路径。

问题（1）

根据旁路缓存模式的读数据原理，填写下图中（1）到（5）的空白。



思路解析

旁路模式在开发中用的比较多，其核心是将缓存模块设置为I/O路径的一个非强制性组成部分。在这种模式下，缓存是一个附加组件。在读操作中，首先根据Key在缓存中查找所需的数据。如果成功找到（命中），则直接返回数据；若未命中，则会从存储模块获取，并在适当的时候把数据写入缓存。

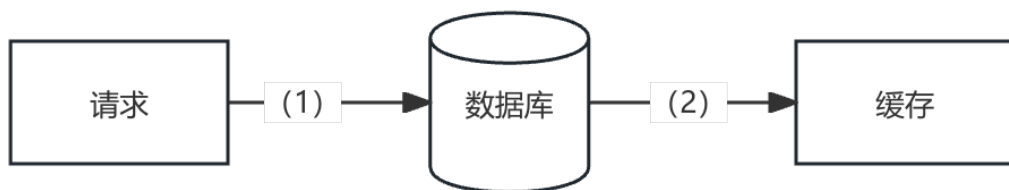
另外还有两种模式凯恩建议大家了解。

读写穿透（Read/Write Through） 模式是指所有读写操作都经过缓存，缓存负责同步数据库。该模式保证了缓存与数据库的数据一致性，适合读写均衡且对一致性要求较高的系统，例如权限管理或账户信息场景。但它高度依赖缓存系统，一旦缓存出现故障，将严重影响整体服务稳定性。在日常开发中，可以尝试使用 Redisson + RMapLoader/Writer 的组合。

写回缓存（Write Back / Write Behind） 模式是将写操作先写入缓存，再异步写入数据库，能显著提升写入性能，适用于写多读少、对实时一致性要求不高的场景，如日志收集或行为打点系统。缺点是数据可能在写入数据库前丢失，存在一致性风险，需配合持久化机制确保可靠性。一般和消息队列进行配合。

问题（2）

根据旁路缓存模式写数据原理，填写下图中（1）（2）的空白。



思路解析

旁路模式的写操作流程如下所示：（1）接收到写请求后，首先更新数据库中的数据。（2）数据库中的数据更新完毕后，会发请求给缓存，删除相关联的缓存条目。这里应当删除相应的缓存条目，而非更新缓存。这个主要是出于缓存数据库一致性的考虑了解即可。

问题（3）

王工使用了多线程技术进行缓存处理，线程1负责写入，线程2负责读取，可能存在数据不一致性问题，请解释其原因，并给出3个以上的解决办法。

思路解析

这里会有一个同步问题。假设线程1做了一个写入数据库的操作，此时数据库的值是新的，缓存是旧的，我当然希望缓存也是立刻变成新的，这样肯定不会有业务问题。但是我们是一个高并发的系统，可能你在写完数据库之后，还没来得及改缓存，后面的一个线程就来读，此时它读的是缓存的就旧值，这样就可能发生业务问题。

所以这里的解决办法：**一种是基于强一致性**，也就是保证数据库中的值和缓存中的数据任何情况下都一致，假如不一致，缓存没改完前不给别人读，但是这种方法需要用分布式锁/事务/队列串行化，会显著影响性能，不太适合高并发场景。**另外一种是最最终一致性**，允许数据库和缓存短暂不一致，但是可以确保过了一段时间，数据库和缓存的值肯定是一样的，都是最新的。我们要做的就是缩短这个不一致的时间。

答案-问题（1）

- （1）向缓存请求读取该商品信息
- （2）有缓存目标数据则直接返回，若命中则返回该商品信息
- （3）没有则提交给数据库进行查询，若未命中则访问数据库查询该商品信息
- （4）将查询到的数据库数据更新到缓存
- （5）将查询到的数据库目标数据返回

答案-问题（2）

- （1）更新数据库中的目标商品信息
- （2）删除缓存中的目标商品信息

答案-问题（3）

旁路模式在多线程场景下：线程1 正在写入数据库，但还没来得及删除缓存；线程2 此时仍然从旧缓存读取，拿到过期数据。

解决方案如下：

（1）延迟双删（最终一致性）。在写入数据时，先删除缓存中的数据，然后更新数据源中的数据，最后在经过一段短暂的延迟后再次删除缓存（记住结论即可，推论比较复杂，这里就很玄学了，什么叫短暂的时间，多短多长，不同的时间设置，在不同的业务场景也有可能不一致，概率高低罢了）。

（2）先写数据库再删除缓存（最终一致性）（会有失效的情况，但是概率很低，就是缓存失效和

写数据库同时发生的时候，这时候会有旧数据填充到缓存里，记住结论即可，实践中常用这个方法，因为最简单）。

(3) 分布式锁（强一致性）。写操作的过程为获取写锁、更新数据库、删除缓存、释放写锁。读数据的时候也要对访问数据库的操作获取读锁。缓存读取完毕之后，释放读锁，确保读写操作序列化。这里的锁可以通过 Redis SET lock-key unique-value NX PX 10000 命令设置（一般以数据为Key）。

(4) 消息队列进行序列化读写操作（强一致性）。消息队列进行读写操作。把同一业务键的写请求路由到同一消息队列，天然顺序、无乱序覆盖。

(5) 订阅binlog日志（最终一致性）。在写入数据时，先数据源中的数据。同时可以使用阿里的canal将binlog日志采集发送到MQ队列里面，然后通过ACK机制确认处理这条更新消息，修改缓存，保证数据缓存一致性。因为队列消费是按顺序的，所以缓存最终会依据顺序修改达到最后的状态。

(2024年11月系统架构设计师真题-第3题)

阅读以下关于数据库缓存架构的叙述，在答题纸上回答问题1-3。

【说明】

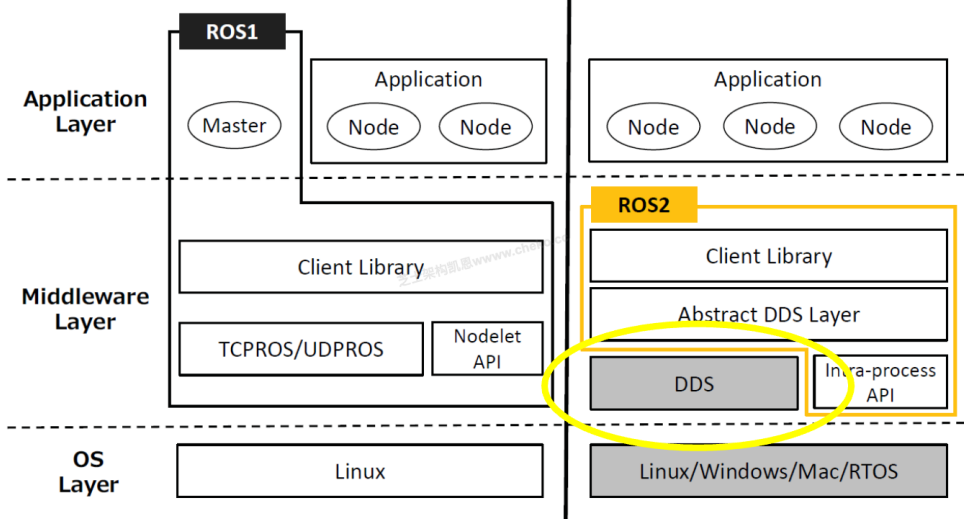
某智能科技公司承接了一项开发服务型机器人的重要项目，该机器人旨在应用于大型商场，为顾客提供导购、信息咨询以及简单的货物搬运等服务。在项目开发过程中，选用了嵌入式系统架构，并决定采用机器人操作系统 ROS 来构建机器人的软件平台。

问题（1）

请用150字说明ROS定义和特点。ROS2 与 ROS1 相比哪些地方做了改进，请用150 字描述。

思路解析

这道题本质上是考察你对 ROS 基础知识的理解，以及对 ROS2 相较于 ROS1 技术演进的掌握。ROS1相关内容书本有简单介绍，但是ROS2没有，要看考生行业积累。



问题（2）

请将符合的描述的 ROS 的通信机制名称填入下方空格处。

- (1) 是一种单向通信模型，在通信双方中，发布方发布数据，订阅方订阅数据，数据流单向的由发布方传输到订阅方。
- (2) 是一种基于请求响应的通信模型，在通信双方中，客户端发送请求数据到服务端，服务端响应结果给客户端。
- (3) 是一种带有连续反馈的通信模型，在通信双方中，客户端发送请求数据到服务端，服务端响应结果给客户端，但是在服务端接收到请求到产生最终响应的过程中，会发送中间连续的反馈（进

度)信息到客户端。

(4) 是一种基于共享的通信模型,在通信双方中,服务端可以设置数据,而客户端可以连接服务端并操作服务端数据。

思路解析

(1) 发布-订阅机制(Publish/Subscribe)

“单向通信模型”、“发布方”、“订阅方”、“数据单向传输”都给出了提示。这是 ROS 最基本的通信机制之一。节点通过发布(publish)消息到某个 topic,订阅者(subscriber)监听该 topic 并接收消息。特点是**解耦、多对多、实时传输**。这里使用“话题通信”或“发布-订阅机制”都可以,关键是体现出你理解这个模型的核心特征:**基于 topic,数据单向流动,解耦式通信**。

(2) 服务机制(Service)

“请求响应”、“客户端发送请求”、“服务端响应结果”是关键。在服务机制模式下,类似传统 RPC 通信模式,客户端调用服务(service),服务端返回结果,适用于**同步、一对一通信**。特点是**阻塞式、同步请求响应**。

(3) 动作机制(Action)

题干中提到“连续反馈”、“中间反馈”、“最终响应”,适用于需要持续执行、状态反馈的任务,如导航或机械臂运动。基于 service+topic 实现,提供中间状态(feedback)、最终结果(result)等。特点是**支持异步、带进度反馈**。

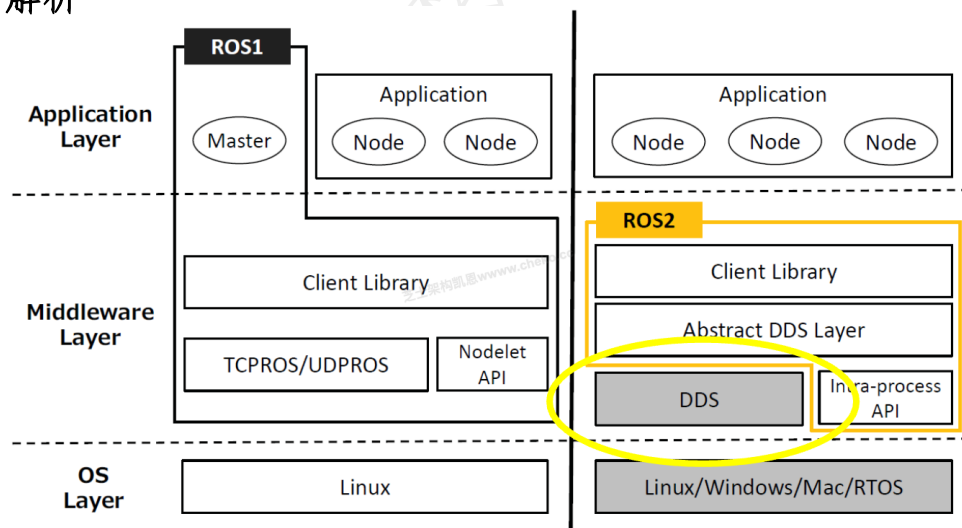
(4) 参数机制(Parameter Server)

题干中提到“共享”、“服务端设置数据”、“客户端连接与操作”。ROS 提供了一个参数服务器,用于存储运行时可共享的配置参数。节点可读写这些参数。特点是**集中式存储、配置共享、适合静态参数**。

问题(3)

立足系统架构,ROS2 可以划分为三层。请解析 ROS2架构每一层含义。

思路解析



ROS2 的系统架构可以清晰划分为三层结构:操作系统层(OS Layer)、中间件层(Middleware Layer)和应用层(Application Layer)。每层功能如下:

操作系统层提供基础运行环境,支持多平台和实时系统;**中间件层**采用 DDS 实现高性能通信,并通过抽象层屏蔽具体实现,支持进程内通信优化;**应用层**包括用户编写的节点和功能包,通过客户端库与中间件交互,实现机器人系统的感知、控制与决策等功能。

答案-问题(1)

(1) ROS(机器人操作系统)是一个开源的机器人软件平台,提供了一系列工具、库和约定,用于

帮助开发者创建机器人应用程序，能实现机器人各模块间的通信与协作。特点是具有分布式架构，可使不同节点协同工作；有丰富的开源库和工具，能减少开发成本与周期；还具备良好的可扩展性和兼容性，方便添加新功能与硬件。

(2) ROS2 在通信机制上更高效，采用数据分发服务 (Data Distribution Service) DDS 提高实时性和可靠性。增强了跨平台支持，可更好适配多种系统。改进了软件架构，让系统更稳定、灵活，且安全性提升。同时，解决了 ROS1 中的一些痛点，如多机通信更便捷等，更适合复杂机器人系统开发。

答案-问题 (2)

(1) 话题通信 / 发布-订阅机制 (2) 服务通信 (3) 动作通信 (4) 参数服务

答案-问题 (3)

ROS2 从系统架构角度可分为三层：通信层、中间层和应用层。通信层基于 DDS 实现节点间的高性能、实时分布式通信；中间层提供如 rclcpp、rclpy 等客户端库，封装底层通信接口，供用户用主流语言开发节点；应用层则包含具体的功能包与节点，用于实现感知、导航、控制等高层逻辑，支撑整个机器人系统的业务运行。

第19章——分布式缓存与高并发
(2020年11月系统架构设计师真题-第4题)

阅读以下关于数据库缓存的叙述，在答题纸上回答问题1至问题3。

[说明]

某互联网文化发展公司因业务发展，需要建立网上社区平台，为用户提供一个对网络文化产品(如互联网小说、电影、漫画等)进行评论、交流的平台。该平台的部分功能如下：

- (a) 用户帖子的评论计数器；
- (b) 支持粉丝列表功能；
- (c) 支持标签管理；
- (d) 支持共同好友功能等；
- (e) 提供排名功能，如当天最热前10名帖子排名、热搜榜前5排名等；
- (f) 用户信息的结构化存储；
- (g) 提供好友信息的发布/订阅功能。

该系统在性能上需要考虑高性能、高并发，以支持大量用户的同时访问。开发团队经过综合考虑，在数据管理上决定采用Redis+数据库(缓存+数据库)的解决方案。

问题 (1)

Redis支持丰富的数据类型，并能够提供一些常见功能需求的解决方案。请选择题干描述的(a)~(g)功能选项，填入表4-1中(1)~(5)的空白处。

表 4-1 Redis 数据类型与业务功能对照表

数据类型	存储的值	可实现的业务功能
STRING	字符串、整数或浮点数	(1)
LIST	列表	(2)
SET	无序集合	(3)
HASH	包括键值对的无序散列表	(4)
ZSET	有序集合	(5)

思路解析

(1) (a) 依据：STRING 类型在 Redis 中可用于存储简单的字符串、整数或浮点数。用户帖子的评论计数器，主要是对整数进行计数操作，利用 STRING 类型的原子自增 (INCR)、自减

(DECR) 等命令, 能方便高效地实现评论数量的统计, 所以 (1) 对应 (a)。

(2) (b)、(g) 对于 (b): LIST 类型是一个双向链表, 可以在链表两端进行插入和删除操作。粉丝列表功能中, 可将粉丝信息按添加顺序依次存储在 LIST 中, 能满足粉丝列表这种具有顺序性的数据存储需求, 便于按顺序遍历粉丝列表等操作, 所以 LIST 可对应 (b)。有的同学说粉丝要算共同好友怎么办, 的确有争议, 选 SET 也可以, 实际还是要看具体的应用场景。针对这题, 题干出现列表, 我们就选列表, 绝对没错。

对于 (g): 好友信息的发布 / 订阅功能, 发布的好友信息可按时间顺序等依次存入 LIST。比如新发布的好友动态可以从链表一端插入, 在订阅查看时能按顺序依次获取, 符合 LIST 类型顺序存储和操作的特性, 因此 (2) 也对应 (g)。

(3) (c)、(d)。对于 (c): SET 类型是无序且不重复的集合。在标签管理功能中, 标签集合不需要考虑顺序, 且每个标签应该是唯一的, 使用 SET 类型可以利用其自动去重特性, 方便地实现标签的存储、添加、删除以及判断某个标签是否存在等操作, 所以 SET 对应 (c)。这里标签用顺序是否可以, 凯恩认为也可以。题干描述不清楚, 留给大家讨论。

对于 (d): 共同好友功能中, 共同好友组成的集合不需要有特定顺序, 且好友个体不能重复, SET 类型的无序去重特性正好适用于存储共同好友集合, 能有效进行共同好友的相关操作, 所以 SET 也对应 (d)。

(4) (f)。依据: HASH 类型是由字段和值组成的键值对集合, 适合存储结构化数据。用户信息包含姓名、年龄、联系方式等多个属性, 以 HASH 类型存储时, 可将用户 ID 作为键, 用户各项属性作为字段和对应的值, 方便对用户信息的各个属性进行独立的增删改查操作, 所以 (4) 对应 (f)。

(5) (e)。依据: ZSET 类型是有序集合, 每个元素都关联一个分数, Redis 会根据分数对元素进行排序。在提供排名功能 (如当天最热前 10 名帖子排名、热搜榜前 5 排名等) 中, 可将帖子或热搜词条作为元素, 将其热度值、搜索次数等作为分数存储在 ZSET 中, 通过对分数的排序就能轻松获取排名情况, 所以 (5) 对应 (e)。

问题 (2)

该网上社区平台需要为用户提供 7×24 小时的不间断服务。同时在系统出现宕机等故障时, 能在最短时间内通过重启等方式重新建立服务。为此, 开发团队选择了 Redis 持久化支持。Redis 有两种持久化方式, 分别是 RDB (Redis DataBase) 持久化方式和 AOF (Append Only File) 持久化方式。开发团队最终选择了 RDB 方式。请用 200 字以内的文字, 从磁盘更新频率、数据安全、数据一致性、重启性能和数据文件大小五个方面比较两种方式, 并简要说明开发团队选择 RDB 的原因。

思路解析

磁盘更新频率: AOF 比 RDB 文件更新频率高。数据安全: "AOF 比 RDB 更安全。数据一致性: RDB 间隔一段时间存储, 可能发生数据丢失和不一致; AOF 通过 append 模式写文件, 即使发生服务器宕机, 也可通过 `redis-check-aof` 工具解决数据一致性问题。重启性能: RDB 性能比 AOF 好。数据文件大小: AOF 文件比 RDB 文件大。

综合上述五个方面的比较, 考虑在系统出现宕机等故障时, 需要在最短时间内通过重启等方式重新建立服务, 因此开发团队最终选择了 RDB 方式。

有的同学提出 AOF 有多种刷新模式, 这里回答不准确, 在 Redis 中, AOF 磁盘刷新方式主要由配置参数 `appendfsync` 控制, 该参数决定了 Redis 将数据刷新 (`fsync`) 到磁盘的频率。下面是 AOF 的三种磁盘刷新方式:

(1) **appendfsync always (总是刷新)**: 在每次写操作后, Redis 都会立即将 AOF 缓冲区的数据写入并同步到磁盘。优点: 提供最高的数据安全性, 确保在系统崩溃时数据不会丢失。缺点: 性能较低, 因为频繁的磁盘 I/O 操作会增加系统开销。

(2) **appendfsync everysec (每秒刷新一次)**: 描述: Redis 每秒钟在后台线程中将 AOF 缓冲区的数据写入并同步到磁盘。优点: 在性能和数据安全性之间提供了良好的平衡。大多数情况下, 即

使系统崩溃，最多只会丢失一秒钟的数据。缺点：在极端情况下（如系统故障或断电），可能会丢失最近一秒钟的数据。

(3) appendfsync no (不主动刷新)：描述：Redis 不主动调用 fsync，而是依赖操作系统自行决定何时将数据从缓冲区刷新到磁盘。优点：性能最佳，因为减少了磁盘 I/O 操作。缺点：数据安全性最低，可能会在系统崩溃时丢失大量数据，因为最后的写操作可能尚未同步到磁盘。

问题 (3)

缓存中存储当前的热点数据，Redis 为每个KEY值都设置了过期时间，以提高缓存命中率。为了清除非热点数据，Redis 选择"定期删除+惰性删除"策略。如果该策略失效，Redis内存使用率会越来越高，一般应采用内存淘汰机制来解决。请用100字以内的文字简要描述该策略的失效场景，并给出三种内存淘汰机制。

思路解析

Redis 默认使用“定期删除+惰性删除”策略。定期删除是后台线程随机抽样过期键并清理，惰性删除则在访问时判断是否过期并删除。但若**抽样未覆盖到过期键，且这些键长期无人访问**，就会出现过期数据残留，内存占用率上升。

此时必须启用**内存淘汰机制 (Eviction Policy)**，例如：①LRU，从最近最少使用的数据中淘汰，保证热点数据留存；②LFU，从访问频率最低的数据中淘汰，更精准地维护活跃数据；③随机淘汰，简单高效，适用于性能优先的场景。通过合理配置淘汰机制，能确保内存使用稳定，避免 OOM。

答案-问题 (1)

- (1) (a)
- (2) (b)、(g)
- (3) (c)、(d)
- (4) (f)
- (5) (e)

答案-问题 (2)

RDB持久化方式和AOF持久化方式在以下面有所不同：

(1) 磁盘更新频率：RDB方式在指定时间间隔内将内存中的数据快照保存到磁盘，而AOF方式则实时记录每次写操作到磁盘。

(2) 数据安全：AOF方式通过追加日志的方式记录每次写操作，可以更好地保证数据的安全性，但可能会增加磁盘IO负担。

(3) 数据一致性：RDB方式在快照时刻保存数据，可能会导致部分数据丢失，而AOF方式记录每次写操作，数据更加完整和一致。

(4) 重启性能：RDB方式在重启时加载快照文件，速度较快；而AOF方式需要重新执行日志文件中的写操作，重启时间可能较长。

(5) 数据文件大小：AOF方式通常比RDB方式的数据文件更大，因为AOF记录了每次写操作。

开发团队选择RDB方式可能是因为它在重启性能方面更优，能够在最短时间内重新建立服务，符合要求的7x24小时不间断服务需求。同时，RDB方式也相对简单，易于管理和维护

答案-问题 (3)

Redis 的过期键 (Expired Keys) 是自动删除的，Redis 会在每个键的过期时间秒数到达时检查该键是否过期，如果是，它会自动从 Redis 中删除该键。Redis 内部通过使用时间轮算法和惰性删除方法来实现过期键的删除。具体来说，过期键的定期删除策略由redis.c/activeExpireCycle函数实现，每当Redis的服务器周期性操作redis.c/serverCron函数执行时，activeExpireCycle函数就会被调用，它在规定的时间内，分多次遍历服务器中的各个数据库，从数据库的expires字典中随机检

查一部分键的过期时间，并删除其中的过期键。然后，Redis 会启动一个后台线程，使用惰性删除方法来删除已过期的键。惰性删除指的是，当需要访问一个键时，Redis 判断该键是否过期，如果过期就立即删除。

失效场景：是指"定期删除"没删除KEY，"惰性删除"也没生效。因为大量过期的key在定期删除策略下没有及时删除 + 外部在这段时间内没有要查询这个key的动作，导致无法做惰性删除，所以还占用着内存。对此，可采用内存淘汰机制解决：（1）从已设置过期时间的数据集最近最少使用的数据淘汰。（2）从已设置过期时间的数据集将要过期的数据淘汰。（3）从已设置过期时间的数据集任意选择数据淘汰。（4）从数据集最近最少使用的数据淘汰。（5）从数据集任意选择数据淘汰。

(2019年11月系统架构设计师真题-第4题)

阅读以下关于分布式数据库缓存设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某初创企业的主营业务是为用户提供高度个性化的商品订购业务，其业务系统支持PC端、手机App等多种访问方式。系统上线后受到用户普遍欢迎，在线用户数和订单数量迅速增长，原有的关系数据库服务器不能满足高速并发的业务要求。

为了减轻数据库服务器的压力，该企业采用了分布式缓存系统，将应用系统经常使用的数据放置在内存，降低对数据库服务器的查询请求，提高了系统性能。在使用缓存系统的过程中，企业碰到了一系列技术问题。

问题（1）

该系统使用过程中，由于同样的数据分别存在于数据库和缓存系统中，必然会造成数据同步或数据不一致性的问题。该企业团队为解决这个问题，提出了如下解决思路：应用程序读数据时，首先读缓存，当该数据不在缓存时，再读取数据库；应用程序写数据时，先写缓存，成功后再写数据库；或者先写数据库，再写缓存。

王工认为该解决思路并未解决数据同步或数据不一致性的问题，请用100字以内的文字解释其原因。

王工给出了一种可以解决该问题的数据读写步骤如下：

读数据操作的基本步骤：

1. 根据key读缓存；
2. 读取成功则直接返回；
3. 若key不在缓存中时，根据key (a)；
4. 读取成功后，(b)；
5. 成功返回。

写数据操作的基本步骤：

1. 根据key值写(c)；
2. 成功后(d)；
3. 成功返回。

请填写完善上述步骤中(a)~(d)处的空白内容。

思路解析

在原有方案中，应用程序写数据时，先写缓存，成功后再写数据库；或者先写数据库，再写缓存。这里存在双写不一致问题。不管先写缓存还是数据库，都会存在一方写成功，另一方写失败的问题，从而造成数据不一致。当多个请求发生时，也可能产生读写冲突的并发问题。

王工的解决思路是（同样是旁路模式，但是写操作的细节进行了改进，变成了删除缓存）：读操作的顺序是，先读缓存，如果数据在缓存中，则直接返回，无须数据库操作；如果数据不在缓存，则读数据库，如成功，则更新缓存，如失败，则返回无此数据。

读操作主要解决查询效率问题。写操作的顺序是先写数据库，如失败，则返回失败；如成功，则更

新缓存。

Cache-Aside Pattern 更新缓存的方式有：如缓存中无此key值，则在缓存中不作处理；如缓存中存在此key值，则删除key值或使该key值失效。写操作的顺序主要防止数据库写操作失败，缓存更新为内存操作，失败的概率很小。同时删除key或使key失效，则在下一次查询该key值时，会发起数据库读操作，并同步更新缓存中的key值，从而最大程度上避免双写不一致问题。

问题（2）

缓存系统一般以key/value形式存储数据，在系统运维中发现，部分针对缓存的查询，未在缓存系统中找到对应的key，从而引发了大量对数据库服务器的查询请求，最严重时甚至导致了数据库服务器的宕机。

经过运维人员的深入分析，发现存在两种情况：

- （1）用户请求的key值在系统中不存在时，会查询数据库系统，加大了数据库服务器的压力；
- （2）系统运行期间，发生了黑客攻击，以大量系统不存在的随机key发起了查询请求，从而导致了数据库服务器的宕机。经过研究，研发团队决定，当在数据库中也未查找到该key时，在缓存系统中为key设置空值，防止对数据库服务器发起重复查询。

请用100字以内文字说明该设置空值方案存在的问题，并给出解决思路。

思路解析

研发团队采用“为空值设置缓存”的方式，防止对数据库的重复查询，但这一方案存在隐患：系统不存在的key值理论上无限的，如果黑客或恶意用户提交大量随机key，每个key都会被写入缓存为空值，最终导致缓存内存被迅速耗尽，从而影响正常业务请求，严重时甚至会拖垮缓存系统本身。

解决该问题的核心思路是在查询缓存之前过滤掉无效key。常见方法是布隆过滤器：它基于位数组和多哈希函数，能够高效判断某个key是否可能存在于系统中，如果布隆过滤器判断key不存在，则无需访问数据库和缓存，从源头阻断恶意请求。此外，还可采用字典表或位图记录已存在key的范围和规则。这样，系统只对合理key发起查询，从而避免无效key导致缓存膨胀，保证整体稳定性和性能。

问题（3）

缓存系统中的key一般会存在有效期，超过有效期则key失效；有时也会根据LRU算法将某些key移出内存。当应用软件查询key时，如key失效或不在内存，会重新读取数据库，并更新缓存中的key。

运维团队发现在某些情况下，若大量的key设置了相同的失效时间，导致缓存在同一时刻众多key同时失效，或者瞬间产生对缓存系统不存在key的大量访问，或者缓存系统重启等原因，都会造成数据库服务器请求瞬时爆量，引起大量缓存更新操作，导致整个系统性能急剧下降，进而造成整个系统崩溃。请用100字以内文字，给出解决该问题的两种不同思路。

思路解析

当大量缓存key设置了相同过期时间，或者缓存系统重启导致key大量失效，会出现“缓存雪崩”现象：瞬间有海量请求绕过缓存直击数据库，导致数据库QPS暴增，甚至直接宕机。

本题中的场景就是典型的缓存雪崩问题。其原因在于缓存失效集中触发数据库访问，加上并发请求数大，数据库无法承受瞬时流量。

解决思路主要有三类：①串行化更新，通过加锁或排队控制同一key的并发写请求，避免重复写数据库刷新缓存；②过期时间随机化，为不同key设置不同或带随机偏移的TTL，使缓存过期时间分布更均匀，减少同一时刻的大规模失效；③多级缓存，在应用节点本地维护L1缓存，分布式缓存作为L2，双层缓存共同抵御突发流量，即使部分失效，也能通过局部缓存抵消部分请求压力。结合实际业务特点，往往需要综合使用，例如在热点数据上启用多级缓存与锁机制，在长尾数据上采用过期时间分散策略，从而整体上避免缓存雪崩，保障系统性能与可用性。

答案-问题（1）

原方案采用“写缓存再写数据库”或“写数据库再写缓存”，看似能保证缓存与数据库的数据一致，但实际上存在**双写不一致问题**。

例如，先写缓存后写数据库时，如果缓存写成功而数据库写失败，会导致数据不一致；反之，若先写数据库再写缓存，缓存更新失败时也会造成矛盾。

此外，当多个请求同时发生时，可能出现读写冲突，即一个请求刚写数据库但缓存未更新，另一个请求却从缓存读到旧数据，造成脏读。这些问题说明，双写方案并不能彻底解决一致性。

王工的改进方案采用**缓存淘汰机制**（Cache-Aside Pattern）。

读操作：优先从缓存读，若未命中，则从数据库读并更新缓存；写操作：先写数据库，再删除或使缓存失效。

这种方式的优势在于避免了缓存与数据库的双写，只需保证数据库正确写入即可。缓存失效后，下一次读请求会触发数据库读取并刷新缓存，从而实现最终一致性。

由于删除缓存的操作是轻量级内存操作，失败概率极低；即便出现偶发不一致，也会在下一次读取时自动修复。

因此，该方案在保证高性能的同时，极大地降低了缓存与数据库不一致的风险，成为业界常见的实践。

- （a）从数据库中读取数据或读数据库
- （b）更新缓存中key值或更新缓存
- （c）数据库
- （d）删除缓存key或使缓存key失效或更新缓存（key值）

答案-问题（2）

存在问题：不在系统中的key值是无限的，如果均设置key值为空，会造成内存资源的极大浪费，引起性能急剧下降。解决思路：查询缓存之前，对key值进行过滤，只允许系统中存在的key进行后续操作（例如采用布隆过滤器进行过滤）。

答案-问题（3）

思路 1：缓存失效后，通过加排它锁或者队列方式控制数据库写缓存的线程数量，使得缓存更新串行化；

思路 2：给不同key设置随机或不同的失效时间，使失效时间的分布尽量均匀；

思路 3：设置两级或多级缓存，避免访问数据库服务器。

（2018年11月系统架构设计师真题-第4题）

阅读以下关于分布式数据库缓存设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某企业是为城市高端用户提供高品质蔬菜生鲜服务的初创企业，创业初期为快速开展业务，该企业采用轻量型的开发架构（脚本语言+关系型数据库）研制了一套业务系统。业务开展后受到用户普遍欢迎，用户数和业务数量迅速增长，原有的数据库服务器已不能满足高度并发的业务要求。为此，该企业成立了专门的研发团队来解决该问题。

张工建议重新开发整个系统，采用新的服务器和数据架构，解决当前问题的同时为日后的扩展提供支持。但是，李工认为张工的方案开发周期过长，投入过大，当前应该在改动尽量小的前提下解决该问题。李工认为访问量很大的只是部分数据，建议采用缓存工具MemCache来减轻数据库服务器的压力，这样开发量小，开发周期短，比较适合初创公司，同时将来也可以通过集群进行扩展。然而，刘工又认为李工的方案中存在数据可靠性和一致性问题，在宕机时容易丢失交易数据，建议采用Redis来解决问题。经过充分讨论，该公司最终决定采用刘工的方案。

问题（1）

在李工和刘工的方案中，均采用分布式数据库缓存技术来解决问题。请用100字以内的文字说明分布式数据库缓存的基本概念。

表4-1中对MemCache和Redis两种工具的优缺点进行了比较，请补充完善表 4-1中的空（1）~（6）。

表 4-1 MemCache 与 Redis 能力比较

	Memcache	Redis
数据类型	简单 key/value 结构	(1)
持久性	(2)	支持
分布式存储	(3)	多种但是，主从、Sentinel\Cluster 等
多线程支持	支持	(4)
内存管理	(5)	无
事务支持	(6)	有限支持

思路解析

分布式缓存的核心思想是利用“内存 + 分布式”的架构手段解决高并发下的数据库性能瓶颈。数据库作为系统的权威存储，常常面临读写请求的洪峰，而其中大部分请求集中在少量热点数据（如热门商品、用户会话）。通过缓存，将这些热点数据存放到内存中，让应用优先读取缓存，减少数据库访问频率，从而极大地提升系统性能。分布式部署则通过分片、复制和一致性哈希等机制，把缓存负载分散到多台节点，实现容量扩展和高可用。

Redis 与 MemCache 的差异主要体现在：

- ①**数据结构**，MemCache 仅支持简单的 key-value，而 Redis 支持 string、list、hash、set、zset 等复杂结构，更适合处理多样化业务场景；
- ②**持久化**，MemCache 不支持持久化，掉电数据全丢，而 Redis 提供 RDB、AOF 等机制保障数据可靠性；
- ③**分片机制**，MemCache 一般依赖客户端一致性哈希，而 Redis 提供 Cluster 模式实现服务端分片；
- ④**内存管理**，Redis 的早期版本并不支持高级的内存管理功能，但从Redis 3.2 版本开始引入了内存淘汰策略等机制；
- ⑤**事务支持**，Redis 提供事务功能，而 MemCache 不支持。正因如此，Redis 常被用于对可靠性、一致性有要求的场景，而 MemCache 更适合纯缓存、读多写少的轻量级应用。

问题（2）

刘工认为李工的方案存在数据可靠性和一致性的问题，请用100字以内的文字解释说明。为避免数据可靠性和一致性的问题，刘工的方案采用Redis作为数据库缓存，请用200字以内的文字说明基本的Redis与原有关系数据库的数据同步方案。

思路解析

李工的方案基于 MemCache，虽然开发量小，但存在严重缺陷。

第一，**可靠性差**：MemCache 所有数据存于内存，不支持持久化，服务器宕机或重启后缓存数据完全丢失，业务可能出现数据丢失或重新加载数据库的性能抖动。

第二，**一致性差**：MemCache 不支持事务，多个并发请求可能导致缓存与数据库内容不一致，尤其在电商下单、支付场景中风险较大。这些问题使得李工方案无法满足核心业务的可靠性需求。

Redis 的方案则通过“缓存 + 数据库双存储”的方式实现优化。典型模式是 **Cache-Aside（旁路缓存）**：

- 读操作**：应用程序先访问 Redis，如果命中则直接返回；若未命中，则访问数据库，取回数据后写入 Redis，以供后续使用。
- 写操作**：先更新数据库，成功后再删除或更新 Redis 缓存，确保数据库作为权威数据源，避免双

写不一致。

此外，Redis 通过持久化机制（RDB 快照、AOF 日志）提升数据可靠性，还能结合主从复制、哨兵模式实现高可用。这样即能兼顾高性能，又能保障交易数据安全，解决 MemCache 的短板。

问题（3）

请用300字以内的文字，说明Redis分布式存储的两种常见方案，并解释说明Redis集群切片的几种常见方式。

思路解析

Redis 单机虽高效，但受限于内存和并发能力，必须通过分布式扩展支撑高并发业务。三种常见方案分别针对不同场景：

主从模式主要解决读写分离和数据备份，但主库宕机时需人工切换；

哨兵模式在主从基础上增加自动化监控与故障转移，提升高可用性；

集群模式则通过内置的 slot 分片机制，支持多主多从、自动负载均衡，是互联网企业应对大规模业务的主流方案。

在 **数据分片方式**上，**客户端分片**逻辑简单但耦合度高，需要客户端维护分片策略；**中间件分片**通过代理层屏蔽复杂性，应用透明，但会引入转发开销；**Redis Cluster**采用 16384 个 slot，服务端负责管理 slot 与节点映射，客户端根据 key 的 CRC16 计算 slot 并路由到对应节点，若节点迁移则通过 MOVED/ASK 重定向实现动态调整。

答案-问题（1）

分布式数据库缓存指的是在高并发环境下，为了减轻数据库压力和提高系统响应时间，在数据库系统和应用系统之间增加的独立缓存系统。

（1）key/value, list, set, string, sorted 或 丰富/多种数据结构

（2）不支持

（3）客户端哈希分片/一致性哈希

（4）Redis 5.0 及以前版本不支持

（5）有，私有内存池

（6）不支持

答案-问题（2）

（1）MemCache 没有持久化功能，所以掉电数据会全部丢失，而且无法直接恢复，这存在可靠性问题。

（2）MemCache 不支持事务，所以操作过程中可能产生数据的不一致性。

同步方案：读取数据时，先读取 Redis 中的数据，如果 Redis 没有，则从原数据库中读取，并同步更新 Redis 中的数据。写回时，写入到原数据库中，并同步更新至 Redis 中。

答案-问题（3）

Redis 分布式存储的常见方案：主从模式（Master/Slave）、哨兵模式（Sentinel）、集群模式（Cluster）

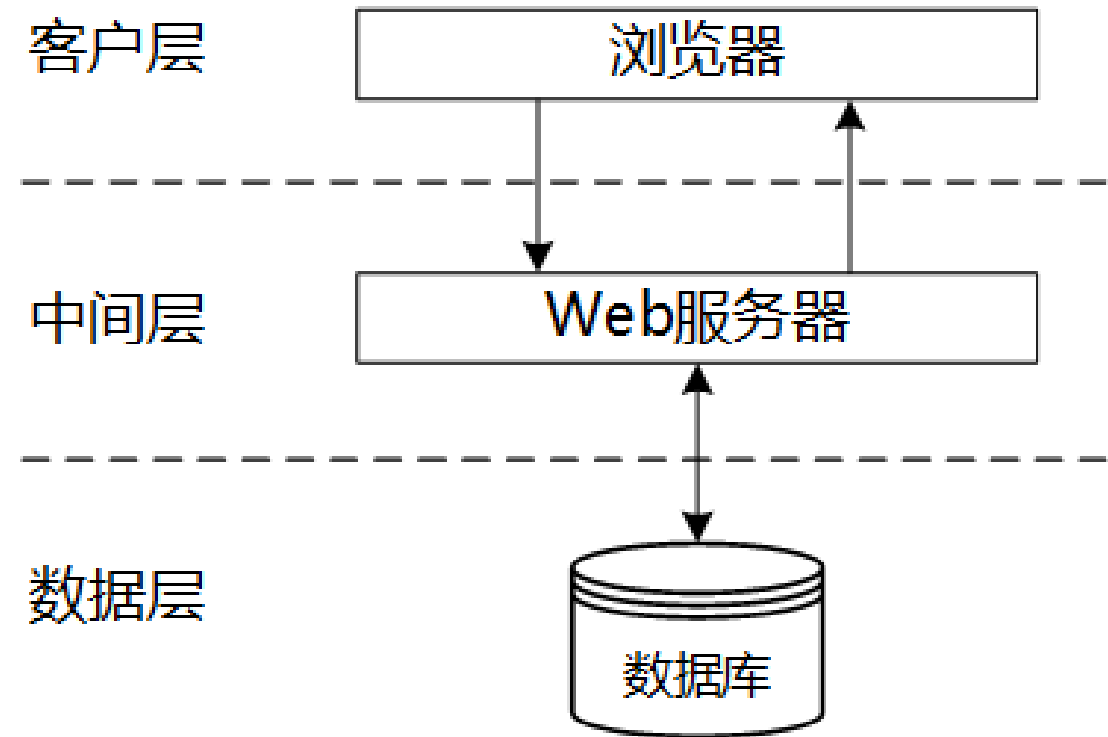
Redis 集群切片的常见方式：（1）客户端分片，即在客户端就通过 key 的 hash 值对应到不同的服务器。（2）中间件实现分片。在应用软件和 Redis 中间，例如：Twemproxy、Codis 等，由中间件实现服务到后台 Redis 节点的路由分派。（3）客户端服务端协作分片。Redis Cluster 模式，客户端可采用一致性哈希，服务端提供错误节点的重定向服务 slot 上。不同的 slot 对应到不同服务器。

（2014年11月系统架构设计师真题-第5题）

请详细阅读有关Web应用架构设计方面的说明，在答题纸上回答问题1至问题3。

【说明】

某软件公司开发运维了一个社交网站系统，该系统基于开源软件平台 LAMP（Linux+Apache+MySQL+PHP）构建，运行一段时间以来，随着用户数量及访问量的增加，系统在Web服务器负载、磁盘I/O等方面出现了明显瓶颈，已不能满足大量客户端并发访问的要求，因此公司成立了专门的项目组，拟对系统架构进行调整以提高系统并发处理能力。目前系统采用了传统的三层结构，系统架构如图5-1所示。



问题（1）

针对目前出现的Web服务器负载过大问题，项目组决定在客户端与中间层Web服务器之间引入负载均衡器，通过中间层Web服务器集群来提高Web请求的并发处理能力。在讨论拟采用的负载均衡机制时，王工提出采用基于DNS的负载均衡机制，而李工则认为应采用基于反向代理的负载均衡机制，项目组经过讨论，最终确定采用李工提出的方案。请用200字以内的文字，分别简要说明两个机制的基本原理，并从系统执行效率、安全性及简易性等方面将两种机制进行对比，将对比结果填入表5-1中。

表 5-1 两种负载均衡机制对比分析表

特性		基于 DNS 的负载均衡	基于反向代理 的负载均衡
系统执行效率	是否考虑内容服务器性能差异及实时负载情况	(1)	(2)
	是否可对内部服务器静态资源进行缓存	(3)	(4)
安全性	是否能屏蔽客户端对真实 Web 服务器的直接访问	(5)	(6)
简易性	是否具有实现简单、容易实施及低成本的特性	(7)	(8)

注：请在表格（1）~（8）处填入“是”或“否”

思路解析

DNS负载均衡的原理是利用DNS服务器为一个域名配置多个IP地址，在解析时通过轮询或简单策略将不同的请求分配给不同的服务器节点。这种方式部署简单，几乎不需要额外的硬件或软件支持，适合早期阶段的中小型网站。但是它的缺点也十分明显：其调度策略非常有限，通常只能基于轮询，无法感知后端服务器的实时运行状态。例如当某个节点宕机时，DNS服务器依旧可能将请求分配到该节点，导致客户端访问失败。此外，由于DNS解析结果会缓存于本地和各级DNS服务器，更新延迟较大，不能快速响应负载变化。另外，DNS方式要求每个节点都有公网IP，浪费资源且降低安全性。**反向代理负载均衡**则通过在Web服务器集群前增加一个代理层，所有请求首先到达代理，再由代理根

据后端节点的实际负载情况进行智能调度。例如可以基于当前连接数、CPU利用率等指标选择最优的节点。这不仅提升了调度的灵活性和执行效率，还能屏蔽后端节点的真实地址，从而增强安全性，防止直接攻击后端服务器。同时反向代理服务器本身还可以结合缓存机制，例如缓存静态资源，进一步提高性能。不过，它的缺点是部署和维护成本较高，当并发量极大时代理服务器本身可能成为瓶颈。

DNS负载均衡适合小规模、低成本场景，但在高并发和高安全需求下，反向代理机制更优。因此，项目组选择了李工提出的反向代理方案。

问题（2）

针对并发数据库访问所带来的磁盘I/O瓶颈问题，项目组决定在数据层引入数据库扩展机制。经过调研得知系统数据库中存储的主要数据为以用户标识为索引的社交网络数据，且系统运行时发生的大部分数据库操作为查询操作。经过讨论，项目组决定引入数据库分区和MySQL主从复制两种扩展机制。数据库分区可采用水平分区和垂直分区两种方式，请用350字以内的文字说明在本系统中应采用哪种方式及其原因，并分析引入主从复制机制给系统带来的好处。

思路解析

数据库扩展常用方法包括 **分区（Partitioning）** 和 **主从复制（Replication）**。

在本案例中，数据库中存储的数据是以用户标识为索引的社交网络数据，访问模式主要是“按用户查询”。此类业务特点决定了最合适的分区方式是 **水平分区**。

水平分区的原理是根据某个分区键（例如用户ID范围或哈希值），将一张大表拆分成多张子表（或存放在不同数据库节点上）。例如用户ID为11000000的数据存储在分区1，10000012000000的数据存储在分区2。这样，每次查询只需访问特定的分区，避免了全表扫描和单表过大的索引性能问题，极大提高了查询效率。而**垂直分区**则是根据列进行拆分，例如把用户基本信息和好友关系分开存储。但在本案例中，所有数据都是基于用户ID进行查询，使用垂直分区反而会增加联合查询的复杂度，因此不合适。

主从复制机制进一步提升了系统性能与可靠性。MySQL主从复制的典型模式是：主库处理所有写操作，同时通过binlog日志将更新操作同步到多个从库。从库则主要承担查询操作。由于本系统绝大部分操作是查询，因此读写分离非常适用：主库压力显著减轻，从库并行响应大量查询，提升吞吐量。另一方面，从库可以作为冗余备份，当某个数据库节点故障时，系统依旧能通过其他副本提供服务，显著提升了可用性和容错性。此外，该机制还有助于系统水平扩展，当访问量增加时，可以通过增加更多的从库来满足需求。

在本系统中采用水平分区+主从复制的组合，是提升数据库性能与可靠性的最佳选择。

问题（3）

为进一步提高数据库访问效率，项目组决定在中间层与数据层之间引入缓存机制。赵工开始提出可直接使用MySQL的查询缓存（query cache）机制，但项目组经过分析好友动态显示等典型业务的操作需求，同时考虑已引入的数据库扩展机制，认为查询缓存尚不能很好地提升系统的查询操作效率，项目组最终决定在中间层与数据层之间引入Memcached分布式缓存机制。

（a）请补充下述关于引入Memcached后系统访问数据库的基本过程：系统需要读取后台数据时，先检查数据是否存在于（1）中，若存在则直接从其中读取，若不存在则从（2）中读取并保存在（3）中；当（4）中数据发生更新时，需要将更新后的内容同步到（5）实例中。（备选答案：数据库、Memcached缓存）

（b）请结合已知信息从缓存架构、缓存有效性及缓存数据类型等方面分析使用Memcached代替数据库查询缓存的原因。

思路解析

在Web系统架构优化中，**缓存机制**是缓解数据库压力、提升响应速度的关键措施。本案例中，项目组

在中间层与数据库之间引入了Memcached分布式缓存，而不是依赖MySQL自带的查询缓存。

首先来看缓存的基本访问流程：当系统需要读取数据时，先检查Memcached缓存中是否已有数据，如果有则直接返回，避免访问数据库；如果没有，则访问数据库获取，并将结果写入缓存，供下次快速访问。当数据库中的数据发生更新时，必须同步更新缓存中的副本，确保数据一致性。这个流程显著减少了数据库的直接访问次数，从而降低了磁盘I/O压力。

对比MySQL查询缓存与Memcached：

缓存架构方面：MySQL查询缓存属于数据库内部机制，只在单个数据库实例内生效，无法跨节点共享；而Memcached是独立的分布式缓存服务，可以部署在多台服务器上，具备良好的扩展性，非常适合互联网级高并发场景。

缓存有效性方面：MySQL查询缓存一旦表发生更新，与该表相关的所有缓存查询结果都会被清空，导致缓存命中率低；而Memcached采用灵活的过期时间与手动更新机制，可以根据业务特性精细化地管理缓存数据，使缓存利用率更高。

缓存数据类型方面：MySQL查询缓存仅缓存SQL语句的结果集，局限性大，不能满足复杂业务；Memcached以键值对方式存储数据，可以缓存对象、JSON、序列化数据等，特别适合社交网络中复杂多样的数据访问场景，如好友动态流、用户信息缓存等。

Memcached能够为本系统提供更高效、更灵活、更可扩展的缓存服务，在性能和适用性上远优于MySQL的查询缓存。

答案-问题（1）

基于DNS的负载均衡是在DNS服务器中为同一个主机名配置多个IP地址，在应答DNS查询时，DNS服务器对每个查询将以DNS文件中主机记录的IP地址按顺序返回不同的解析结果，将客户端的访问引导到不同的节点上去，使得不同的客户端访问不同的节点，从而达到负载均衡的目的。

反向代理负载均衡。反向代理负载均衡是将来自Internet上的连接请求以反向代理的方式动态地转发给内部网络上的多个节点进行处理，从而达到负载均衡的目的。

- (1) 否
- (2) 是
- (3) 否
- (4) 是
- (5) 否
- (6) 是
- (7) 是
- (8) 否

答案-问题（2）

本系统应采用水平分区，因为社交网络数据库的数据表记录数量非常庞大，而且记录的访问，大多集中于本地区域，所以水平分区能极大提高处理效率。

主从复制机制使得同样的数据，存在多个副本，这样让用户查询数据时，可以选择该数据最近的副本进行访问，提高效率，降低资源使用时的冲突。另一方面主从复制机制中一台数据库服务器故障不会导致站点无法访问。

答案-问题（3）

- (1) Memcached缓存
- (2) 数据库
- (3) Memcached缓存
- (4) 数据库
- (5) Memcached缓存

Memcached相比数据库查询缓存：缓存架构：数据库查询缓存只是将查询结果进行缓存，适用面很

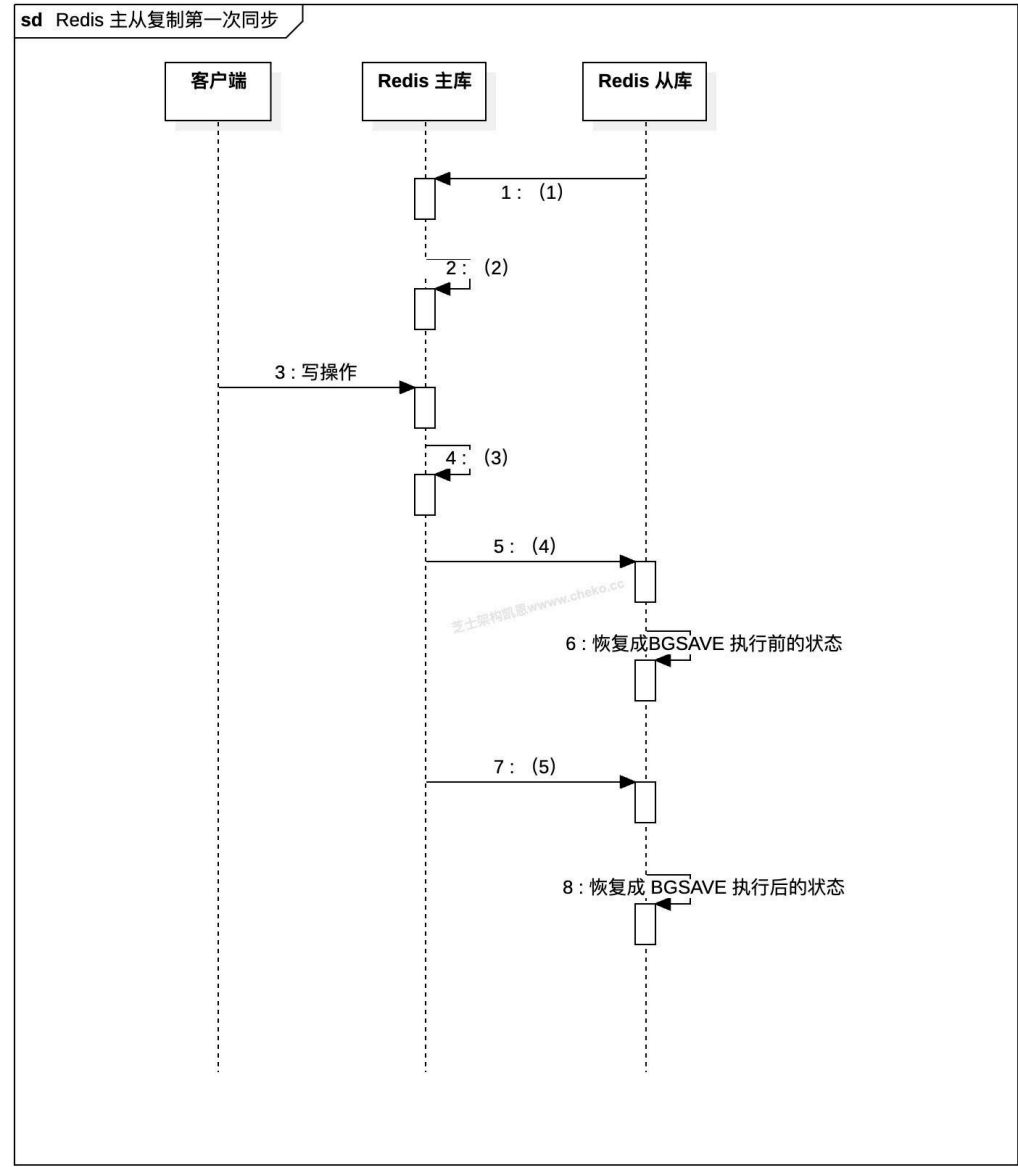
窄，而Memcached缓存是将数据库中的表进行缓存，对于在这些表之上的操作均可适用。缓存有效性：Memcached缓存时效较长，只要未更新，就属于有效状态，而数据库查询缓存时效较短（具体时效与配置有关），所以在此方面Memcached缓存有优势。缓存数据类型：Memcached缓存数据为键值对，而数据库查询缓存为元组。

(2025年5月系统架构设计师真题-第3题)

某互联网企业开发并部署了一套高并发电商系统，系统中的商品信息、用户购物车、热销榜单等高频数据均存储在 Redis 中以提高访问效率。随着业务规模扩大，单实例 Redis 出现了性能瓶颈和单点故障风险。为提升系统的可用性、扩展性和数据容灾能力，技术团队决定采用 Redis 的主从复制架构，将写操作集中在主节点（Master），将读操作分散到多个从节点（Slave）中。同时，系统还需支持主从自动同步机制，确保在主库故障后可以快速切换至从库，保障业务连续性。

问题（1）

请根据 Redis 的主从架构原理和下图的同步流程，在图示中填入正确的关键步骤和指令。



思路解析

当 Redis 从库（Slave）第一次与主库（Master）建立复制关系时，会进行一次完整的数据同步，这一过程称为“全量同步”。其主要流程如下：

从库发起同步请求。从库在启动并配置主库地址后，主动向主库发送 PSYNC ? -1 命令（老版本使用 SYNC），表明自己是第一次同步，请求主库发送完整的数据副本。

主库执行 BGSAVE 生成快照。主库接收到同步请求后，会启动一个后台子进程执行 BGSAVE 命令，生成当前数据库的 RDB 快照文件（即数据库当前状态的完整镜像）。此时，主库仍然可以继续响应客户端的写入请求。

写操作被追加到复制缓冲区。由于 BGSAVE 执行期间主库仍然对外提供服务，客户端可能继续写入数据。为了不丢失这部分写操作，主库会将这些命令追加到复制积压缓冲区（replication backlog）中，准备后续发送给从库。

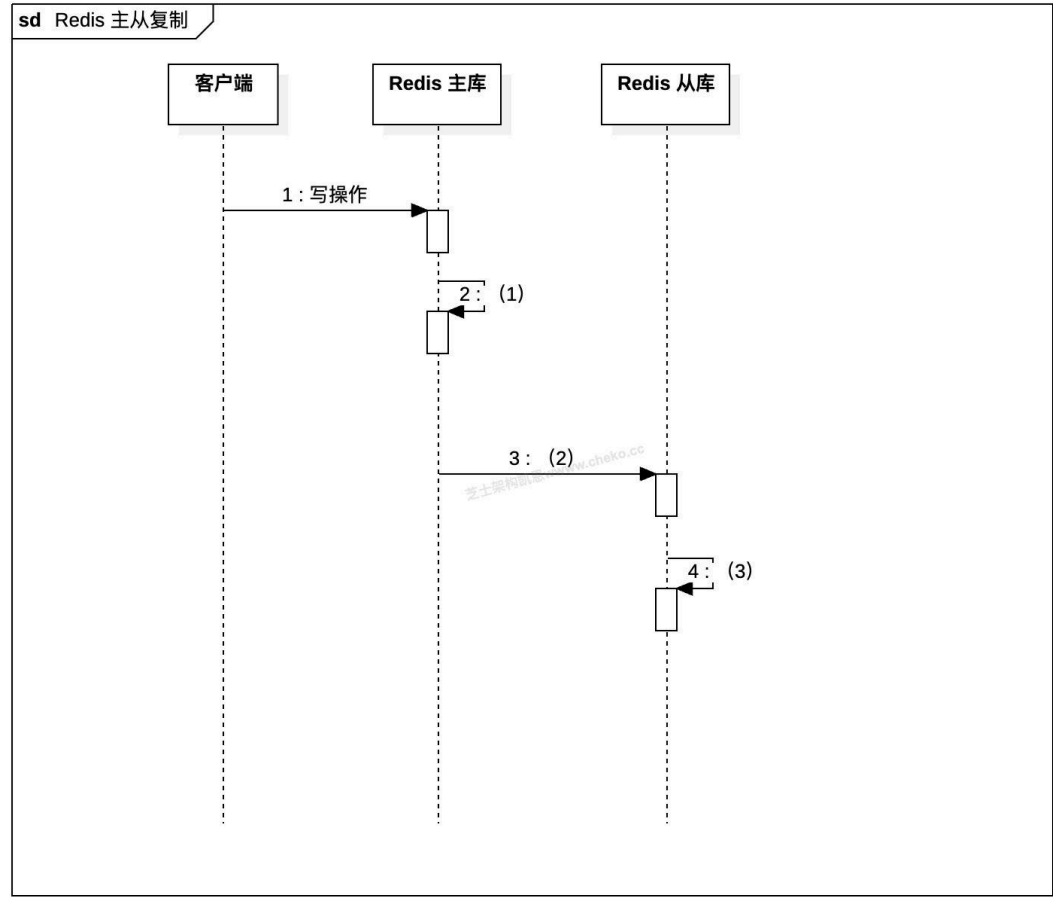
主库发送 RDB 快照给从库。当 BGSAVE 完成后，主库将生成的 RDB 文件通过网络发送给从库。从库收到文件后会清空原有数据，并加载这份 RDB 快照，数据状态回滚至主库执行 BGSAVE 时的快照状态。

主库发送增量命令流。RDB 加载完成后，从库继续接收主库发送的 backlog 缓冲区中的写命令。这些命令是在 BGSAVE 执行期间产生的，用于追平主从之间的数据差异。

同步完成。从库执行完 backlog 中的所有写操作后，主从数据状态保持一致。此时主从之间进入“命令传播阶段”，即主库每次收到写命令时，都会实时同步给从库，从库保持实时追随主库的数据变更。

问题（2）

主从库第一次同步完成之后，后面是如何同步的，请在图示中填入正确的关键步骤和指令。



思路解析

在 Redis 主从架构中，主从库之间的数据一致性通过“命令传播”的机制持续保持。

当客户端向主库发送写操作时，主库不仅会在本地执行这些命令、更新内存，还会将这些写操作记录到两个地方：一个是用于持久化的 AOF 文件，另一个是复制积压缓冲区（backlog buffer），这是主库内存中专门用于存储最近一段命令历史的数据结构。如果从库网络中断后重连，可以通过

backlog 中的内容快速补齐，避免重新做全量同步。

接下来，主库还会把这些写命令放入每个从库对应的复制缓冲区（replication buffer）中，通过网络将命令推送给从库。从库收到后会逐条执行，确保数据状态与主库保持一致。整个过程从库则紧跟其后，不断执行主库发来的操作指令。

问题（3）

Redis 的数据持久化机制在生产环境中至关重要，当系统遭遇故障时需依赖持久化数据进行快速恢复。请详细阐述 Redis 的两种主要持久化方式，并对比分析其优缺点。

思路解析

Redis 主要有两种数据持久化方式：RDB（快照持久化）和 AOF（追加日志）。RDB 通过定期将内存中的数据生成快照保存为二进制文件，优点是文件体积小、恢复速度快，适用于启动速度要求高或需要定时备份的系统场景。例如在电商平台中，凌晨业务低谷期可定时生成 RDB 快照，作为整站数据的每日备份。但由于其是间隔性保存，一旦系统在两次快照之间发生异常宕机，可能会造成数分钟的数据丢失，如用户刚刚下单的数据未被记录。

AOF 则是将每一次写命令以日志形式追加写入磁盘，可实现几乎实时的持久化，数据丢失极少，适用于数据精度要求高的业务场景，如金融系统中的转账、扣款等操作，哪怕丢失一笔都可能造成严重后果。但 AOF 文件体积相对更大，恢复时间更长，并且长时间运行后需执行日志重写以压缩文件体积。实际生产中，为了兼顾性能和数据安全，通常推荐开启 RDB + AOF 混合持久化模式，以实现恢复速度与数据完整性的双重保障。

答案-问题（1）

- （1）从库向主库发送 PSYNC ? -1 或 SYNC 请求，表示首次同步请求
- （2）BGSAVE 命令，主库收到请求后执行 BGSAVE，生成 RDB 快照
- （3）主库将写操作追加到 backlog，用于稍后的增量复制
- （4）主库将 RDB 文件通过 socket 传输给从库（传输全量快照）
- （5）主库将 backlog 中积压的写命令（增量）发送给从库

答案-问题（2）

- （1）主库将写命令写入 AOF 日志和复制积压缓冲区（replication backlog buffer）
- （2）主库将写命令通过 replication buffer（复制缓冲区）发送给从库
- （3）从库执行主库发来的命令，应用到本地数据

答案-问题（3）

Redis 主要有两种数据持久化方式：RDB（快照持久化）和 AOF（追加日志）。

RDB 是通过定期生成内存快照的方式将数据保存为二进制文件，优点是文件体积小、恢复速度快，适合用于灾备和冷启动场景，但它是定时保存，可能在故障时丢失最近修改的数据。

AOF 则是将每一次写命令以日志形式追加记录到磁盘，优点是数据丢失更少、可读性高，但缺点是文件更大、恢复速度相对慢，且长期运行后需要重写以控制体积。

（2025年11月系统架构设计师真题-第3题）

某互联网电商平台采用 MySQL + Redis 的缓存架构。核心接口为“查询商品详情”，日常峰值 QPS 达到几万。系统目前使用 Cache-Aside 模式：先查 Redis 缓存，命中则直接返回；未命中时访问数据库，并把结果写回 Redis，设置过期时间（TTL）。

近期运维发现：某些热点商品在缓存失效瞬间，会出现大量请求同时击穿缓存、直接访问数据库，导致数据库连接数飙升、接口响应超时，影响系统整体可用性。架构组组织评审，提出了两种优

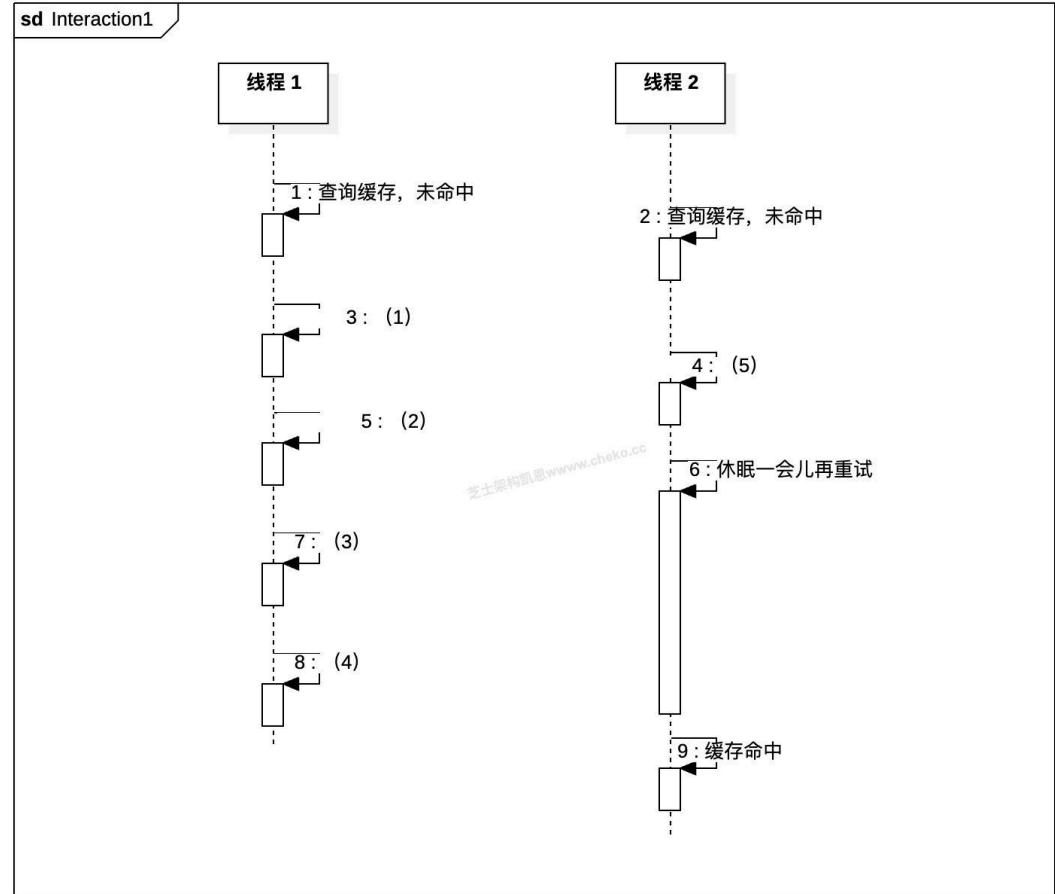
化思路：

方案一（王工）：基于互斥锁的物理过期方案。仍然依赖 Redis TTL，缓存正常按过期时间删除。当缓存未命中时，只有拿到互斥锁的线程才允许去数据库重建缓存，其它线程等待或重试。

方案二（李工）：逻辑过期方案（不依赖互斥锁做强同步，可返回过期数据）：Redis 中存储“业务数据 + 逻辑过期时间字段”，可以不设置或设置很长 TTL。访问时若发现逻辑过期，可以先返回旧数据，同时由获取到非互斥锁的线程或后台异步线程去数据库重建缓存。

问题（1）

请根据王工“基于互斥锁的缓存查询过程”（物理过期方案）的主要处理步骤，把下方的序列图补充完整。



思路解析

本题考察的是 基于互斥锁防止缓存击穿 的典型流程，以及如何用序列图刻画两个并发线程在同一热点 Key 上的协作关系。关键点是：只有一个线程真正访问数据库并重建缓存，其它线程都通过等待与重试的方式复用已经重建好的缓存，从而保护数据库不被突发流量冲击。

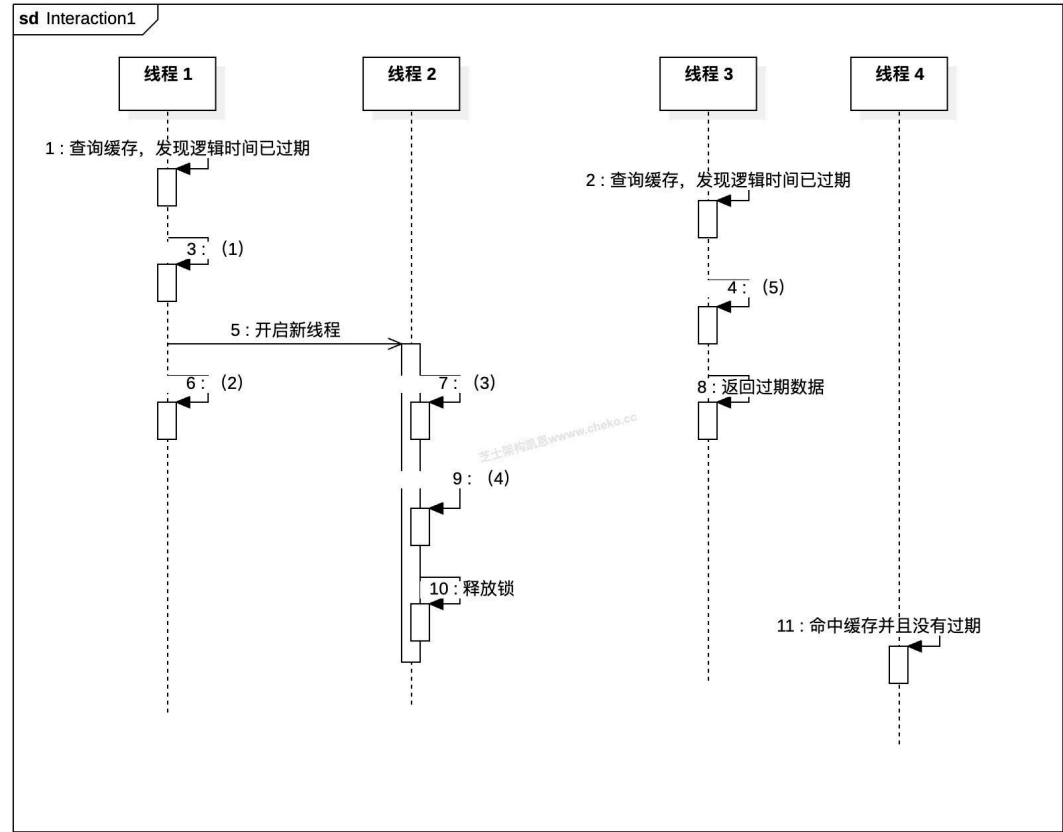
在补图时，应先理清时间线：线程 1 先到达，线程 2 后到达。两者在步骤 1 和 2 都做了“查询缓存”，因为缓存物理过期，结果都是未命中。接下来线程 1 在步骤 3 成功获得互斥锁（一般用 Redis SETNX 实现），说明它成为“重建者”；线程 2 在步骤 4 获取锁失败，因此不能再去数据库，只能进入步骤 6“休眠一会儿再重试”。这体现了互斥锁的核心目的：串行化对数据库的访问。随后线程 1 在持锁状态下执行步骤 5“查询数据库重建缓存数据”，拿到结果后在步骤 7 将数据写入缓存，最后在步骤 8 释放锁。到这里，缓存中已经重新有了最新的数据。

而线程 2 在休眠一段时间后被唤醒，根据王工的流程，不是直接再去加锁，而是先重新查询缓存。若缓存已经被线程 1 重建，则这次查询会命中，这就是序列图中步骤 9“缓存命中”。线程 2 直接返回数据，不需要访问数据库，也不再参与锁竞争。这部分在画图时容易漏掉：6 后面必须有“再次查询缓存”的消息线，才能解释为何 9 是“命中”，也体现了“等待别人重建好缓存后自己复用”的设计思想。

通过这个序列图可以看出：互斥锁方案的优点是数据库只被一个线程访问，避免热点 key 击穿；缺点是其他线程在等待期间被阻塞或轮询，对响应时间有一定影响。因此在题目的大案例中，还需要和逻辑过期方案结合“高可用”要求进行权衡。

问题（2）

请给出李工“逻辑过期方案（可返回过期数据）”的缓存查询过程的主要处理步骤，把下方的序列图补充完整。



思路解析

李工的逻辑过期方案核心思想是：缓存永远尽量有值，只是区分“是否过期”。在 Redis 中保存 {data, expireTime} 结构，真正判断是否过期的是 expireTime 字段，而不是 Redis TTL。序列图中，线程1先查询缓存，发现逻辑时间已过期，但缓存里仍有旧数据可用。此时线程1尝试获取互斥锁，步骤3成功后，说明它成为“刷新发起者”。为了不阻塞当前请求，它在步骤5中启动一个新线程（线程2）去后台刷新缓存，自己在步骤6就把当前这份“过期数据”返回给调用方，实现快速响应。

线程2在后台按照步骤7从数据库查询最新数据，再在步骤9把数据写回缓存并重置逻辑过期时间，最后在步骤10释放锁。与此同时，线程3也在访问同一缓存，它在步骤2也发现逻辑过期，但在步骤4获取锁失败，说明已经有别的线程在刷新，于是它直接在步骤8返回旧数据，不再访问数据库，避免出现缓存击穿。

当线程2刷新完毕后，后续新的请求（如线程4）在步骤11查询时，就会命中已更新且未过期的缓存数据。整个流程体现了逻辑过期方案的特点：主流程永远可以返回数据，只是短时间内可能是旧数据；真正访问数据库的只有极少数负责刷新的线程，从而在高并发场景下显著提升系统可用性和整体性能。

问题（3）

从实现复杂度、数据一致性、性能影响、内存占用、适用场景等维度分析两种方法的优劣，

思路解析

王工方案本质上是对传统 Cache-Aside 模式加上一把 **互斥锁（分布式锁）**，避免同一个热点 key 过期时被多个线程同时回源数据库。题中典型实现是借助 SETNX 或 SET key value NX EX 命令加锁：当缓存未命中时，线程先尝试加锁，只有加锁成功的线程才去数据库查询并重建缓存，完成后删除锁；加锁失败的线程采取“**休眠 + 重试**”的方式，不断检测缓存是否已经被填充。该模式能有效避免缓存击穿带来的数据库瞬时压力，是业界常见的“互斥重建缓存”方案。实现时还要注意锁的 **过期时间设置**，防止线程异常退出造成死锁，同时重试间隔要合理，既不能太频繁导致无谓的 Redis 压力，也不能过长引起延迟过大。

但需要注意，这一模式强调的是**强一致性**：在重建期间，如果没有旧值可用，其他线程会一直等待或轮询，业务请求无法立即返回，接口 RT 会显著升高，极端情况下可能出现大量请求同时卡在等待锁或等待缓存的状态，对系统可用性不利。此外，若热点 key 特别多或锁粒度过粗，还可能形成锁竞争热点，成为新的瓶颈。因此，这种方案更适合读压力不是特别大、但对数据一致性非常敏感的业务场景，如核心交易流水、账户余额等，对高可用高吞吐的读取接口并不一定是最佳选择。

李工提出的逻辑过期方案的核心思想是：把“**是否过期**”的判断从 Redis 的 TTL 转移到业务数据本身。缓存中不再只存业务数据，而是存 {data, expireTime} 这样的结构，expireTime 表示业务层认为数据失效的时间点。查询时，只要 Redis 能命中这条记录，就一定能拿到一份可用数据；如果当前时间未超过 expireTime，说明数据在逻辑上未过期，直接返回即可；如果已经超过了该时间，则说明数据“逻辑过期”。此时，为了**保证高可用**，系统可以先将这份“过期但不为空”的旧数据返回给用户，确保请求快速完成；与此同时，系统异步触发一次缓存刷新流程（通过一个非互斥锁或轻量锁控制只有少量线程真正执行刷新），在后台从数据库加载最新数据并更新缓存。这样，新旧数据在短时间内会存在一定不一致，但系统整体处于“有旧数据可用”的状态，请求不会因为缓存重建而大面积阻塞。

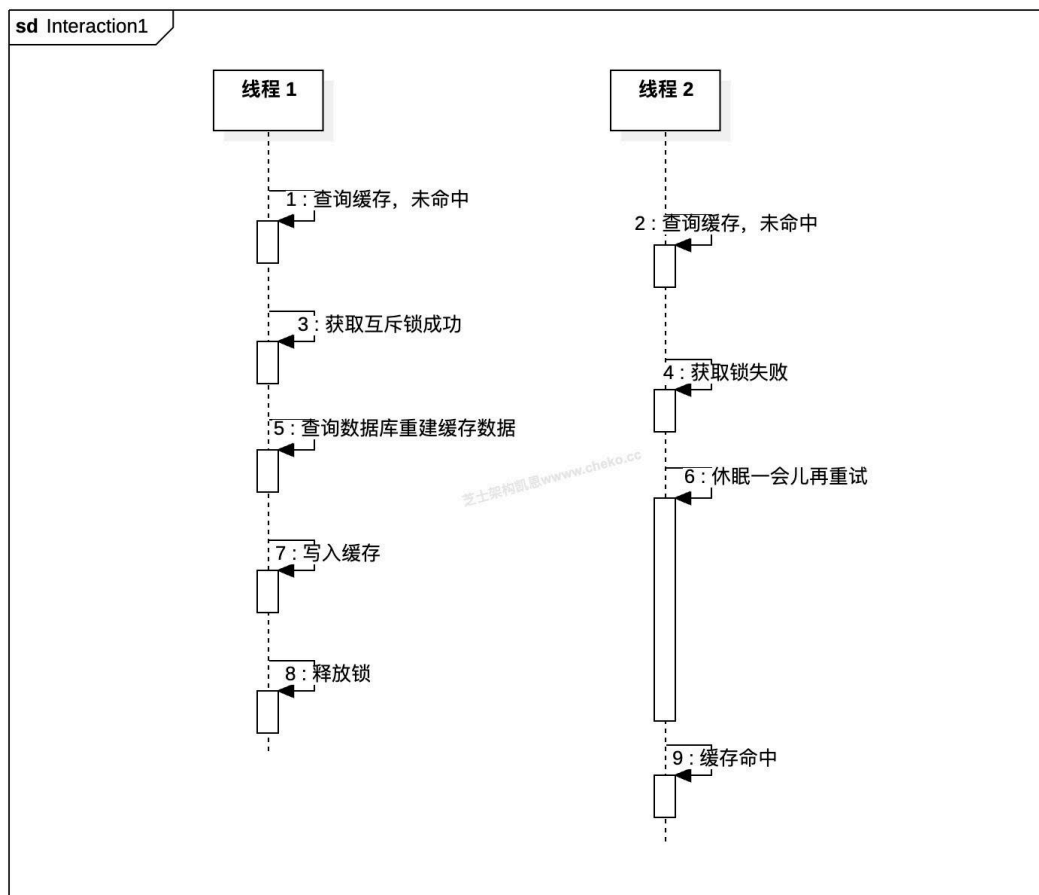
逻辑过期方案带来的代价是**实现复杂度和一致性模型的变化**。开发人员需要设计缓存结构、维护逻辑过期时间、实现异步刷新和刷新失败重试等机制，还要在业务层充分评估“短暂返回过期数据”对业务的影响。因此，此方案通常适合新闻、商品详情、排行榜等读多写少、对实时性要求不是毫秒级严格的场景。它的优势在于：缓存重建时不会阻塞主流程，对高并发场景下的系统可用性和性能提升非常明显，是当前应对缓存击穿的推荐做法之一。

综合来看，两种方案形成了一种典型的**一致性-可用性-性能**之间的权衡。互斥锁物理过期方案依赖 Redis 的 TTL 控制数据生命周期，当某个 key 过期后，外部看起来数据彻底消失，只能等待新的缓存构建完成才能再次提供服务，因此能保证“只要有数据就是最新的”这种强一致体验；但这也意味着在缓存重建期间大量请求可能被阻塞或轮询，尤其是热点 key 占比高时，锁竞争会很激烈，接口响应时间抖动严重，系统整体可用性下降。其优点在于逻辑简单、运维成本低，缺点是**高并发下性能风险较大**。

逻辑过期方案则选择牺牲短暂的一致性，换取系统高可用和高性能。由于逻辑层面允许先返回过期数据，再在后台刷新缓存，用户请求几乎不会因为缓存重建而被阻塞；只有极少数负责刷新的线程会访问数据库，数据库压力也被平滑掉。代价是需要额外字段和代码来维护逻辑过期时间，还要接受“在某个刷新窗口内读到旧数据”的事实，这在金融等强一致业务中可能无法接受。结合题干“**要求高可用**”这一前提，系统更看重在热点 key 失效、网络波动等异常场景下依然能够快速响应，而不是每次都返回绝对最新数据。因此，本题应推荐**优先采用逻辑过期方案**，并在关键业务上适当缩短逻辑过期时间、增加监控和降级策略，以在可接受的一致性范围内获得更高的吞吐和更好的用户体验。

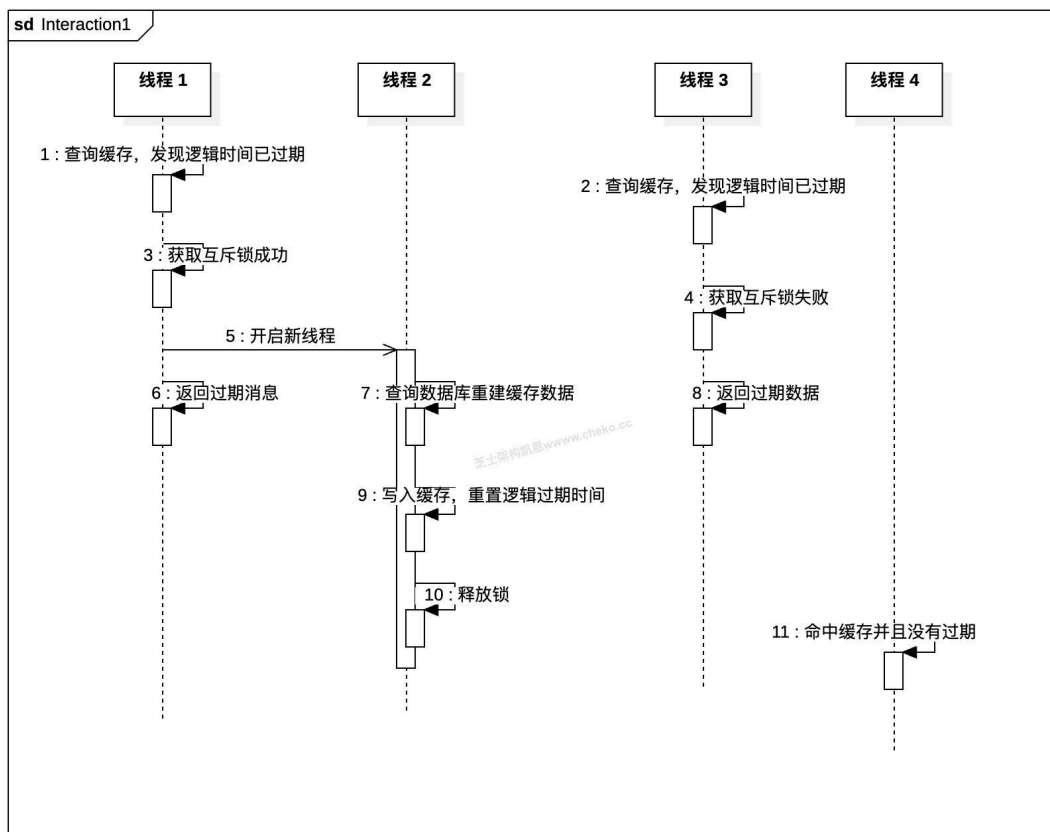
答案-问题（1）

（1）获取互斥锁成功（2）查询数据库重建缓存数据（3）写入缓存（4）释放锁（5）获取锁失败



答案-问题（2）

（1）获取互斥锁成功（2）返回过期消息（3）查询数据库重置缓存（4）写入缓存重置逻辑过期时间（5）获取互斥锁失败



答案-问题（3）

逻辑过期方案的实现复杂度较高，需额外维护过期时间字段并通过异步更新机制保障数据有效性，其数据一致性表现为最终一致，允许系统在短时间内返回旧数据，但因更新操作在后台异步执行，对核心业务主流程的性能影响较小；而物理过期（互斥锁）方案的实现难度更低，主要依托 Redis 的 TTL 机制与锁控制实现数据过期管理，能实现强一致性保障，确保数据更新后才对外可见，不过在缓存击穿场景下，可能出现大量请求阻塞或重试的情况，对并发处理效率产生影响。在内存占用方面，逻辑过期方案需额外存储过期时间等辅助字段，会增加一定的内存开销，适用于高并发读场景，且对数据一致性要求相对宽松的业务场景；物理过期（互斥锁）方案仅需存储核心业务数据，内存利用效率更高，更适合对数据一致性要求严格、整体并发压力适中的应用场景。在题目明确“**要求高可用**”的前提下，建议**优先选择逻辑过期方案**：因为逻辑过期即使在数据已过期、正在重建期间，仍能用旧数据响应请求，避免大面积阻塞和超时，能更好支撑高并发和高可用；而互斥锁方案在缓存重建阶段会阻塞许多线程，请求体验较差，容易在热点 key 上形成“雪崩”式排队。

（2026年5月系统架构设计师真题-第5题）

阅读以下关于 Redis 缓存与并发控制的叙述，在答题纸上回答问题1-3。

【说明】

某互联网平台在大促活动中提供优惠券领取和限量商品抢购功能。活动开始后的前几分钟，请求量会瞬间上升，大量用户同时访问库存、优惠券余量和订单创建接口。系统当前使用 Redis 缓存活动信息和库存数量，并通过数据库最终保存订单和扣减结果。在一次压测中，项目组发现热点商品库存被多个线程同时读取和扣减，偶尔出现重复领取、库存扣减不一致和数据库写入冲突等问题。

为解决高并发下共享资源竞争问题，张工建议采用悲观锁控制临界区，确保同一时刻只有一个请求能够修改关键资源；李工建议采用乐观锁，通过版本号或 CAS 方式减少阻塞，提高并发吞吐。架构师进一步提出，在 Redis 场景中可以结合分布式锁、原子递减、Lua 脚本、限流、队列削峰和异步

落库等机制，综合保障库存、优惠券和订单数据的一致性。

项目组需要比较不同锁机制的适用场景，并给出 Redis 高并发控制的设计方案。

问题（1）

说明悲观锁的常见分类，并给出两类设计方案。

思路解析

本问考查悲观锁的概念、分类和工程化方案。悲观锁的基本假设是共享资源发生并发冲突的概率较高，因此在访问资源前先加锁，使同一时刻只有持锁请求能进入临界区。常见分类可以从多个角度写：按读写能力分为共享锁和排他锁，或读锁和写锁；按数据库粒度分为行锁、表锁和间隙锁；按部署范围分为本地互斥锁、数据库锁和分布式锁。共享锁适合多个读请求并发读取但不允许写，排他锁适合库存扣减、账户余额变更、优惠券领取这类必须独占修改的场景。

两类设计方案可以结合题干写。第一类是数据库悲观锁方案，使用事务和 `SELECT ... FOR UPDATE` 锁定库存行，在同一事务内完成查询余量、校验资格、扣减库存和写入订单。这种方案语义清楚、一致性强，但热点商品会造成数据库行锁竞争，吞吐量有限。第二类是 Redis 分布式锁方案，使用 `SET key value NX PX` 获取锁，其中 `NX` 保证不存在才设置，`PX` 设置过期时间防止死锁，`value` 使用唯一请求标识防止误删他人锁，释放锁时用 Lua 脚本先校验 `value` 再删除，保证判断和删除原子执行。如果业务耗时可能超过锁过期时间，还要考虑续期或缩短临界区。答题时把这些细节写出来，比只说“用 Redis 加锁”更符合架构师案例题要求。

还可以补充锁粒度和临界区设计。库存活动中不应把整个下单流程都放进锁里，否则支付、风控、远程调用都会拖长持锁时间，造成大量请求等待。更合理的做法是只锁定必须互斥的短操作，例如校验并扣减某个活动商品库存，后续订单创建、通知和落库尽量异步化。锁 `key` 也要有合适粒度，可以按活动ID和商品ID组织，而不是全站共用一把大锁。如果是 Redis 集群或主从切换场景，还需要理解单实例锁、`RedLock` 或基于数据库唯一约束的取舍，但考试中写清 `SET NX PX`、唯一 `value`、Lua 释放和超时控制通常已覆盖主要得分点。

问题（2）

比较乐观锁和悲观锁的优缺点。

思路解析

比较乐观锁和悲观锁时，建议按假设、实现方式、优点、缺点和适用场景展开。乐观锁假设冲突较少，读取数据时不阻塞其他请求，提交更新时通过版本号、时间戳、CAS 或条件更新判断数据是否被别人修改。例如更新库存时带上 `version` 或当前库存条件，只有版本未变化才更新成功。它的优点是阻塞少、吞吐量高、不会长时间占用锁，适合读多写少、冲突概率不高的场景；缺点是冲突高时会大量失败和重试，重试会消耗 CPU、数据库连接和用户等待时间，还要处理重试次数、失败提示、幂等和补偿。

悲观锁假设冲突较多，进入临界区前先锁定资源。它的优点是互斥关系明确，可以强约束库存扣减、账户余额、优惠券领取等不能出错的操作；缺点是会阻塞其他请求，降低并发吞吐，锁粒度过大会放大等待，锁顺序不当可能死锁，锁超时或释放失败又可能造成异常。在秒杀或大促活动中，纯乐观锁面对少量库存和海量请求时可能出现大量 CAS 失败；纯悲观锁又会把请求堆在热点锁上。所以工程上常采用组合策略：入口限流减少竞争，Redis 原子扣减快速判断库存，必要时用短时分布式锁保护复杂临界区，再通过 MQ 异步落库和幂等表保证最终一致。

在答题表达上，乐观锁和悲观锁不要写成绝对优劣。乐观锁适合冲突少，因为失败重试成本低；悲观锁适合冲突多且一致性要求强，因为它直接限制并发进入临界区。但两者在高并发秒杀中都不能单独解决全部问题：乐观锁可能把压力转化成大量失败重试，悲观锁可能把压力转化成锁等待和超时。因此架构师更关注组合设计和权衡，例如用 Redis 原子扣减挡住大部分请求，用数据库唯一约束防重复订单，用乐观版本控制处理最终落库冲突，用短粒度悲观锁保护复杂互斥逻辑。

问题（3）

在 Redis 场景下给出高并发控制方案。

思路解析

Redis 高并发控制不能只靠一个分布式锁，而应把流量治理、原子操作、幂等、异步削峰和对账补偿组合起来。入口层先做限流、风控和资格校验，过滤未登录、无资格、重复点击和明显超频请求，避免所有流量都打到库存扣减链路。库存或优惠券余量在 Redis 中可用 DECR、INCRBY 或 Lua 脚本完成“判断余量、扣减余量、记录领取标识”的一体化操作，因为 Lua 在 Redis 单线程执行模型下具备原子性，可以避免先查库存再扣库存之间被其他请求插入。

对不能单靠原子扣减解决的共享资源，例如创建订单前需要同时检查用户资格、活动状态和互斥领取关系，可以使用 Redis 分布式锁保护临界区。锁要使用 SET key value NX PX，设置唯一 value 和合理过期时间，释放时用 Lua 校验 value 后删除，避免锁超时时误删其他请求的新锁。对于耗时链路，要么缩短持锁范围，要么设计续期机制，并保证异常路径也能释放或自然过期。

业务层还必须做幂等控制，可用用户 ID、活动 ID、商品 ID、订单号或请求流水号记录领取状态，防止重复下单和重复消费。Redis 预扣成功后，可以把请求写入 MQ，由消费者异步创建订单和落库，通过队列削峰保护数据库。最后需要补偿和对账任务，比对 Redis 余量、订单表、库存表和支付状态，处理消息重复、消费失败、超时未支付和库存回滚。常见错误包括锁没有过期时间、释放锁不校验 value、库存检查和扣减不是原子操作、MQ 消费不幂等、Redis 成功但数据库失败后没有补偿。把这些风险和措施对应起来，才是完整的 Redis 高并发方案。

还应补充失败处理路径。比如 Redis 预扣成功但 MQ 发送失败时，要回滚库存或记录补偿任务；MQ 消费成功但数据库写入失败时，要重试并保证消费幂等；用户超时未支付时，要释放库存并更新订单状态；活动结束后，要通过对账任务校验 Redis 剩余量、订单成功量、优惠券领取记录和数据库库存是否一致。热点 key 也是 Redis 高并发的典型问题，可以通过本地缓存活动配置、库存分片、预热、限流和排队减少单 key 压力。案例题如果能写出“入口限流、Redis 原子、短锁保护、MQ 削峰、幂等消费、补偿对账”这一整条链路，就基本覆盖了高并发控制的主要架构点。

答案-问题（1）

悲观锁认为并发冲突较可能发生，因此访问共享资源前先加锁，其他请求必须等待或失败。

常见分类包括：共享锁和排他锁，读锁和写锁，数据库行锁和表锁，本地互斥锁和分布式锁。共享锁/读锁允许多个读请求并发访问，排他锁/写锁会独占资源，适用于库存扣减、优惠券领取等需要强约束的临界区。

两类设计方案可以这样写：第一，数据库悲观锁方案，利用行锁或 SELECT ... FOR UPDATE 锁定库存记录，在事务中完成校验和扣减；第二，Redis 分布式锁方案，使用 SET key value NX PX 获取锁，value 使用唯一请求标识，释放锁时通过 Lua 脚本校验 value 后删除，防止误删他人锁。还可以扩展读写锁方案，对读多写少资源区分读锁和写锁。

答案-问题（2）

乐观锁假设并发冲突较少，访问资源时不先阻塞其他请求，提交更新时通过版本号、时间戳或 CAS 判断数据是否被其他请求修改。优点是无锁或少锁、吞吐量高、不会长时间阻塞，适合读多写少、冲突概率低的场景；缺点是冲突高时会频繁失败和重试，导致 CPU 和数据库压力上升，还需要处理重试次数、用户提示和幂等问题。

悲观锁假设冲突较多，访问资源前先加锁。优点是互斥关系明确，能强约束临界区，适合库存扣减、账户余额、优惠券领取等强一致场景；缺点是会阻塞其他请求，降低并发吞吐，锁粒度过大可能造成性能下降，设计不当还会出现死锁、锁超时、锁泄漏或热点锁竞争。

答案-问题（3）

Redis 高并发控制可采用组合方案：

入口限流和资格校验：在网关或活动服务中限制请求速率，过滤未登录、重复请求、无资格用户，减少进入库存扣减链路的流量。

Redis 原子扣减：库存或优惠券余量可使用 DECR、INCRBY 或 Lua 脚本完成检查与扣减的一体化操作，保证判断余量和扣减余量的原子性。

分布式锁保护临界区：对不能只靠原子扣减解决的共享资源，使用 SET key value NX PX 加锁，释放时用 Lua 校验唯一 value 后删除，避免锁误删；锁要设置合理过期时间，必要时做续期。

幂等控制：使用用户ID、活动ID、订单号或请求流水号记录领取状态，防止重复领取和重复下单。

队列削峰和异步落库：Redis 预扣成功后写入 MQ，由消费者异步创建订单和落库，失败时进行补偿或回滚库存。

最终一致性校验：通过对账任务比对 Redis、订单表和库存表，处理超时未支付、消费失败和重复消息。

第20章——关系数据库设计与规范化 (2021年11月系统架构设计师真题-第4题)

阅读以下关于数据库设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某医药销售企业因业务发展，需要建立线上药品销售系统，为用户提供便捷的互联网药品销售服务。该系统除了常规药品展示、订单、用户交流与反馈功能外，还需要提供当前热销产品排名、评价分类管理等功能。通过对需求的分析，在数据管理上初步决定采用关系数据库（MySQL）和数据库缓存（Redis）的混合架构实现。

经过规范化设计之后，该系统的部分数据库表结构如下所示。

供应商（供应商ID，供应商名称，联系方式，供应商地址）；

药品（药品ID，药品名称，药品型号，药品价格，供应商ID）；

药品库存（药品ID，当前库存数量）；

订单（订单号码，药品ID，供应商ID，药品数量，订单金额）。

问题（1）

在系统初步运行后，发现系统数据访问性能较差。经过分析，刘工认为原来数据库规范化设计后，关系表过于细分，造成了大量的多表关联查询，影响了性能。例如当用户查询商品信息时，需要同时显示该药品的信息、供应商的信息、当前库存等信息。

为此，刘工认为可以采用反规范化设计来改造药品关系的结构，以提高查询性能。修改后的药品关系结构为：

药品（药品ID，药品名称，药品型号，药品价格，供应商ID，供应商名称，当前库存数量）；

请用200字以内的文字说明常见的反规范化设计方法，并说明用户查询商品信息应该采用哪种反规范化设计方法。

思路解析

在数据库设计中，规范化能够减少数据冗余、提高一致性，但带来的副作用就是多表关联查询过多，降低性能。反规范化设计的目的是在一定程度上增加冗余数据或改变表结构，从而提高查询性能。常见方法包括：

增加冗余列：在多个表中保存相同字段，避免频繁连接。

增加派生列：将通过计算得到的数据直接存储，减少运行时计算。

重新组表：合并频繁关联的表，减少 JOIN 操作。

分割表：通过横向或纵向切分提高查询效率。

在本案例中，用户查询商品信息时需要同时展示药品信息、供应商信息和库存信息。若采用规范化

结构，必然要对三张表进行 JOIN 操作，性能较差。因此，应采用 **增加冗余列** 的反规范化方法，把供应商名称、库存数量冗余到药品表中。这样用户只需查询药品表即可一次性获得完整信息，大大降低了数据库的查询压力。

问题（2）

王工认为，反规范化设计可提高查询的性能，但必然会带来数据的不一致性问题。请用200字以内的文字说明在反规范化设计中，解决数据不一致性问题的三种常见方法，并说明该系统应该采用哪种方法。

思路解析

反规范化虽然提升了性能，但冗余数据容易与源数据不一致。例如，药品表冗余了供应商名称，如果供应商表中的供应商名称发生修改，而药品表未同步更新，就会出现数据不一致。为解决这一问题，常见方法有：

定期同步更新：通过定时任务批量更新冗余字段，适合实时性要求不高的场景。

触发器同步：在数据库中设置触发器，当源表更新时自动同步冗余字段，实时性高。

应用程序级同步：在业务逻辑中，更新源表数据的同时也更新冗余数据，通常借助事务保持一致性。

在医药销售系统中，药品价格和库存变动频繁，对一致性要求很高，因此不能仅依赖定时任务，而应选择 **触发器或应用程序同步**。触发器能保证在数据更新时自动修改冗余字段，应用程序级同步则可以结合事务管理，确保一次性更新所有相关数据。二者结合能够在保证性能的同时，确保数据一致性。

问题（3）

该系统采用了Redis来实现某些特定功能(如当前热销药品排名等)，同时将药品关系数据放到内存以提高商品查询的性能，但必然会造成Redis和MySQL的数据实时同步问题。

（1）Redis的数据类型包括String、Hash、List、Set和ZSet等，请说明实现当前热销药品排名的功能应该选择使用哪种数据类型。

（2）请用200字以内的文字解释说明解决Redis和MySQL数据实时同步问题的常见方案。

思路解析

（1）Redis 提供了 String、Hash、List、Set、ZSet 等数据类型。其中 **ZSet（有序集合）** 最适合实现热销药品排名，因为它可以通过 score 存储销量，并且支持自动排序、区间查询，非常契合排行榜场景。例如：ZINCRBY sales 1 drugID 可以为某药品销量加 1，然后通过 ZREVRANGE 获取销量前 N 的药品。

（2）Redis 与 MySQL 并存时，常见问题是**缓存与数据库的数据一致性**。常见解决方案有：

应用层双写：应用同时写 MySQL 和 Redis，通过事务保证一致性，适合低并发场景。

基于 Binlog 异步同步：通过 Canal 等工具监听 MySQL Binlog，捕获更新操作并推送到 Redis，适合高并发但可容忍延迟的场景。

消息队列异步更新：写 MySQL 后发送消息到消息队列，由消费者异步更新 Redis，可解耦、抗高并发。

延迟双删策略：更新 MySQL 前后各删除一次 Redis 缓存，确保脏数据概率最小，适合读多写少的场景。

在医药销售系统中，热销药品排名属于高并发访问、但允许轻微延迟的业务，因此推荐使用 **Binlog 异步同步或消息队列方案**，既能保证最终一致性，又不会拖慢数据库的写操作。

答案-问题（1）

（1）增加冗余列：增加冗余列是指在多个表中具有相同的列，它常用来在查询时避免连接操作。

(2) 增加派生列: 增加派生列指增加的列可以通过表中其他数据计算生成。它的作用是在查询时减少计算量, 从而加快查询速度。

(3) 重新组表: 重新组表指如果许多用户需要查看两个表连接出来的结果数据, 则把这两个表重新组成一个表来减少连接而提高性能。

(4) 分割表: 有时对表做分割可以提高性能。

用户查询商品信息应该采用: 增加冗余列。

用户查询商品信息时, 需要显示药品信息(药品表中), 供应商信息(供应商表), 库存信息(库存表中), 此时查询的是药品表, 但表中缺了供应商的信息和库存信息, 可以通过增加冗余列的方式把这些信息并过来。以避免连接操作带来的查询性能下降。

答案-问题(2)

解决数据不一致性问题的常见方法包括定期同步更新、使用触发器和应用程序级别的数据校验。定期同步更新指定期性地将冗余数据与原始数据进行同步更新, 确保数据一致性; 使用触发器可以在数据更新时自动触发相关操作, 保持数据的一致性; 应用程序级别的数据校验则是通过应用程序逻辑来验证数据的一致性。该医药销售系统, 药品数据更新频繁, 采用触发器法和应用程序校验较合适。因为它能在数据更新瞬间自动维护一致性, 无需应用程序过多干预, 也避免了定时任务的延迟性, 可实时保证数据一致性, 不影响系统对药品信息的实时展示和业务操作。

答案-问题(3)

(1) 热销药品排名适合用: ZSet, 即有序集合类型。

(2) 基于 Binlog 异步同步。用 Canal 等工具监听 MySQL 的 Binlog 日志, 解析数据变更事件并推送到 Redis。对应用透明, 能捕获多种操作, 可靠性高, 但有延迟, 部署复杂。适用于高频写且要求最终一致场景。**应用层双写。**业务代码中同步写 MySQL 和 Redis, 靠事务保证原子性。强一致、低延迟, 但双写增加耗时, 事务处理复杂。适用于低频写、强一致场景。**消息队列解耦。**写 MySQL 后发消息到队列, 消费者异步更新 Redis。可解耦、削峰填谷, 但有秒级延迟和消息堆积风险。适用于高并发写、允许短暂不一致场景。**延迟双删。**更新 MySQL 前后各删一次 Redis 缓存。简单有效, 能减少不一致窗口, 但极端并发下仍可能有脏数据。适用于读多写少、对一致性要求不苛刻场景。**结合数据库触发器。**在 MySQL 设触发器, 监听数据变更并调外部程序更新 Redis。会增加数据库负载, 维护困难。适用于遗留系统改造。

第21章——大数据处理与数据挖掘 (2023年11月系统架构设计师真题-第1题)

某网作为某电视台在互联网上的大型门户入口, 某一年成为某奥运会中国大陆地区的特权转播商, 独家全程直播了某奥运会全部的赛事, 积累了庞大稳定的用户群, 这些用户在使用各类服务过程中产生了大量数据, 对这些海量数据进行分析与挖掘, 将会对节目的传播及商业模式变现起到重要的作用。该奥运期间需要对增量数据在当日概览和赛事回顾两个层面上进行分析。

其中, 当日概览模块需要秒级刷新直播在线人数、网站的综合浏览量、页面停留时间、视频的播放次数和平均播放时间等千万级数据量的实时信息, 而传统的分布式架构采用重新计算的方式分析实时数据在不扩充以往集群规模的情况下, 无法在几秒内分析出重要的信息。

赛事回顾模块需要展现自定义时间段内的历史最高在线人数、逐日播放走势、直播最高在线人数和点播视频排行等海量数据的统计信息, 由于该奥运期间产生的数据通常不需要被经常索引、更新, 因此要求采用不可变方式存储所有的历史数据, 以保证历史数据的准确性。

问题(1)

请根据Lambda架构和Kappa架构特点, 填写以下表格。

对比内容	Lambda架构	Kappa架构
复杂度与开发、维护成本	需要维护（1）套系统(引擎),复杂度（3）,开发、维护成本（3）	只需要维护（2）套系统(引擎),复杂度（4）开发、维护成本(4)
计算开销	需要一直运行批处理和实时计算,计算开销大	（5）
实时性	满足实时性	（6）
历史数据处理能力	批式全量处理,吞吐量（7）,历史数据处理能力强	流式全量处理,吞吐量相对较低,历史数据处理能力（8）

#1846px #429px

思路解析

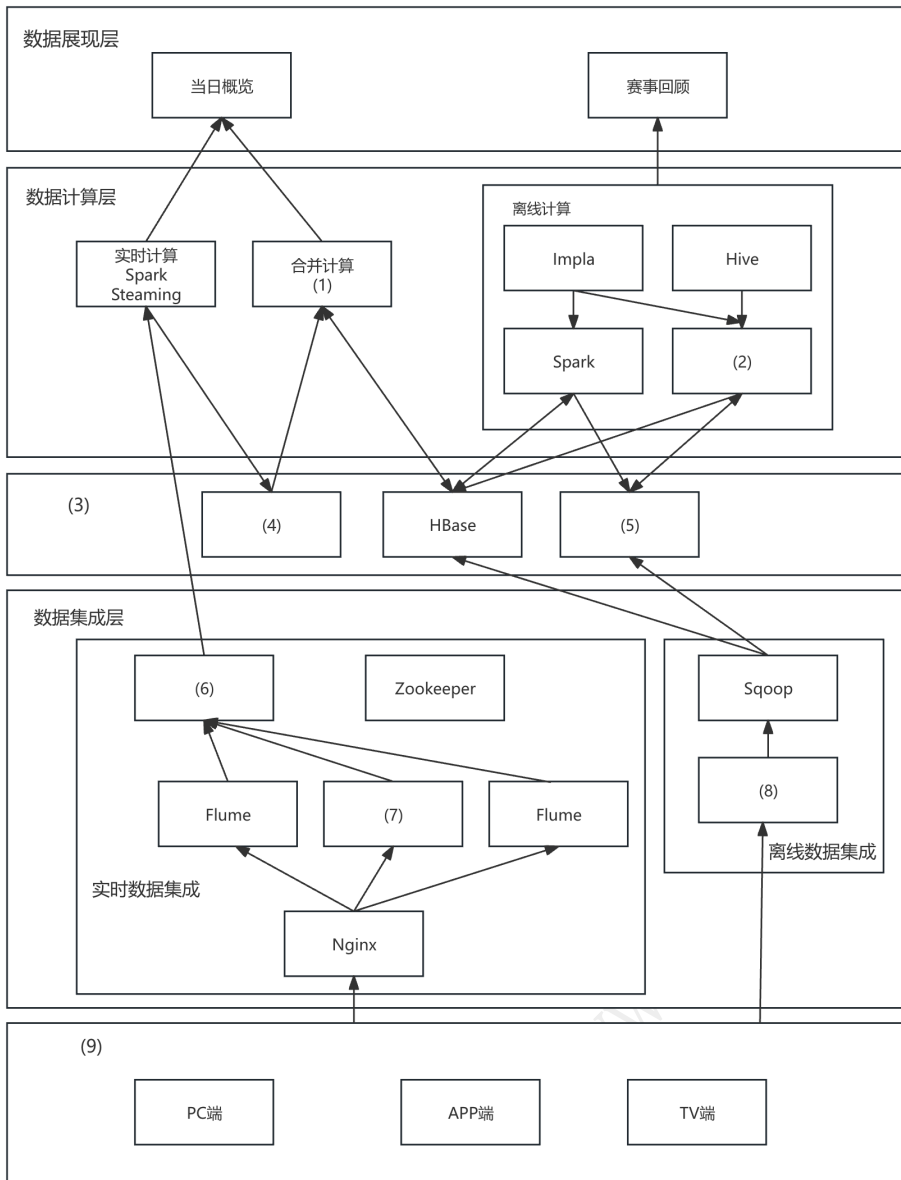
此题出自 19 章大数据架构原图，有关对比类概念类是案例第一题常考的内容，需要好好重视。
表19-1 Lambda 架构和Kappa 架构对比

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统(引擎),复杂度 高,开发、维护成本高	只需要维护一套系统(引擎),复杂度 低,开发、维护成本低
计算开销	需要一直运行批处理和实时计算, 计算开销大	必要时进行全量计算,计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理,吞吐量大,历史数 据处理能力强	流式全量处理,吞吐量相对较低,历史 数据处理能力相对较弱

问题（2）

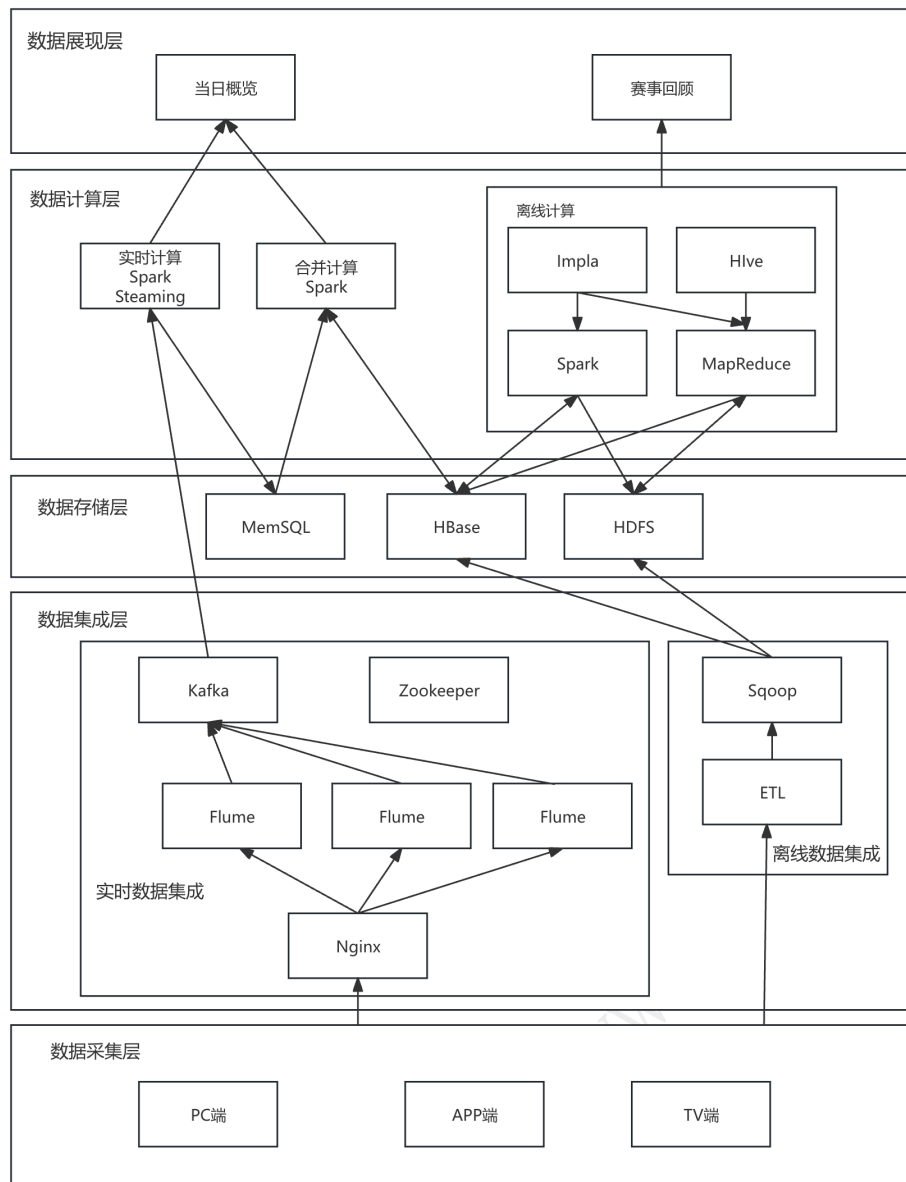
下图给出了某网奥运的大数据架构图，请根据下面的（a）-（n）的相关技术，判断这些技术属于架构图的哪个部分，补充完善下图1的（1）-（9）的空白处。

（a）Nginx;（b）Hbase;（c）Spark Streaming（d）Spark;（e）MapReduce;（f）ETL;（g）MemSQL;（h）HDFS;（i）Sqoop;（j）Flume ;（k）数据存储层;（l）kafka;（m）业务逻辑层;（n）数据采集层



思路解析

此题出自 19 章大数据架构原图，此类架构图也需要好好重视。



问题（3）

大数据的架构包括了Lambda架构和Kappa架构，Lambda架构分解为三层，Kappa架构不同于Lambda同时计算流计算和批计算并合并视图，Kappa只会通过流计算一条的数据链路计算并产生视图。请问该系统的大数据架构是基于哪种架构搭建的大数据平台处理奥运会大规模视频网络观看数据。

思路解析

Lambda架构是一种用于处理大数据的架构设计模式，将数据处理流程分为批处理层、速度层和服务层三个部分。具体来说，Lambda架构包括以下三层：批处理层（Batch Layer）：负责处理大规模数据的批量处理任务，通常使用分布式计算框架（如Hadoop）进行批处理操作，生成批处理视图。速度层（Speed Layer）：负责处理实时数据流，通常使用流处理引擎（如Storm、Spark Streaming）进行实时计算，生成实时视图。服务层（Serving Layer）：负责将批处理层和速度层生成的视图合并，提供统一的查询接口给用户，使用户可以查询批处理和实时处理的结果。

Kappa架构是一种简化的大数据架构设计模式，与Lambda架构不同，Kappa架构只使用流处理引擎来处理数据流，而不区分批处理和实时处理。Kappa架构通过流处理一条数据链路计算并产生视图，简化了架构设计和维护，但也可能在处理大规模数据时性能不如Lambda架构。

答案-问题（1）

- (1) 2
- (2) 1
- (3) 高
- (4) 低
- (5) 必要时进行全量计算，计算开销相对较小
- (6) 满足实时性
- (7) 大
- (8) 弱

答案-问题（2）

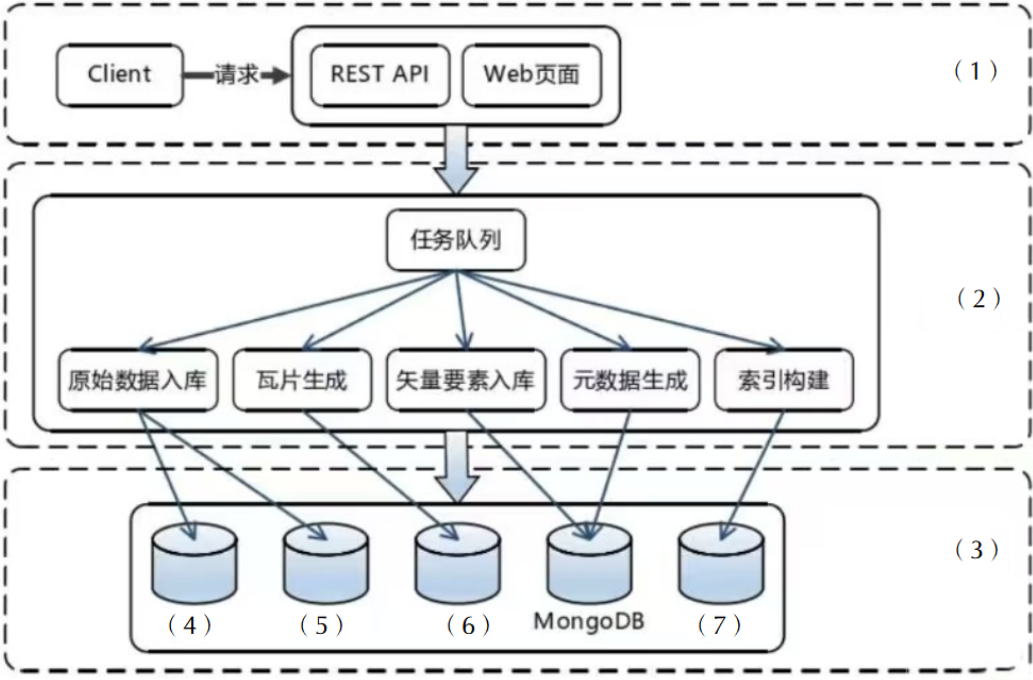
(1)d (2)e (3)k (4)g (5)h (6)l (7)j (8)f (9)n

答案-问题（3）

该系统的大数据架构是基于Lambda架构搭建的大数据平台处理电运会大规模视频网络观看。

(2024年5月系统架构设计师真题-第5题)

某地图服务平台正在建设一个支持大规模地理空间数据处理与检索的系统。系统目标包括：
支持客户端通过 Web 页面或 REST API 访问地理信息服务；
具备任务调度与处理能力，完成如原始数据入库、地图瓦片生成等任务；
能高效存储不同类型的结构化与非结构化数据；
提供快速的检索与空间查询能力；
支持对存储数据按冷热等级分类管理，优化成本与性能。
为满足上述需求，系统整体采用分层架构设计，核心架构图如下所示：



问题（1）

请根据自身作用 把下面的选项填写到上方的系统架构图中。

- (1) 接口层 (2) 处理层 (3) 存储层 (4) MySQL (5) HDFS (6) HBase (7) ES

思路解析

图中 Client 发送请求到 REST API 和 Web 页面，这部分起到对外提供访问接口、接收请求的作用，符合接口层功能，所以（1）对应接口层。

任务队列接收来自接口层的请求，并调度原始数据入库、瓦片生成等任务，这些任务处理操作属于对业务逻辑的处理，因此（2）是处理层。

最下方的数据库用于存储经过处理层处理后的数据，起到数据持久化存储的作用，所以（3）为存储层。

MySQL是关系型数据库，适用于结构化数据存储。在系统中可用于存储一些结构规整、事务性要求较高的数据，如用户信息等基础数据，可对应到（4）。

HDFS分布式文件系统，适合存储海量的、一次写入多次读取的数据，像原始地理空间数据等大文件数据，可由它存储，对应（5）。

HBase基于 Hadoop 的分布式列式数据库，能处理海量数据的随机读写，适合存储像瓦片数据等需要快速查询和更新的数据，对应（6）。

ES开源分布式搜索和分析引擎，在存储层可用于存储索引数据，方便对数据进行快速检索，比如构建的地理空间数据索引等可存储于此，对应（7）。

问题（2）

MongoDB如何存储非结构性数据的，MongoDB 矢量化存储的优点。

思路解析

MongoDB 矢量化存储 适用于系统架构设计、AI 应用、数据库选型等多种场景。矢量化存储是指将高维特征（如图像特征、文本 embedding 向量、音频信号等）以数组的形式存入数据库中，并能基于相似度进行高效检索的能力。

例如：将一张图片的 ResNet（有兴趣可去了解）向量 [0.01, 0.52, -0.33, ...] 存入数据库，支持找出与某张图片最相似的其他图片。MongoDB 可将向量（通常是 float[]）作为字段直接存储在文档中。

问题（3）

该系统中 HDFS 使用热数据、温数据和冷数据存储的原因。

思路解析

在大规模地理空间数据处理中，数据量巨大且访问模式差异明显。如果所有数据都存储在高性能介质（如 SSD）上，将导致成本过高，而若全部放在低成本存储上又会影响查询与处理效率。因此需要根据数据访问频率与业务需求进行分层管理。

热数据（如近期瓦片数据、常用索引）需频繁访问，应存储在高性能存储介质，保证实时性与交互体验。**温数据**（如不常用的历史任务数据）访问相对少，可存储在普通 HDD 以降低成本。**冷数据**（如归档影像、历史版本数据）几乎不访问，可存放在廉价存储介质如磁带或云冷存储。通过这种**冷热分层存储策略**，系统能实现性能与成本的最佳平衡，提高整体资源利用率。

答案-问题（1）

（1）接口层（2）处理层（3）存储层（4）MySQL（5）HDFS（6）HBase（7）ES

答案-问题（2）

MongoDB 是典型的 **NoSQL 文档型数据库**，其数据存储单位为 BSON 文档。相较关系型数据库需要严格的模式定义，MongoDB 无需固定模式，能灵活存储非结构化或半结构化数据，如 JSON 文档、图像元信息、日志、传感器数据等。同时它支持嵌套文档、数组字段与灵活的索引机制，使得数据结构可以与应用场景自然贴合，避免频繁的表结构变更，适应动态性强的业务需求。

在**矢量化存储**方面，MongoDB 支持将高维特征向量（如文本 embedding、图像特征向量）直接存储为字段，并结合向量索引机制实现快速的相似度检索。这在推荐系统、图像搜索、自然语言处理等场景下具有明显优势。例如，将用户兴趣向量存储后，可通过向量相似度检索找到最相关的内容

，提升系统智能化水平。其优势在于 兼容分布式架构、高性能搜索、适配 AI 场景。

答案-问题（3）

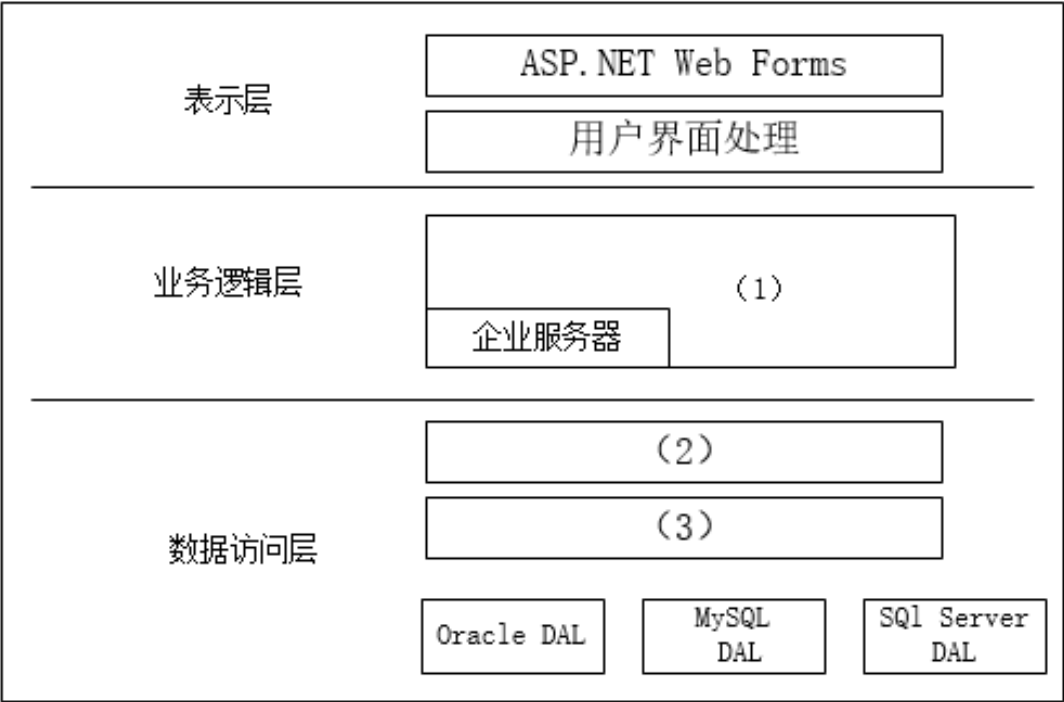
HDFS 采用热数据、温数据和冷数据分层存储，是为了优化性能与降低存储成本。

- 热数据：访问频繁、响应要求高，通常存储在 SSD 上，读取速度快；
 - 温数据：访问较少，对性能要求一般，适合存储在普通硬盘（如 SATA）上；
 - 冷数据：几乎不访问的历史数据或归档数据，存储在廉价介质如磁带或云冷存储上，节省成本。
- 这种分级存储策略可根据数据使用频率匹配不同存储资源，实现资源利用最大化、系统运行更高效。

第22章——数据访问层与 ORM
(2017年11月系统架构设计师真题-第4题)

阅读以下关于数据库设计的叙述，在答题纸上回答问题 1至问题 3。

【说明】
某制造企业为拓展网上销售业务，委托某软件企业开发一套电子商务网站。初期仅解决基本的网上销售、订单等功能需求。该软件企业很快决定基于.NET平台和SQL Server数据库进行开发，但在数据库访问方式上出现了争议。王工认为应该采用程序在线访问的方式访问数据库；而李工认为本企业内部程序员缺乏数据库开发经验，而且应用简单，应该采用 ORM（对象关系映射）方式。最终经过综合考虑，该软件企业采用了李工的建议。
随着业务的发展，该电子商务网站逐渐发展成一个通用的电子商务平台，销售多家制造企业的产品，电子商务平台的功能也日益复杂。目前急需对该电子商务网站进行改造，以支持对多种异构数据库平台的数据访问，同时满足复杂的数据管理需求。该软件企业针对上述需求，对电子商务网站的架构进行了重新设计，新增加了数据访问层，同时采用工厂设计模式解决异构数据库访问的问题。新设计的系统架构如图 4-1所示。



问题（1）

请用300字以内的文字分别说明数据库程序在线访问方式和ORM方式的优缺点，说明该软件企业采用ORM的原因。

思路解析

数据库程序在线访问方式：开发人员直接编写SQL语句，灵活性高，但需要较强的数据库知识，且会带来安全隐患和代码冗余问题。

ORM方式：通过ORM框架自动处理数据库与对象之间的映射，简化开发流程和维护，但可能存在性能问题，特别是在复杂查询方面。

选择ORM的原因：由于该企业开发人员缺乏数据库开发经验，ORM的简便性降低了学习成本，避免了数据库操作的复杂性，同时提高了开发效率。

问题（2）

请用100字以内的文字说明新体系架构中增加数据访问层的原因。请根据图4-1所示，填写图中空白处（1）-（3）。

思路解析

增加数据访问层是为了在系统架构中实现清晰的分层，将数据访问与业务逻辑解耦，提高系统的可维护性和扩展性。数据访问层（DAL）屏蔽了数据库的异构性，简化了数据库操作，并方便了后期的优化和管理。在图4-1中：

- （1）是业务层或组件，负责执行业务逻辑。
- （2）是数据访问接口层，定义统一的数据访问接口。
- （3）是工厂层，负责根据不同的数据库类型实例化相应的数据库访问对象。

问题（3）

应用程序设计中，数据库访问需要良好的封装性和可维护性，因此经常使用工厂设计模式来实现对数据库访问的封装。请解释工厂设计模式，并说明其优点和应用场景；请解释说明工厂模式在数据访问层中的应用。

思路解析

工厂设计模式：用于创建对象的设计模式，主要分为抽象工厂模式和工厂方法模式。工厂方法模式针对一个产品类的创建，而抽象工厂模式用于创建一系列相关的产品。通过封装对象创建过程，使得客户端代码与具体类的实例化解耦，避免了直接依赖具体类，便于扩展和维护。

应用场景：适用于创建一系列相关的对象，而不关心其具体实现类的情况。在数据访问层，工厂模式可以根据不同的数据库类型（如Oracle、MySQL、SQLServer）创建不同的数据库访问对象。通过这种方式，业务逻辑层无需关心具体的数据库类型，只需依赖于统一的接口，大大提高了系统的可维护性和扩展性。

答案-问题（1）

数据库程序的在线访问方式是直接使用SQL语句与数据库进行交互，开发人员需要手动编写SQL查询、更新等操作。优点是对数据库操作更加灵活，可以直接优化SQL语句以提高性能，适用于复杂查询和特定性能要求；缺点是需要开发人员具备较强的数据库知识和编程技能，且容易引入安全风险和代码冗余。

ORM（对象关系映射）方式通过将数据库表映射为面向对象的实体类，由ORM框架自动处理对象与数据库之间的映射关系，开发人员无需编写SQL语句，提高了开发效率和代码可维护性。优点包括减少了开发工作量、降低了数据库操作的复杂度、提高了代码的可读性和可维护性；缺点是可能存在性能损失、对复杂查询支持不足等问题。

该软件企业采用ORM的原因可能是为了提高开发效率、降低维护成本、减少人为错误，以及更好地实现面向对象编程范式。ORM可以帮助开发人员更专注于业务逻辑的实现，而不是关注数据库操作的细节，从而加快软件开发周期并提高整体的开发质量。

答案-问题（2）

增加数据访问层的原因是为了实现数据与业务逻辑的分离，提高系统的可维护性、扩展性和灵活性。数据访问层负责处理数据库操作，封装数据访问细节，使业务逻辑层与具体的数据存储技术解耦，简化开发流程，同时也方便对数据访问进行统一管理和优化，从而提升系统的整体性能和可靠性。

- (1) 执行业务逻辑/业务组件/业务构件
- (2) 数据访问接口层/DAL接口
- (3) 工厂层/DAL工厂/数据访问工厂

答案-问题(3)

工厂模式分抽象工厂与工厂方法，题目中的场景适合采用抽象工厂设计模式。

抽象工厂设计模式提供一个接口，可以创建一系列相关或相互依赖的对象，而无需指定它们具体的类。其优点是可以非常方便地创建一系列的对象，其使用场景也是创建系列对象的情况。在本题中，可以针对Oracle、MySQL、SQLServer分别建立抽象工厂，若指定当前工厂为Oracle工厂，则创建出来的数据库连接、数据集等一系列的对象都是符合Oracle操作要求的。这样便于数据库之间的切换。

(2015年11月系统架构设计师真题-第5题)

阅读以下关于Web应用系统架构设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某信息技术公司计划开发一套在线投票系统，用于为市场调研、信息调查和销售反馈等业务提供服务。该系统计划通过大量宣传和奖品鼓励的方式快速积累用户，当用户规模扩大到一定程度时，开始联系相关企业提供信息服务，并按照信息服务种类和用户投票数量收取费用。

为了降低开发成本和提高开发效率，项目组经过讨论后决定采用轻量级Java EE开发框架设计系统应用架构。在应用架构设计中，除了满足系统主要功能需求，还需要考虑的因素包括：

- (1) 项目开发采用MySQL数据库存储数据，但将来可能移植到其他数据库平台；
- (2) 系统开发过程中尽可能降低或者消除SQL语句开发的工作量；
- (3) 投票系统中数据之间的关系复杂，需要支持数据对象的聚合和继承等关系。

项目组基于MVC模式设计出了投票系统的架构，包括表示层、业务逻辑层、数据持久层和数据层。在具体讨论数据持久层采用哪种技术方案时，老王建议采用成熟的Hibernate框架，小李则认为iBatis更加灵活，更适合作为投票系统数据持久层开发技术。

问题(1)

请用300以内文字说明什么是数据持久层，使用数据持久层能够为项目开发带来哪些好处？

思路解析

数据持久层(Persistence Layer)在Web应用系统中扮演着承上启下的作用。它的核心任务是**持久化数据的管理**，即保证数据从临时的程序运行内存中保存到长期存储介质(数据库)中，并在需要时能够高效读取。数据持久层通常位于**业务逻辑层与数据库层之间**，它将复杂的数据库操作封装起来，对外提供简洁统一的接口。

优势在于：

降低耦合度：通过持久层，业务逻辑层无需关心底层数据库的具体实现，只需通过对象接口操作数据即可。这种分层设计遵循了“关注点分离”原则，使得各层的职责清晰，系统维护性提高。

提高开发效率：传统JDBC需要开发人员手动编写大量的SQL和数据解析代码，而持久层框架(如Hibernate、MyBatis)能自动或半自动完成SQL的生成与执行，大幅度减少重复劳动。

增强系统可扩展性：由于持久层屏蔽了数据库差异，当未来数据库迁移(如从MySQL转向Oracle)时，开发人员只需修改持久层配置，而不必大规模修改业务逻辑。

保障数据一致性与安全性：持久层通常内置事务管理与数据校验机制，避免了开发人员因处理不

当导致的数据不一致或安全隐患。

问题（2）

针对在线投票系统的实际应用需求和要求，项目组应选用哪种技术实现数据持久层？请用200字以内文字说明其采用该技术的原因。

思路解析

结合题干需求分析选择 Hibernate 的理由：

数据库移植性需求： 题干指出“当前使用 MySQL，但未来可能迁移到其他数据库”。Hibernate 提供了 **数据库方言机制**，只需更换配置文件即可支持不同数据库，而不必改动大量 SQL 语句。这一特性使其在移植性方面优于 MyBatis。

减少 SQL 工作量： Hibernate 属于 O/R Mapping 框架，开发人员主要操作对象即可完成持久化操作，大部分情况下不需要编写 SQL。MyBatis 虽然灵活，但高度依赖人工编写 SQL，在复杂业务中会增加开发负担。

对象关系复杂性支持： 在线投票系统的数据模型复杂，存在聚合、继承等关系。Hibernate 在处理对象继承映射、多表关联等方面功能强大，能够自动生成相应 SQL 并保证性能优化。而 MyBatis 更依赖人工配置，维护成本高。

状态管理与事务支持： Hibernate 能够自动管理对象的生命周期（瞬时、持久、游离），并内置事务控制机制。MyBatis 在这方面支持有限，需要开发人员额外处理。

综上，从系统的长远发展与维护角度看，Hibernate 更适合该在线投票系统，而 MyBatis 适合需求变动频繁、需要手写 SQL 精细优化的系统。

问题（3）

数据持久层是Web应用系统框架中重要的组成部分，主流的数据持久层技术分别基于不同的技术方案，请在表5-1中（1）-（4）处分别根据（a）~（d）所列技术的方案类别填入其序号。

技术方案	实现技术
JDBC封装	(1)
SQL Mapping	(2)
O/R Mapping	(3)
Entity Bean	(4)

- （a）BMP，CMP
- （b）iBatis/MyBatis
- （c）Spring JdbcTemplate
- （d）TopLink，JDO，Hibernate

思路解析

四类主流数据持久层技术方案分析：

JDBC 封装技术（答案 c）： Spring JdbcTemplate 属于典型的 **JDBC 封装** 技术。它对 JDBC 的操作进行了二次封装，简化了连接获取、SQL 执行、异常处理和资源释放等繁琐流程，使得开发者能够更高效地进行数据库操作，但仍需编写 SQL。

SQL Mapping 技术（答案 b）： iBatis/MyBatis 属于 **SQL 映射框架**。它通过 XML 或注解将 SQL 语句与 Java 对象映射，开发人员可以灵活控制 SQL，同时通过映射机制减少手动数据解析的负担。优点是灵活、易调优，缺点是 SQL 工作量大，移植性不足。

O/R Mapping 技术（答案 d）：TopLink、JDO、Hibernate 属于 **对象关系映射（O/RM）** 框架。

它们能够自动将对象与关系数据库表进行映射，实现对象与数据库的透明交互。开发者只需操作对象即可完成数据持久化，大大减少 SQL 编写，且支持复杂对象关系。

Entity Bean 技术（答案 a）：BMP（Bean-Managed Persistence）和 CMP（Container-Managed Persistence）属于 **EJB 规范中的实体 Bean 持久化技术**。这种方式较为传统，依赖 J2EE 容器，开发和维护成本较高，但在早期大型企业应用中较为常见。

因此，表格对应为：

(1) → c (JdbcTemplate)

(2) → b (MyBatis)

(3) → d (Hibernate)

(4) → a (Entity Bean 技术)

答案-问题（1）

数据持久层是指应用程序中负责与数据库或其他数据存储系统进行交互的模块或组件。它负责数据的读取、写入、更新和删除操作，以及数据的持久化和管理工作。数据持久层的设计旨在实现数据访问的有效性、可靠性和安全性，同时与业务逻辑层和表示层相互独立，提高系统的灵活性和可维护性。

使用数据持久层可以为项目开发带来多方面的好处。首先，数据持久层将数据访问逻辑与业务逻辑分离，降低了系统的耦合度，使得系统更易于维护和扩展。其次，通过数据持久层的封装和抽象，可以提高数据访问的效率和性能，减少重复代码的编写，提高开发效率。此外，数据持久层还可以提供数据安全性和一致性的保障，确保数据的完整性和可靠性。

答案-问题（2）

项目组应选择Hibernate框架作为数据持久层技术实现。选择Hibernate的原因包括：1. 移植性强，易于在不同数据库平台之间移植；2. 减少SQL语句开发工作量，提供持久层操作方法；3. 提供对象状态管理功能，减少对底层数据库细节的关注，简化开发流程。相比之下，Hibernate相对于MyBatis更具移植性、开发效率和对象管理方面的优势。

答案-问题（3）

(1) (c)

(2) (b)

(3) (d)

(4) (a)

第23章——分布式搜索与 Elasticsearch (2024年11月系统架构设计师真题-第4题)

阅读以下关于数据库缓存架构的叙述，在答题纸上回答问题1-3。

【说明】

某电商企业计划开发一款商品推荐系统，并接入微信小程序，以提升用户购物体验，增加商品销量。该企业组建了项目团队，团队成员包括项目经理、架构师、开发工程师、测试工程师等。在技术选型方面，架构师经过调研和评估，选择了 Elasticsearch 作为搜索引擎，利用其强大的分词功能和分布式架构，实现对海量商品数据的高效存储和检索。

问题（1）

Elasticsearch中内置了很多分词器，请用150字说明Simple, Whitespace, Keyword 分词引擎的特点差异。

思路解析

分词一个非常关键的基础问题，指的是将一段连续的文本拆分成一个个有意义的“词”或“词项（token）”的过程。

例如：

原始文本：我爱北京天安门

分词结果：["我", "爱", "北京", "天安门"]

ES构建索引时拆分文本，也就是将字段值分解为多个词项（tokens），建立倒排索引。如果你查询“北京”，就能直接命中文档中相应的词项，而不是全表检索，效率差、效果差。

假如你不知道什么是分词，实际上根据字面意思Whitespace和Keyword 分词的含义也很好猜出来，拿个 4 分没有问题。其他常见的分词方法如下所示，作为补充拓展。

Standard 分词器原理：默认分词器，基于 Unicode 文本分割算法，自动识别单词边界并将字母转为小写。**示例**："Hello, world!" → ["hello", "world"]**适用场景**：通用英文文本搜索。

原理：默认分词器，基于 Unicode 文本分割算法，自动识别单词边界并将字母转为小写。

示例："Hello, world!" → ["hello", "world"]

适用场景：通用英文文本搜索。

Simple 分词器原理：按非字母字符切分文本，结果转为小写；不识别数字或标点。**示例**

："Hello, world!" → ["hello", "world"]**适用场景**：英文文本中快速获取单词，处理简单清洗任务。

原理：按非字母字符切分文本，结果转为小写；不识别数字或标点。

示例："Hello, world!" → ["hello", "world"]

适用场景：英文文本中快速获取单词，处理简单清洗任务。

Whitespace 分词器原理：仅按空格切分文本，保留大小写和标点，不做任何预处理。**示例**

："Hello WORLD" → ["Hello", "WORLD"]**适用场景**：日志、代码等格式固定文本的处理。

原理：仅按空格切分文本，保留大小写和标点，不做任何预处理。

示例："Hello WORLD" → ["Hello", "WORLD"]

适用场景：日志、代码等格式固定文本的处理。

Keyword 分词器原理：整体保留原始文本为一个词项，不进行分词处理。**示例**："Hello World" → ["Hello World"]**适用场景**：ID、标签、URL 等需精确匹配的字段。

原理：整体保留原始文本为一个词项，不进行分词处理。

示例："Hello World" → ["Hello World"]

适用场景：ID、标签、URL 等需精确匹配的字段。

NGram 分词器原理：将文本按 n 个字符滑动分割，适用于模糊匹配。**示例**："apple", n=2 → ["ap", "pp", "pl", "le"]**适用场景**：搜索建议、拼写纠错、中文子串匹配。

原理：将文本按 n 个字符滑动分割，适用于模糊匹配。

示例："apple", n=2 → ["ap", "pp", "pl", "le"]

适用场景：搜索建议、拼写纠错、中文子串匹配。

CJK 分词器原理：面向中日韩语言的正向最大匹配算法。**示例**："我爱北京天安门" → ["我", "爱", "北京", "天安门"]**适用场景**：中日韩文本搜索。

原理：面向中日韩语言的正向最大匹配算法。

示例："我爱北京天安门" → ["我", "爱", "北京", "天安门"]

适用场景：中日韩文本搜索。

Stop 分词器原理：在分词时过滤停用词（如 "is"、"the" 等），保留有价值词项。**示例**

："The cat is cute" → ["cat", "cute"]**适用场景**：英文搜索优化，减少干扰项。

原理：在分词时过滤停用词（如 "is"、"the" 等），保留有价值词项。

示例："The cat is cute" → ["cat", "cute"]

适用场景：英文搜索优化，减少干扰项。

Pattern 分词器原理：基于正则表达式自定义分割规则。**示例**：文本 "a;b|c"，分隔符正则为 ;|\| → ["a", "b", "c"]**适用场景**：结构化日志、特殊格式字符串。

原理：基于正则表达式自定义分割规则。

示例：文本 "a;b|c"，分隔符正则则为 ;|\| → ["a", "b", "c"]

适用场景：结构化日志、特殊格式字符串。

uax-url-email 分词器原理：扩展分词器，能识别和保留 URL、邮箱等结构化实体。示例

："test@example.com" → ["test@example.com"] **适用场景：**网页索引、邮箱字段搜索。

原理：扩展分词器，能识别和保留 URL、邮箱等结构化实体。

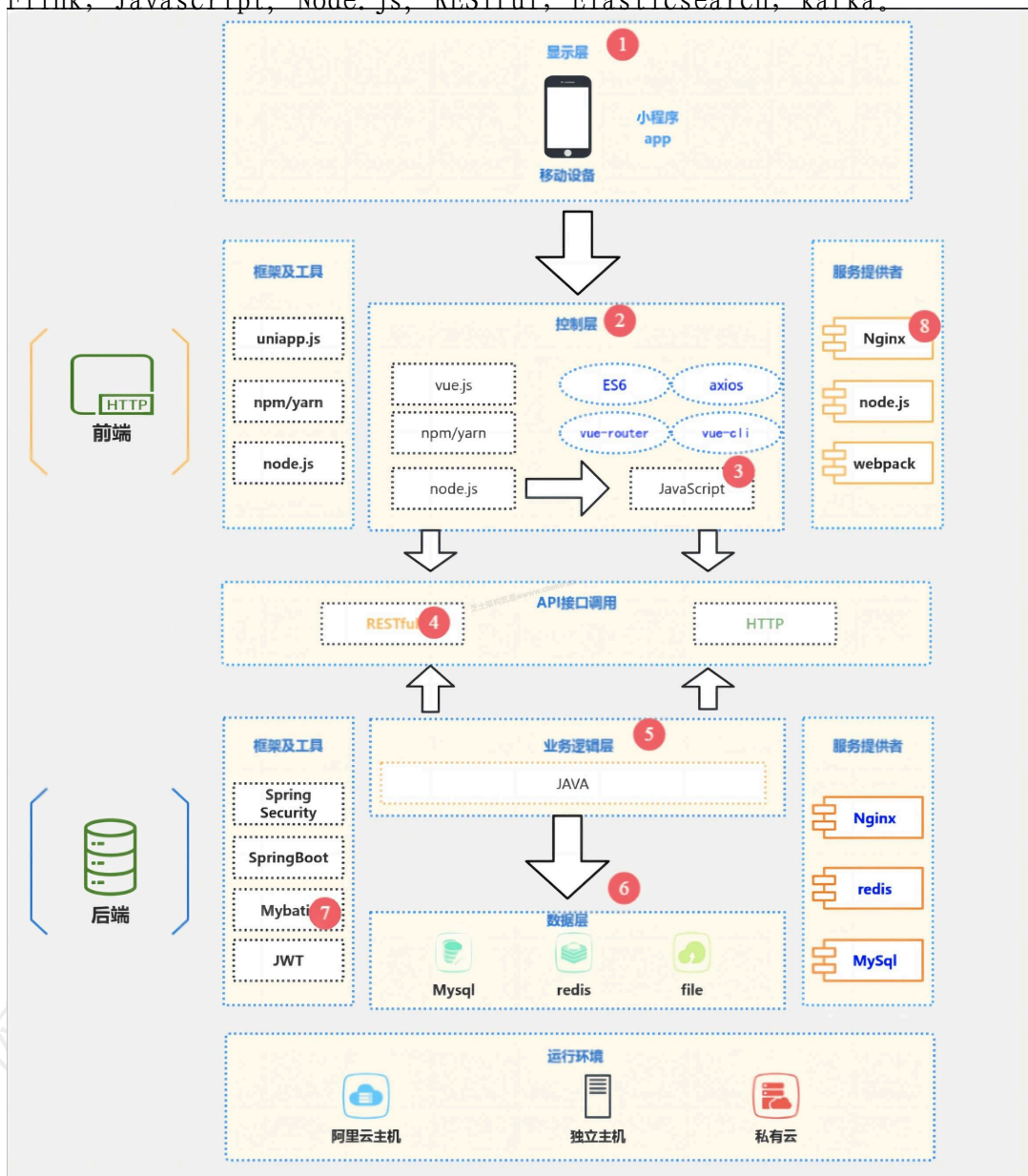
示例："test@example.com" → ["test@example.com"]

适用场景：网页索引、邮箱字段搜索。

问题（2）

请根据下图所示的商品推荐系统架构图，将下列名称填写空白处的内容（原题是红色标注处挖空）。

备选名称：接入层、显示层、数据层、应用层、业务逻辑层、控制层、网络层、Mybatis, Nginx, Flink, Javascript, Node.js, RESTful, Elasticsearch, kafka。



思路解析

这道题不严谨，凯恩逐个选项分析：

编号①“显示层”：表示的是移动端的展示终端，如小程序、App 等。这部分没问题

编号②“控制层”：这里指的是基于 Vue.js 的前端控制逻辑，但“控制层”这个词在后端 MVC

架构中也广泛使用（如 Controller），所以不太准确。凯恩认为更好的说法是“前端逻辑层”或“前端控制层”。

编号③ “JavaScript”：“JavaScript”更像是一种基础语言支撑。Node.js 是 JavaScript 的运行平台 或者说执行环境。这里箭头的语义有点奇怪。

编号④ “RESTful API”：这部分表示前后端接口通信方式，使用 REST 风格，和右侧 HTTP 相对应。

编号⑤ “业务逻辑层”：表示后端核心业务逻辑处理部分，通常由 Java 实现。结构上容易理解。

编号⑥ “数据层”：用于描述后端的数据存储，包括 MySQL、Redis、文件等。这个划分也比较常见，没有歧义。

编号⑦ “MyBatis”：包括 SpringBoot、MyBatis、JWT 等。这个也没歧义。

编号⑧ “服务提供者”：用于展示部署服务中的组件，如 Nginx、Node.js、Webpack 等。但“服务提供者”这个说法容易让人联想到第三方服务商、微服务组件等，表意不够准确。因此凯恩认为这块叫做“部署与运行服务”或“服务组件”更合适。

问题（3）

该方案中使用了RESTful风格，请用150字描述如何使用前后端分离如何实现RESTful。

思路解析

REST (Representational State Transfer) 是一种基于资源的架构风格。RESTful 接口设计强调：

使用 **统一的资源路径（URL）** 表达对象；

通过 **HTTP 方法（GET、POST、PUT、DELETE）** 表达操作；

采用 **无状态通信**，每次请求都自包含信息；

返回结果通常为 **JSON 格式**，结构统一清晰。

示例：

GET /users 获取用户列表

POST /users 创建用户

PUT /users/1 更新 ID 为 1 的用户

DELETE /users/1 删除用户

前后端分离是指：**前端与后端分属不同的项目/服务，各自独立开发、部署、运行，通过接口进行数据交互。**

前端使用 Vue、React 等框架，专注 UI 和交互；

后端负责提供 RESTful API，处理业务逻辑和数据存储；

通信通过 Ajax、Axios 等工具进行 HTTP 请求；

前后端可以并行开发，提升效率，适配多终端需求（如 Web、移动端）。

在实际的答题中，你要结合上面两个部分综合作答。

答案-问题（1）

Simple 分词器：按非字母字符拆分文本，转为小写，适合英文文本，不保留标点和数字。

Whitespace 分词器：仅按空格分词，不做大小写转换或符号过滤，保留原始形式，适用于格式固定的内容。

Keyword 分词器：不分词，整体作为一个词项，常用于精确匹配场景，如ID、标签等。

答案-问题（2）

（1）显示层

（2）控制层

（3）JavaScript

- (4) RESTful
- (5) 业务逻辑层
- (6) 数据层
- (7) MyBaits
- (8) Nginx

答案-问题（3）

在前后端分离架构中，前端通过 HTTP 协议调用后端提供的 RESTful 接口进行数据交互。后端根据资源（如用户、商品）设计统一的 URL 路由，并通过 GET、POST、PUT、DELETE 等方法表示不同操作。前端通过 Ajax 或 Axios 发起请求，后端返回 JSON 格式数据，实现逻辑与视图解耦，提高了开发效率与模块复用性。这种方式契合现代前端框架与微服务架构的协作需求。

第24章——Web 表现层与 MVC

(2013年11月系统架构设计师真题-第4题)

阅读以下有关表现层设计方面的说明，在答题纸上回答问题1至问题3。

【说明】

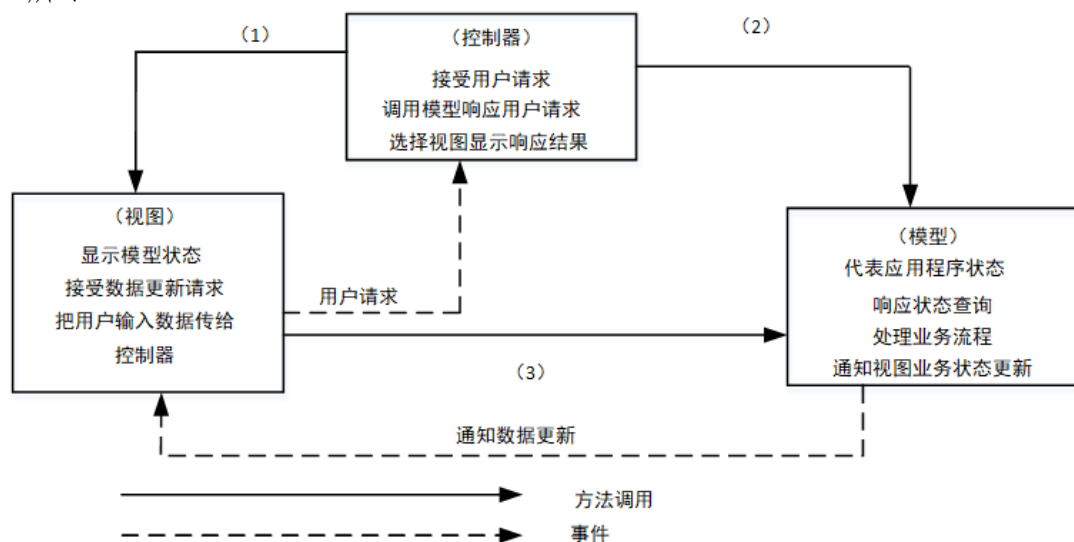
某商业银行欲开发一套个人银行系统，为用户提供常见的金融服务，包括转账、查询、存款变更和个人信息管理等功能。该软件除了业务需求外，还有一些特殊的表现层需求：

- (1) 根据用户级别的不同，界面和可用功能是不同的；
- (2) 支持Web、Windows、手机App等多种不同类型的界面；
- (3) 考虑到将来功能的扩展，需要系统支持界面的定制以及动态生成等功能，以降低系统维护和新功能发布的成本。

经过对需求的讨论，该银行初步决定采用MVC模式设计该个人银行系统的表现层，采用XML作为GUI的描述语言，并应用XML的界面管理技术来实现灵活的界面配置、界面动态生成和界面定制。

问题（1）

MVC模式强制性地将一个应用处理流程按照模型、视图、控制的方式分离，三者的协作关系如图4-1所示。



请填写图4-1中的（1）～（3），并简要说明在该个人银行系统中采用MVC模式对界面设计的作用。

思路解析

MVC（Model-View-Controller）模式是一种经典的表现层架构风格，核心思想是将业务逻辑（模型）、界面展示（视图）、用户交互（控制器）分离。在图 4-1 的交互关系中：控制器负责“选择视图”，即用户操作由控制器解释后映射到某个具体的视图展示；模型负责“业务处理”，即完成数

据存取和业务逻辑计算；视图则通过“状态查询”从模型获取数据并刷新界面。这种分工实现了关注点分离，避免了界面逻辑和业务逻辑耦合。

在个人银行系统中应用 MVC 模式具有显著作用。首先，**支持多界面扩展**：无论是 Web、Windows 客户端，还是手机 App，都可以复用同一套模型逻辑，只需开发不同的视图层即可。其次，**降低维护成本**：随着银行业务扩展（如增加理财、贷款功能），只需扩展模型接口并更新控制器逻辑，而不必大幅修改已有界面。再次，**提升可定制性**：用户可根据身份级别看到不同视图，但底层模型与控制器保持一致，从而实现界面的灵活配置。

问题（2）

请从设计模式的角度，简要说明设计方案采用XML作为GUI描述语言的机制。

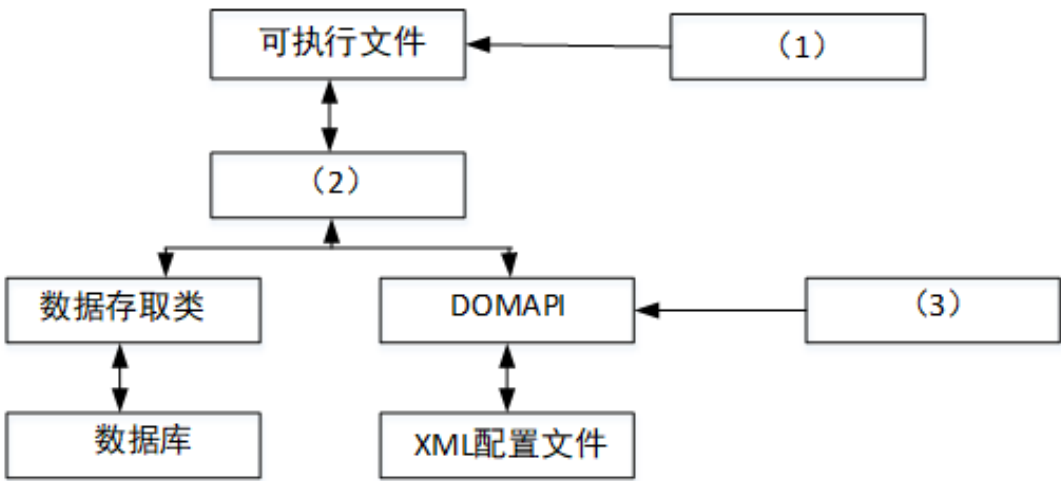
思路解析

在软件工程中，**策略模式（Strategy Pattern）** 是一种定义一系列算法（或行为）的方式，并将每一个算法封装起来，使它们可以相互替换，从而让算法的变化独立于使用它的客户端。在表现层设计中，XML 被用作 GUI 的描述语言，本质上相当于把“界面定义”与“具体实现”解耦，使界面展现逻辑可由不同解析策略决定。

在该银行系统中，GUI 的实现可能基于 **Web 技术（HTML/JS）、Windows 桌面框架（WinForm/WPF）、移动端框架（Android/iOS）** 等。如果采用硬编码方式，界面需要为每个平台单独开发和维护，工作量大且难以统一。而引入 XML 描述语言后，界面结构与布局由 XML 定义；系统根据不同平台加载不同的 **解析器（具体策略类）**，生成相应的视图实例。例如，在 Web 场景中调用 HTML 渲染解析器，在移动端调用 Android XML 渲染解析器。这种机制不仅减少了重复开发，还能在新增展示平台时通过扩展新策略解析器来适配，无需修改原有 XML 文件。由此，XML 描述语言的引入很好地体现了**策略模式的思想：以不变的界面配置，适配多变的表现实现。**

问题（3）

基于XML的界面管理技术可实现灵活的界面配置、界面动态生成和界面定制，其思路是用XML生成配置文件及界面所需的元数据，按不同需求生成界面元素及软件界面，其技术框图如图4-2所示。



请将恰当的内容填入图4-2中的（1）～（3），并简要解释说明其含义。

思路解析

基于 XML 的界面管理技术能够有效支持个人银行系统的灵活性需求。其核心思路是把界面元数据抽象到 XML 配置文件中，系统运行时解析这些元数据并动态生成 GUI，从而支持不同级别用户、不同展示平台以及未来功能扩展。图 4-2 中的三个环节分别代表了 XML 驱动界面的三种应用方式。

首先，**界面配置**是静态层面的定制，主要在系统初始化阶段进行。管理员或开发人员可以通过修改 XML 配置文件，来决定界面初始布局与可用功能，例如为普通用户隐藏高级功能，为 VIP 用户开启

理财服务入口。通过配置机制，系统功能能够被裁剪、重组、扩充，减少了硬编码修改的代价。其次，**界面定制**是动态层面的个性化调整。用户在系统运行时可对菜单、工具栏、键盘命令等属性（文字、图标、位置、大小等）进行修改，并将调整结果保存到配置文件中，实现下次运行时的复用。这保证了系统的**可定制性与用户体验**。最后，**界面动态生成**是运行时的自动化渲染过程，系统通过 DOM API 解析 XML 文件描述的界面元素，再结合数据库中存储的相关数据，在运行中即时构建 GUI。这种机制允许不同平台根据相同的配置文件动态生成界面，支持多端统一与快速扩展。

答案-问题（1）

（1）选择视图 （2）业务处理 （3）状态查询

MVC模式对该个人银行系统的作用：（1）允许多种界面的扩展，视图的变更与增加，与模型无关；（2）易于维护，控制器和视图随着模型的扩展而扩展，只要保持公共接口，控制器和视图的旧版本可以继续使用；（3）可支持功能强大的用户界面。

答案-问题（2）

从设计模式的角度来说，整个XML表现层解析的机制是一种策略模式。在调用显示GUI时，不是直接调用特定的表现技术的API，而是装载GUI对应的XML配置文件，然后根据特定的表现技术的解析器解析XML，得到GUI视图实例对象。这样，对于GUI开发人员来说，GUI视图只需要维护一套XML文件即可。

答案-问题（3）

- （1）界面定制
- （2）界面动态生成
- （3）界面配置

界面配置是对用户界面的静态定义，通过读取配置文件的初始值对界面配置。由界面配置对软件功能进行裁剪、重组和扩充，以实现特殊需求。

界面定制是对用户界面的动态修改过程，在软件运行过程中，用户可按需求和使用习惯，对界面元素（如菜单、工具栏、键盘命令）的属性（如文字、图标、大小、位置等）进行修改。软件运行结束后，界面定制的结果被保存。

界面动态生成是系统通过 DOM API 读取XML 配置文件的表示层信息（初始界面大小、位置等），通过数据存取类读取数据库中的数据层信息，运行时由界面元素动态生成界面。界面配置和定制模块在软件运行前后修改配置文件、更改界面内容。

第25章——响应式 Web 与多终端 (2017年11月系统架构设计师真题-第5题)

阅读以下关于 Web 系统架构设计的叙述，在答题纸上回答问题 1至问题 3。

【说明】

某电子商务企业因发展良好，客户量逐步增大，企业业务不断扩充，导致其原有的B2C商品交易平台已不能满足现有业务需求。因此，该企业委托某软件公司重新开发一套商品交易平台。该企业要求新平台应可适应客户从手机、平板设备、电脑等不同终端设备访问系统，同时满足电商定期开展"秒杀"、"限时促销"等活动的系统高并发访问量的需求。面对系统需求，软件公司召开项目组讨论会议，制定系统设计方案。讨论会议上，王工提出可以应用响应式Web设计满足客户从不同设备正确访问系统的需求。同时，采用增加镜像站点、CDN内容分发等方式解决高并发访问量带来的问题。李工在王工的提议上补充，仅仅依靠上述外网加速技术不能完全解决高用户并发访问问题，如果访问量持续增加，系统仍存在崩溃可能。李工提出应同时结合负载均衡、缓存服务器、Web 应用服务器、分布式文件系统、分布式数据库等方法设计系统架构。经过项目组讨论，最终决定综合王工和李工的思路，完成新系统的架构设计。

问题（1）

请用200字以内的文字描述什么是"响应式 Web 设计"，并列举2个响应式Web设计的实现方式。

思路解析

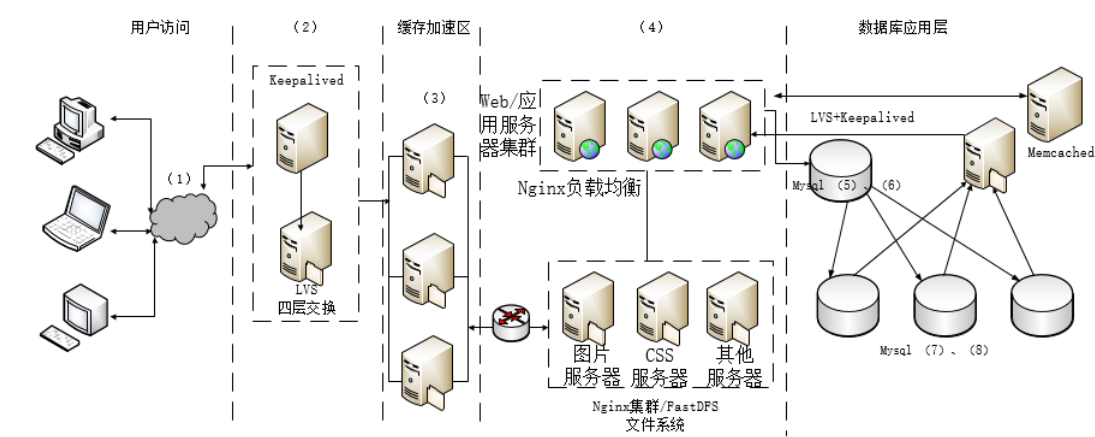
响应式Web设计是针对各种设备的浏览器优化设计，确保网页在不同终端（如PC、手机、平板等）上都能有良好的浏览体验。通过这种设计，网站能够根据用户的设备特性（如屏幕尺寸、分辨率）进行自动适配，达到自适应的效果。

流式布局（Fluid Layout）：使用相对单位（如百分比）来设计网页元素，使得网页可以根据设备屏幕大小自动调整。

媒体查询（Media Queries）：CSS3的功能，允许开发者根据设备的特征（如宽度、分辨率）加载不同的样式，从而使页面在各种设备上呈现最佳显示效果。

问题（2）

综合王工和李工的提议，项目组完成了新商品交易平台的系统架构设计方案。新系统架构图如图 5-1 所示。请从选项（a）-（j）中为架构图中（1）-（8）处空白选择相应的内容，补充支持高并发的 Web 应用系统架构设计图。



- (a) Web应用层
- (b) 界面层
- (c) 负载均衡层
- (d) CDN 内容分发
- (e) 主数据库
- (f) 缓存服务器集群
- (g) 从数据库
- (h) 写操作
- (i) 读操作
- (j) 文件服务器集群

思路解析

在该架构设计中，考虑到高并发访问的问题，王工和李工的方案结合了多种技术和方法：

CDN内容分发（d）：通过将静态内容分发到离用户更近的边缘服务器，减少了中心服务器的负担，提高了系统的响应速度。

负载均衡层（c）：通过负载均衡技术，将用户请求均匀地分配到不同的服务器节点上，从而有效处理高并发请求，避免单点故障。

缓存服务器集群（f）：通过设置缓存服务器，减少数据库的访问频率，提高数据访问速度。缓存服务器集群可存储常用数据，加快响应时间。

Web应用层（a）：这是处理用户请求和业务逻辑的核心层，它负责与后端服务（如数据库、缓存

) 交互, 处理用户请求。

主数据库 (e): 用于存储核心业务数据, 包括用户信息、商品信息等。

写操作 (h): 所有写操作 (如数据更新、订单生成等) 通常写入主数据库。

从数据库 (g): 从数据库主要用于读取操作, 缓解主数据库的负担, 提供更高的查询性能。

读操作 (i): 读操作通常从从数据库获取, 减少了主数据库的负担, 提升了系统的并发处理能力。

问题 (3)

根据李工的提议, 新的B2C商品交易平台引入了主从复制机制。请针对B2C商品交易平台的特点, 简要叙述引入该机制的好处。

思路解析

主从复制机制是指将数据库的读操作和写操作分离, 主数据库处理所有的写操作 (如插入、更新、删除), 从数据库用于处理读操作 (如查询)。这种机制在B2C商品交易平台中具有以下优势:

提升性能: 通过读写分离, 主数据库承担写操作, 而从数据库专门处理读请求, 减少了主数据库的负载, 从而提升了系统的整体性能。

可扩展性更优: 随着访问量的增加, 可以通过增加更多的从数据库来应对更高的读请求, 避免了系统性能瓶颈。

提升可用性: 如果一台从数据库发生故障, 系统仍能继续提供服务, 因为写操作依赖主数据库, 而读取操作可以从其他健康的从数据库获取。

负载均衡: 多个从数据库可以分担读请求的压力, 平衡各个节点的负载, 提高系统处理能力。

数据安全性: 数据在主数据库和从数据库之间进行复制, 增加了数据的冗余备份, 防止了数据丢失, 确保系统的稳定性和可靠性。

答案-问题 (1)

响应式Web设计是一种网页设计方法, 旨在使网站能够在不同设备上提供最佳的用户体验。通过响应式设计, 网页能根据访问设备的屏幕大小、分辨率和方向等特征, 自动调整布局和内容展示方式, 确保页面内容在各种设备上都能清晰、易读和易用。这种设计方法能够提高网站的可访问性和用户满意度, 适应了多样化的设备和屏幕尺寸。

实现响应式Web设计的方式包括流式布局 (Fluid Layout) 和媒体查询 (Media Queries)。流式布局通过使用百分比或相对单位定义元素宽度, 使页面能根据屏幕大小自动调整布局, 适应不同设备的显示尺寸。而媒体查询则利用CSS3中的功能, 根据设备特性 (如屏幕宽度、分辨率等) 应用不同样式表, 以在不同设备上呈现不同布局和样式, 实现页面的响应式展示。

答案-问题 (2)

(1) (d) (2) (c) (3) (f) (4) (a)
(5) (6) (e) (h) (7) (8) (g) (i)

答案-问题 (3)

引入主从复制机制对B2C商品交易平台的好处包括: 提升性能, 通过读写分离提高并发度; 可扩展性更优, 快速增加从服务器应对访问量增加; 提升可用性, 一台从服务器故障不影响系统正常工作; 实现负载均衡, 分担任务; 提升数据安全性, 数据冗余存放多份, 避免数据丢失。

第26章——Java Web 框架与应用服务器 (2016年11月系统架构设计师真题-第4题)

阅读以下关于应用服务器的叙述, 在答题纸上回答问题1至问题3。

【说明】

某电子产品制造公司，几年前开发建设了企业网站系统，实现了企业宣传、产品介绍、客服以及售后服务等基本功能。该网站技术上采用了Web服务器、动态脚本语言PHP。随着市场销售渠道变化以及企业业务的急剧拓展，该公司急需建立完善的电子商务平台。

公司张工建议对原有网站系统进行扩展，增加新的功能（包括订单系统、支付系统、库存管理等），这样有利于降低成本、快速上线；而王工则认为原有网站系统在技术上存在先天不足，不能满足企业业务的快速发展，尤其是企业业务将服务全球，需要提供24小时不间断服务，系统在大负荷和长时间运行下的稳定性至关重要。建议采用应用服务器的Web开发方法，例如J2EE，为企业重新开发新的电子商务平台。

问题（1）

王工认为原有网站在技术上存在先天不足，不能满足企业业务的快速发展，根据你的理解，请用300字以内的文字说明原系统存在哪几个方面的不足。

思路解析

原有系统采用的技术路线较为简单，只依赖Web服务器与PHP动态脚本语言，这类方案非常适合中小型网站，能快速开发和上线，但不足之处十分明显。

首先，在架构层面，PHP主要面向过程开发，其扩展性和代码复用性较差，难以适应大型电子商务平台不断增长的功能需求。相比之下，Java的面向对象机制和J2EE的多层体系架构能够提供更强的模块化与灵活性。

其次，在稳定性方面，PHP适合轻量型应用，一旦系统需要24小时不间断运行，并处理大量并发请求，PHP的运行环境容易成为瓶颈，而J2EE提供了集群、事务管理等成熟机制，更能保证系统稳定性。

第三，在数据库访问层，PHP缺乏统一的访问标准，修改或扩展数据库支持时需要大量手工代码，而Java提供JDBC标准接口，支持多种数据库切换与扩展。

问题（2）

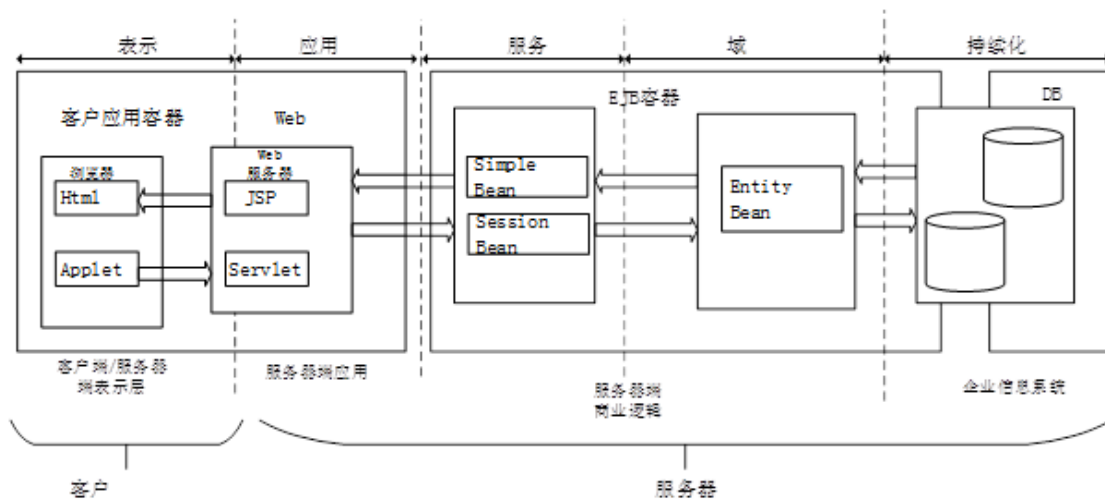
请简要说明应用服务器的概念，并重点说明应用服务器如何来保障系统在大负荷和长时间运行下的稳定性以及可扩展性。

思路解析

应用服务器（Application Server）是现代企业级系统的核心基础设施，通常作为中间件运行在操作系统与应用程序之间，提供统一的运行环境和多种服务支持。它不仅仅是一个Web服务器，还包括会话管理、事务管理、安全认证、消息中间件、对象持久化等功能。对于电子商务类系统，应用服务器承担了“容器”的角色，可以将不同层次的应用逻辑分别部署和托管，并且屏蔽底层复杂性，开发人员只需关注业务逻辑的实现。

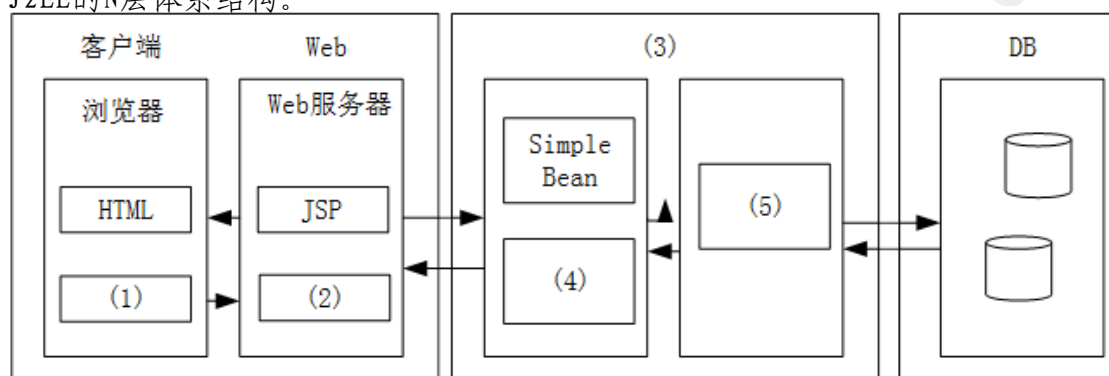
在稳定性和扩展性方面，应用服务器的优势更为突出。**第一**，应用服务器通常支持**集群与负载均衡**，能够将用户请求分散到多台机器处理，从而消除单点瓶颈，并在某一节点故障时自动切换，提高系统可用性。**第二**，它支持**资源池化管理**，如数据库连接池、线程池，通过复用机制减少频繁创建资源带来的性能损耗。**第三**，应用服务器提供**运行时监控与调优**功能，可以根据实时负载情况动态调整资源分配。**第四**，某些应用服务器还支持**自动扩展（Auto Scaling）**，根据系统压力动态增加或减少实例数量，从而保障系统在大规模并发和长时间运行下依旧保持稳定和高效。

因此，应用服务器不仅仅是一个运行环境，更是保障系统在高并发、大流量和长周期运行场景下的关键支撑平台。



问题（3）

J2EE平台采用了多层分布式应用程序模型，实现不同逻辑功能的应用程序被封装到不同的构件中，处于不同层次的构件可被分别部署到不同的机器中。请填写图4-1中（1）~（5）处的空白，完成J2EE的N层体系结构。



思路解析

J2EE（Java 2 Platform, Enterprise Edition）平台采用多层分布式应用模型，将不同逻辑功能划分为不同层次，并封装到不同构件中，进而支持跨平台、跨网络的部署。典型的 J2EE N层架构包括：客户端层、Web层、业务逻辑层和数据持久层。

具体来说：

（1）Applet：属于客户端层，运行在浏览器环境中，负责与用户交互。

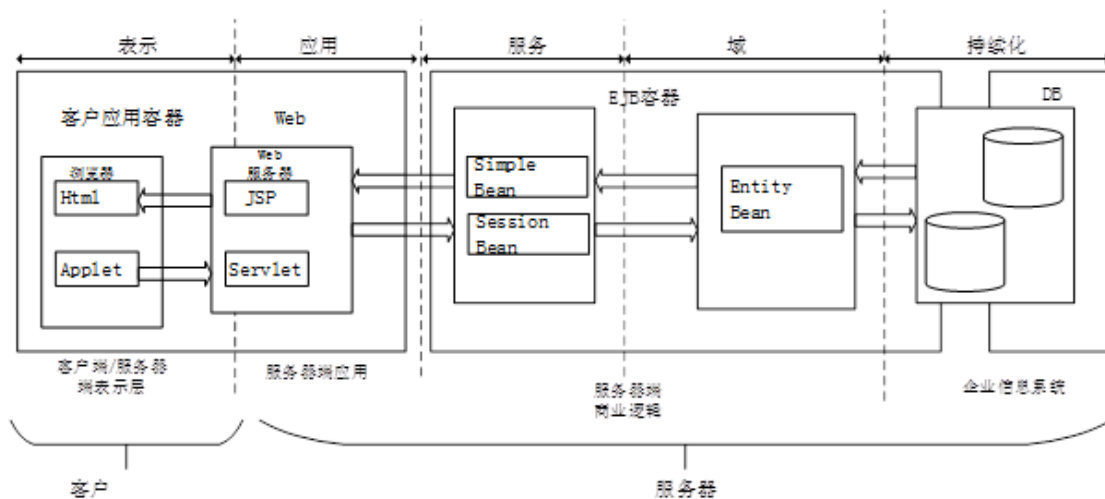
（2）Servlet：位于Web层，处理来自客户端的请求，并与后端业务逻辑层交互。

（3）EJB容器：运行在应用服务器中，用于托管企业级Bean组件，提供事务管理、安全性、分布式服务等功能。

（4）Session Bean：属于业务逻辑层，用于处理用户会话和具体的业务逻辑。

（5）Entity Bean：属于持久层，主要用于表示数据库中的数据实体，实现对象与关系数据库的映射。

这种分层模式的最大好处是 **解耦与可扩展性**。客户端只负责交互，不涉及业务逻辑；Web层主要处理协议与请求转发；业务逻辑层通过Session Bean实现复杂的业务规则；数据访问则由Entity Bean负责。这样一来，系统不仅便于扩展和维护，而且不同层次的组件可以分布在不同物理服务器上，实现负载分担和容错能力。这也是王工推荐使用 J2EE 应用服务器的重要原因。



答案-问题（1）

PHP适合简单的两层或三层架构，但在多层网络架构方面不如Java强大。PHP是面向过程的语言，可修改性和复用性较差，而Java是面向对象的，更易于维护和扩展。在可靠性方面，J2EE平台有成熟解决方案，而PHP缺乏。PHP在数据库访问方面不如Java统一，修改数据库连接工作量大。虽然PHP适合小型项目，但在大型项目中稳定性、可维护性、扩展性和安全性方面不如Java表现出色。综合来看，针对企业电子商务平台的需求，使用J2EE平台重新开发可能更适合，以提升系统的性能、可靠性和安全性。

答案-问题（2）

应用服务器是一种软件框架，用于托管、管理和运行应用程序，提供各种服务和功能，如处理用户请求、管理数据库连接、执行业务逻辑等。应用服务器通过负载均衡、集群部署、资源管理、监控调优和自动扩展等功能来保障系统在大负荷和长时间运行下的稳定性和可扩展性。负载均衡可分发请求，集群部署提高可用性，资源管理避免瓶颈，监控调优实时调整，自动扩展根据负载动态调整服务器实例，确保系统高性能、稳定运行。

答案-问题（3）

(1) Applet (2) Servlet (3) EJB容器 (4) Session Bean (5) Entity Bean

(2023年11月系统架构设计师真题-第2题)

某公司正在开发一套基于 **Java Web** 的企业应用系统，主要涉及 **数据持久化**、**用户认证授权** 和 **跨域访问支持**。在系统设计中，开发团队选择了 **Hibernate** 作为数据持久层框架，并在用户认证中引入 **JWT (JSON Web Token)** 技术。

问题（1）

JWT 是由哪三个部分组成，请写出对应的组成和作用。

思路解析

JWT 的结构由三部分组成，格式为 **Header.Payload.Signature**。

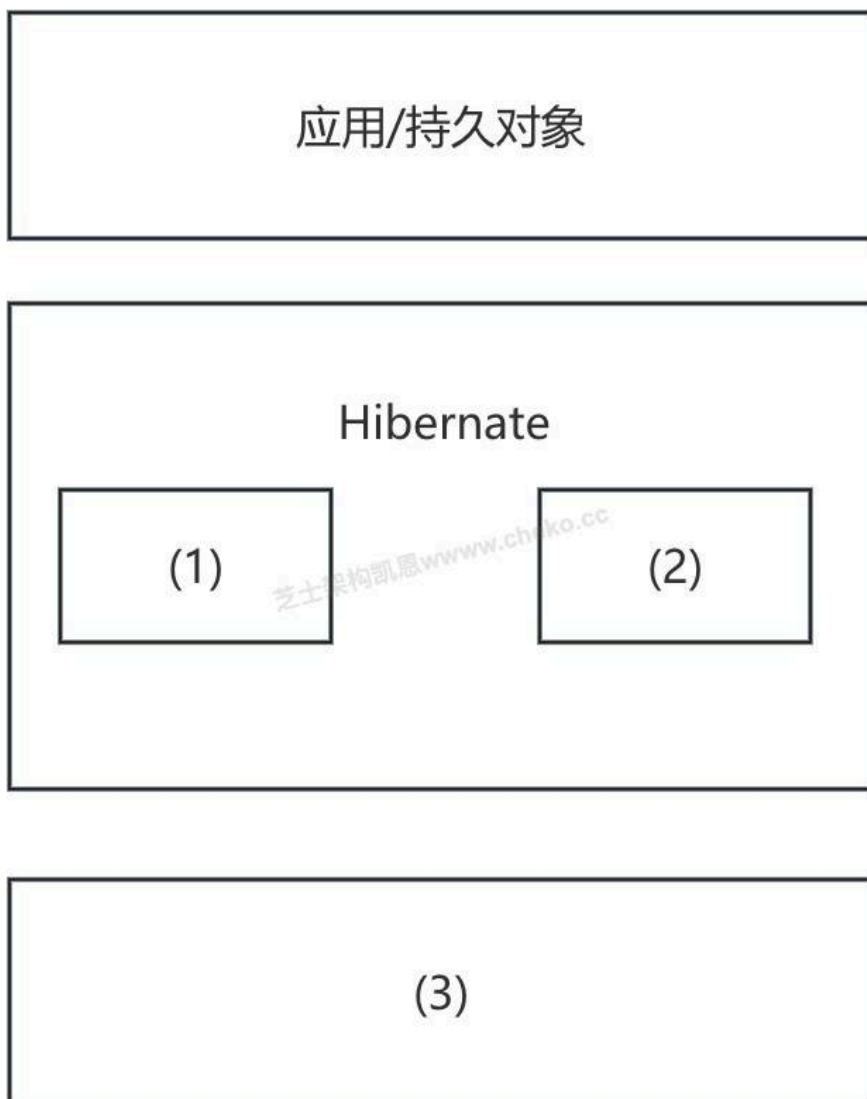
Header（头部）：主要说明使用的签名算法（如 **HMAC SHA256**）和类型（**JWT**），其作用是解析方提供校验依据。

Payload（载荷）：存放用户相关信息以及声明（**Claims**），如用户 **ID**、角色、签发时间、过期时间等。需要注意的是，载荷部分本身未加密，仅做 **Base64** 编码，因此不应存放敏感信息。

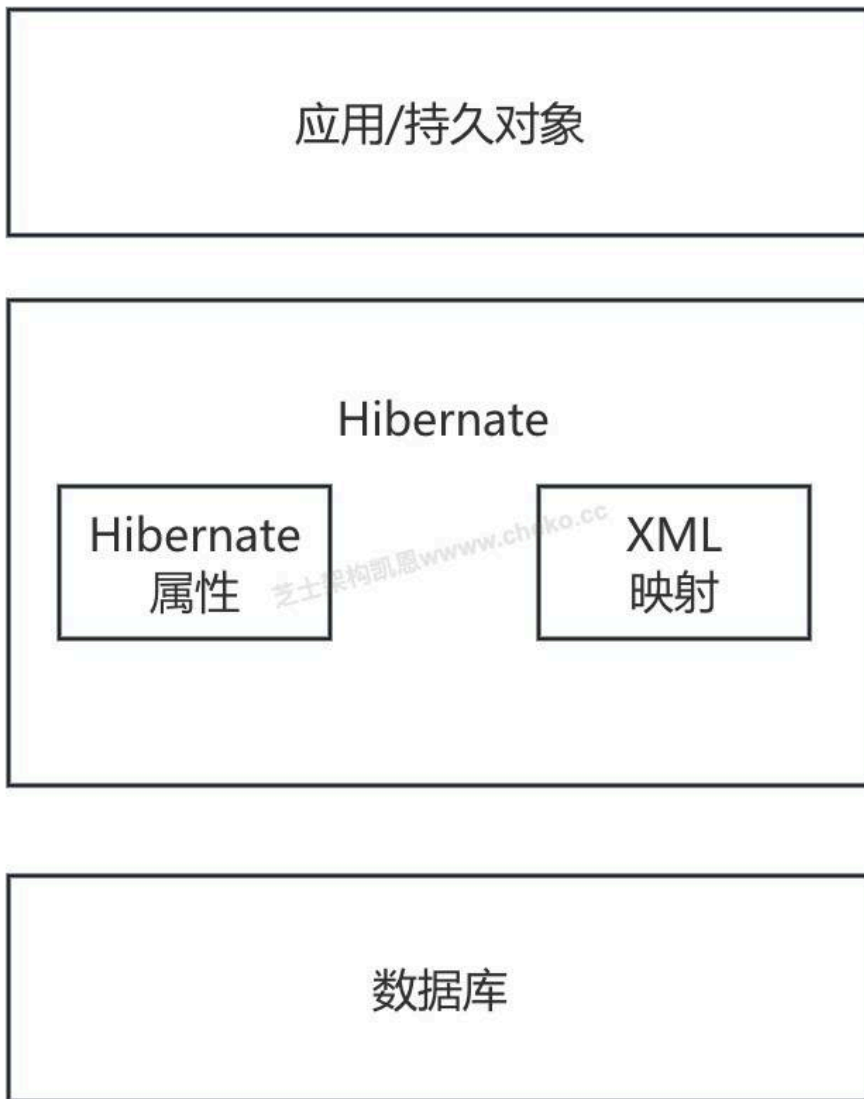
Signature（签名）：使用密钥和 **Header+Payload** 生成，验证 **JWT** 是否完整，确保令牌未被伪造。

问题（2）

请完善下方 Hibernate 的体系架构图。



思路解析



Hibernate 的体系架构，核心思想就是 应用层 → Hibernate 层 → 数据库层，其中 Hibernate 层承担了 ORM（对象关系映射）的关键作用。

(1) **Hibernate 属性**：指 Hibernate 的 配置管理部分，主要由 hibernate.cfg.xml 或 hibernate.properties 文件来定义。内容包括：数据库连接信息（驱动、URL、用户名、密码）、方言（Dialect）、缓存策略、SQL 显示开关等。为 Hibernate 提供运行所需的环境配置。

(2) **XML 映射**：指 Hibernate 的 对象关系映射配置，核心是 .hbm.xml 文件，或者基于 JPA 注解的映射。描述 Java 类与数据库表、类属性与表字段之间的对应关系。让 Hibernate 能够将 Java 对象自动转换为数据库记录，反之亦然。

(3) **数据库**：Hibernate 最终通过 JDBC 与数据库交互。它屏蔽了具体数据库的差异，开发人员只需面向对象编程，而不必直接编写大量 SQL。

答案-问题（1）

Header（头部）：描述算法和类型，如 {"alg": "HS256", "typ": "JWT"}。

Payload（载荷）：存放用户信息和声明，如用户 ID、角色、过期时间。

Signature（签名）：由 Header+Payload+密钥生成，防止数据篡改。

答案-问题（2）

(1) Hibernate 属性 (2) XML 映射 (3) 数据库

第27章——嵌入式开放式架构 (2020年11月系统架构设计师真题-第3题)

阅读以下关于开放式嵌入式软件架构设计的相关描述，回答问题1至问题3。

[说明]

某公司一直从事宇航系统研制任务，随着宇航产品综合化、网络化技术发展的需要，公司的业务量急剧增加，研制新的软件架构已迫在眉睫。公司架构师王工广泛调研了多种现代架构的基础，建议采用基于FACE (Future Airborne Capability Environment)的宇航系统开放式软件架构，以实现宇航系统的跨平台复用，实现宇航软件高质量、低成本的开发。公司领导肯定了王工的提案，并指出公司要全面实施基于FACE的开放式软件架构，应注意每个具体项目在实施中如何有效实现从需求到架构设计的关系，掌握基于软件需求的软件架构设计方法，并做好开放式软件架构中各段间的接口标准化设计工作。

问题(1)

王工指出，软件开发中需求分析是根本，架构设计是核心，不考虑软件需求便进行软件架构设计很可能导致架构设计的失败，因此，如何把软件需求映射到软件架构至关重要。请从描述语言、非功能性需求描述、需求和架构的一致性等三个方面，用 300字以内的文字说明软件需求到架构的映射存在哪些难点。

思路解析

软件需求是指为用户解决某一问题或达到某一目标所需的软件功能；系统或系统构件为了满足合同、规约、标准或其他正式实行的文档而必须满足或具备的软件功能。

软件需求包括三个不同的层次：业务需求、用户需求和功能需求；软件需求规格说明还应包括非功能需求，它描述了系统展现给用户的行为和执行的操作等。它包括产品必须遵从的标准、规范和合约；外部界面的具体细节；性能要求；设计或实现的约束条件及质量属性。

架构，又名软件架构，是有关软件整体结构与组件的抽象描述，用于指导大型软件系统各个方面的设计。

通常在软件开发过程中，需求会随着开发深入而有所变化，而架构又不能完全地将需求全部反映出来，因此，如何把软件需求映射到软件架构是至关重要的一个问题。

(1) 从描述语言方面来讲：软件需求是频繁获取的非正规的自然语言，而软件架构常用的是一种正式语言。

(2) 从非功能性需求描述方面来讲：系统属性中描述的非功能性需求通常很难在架构模型中形成规约。

(3) 从需求和架构的一致性方面来讲：从软件需求映射到软件架构的过程中，保持一致性和可追溯性很难，且复杂程度很高，因为单一的软件需求可能定位到多个软件架构的关注点。反之，架构元素也可能有多个软件需求。

问题(2)

图3-1是王工给出的FACE架构布局，包括操作系统、I/O 服务、平台服务、传输服务和可移植组件等5个段；操作系统、I/O 和传输等3个标准接口。请分析图3-1给出的FACE架构的相关信息，用300字以内的文字简要说明FACE5个段的含义。

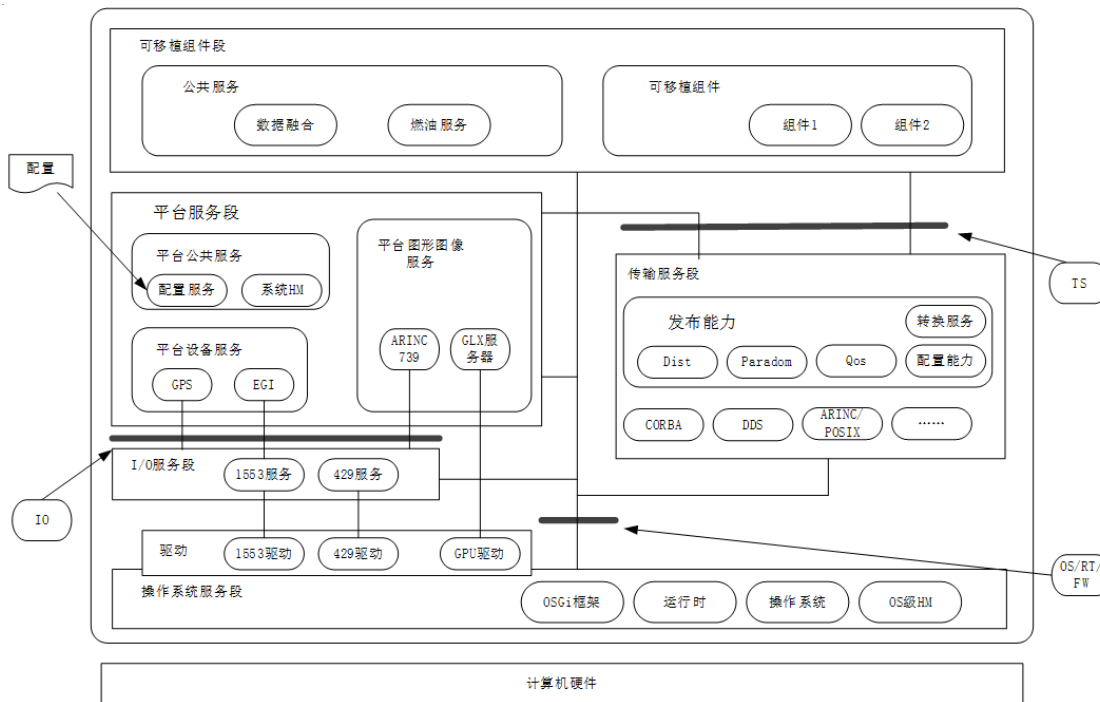


图 3-1 FACE 架构

思路解析

FACE架构布局包括操作系统、I/O服务、平台服务、传输服务和可移植组件等5个段，其中操作系统、I/O和传输是3个标准接口。简要说明FACE5个段的含义如下：

操作系统：这个段指的是操作系统提供的基本功能和服务，包括处理器管理、内存管理、文件系统等。操作系统提供了软件与硬件之间的接口，为应用程序提供运行环境。

I/O服务：I/O服务段包括输入/输出服务，用于管理系统与外部设备之间的数据传输。这个段涵盖了与外部设备交互的功能，如文件读写、网络通信等。

平台服务：平台服务段提供了一系列通用的服务和功能，如安全性、日志记录、配置管理等。这个段的目的是提供一些通用的服务，以支持系统的稳定性和可靠性。

传输服务：传输服务段涵盖了数据传输和通信相关的功能，包括网络通信、数据传输协议等。这个段负责处理数据在系统内部和系统外部之间的传输。

可移植组件：可移植组件段包括可以在不同平台上移植和重用的组件，如库、工具等。这个段的目的是提供可重用的组件，以便在不同系统中实现相似的功能。

问题（3）

FACE架构的核心能力是可支持应用程序的跨平台执行和可移植性，要达到可移植能力，必须解决应用程序的紧耦合和封装的障碍。请用200字以内的文字简要说明在可移植性上，应用程序的紧耦合和封装问题的主要表现分别是什么，并给出解决方案。

思路解析

在可移植性上，应用程序的紧耦合和封装问题主要表现为：紧耦合：应用程序中各个模块之间存在较强的依赖关系，一个模块的修改可能会影响其他模块，导致难以将模块独立地移植到其他平台。

封装问题：应用程序的某些功能或数据没有良好的封装，导致与特定平台相关的部分与通用部分混合在一起，使得移植时难以区分和处理。

解决方案包括：松耦合设计：通过使用接口、抽象类等方式降低模块之间的耦合度，使模块之间的依赖关系更加灵活，便于在不同平台上移植和重用。良好的封装：将与特定平台相关的部分进行良好的封装，使其与通用部分分离，提高模块的独立性和可移植性。通过封装，可以隐藏平台相关细节，使得应用程序更易于移植到不同平台上。

答案-问题（1）

- (1) 需求和架构描述语言存在差异: 软件需求是频繁获取的非正规的自然语言, 而软件架构常用的是正式语言。
- (2) 非功能属性难于在架构中描述: 系统属性中描述的非功能性需求通常很难在架构模型中形成规范。
- (3) 需求和架构的一致性难以保障: 从软件需求映射到软件架构的过程中, 保持一致性和可追溯性很难, 且复杂程度很高, 因为单一的软件需求可能定位到多个软件架构的关注点。反之, 架构元素也可能有多个软件需求。

答案-问题（2）

操作系统服务段: 为FACE架构其他段提供操作系统、运行时和操作系统级健康监控等服务。通过开放式OSGi框架为上层功能提供OS标准接口, 并可实现上层组件的即插即用能力。I/O服务段: 主要针对专用I/O设备进行抽象, 屏蔽平台服务段软件与硬件设备的关系。由于图形服务软件和GPU处理器紧密相关, 因此I/O服务段不对GPU驱动进行抽象。平台服务段: 主要是指用户需要的共性软件, 如: 系统级健康监控(HM)、配置、日志和流媒体等服务。本段可包括平台公共服务、平台设备服务和平台图像服务等三类。传输服务段: 主要为上层可移植组件段提供平台性的数据交换服务。可移植组件将通过传输服务段提供的服务实现交换, 禁止组件间直接调用。可移植组件段: 提供了多组件使用能力和功能服务。主要包括公共服务和可移植组件两类。

答案-问题（3）

紧耦合问题主要表现在: I/O 问题、业务逻辑问题和表现问题。解决方案: 可采用分离原则, 通过隔离实现硬件特定信息和少数模块的代码, 减少耦合性。封装问题主要表现在: ICD 硬编码问题、组件的紧耦合问题、直接调用问题。解决方案: 可以通过提供数据源或槽的软件服务的方法, 将紧耦合组件分解出应用程序, 并将平台相关部分加入计算环境中, 在计算平台内提供数据源或槽的软件服务, 并实现接口标准化。

第28章——实时系统与 RTOS (2018年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式实时系统相关技术的叙述, 在答题纸上回答问题1和问题2。

【说明】

某公司长期从事宇航领域嵌入式实时系统的软件研制任务。公司为了适应未来嵌入式系统网络化、智能化和综合化的技术发展需要, 决定重新考虑新产品的架构问题, 经理将论证工作交给王工负责。王工经调研和分析, 完成了新产品架构设计方案, 提交公司高层讨论。

问题（1）

王工提交的设计方案中指出: 由于公司目前研制的嵌入式实时产品属于简单型系统, 其嵌入式子系统相互独立, 功能单一, 时序简单。而未来满足网络化、智能化和综合化的嵌入式实时系统将是一种复杂系统, 其核心特征体现为实时任务的机理、状态和行为的复杂性。简单任务和复杂任务的特征区分主要表现在十个方面。请参考表3-1给出的实时任务特征分类, 用题干中给出的

(a) ~ (t) 20个实时任务特征描述, 补充完善表3-1给出的空 (1) ~ (14)。

- (a) 任务属性不会随时间变化而改变;
- (b) 任务的属性与时间相关;
- (c) 任务仅可以从非连续集中获取特征变量;
- (d) 任务变量域是连续的;
- (e) 功能原理不依赖于上下文;
- (f) 功能原理依赖于上下文;
- (g) 任务行为可以用step-by-step顺序分析方法来理解;

- (h) 许多任务在产生访问活动时相互间是并发处理的，很难用step-by-step方法分析；
- (i) 因果关系相互影响；
- (j) 行为特征依赖于大量的反馈机制；
- (k) 系统内构成、策略和描述是相似的；
- (l) 系统内存在许多不同的构成、策略和描述；
- (m) 功能关系是非线性的；
- (n) 功能关系是线性的；
- (o) 不同的子任务是相互独立的，任务内部仅存在少量的交互操作；
- (p) 不同的子任务有很高的交互操作，要把一个单任务的行为隔离开是困难的；
- (q) 域特征有非常整齐的原则和规则；
- (r) 许多不同的上下文依赖于规则；
- (s) 原理和规则在表面属性上很容易被识别；
- (t) 原理被覆盖、抽象，而不会在表面属性上被识别。

特征分类	简单任务 (simple task)	复杂任务 (complex task)
静态/动态	(a)	(b)
连续/非连续	(1)	(2)
子系统的独立性	(3)	(4)
顺序/并行执行	(5)	(6)
单一性/混合性	(7)	(8)
工作原理	(9)	(10)
线性/非线性	(11)	(12)
上下文相关性	(13)	(14)
规律/不规律	(q)	(r)
表面属性	(s)	(t)

思路解析

本题考查**嵌入式实时任务特征分类**。在嵌入式实时系统中，任务可以分为简单任务与复杂任务。简单任务多见于独立型嵌入式设备（如单功能传感器控制），而复杂任务往往出现在网络化、智能化装备中（如飞控系统、卫星姿态控制）。它们的区别主要体现在十个方面：①静态/动态特性：简单任务属性固定，不随时间改变，而复杂任务与时间紧密相关；②非连续/连续性：简单任务特征变量不连续，而复杂任务变量域连续；③独立/交互性：简单子任务相互独立，复杂任务交互性强；④顺序/并行性：简单任务可逐步分析，而复杂任务常有并发处理；⑤相似/多样性：简单任务结构策略相似，复杂任务存在多种构成与描述；⑥因果/反馈性：简单任务因果关系明确，复杂任务依赖大量反馈机制；⑦线性/非线性：简单任务功能关系线性，复杂任务呈非线性；⑧上下文无关/相关：简单任务原理与上下文无关，复杂任务需依赖上下文；⑨规则性：简单任务规则整齐易识别，复杂任务规则受上下文影响，不规律。

因此，在表 3-1 中，(1) ~ (14) 的匹配结果正体现了这些对比特征。例如，简单任务的“功能原理不依赖上下文 (e)”对应复杂任务的“功能原理依赖上下文 (f)”；简单任务“顺序分析 (g)”对应复杂任务“并发分析困难 (h)”；简单任务“线性关系 (n)”对应复杂任务“非线性关系 (m)”。通过这种对比，可以看出复杂实时任务具备高动态性、高交互性与强反馈依赖，这正是未来网络化、智能化嵌入式系统的本质特征。

问题 (2)

工设计方案中指出：要满足未来网络化、智能化和综合化的需求，应该设计一种能够充分表达嵌入式系统行为的、且具有一定通用性的通信架构，以避免复杂任务的某些特征带来的通信复杂性。通常为了实现嵌入式系统中计算组件间的通信，在架构上需要一种简单的架构风格，用于屏蔽不同协议、不同硬件和不同结构组成所带来的复杂性。图3-1给出了一种“腰 (Waistline)”型通信模式的架构风格。腰型架构的关键是基本消息通信 (BMTS)，通常BMTS的消息与时间属性相关，支持事件

触发消息、速率约束消息和时间触发消息。

请用400以内的文字说明基于BMTS的消息通信网络的主要特征和上述三种消息的基本含义，并举例给出两种具有时间触发消息能力的网络总线。



思路解析

BMTS (Basic Message Transmission System, 基本消息传输系统) 是一种“腰型架构”的核心机制，其目标是屏蔽底层协议、硬件差异，为上层计算组件提供统一的消息通信接口。其特征包括：①支持一对一或一对多消息传输；②在实时环境下保持低延迟和小抖动；③适应多种消息模式，满足不同实时任务需求。

在 BMTS 中，**事件触发消息**适合突发事件处理，如传感器告警；**速率约束消息**适合周期不严格但有速率上限的场景，如状态更新；**时间触发消息**则适合高可靠、强实时的场景，如飞控指令分发。当前具有时间触发能力的总线包括 **TTE**、**AFDX** 和 **FC**，它们广泛应用于航空航天和高可靠嵌入式系统，保证任务在严格时间约束下执行，是未来嵌入式实时系统实现网络化、智能化的基础通信技术。

答案-问题 (1)

- (1) (c) (2) (d) (3) (o) (4) (p)
(5) (g) (6) (h) (7) (k) (8) (l)
(9) (i) (10) (j) (11) (n) (12) (m)
(13) (e) (14) (f)

答案-问题 (2)

BMTS的消息通信网络主要特征为：一个计算组件传输消息到另一个或多个接收组件，传输可靠性高，延迟低，抖动微小。

事件触发消息：以事件作为触发方式，事件发生便触发相应消息。

速率约束消息：消息偶发性产生，而不考虑发送者承诺消息不超出最大消息速率。

时间触发消息：发送者和接收者遵循一个精确的时间片周期完成消息的发送与接收。

具有时间触发消息能力的网络总线：航空电子全双工交换式以太网 (Avionics Full Duplex Switched Ethernet, AFDX)，时间触发以太网 (Time-Triggered Ethernet, TTE)，FC (Fiber Channel) 总线

(2016年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式实时系统设计的描述，回答问题1至问题3。

【说明】

嵌入式系统是当前航空、航天、船舶及工业、医疗等领域的核心技术，嵌入式系统可包括实时系统与非实时系统两种。某宇航公司长期从事航空航天飞行器电子设备的研制工作，随着业务的扩大，需要大量大学毕业生补充到科研生产部门。按照公司规定，大学毕业生必须进行相关基础知识培训，为此，公司经理安排王工对他们进行了长达一个月的培训。

问题（1）

王工在培训中指出：嵌入式系统主要负责对设备的各种传感器进行管理与控制。而航空航天飞行器的电子设备由于对时间具有很强的敏感性，通常由嵌入式实时系统进行管控，请用300字以内文字说明什么是实时系统，实时系统有哪些主要特性。

思路解析

实时系统的定义：实时系统是指能够在 **特定时间约束内完成指定功能** 的计算机系统，它不仅要保证功能正确，还要保证在时间限制下完成。例如，飞行控制系统必须在几毫秒内对姿态变化作出响应，否则飞行器可能失控。实时系统可分为 **硬实时系统** 和 **软实时系统**。硬实时系统必须严格满足时间要求（如火箭点火控制），任何延迟都不可接受；软实时系统允许偶尔违反时间约束（如多媒体播放），但总体性能需可接受。

实时系统的主要特性：

时间约束性：这是实时系统的核心，即任务必须在截止时间内完成，否则会导致功能失效或灾难性后果。

可预测性：系统必须能够对执行时间和行为进行预测，调度策略应确保任务在预期范围内完成。

高可靠性：系统通常应用于航空、医疗等安全关键场景，必须具备长时间稳定运行的能力。

与外部环境交互性：实时系统通常依赖传感器与执行器，必须能够即时响应外部环境变化。

多任务性：需要同时处理多种并发任务，例如数据采集、信号处理和控制指令。

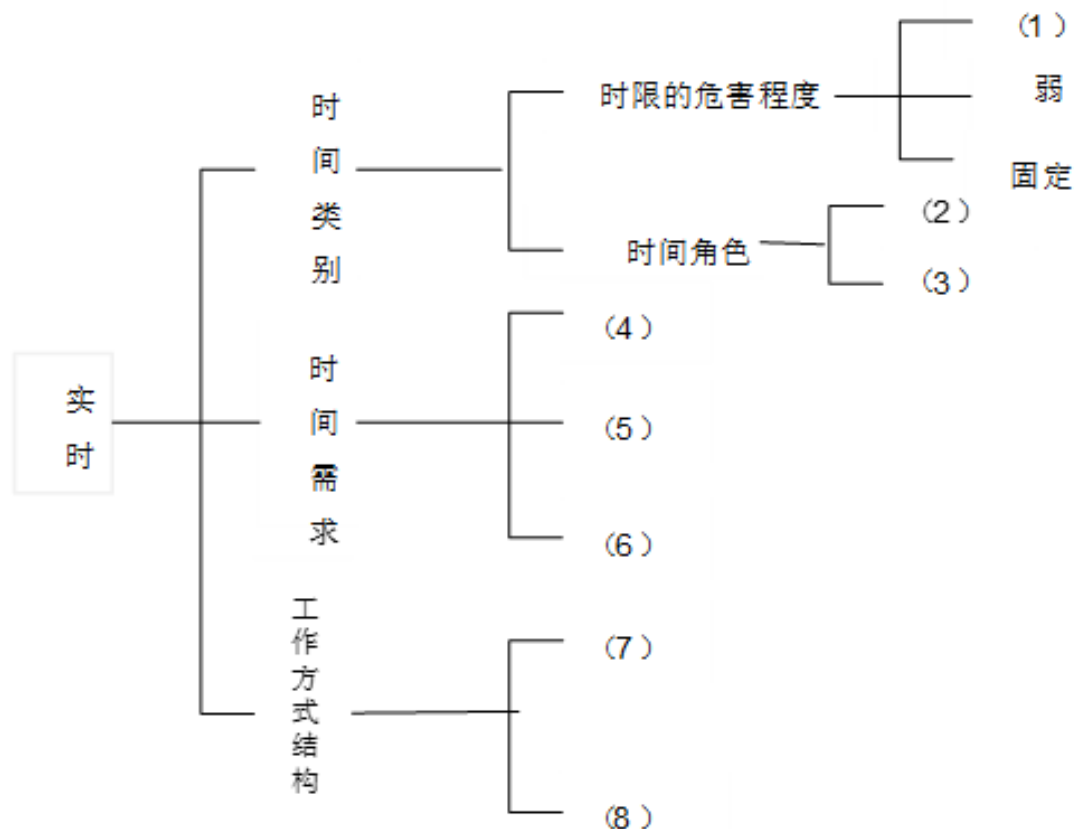
复杂约束性：涉及资源限制、优先级分配和实时调度，约束条件复杂。

短暂超载处理：在某些情况下，实时系统必须能应对短时间的超载任务，仍保持基本功能。

问题（2）

实时系统根据应用场景、时间特征以及工作方式的不同，存在多种实时特性，大致有三种分类方法，即时间类别、时间需求和工作方式结构。根据自己所掌握的"实时性"知识，将图3-1给出的实时特性按三种分类方式，填写图3-1中（1）~（8）处空白。

备选答案：时限的危害程度；时间角色；弱；时间响应；固定；时限/反应时间；时间明确；输入/输出激励；时间触发；强；周期/零星/非周期；事件触发。



思路解析

时间类别分类 (1) : 在“时限的危害程度”维度下，实时系统可分为 **强实时** 与 **弱实时**。强实时要求任务严格按照时完成，否则会造成严重后果（如导弹制导）。因此 (1) 填“强”。

时间角色分类 (2) (3) 实时系统在不同任务中可承担不同时间角色：

(2) **时间响应**：表示系统必须对外部事件在时间约束内做出反应，例如传感器检测到异常信号后立即调整飞行姿态。

(3) **时间明确**：系统的行为是可预测的，执行时间确定，例如任务调度在固定周期执行。

时间需求分类 (4) (5) (6)

(4) **时限/反应时间**：系统必须在任务规定的截止时间前完成。

(5) **输入/输出激励**：实时任务通常由外部输入触发，并要求在输出端及时响应。

(6) **周期/零星/非周期**：任务的触发方式多样，有些是周期性的（如采样），有些是零星或非周期事件（如外部中断）。

工作方式结构分类 (7) (8)

(7) **时间触发**：系统按照预设时间点启动任务，例如定时任务。

(8) **事件触发**：由外部事件（如中断信号）引起任务执行。

综上，时间角色你需要理解成系统中和时序相关的两种设计。响应任务和定时任务。时间需求是针对这两种设计的评价方式。

问题 (3)

可靠性是实时系统的关键特性之一，区分软件的错误 (Error)、缺陷 (Defect)、故障

(Fault) 和失效 (Failure) 概念是软件可靠性设计工作的基础。请简要说明错误、缺陷、故障和失效的定义；并在图3-2中标出错误、缺陷和失效出现阶段，说明缺陷、故障和失效的表现形式，填写图3-2中 (1) ~ (6) 处的空白。

思路解析

错误 (Error) : 错误是人类活动中的失误或思维偏差，通常发生在 **需求分析、设计或编码阶段**。

例如开发人员误解需求或实现逻辑错误。错误本身并不会直接影响用户，但会导致缺陷的引入。

缺陷 (Defect)：缺陷是由错误引入到软件制品中的 **潜在问题**，它存在于程序代码、设计文档或数据库中。例如一个数组越界问题就是典型缺陷。缺陷存在于软件制品之中，即（1）“存在”，并由错误（2）“引起”。

故障 (Fault)：当缺陷在运行时被激活时，就会形成 **故障**。故障属于系统的内部状态，例如内存溢出、变量取值错误。故障通常表现为软件运行中不正确的中间状态，它存在（5）“在产品中”。

失效 (Failure)：失效是故障在外部表现出的 **可见错误行为**，例如程序崩溃、输出结果错误等，这是用户（3）“经历”到的结果。失效发生（6）“在运行时”，表现为系统未能满足预期需求。

因此四者的关系可描述为：**错误 → 缺陷 → 故障 → 失效**。

错误（产生阶段：开发过程中，4）；

缺陷（存在于产品中，5）；

故障（被激活后导致内部异常）；

失效（运行时被用户感知，6）。

答案-问题（1）

实时系统是指向系统发出指令后，在一个极短的时间内，系统回复结果。实时系统的特性：

- （1）时间约束性（及时性）
- （2）可预测性
- （3）高可靠性
- （4）与外部环境的交互作用性
- （5）多任务类型
- （6）约束的复杂性
- （7）具有短暂超载的特点

答案-问题（2）

- （1）强
- （2）（3）时间响应、时间明确
- （4）（5）（6）时限/反应时间、输入/输出激励、周期/零星/非周期
- （7）（8）时间触发、事件触发

答案-问题（3）

软件错误：软件错误是指在软件生存期内的不希望或不可接受的人为错误，其结果是导致软件缺陷的产生。

软件缺陷：软件缺陷是指存在于软件（文档、数据、程序）之中的那些不希望或不可接受的偏差。

软件故障：软件故障是指软件运行过程中出现的一种不希望或不可接受的内部状态。

软件失效：软件失效是指软件运行时产生的一种不希望或不可接受的外部行为结果。

- （1）存在
- （2）引起
- （3）用户经历
- （4）在开发过程中
- （5）在产品中
- （6）在运行时

第29章——物联网与 CPS (2019年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式系统开放式架构相关技术的描述，在答题纸上回答问题1至问题3。

【说明】

信息物理系统（Cyber Physical Systems，CPS）技术已成为未来宇航装备发展的重点关键技术之一。某公司长期从事嵌入式系统的研制工作，随着公司业务范围不断扩展，公司决定进入宇航装备的研制领域。为了做好前期准备，公司决定让王工程师负责编制公司进军宇航装备领域的战略规划。王工经调研和分析，认为未来宇航装备将向着网络化、智能化和综合化的目标发展，CPS 将会是宇航装备的核心技术，公司应构建基于CPS技术的新产品架构，实现超前的技术战略储备。

问题（1）

通常CPS结构分为感知层、网络层和控制层，请用300字以内文字说明CPS的定义，并简要说明各层的含义。

思路解析

信息物理系统（CPS）是未来智能装备的核心基础，其本质是通过3C 技术（计算、通信、控制）的深度融合，实现物理实体与网络空间的高效交互。与传统嵌入式系统相比，CPS 不仅关注单一设备的控制，还强调多个子系统之间的网络化协同，使系统具备实时感知、动态控制与自适应能力。这使得 CPS 在宇航、工业控制、智能制造、智能交通等领域具有广阔应用前景。

CPS 的三层结构明确了不同层次的功能分工。**感知层**主要是各种传感器和末端设备，负责将物理环境的数据采集并上报，是数据来源的基础；**网络层**是物理世界与信息世界的桥梁，保证信息能够低时延、可靠、安全地传输，是实现远程控制与分布式协作的关键；**控制层**则是系统的大脑，负责数据分析、逻辑判断和决策，并向执行机构下发指令。三层协作，构成了一个闭环的感知—传输—控制系统，确保 CPS 在复杂应用环境中具备高效、智能和安全的运行能力。

问题（2）

王工在提交的战略规划中指出：飞行器中的电子设备是一个大型分布式系统，其传感器、控制器和采集器分布在飞机各个部位，相互间采用高速总线互连，实现子系统间的数据交换，而飞行员或地面指挥系统根据飞行数据的汇总决策飞行任务的执行。图3-1给出了飞行器系统功能组成图。

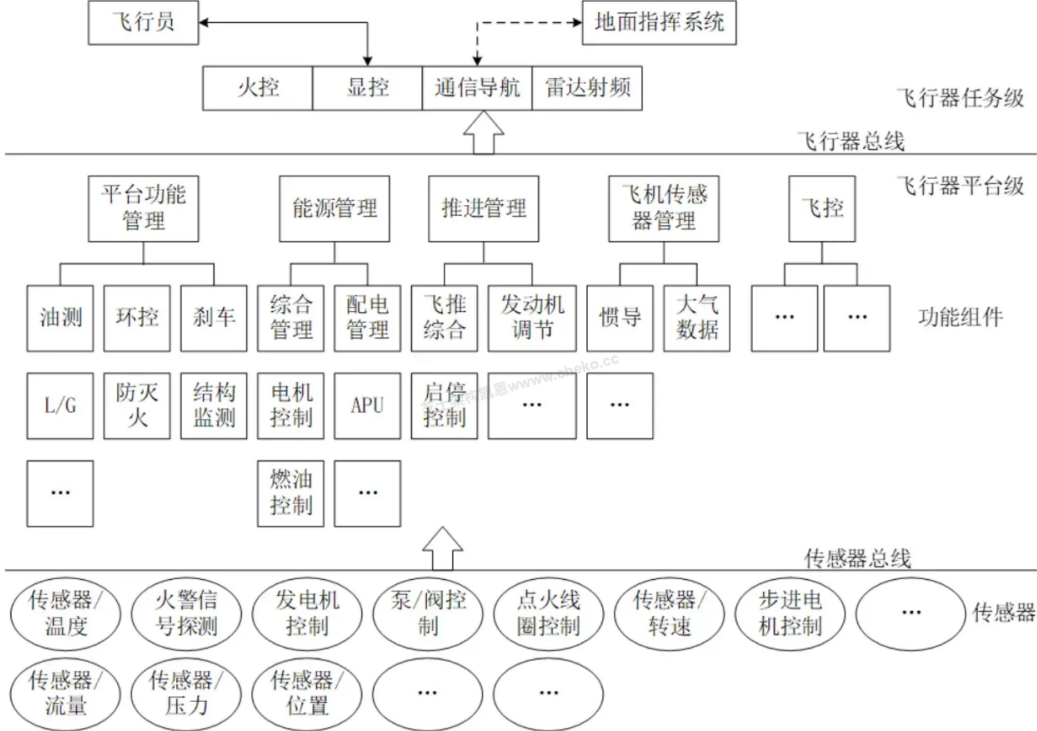


图 3-1 飞行器系统功能组成图

请参考图3-1给出的功能图，依据你所掌握的CPS知识，说明以下所列的功能分别属于CPS结构中的哪层，哪项功能不属于CPS任何一层。

1. 飞行传感器管理

2. 步进电机控制
3. 显控
4. 发电机控制
5. 环控
6. 配电管理
7. 转速传感器
8. 传感器总线
9. 飞行员
10. 火警信号探测

思路解析

飞行器电子设备是一个典型的 **分布式 CPS 系统**。其传感器、控制器、采集器分布在机体各处，通过高速总线互联，实现对飞行状态、环境参数、设备运行的实时监测与控制。

根据三层架构，可将图 3-1 中的功能逐一对应到 CPS 层次。**感知层**负责对物理环境进行直接交互，例如电机控制、发电机运行、转速测量、火警信号探测等，均是传感器或执行器的职能；**网络层**则体现在数据传输渠道，这里是传感器总线，起到连接不同部件的桥梁作用；**控制层**则负责综合管理与决策，包括飞行传感器管理、显示控制（显控）、环境控制（环控）、配电管理等，它们基于感知数据进行分析并下发指令。

特别需要注意的是“**飞行员**”。飞行员作为系统的操作者，是通过人机交互接口与 CPS 进行交互的对象，但不属于 CPS 系统本身的层次。因此，应将飞行员排除在 CPS 三层之外，而视为外部控制主体。这也说明 CPS 不仅包括设备与网络本身，还需要与**人机交互**、**地面指挥系统**结合，才能完成复杂任务。

问题（3）

王工在提交的战略规划中指出：未来宇航领域装备将呈现网络化、智能化和综合化等特征，形成集群式的协同能力，安全性尤为重要。在宇航领域的CPS系统中，不同层面上都会存在一定的安全威胁。请用100字以内文字说明CPS系统会存在哪三类安全威胁，并对每类安全威胁至少举出两个例子说明。

思路解析

CPS 的核心问题之一是**系统安全**。由于 CPS 融合了物理世界与网络空间，任何安全漏洞都可能导致严重的物理后果。安全威胁主要体现在三层：

①**感知层**，作为数据入口，面临物理节点被捕获、传感器数据篡改或窃听的问题，例如攻击者篡改飞行器温度传感器数据，会导致错误决策；

②**网络层**，面临拒绝服务（DoS）、选择性转发或路由误导攻击，导致数据无法可靠传输，例如攻击者阻塞飞控总线会导致飞行系统瘫痪；

③**控制层**，作为决策大脑，若被注入恶意代码或非授权访问，可能直接造成任务失败或设备损坏，例如攻击者植入木马程序窃取飞行数据，或绕过权限系统操纵飞行任务。CPS 安全不仅要求传统的信息安全手段，还必须结合物理安全、冗余设计和容错控制，才能保障宇航装备的稳定运行。

答案-问题（1）

信息物理系统（Cyber Physical Systems, CPS）作为计算进程和物理进程的统一体，是集计算、通信与控制于一体的下一代智能系统。信息物理系统通过人机交互接口实现和物理进程的交互，使用网络化空间，以远程的、可靠的、实时的、安全的、协作的方式操控一个物理实体。

感知层：主要由传感器、控制器和采集器等设备组成，它属于信息物理系统中的末端设备。

网络层：主要是连接信息世界和物理世界的桥梁，实现的是数据传输，为系统提供实时的网络服务，保证网络分组传输的实时可靠。

控制层：主要是根据认知结果及物理设备传回来的数据进行相应的分析，将相应的结果返回给客户端。

答案-问题（2）

感知层：2、4、7、10

网络层：8

控制层：1、3、5、6

不属于 CPS 结构中的功能：9

答案-问题（3）

（1）感知层安全威胁：感知数据破坏、信息窃听、节点捕获。

（2）网络层安全威胁：拒绝服务攻击、选择性转发、方向误导攻击。

（3）控制层安全威胁：用户隐私泄露、恶意代码、非授权访问。

第30章——软件 FMEA 与故障诊断 (2022年11月系统架构设计师真题-第3题)

阅读以下关于嵌入式系统故障检测和诊断的相关描述，在答题纸上回答问题1至问题3。

【说明】

系统的故障检测和诊断是宇航系统提高装备可靠性的主要技术之一，随着装备信息化的发展，分布式架构下的资源配置越来越多、资源布局也越来越分散，这对系统的故障检测和诊断方法提出了新的要求。为了适应宇航装备的分布式综合化电子系统的发展，解决由于系统资源部署的分散性，造成系统状态的综合和监控困难的问题，公司领导安排张工进行研究。张工经过分析、调研提出了针对分布式综合化电子系统架构的故障检测和诊断的方案。

问题（1）

张工提出：宇航装备的软件架构可采用四层的层次化体系结构，即模块支持层、操作系统层、分布式中间件层和功能应用层。为了有效、方便地实现分布式系统的故障检测和诊断能力，方案建议将系统的故障检测和诊断能力构建在分布式中间件内，通过使用心跳或者超时探测技术来实现故障检测器。请用300字以内的文字分别说明心跳检测和超时探测技术的基本原理及特点。

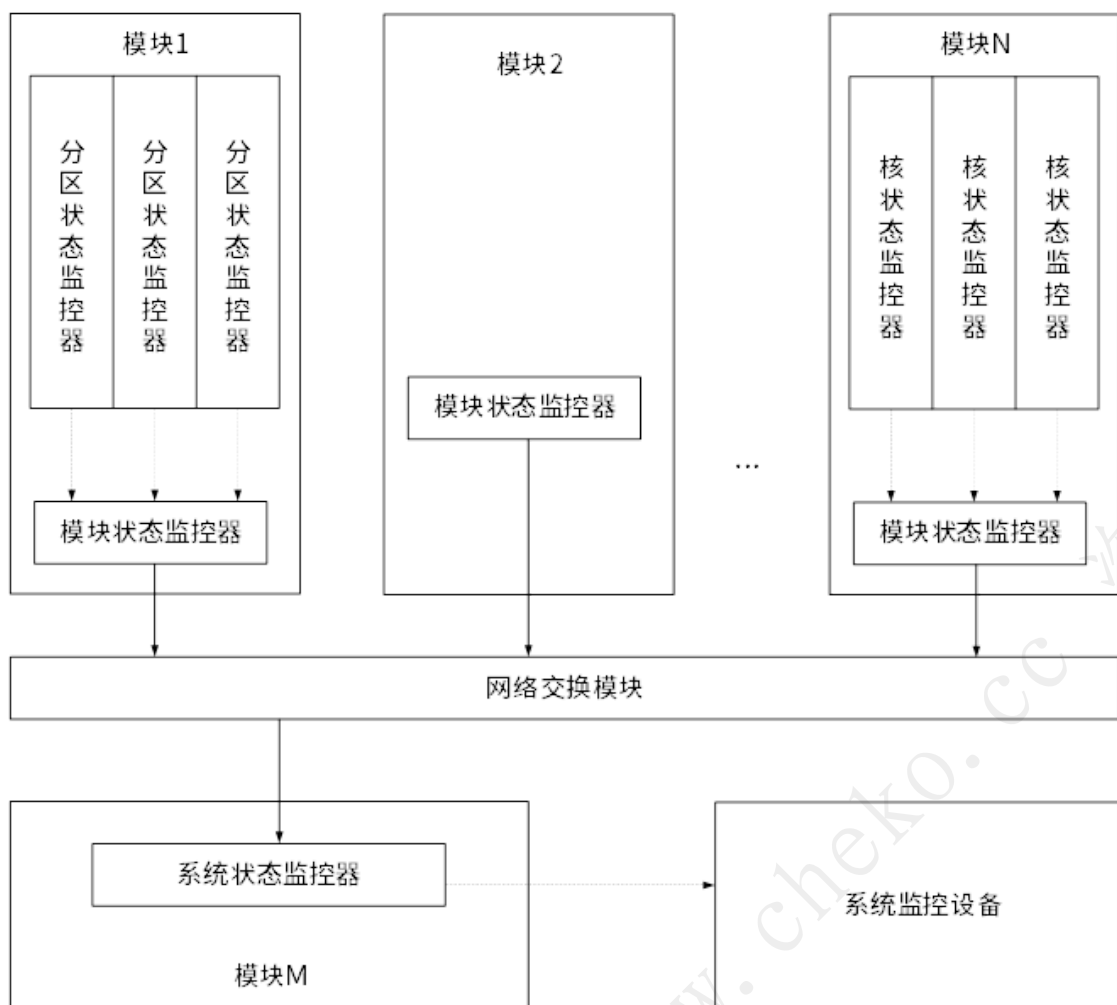
思路解析

心跳检测是一种主动式故障检测机制，系统中的每个节点会定期向监控器发送“心跳”信号，表示自身正常运行。若监控器在设定时间内未收到心跳信号，则判断该节点可能发生故障。该方法实现简单、实时性强，常用于判断节点是否存活，但对网络稳定性依赖较大，可能因丢包或网络延迟引发误判。

超时探测是一种被动式故障检测方式，监控端在发起请求后设定超时时间，若在规定时间内未收到响应，则认为目标节点或服务异常。超时机制适用于检测服务调用是否正常，能够发现“卡顿”或“半失效”问题，对通信实时性要求较低，但超时时间设置不当可能影响检测的准确性或效率。

问题（2）

张工针对分布式综合化电子系统的架构特征，给出了初步设计方案，指出每个节点的故障监测与诊断器主要负责监控系统中所有的故障信息，并将故障信息进行综合分析判断，使用故障诊断器分析出故障原因，给出解决方案和措施。系统可以给模块的每个处理机器核配置核状态监控器、给每个分区配置分区状态监控器、给每个模块配置模块状态监控器、给系统配置系统状态监控器，如图3-1所示。



请根据下面给出的分布式综合化电子系统可能产生的故障(a)~(h)，判断这些故障分别属于哪类监控器检测的范围，完善表3-1的(1)~(8)的空白。

- (a) 应用程序除零
- (b) 看门狗故障
- (c) 任务超时
- (d) 网络诊断故障
- (e) BIT检测故障
- (f) 分区堆栈溢出
- (g) 操作系统异常
- (h) 模块掉电

表 3-1 故障分类

核状态监控器	(1)、(2)
分区状态监控器	(3)
模块状态监控器	(4)、(5)、(6)
系统状态监控器	(7)、(8)

思路解析

- b、e：看门狗与BIT检测属于底层硬件层面，归属“核”监控；
- f：堆栈溢出发生于分区内部，归属“分区监控”；
- a、d、h：程序错误、网络异常、模块掉电均作用于模块整体；
- g、c：操作系统层面事件归“系统监控”；

问题(3)

张工在方案中指出，本系统的故障诊断采用故障诊断器实现，它可综合多种故障信息和系统状态

，依据智能决策数据库提供的决策策略判定出故障类型和处理方法。智能决策数据库中的策略可以对故障开展定性或定量分析。通常，在定量分析中，普遍采用基于解析模型的方法和数据驱动的方法。张工在方案中提出该系统定量分析时应采用基于解析模型的方法。但是此提议受到王工的反驳，王工指出采用数据驱动的方法更适合分布式综合化电子系统架构的设计。请用300字以内的文字，说明数据驱动方法的基本概念，以及王工提出采用此方法的理由。

思路解析

数据驱动的方法是一种不依赖系统精确数学模型、而依托历史或实时运行数据进行故障识别和预测的技术路线，其核心是通过采集大量系统运行状态数据，利用机器学习、统计建模或信号处理等方法从数据中提取特征、构建模型，识别异常模式、定位故障源。相比于解析模型方法，数据驱动方法具有更强的适应性与可扩展性。

张工提出的基于解析模型方案，虽然在理论上可以通过系统输入输出之间的数学关系推导故障特征，但这要求建立系统的精确数学模型，而分布式综合化电子系统由于涉及模块众多、结构复杂、耦合性强、运行环境多变，很难用统一的解析模型准确刻画，建模代价高、维护困难、泛化能力弱。

而王工认为，数据驱动方法更贴合当前系统的实际需求和工程可行性。尤其是在宇航装备等高度集成的复杂系统中，已部署大量传感器、监控器、日志记录器，为数据驱动方法提供了丰富的数据基础。此外，数据驱动方法可实现在线学习与自适应调整，不依赖人工规则设计，适用于多种未知故障类型的检测和诊断。综上所述，王工建议采用数据驱动方法是基于工程实践的合理选择，更能满足分布式综合化电子系统对智能化、自主化故障诊断的需求。

答案-问题（1）

心跳检测：系统中各节点周期性向监控器发送“心跳”信号，若在设定时间未收到心跳，即认为节点失效。优点是实现简单，实时性强；缺点是对通信依赖高，易受网络波动干扰。

超时机制：监控器在预设时间内等待节点响应，如未收到回复即判定超时并上报故障。优点是适用于异步通信，缺点是设置合理的超时时间具有挑战性。

答案-问题（2）

- (1) b (2) e
- (3) f
- (4) a (5) d (6) h
- (7) g (8) c

答案-问题（3）

数据驱动方法是基于系统运行中的历史或实时数据，通过机器学习、统计分析或信号处理等手段，无需精确建模即可实现故障识别和预测。适用于复杂、高耦合、难建模系统。

王工理由：宇航系统结构复杂、组件多样，难以构建精确解析模型；数据驱动方法依赖实测数据，适应性强、部署灵活，更适合分布式架构系统。

(2013年11月系统架构设计师真题-第3题)

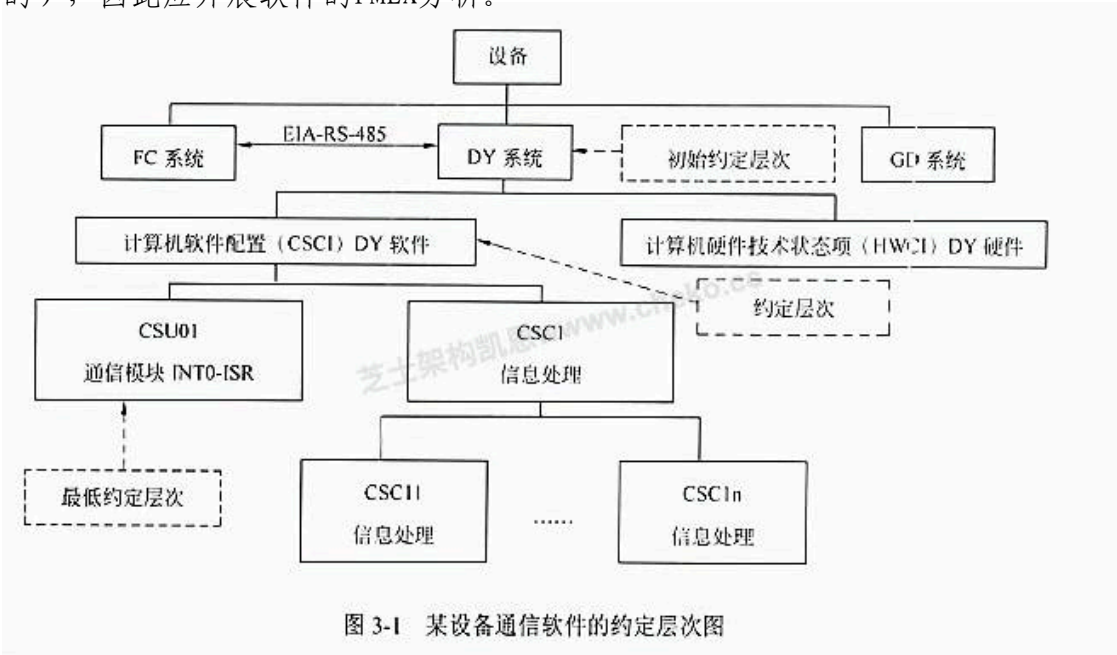
阅读以下有关嵌入式软件FMEA方法和相关案例的说明，在答题纸上回答问题1至问题3。

【说明】

故障（失效）模型影响分析FMEA是分析产品所有可能的故障模式及其可能产生的影响，并按每个故障模式产生影响的严重程度及其发生概率予以分类的一种归纳分析方法。近年来，FMEA方法已被广泛用于安全关键系统的嵌入式软件可靠性分析工作。

某软件公司承担了一项通信软件的开发项目。该项目由FC系统、DY系统和GD系统组成，而DY系统

(TMS320C25S) 软件负责按系统的通信协议完成与FC系统的通信，图3-1给出了该通信软件的约定层次图。公司高层将项目交给王工程师，王工认为此项目是安全关键系统，安全等级应为Ⅱ类（致命的），因此应开展软件的FMEA分析。



问题（1）

请阅读以下有关FMEA的描述，将恰当的内容填入（1）～（7）。

FMEA是FMA（故障模式分析）和FEA（故障影响分析）的组合，它对系统各种可能的风险进行评价、分析后，在现有技术的基础上消除这些风险或将这些风险降低到可接受的水平。为达到最佳效益，FMEA必须在产品研制初期进行。

FMEA实际是一组系列化的活动，其主要活动包括：

- （1）（ ）；
- （2）（ ）；
- （3）（ ）。

由于产品故障可能与设计、制造过程、使用、承包商/供应商以及服务有关，因此FMEA又细分为（4）FMEA、（5）FMEA、（6）FMEA和（7）FMEA四类。

思路解析

FMEA（Failure Mode and Effects Analysis） 是一种系统性、前瞻性的可靠性分析方法，它的目标是识别出产品或系统可能存在的所有潜在故障模式，分析这些故障对系统运行和用户的影响，并据此采取措施消除或降低风险。它实际上是 **FMA（故障模式分析）** 与 **FEA（故障影响分析）** 的结合。实施FMEA的最佳时机是在产品研制的早期阶段，此时设计还未完全固化，修改和优化的成本最低，可以用最小的代价消除潜在风险。

具体来说，FMEA包括三个主要活动：

- 识别潜在故障模式：**分析系统、子系统、模块、过程等可能会出现的失效情况。例如在嵌入式通信模块中，可能出现“接收数据不完整”“发送失败”“中断未触发”等故障模式。
- 风险量化评估：**根据严酷度（Severity）、发生概率（Occurrence）、检测难度（Detection）三个维度对每个故障模式进行量化，常用的指标是 **RPN（Risk Priority Number）**，即 $RPN = S \times O \times D$ ，用于排序和确定优先处理的故障模式。
- 提出改进措施：**分析故障起因和机理，制定相应的预防和改进措施，例如增加超时检测、冗余机制、完善异常处理等。

根据产品生命周期中可能出现的不同环节，FMEA可以分为 **设计FMEA**（关注设计阶段缺陷）、**过程FMEA**（关注制造或开发过程中的失效）、**使用FMEA**（关注用户使用过程中的问题）、**服务FMEA**（关注售后服务与维护阶段可能出现的故障）。这种分类保证了FMEA在产品从开发、生产到使用和维护

的全生命周期中都能发挥作用。

问题（2）

从图3-1可以看出，CSU01通信模块是该项目的关键模块，主要功能定义为：总线通信控制器自动完成一帧数据的接收，存入数据缓冲区，并产生中断(INT0)通知CPU从数据缓冲区中读取数据；CPU读完数据后，将准备好的发送数据写至数据缓存区，写完后通知总线通信控制器自动完成一帧数据的发送。CRC校验由外部电路完成判别，其结果通过数据线上的相应位进行标识。针对CSU01通信模块，简要描述实施FMEA的具体内容，填写完成表3-1的（1）~（5）。

序号	主要步骤	具体内容
1	故障模式确定	（1）
2	故障原因分析	（2）
3	故障影响分析	（3）
4	危害性分析	（4）
5	改进措施	（5）

思路解析

在嵌入式通信软件的可靠性设计中，**关键模块的FMEA分析**至关重要。本案例中的 CSU01 通信模块，是 DY 系统的核心组成部分，它负责数据帧的接收与发送，同时与 CPU、总线控制器和外部 CRC 校验逻辑交互。由于它处在协议实现的中心位置，一旦失效会直接导致整个系统的通信中断，因此是实施 FMEA 的重点对象。

实施步骤如下：

- 确定故障模式：**根据 CSU01 模块的功能定义，可从“接收数据”和“发送数据”两个核心功能点入手，定义潜在故障模式，例如“接收帧丢失”“发送超时”“缓冲区覆盖”等。
- 分析故障原因：**潜在原因可以归结为三大类：①总线通信控制器的硬件问题，如控制寄存器异常；②对方设备发送问题，如数据格式错误；③自身程序错误，如中断处理逻辑缺陷。
- 分析故障影响：**采用分层思路，先分析对 CSU01 模块本身的影响（如中断无法触发），再上升到对 DY 软件的影响（如通信协议无法继续），最后扩展到对整个系统的影响（如与 FC 系统的通信中断）。
- 危害性分析：**利用 **RPN（风险优先数）** 方法，将每个故障模式按照严重度、发生概率和检测难度进行量化，计算出 RPN 值，从而区分需要重点关注的故障。
- 提出改进措施：**根据风险大小和影响范围，提出具体措施，例如增加数据冗余校验、增加中断超时处理、在缓冲区操作时加入边界检查等。

问题（3）

表3-2给出针对该项目的CSU01通信模块的软件故障（失效）模型影响分析FMECA表（局部），请根据此题描述情况填写表3-2中的（1）~（7）。注：表3-2中的SRPN（软件风险优先数）=SES（软件故障模式的严酷度等级）×SOPR（软件故障模式的发生概率等级）×SDDR（软件故障模式的被检测难度等级）。

序号	单元	功能	故障模式	故障原因	故障影响			危害性分析				改进措施
					局部影响	高一层次影响	最终影响	SE SR	SO PR	SD DR	SR PN	
1	INTO-ISR	数据接收	通信接口非接收状态	(1)	模块单元无法进入	无法产生 INTO 中断	通信功能丧失	8	7	4	224	初始化时写 0C300H 地址单元后,读 0C300H 的 D7 位,直到确认通信接口为接收状态。判别中加以计数限制,以保证规定时间到时,记录故障标志并报错
2			中断允许处于禁止状态	程序使用 DINT 和 EINT 不当	模块单元无法进入	(2)	通信功能丧失	8	7	4	224	严格检查 DINT 和 EINT 的语句位置
3			CRC 错误	(3)	接收数据异常	接收数据错误	(4)	7	5	6	210	首先读 0C200H 地址单元的 D0 位,判别 CRC 是否正确,若 CRC 错误,则放弃此帧
4		数据发送	尚未发送就强行设置接收状态	(5)	影响发送数据的正确性	发送数据错误	通信错误	7	6	5	(6)	写 0C200H 地址单元后,读 0C200H 地址单元的 D7 位,判别是否已发送完,再通过写 0C300H 地址单元设置通信接口为接收状态。注:此措施与模式 10 和 11 相结合
5			(7)	总线通信控制器错误	发送数据失败,如果程序处理不当可能造成死循环	发送数据失败,处理不当此单元可能无法退出	通信功能丧失,处理不当可能死机	8	6	7	336	写 0C200H 地址单元后,读 0C200H 地址单元的 D7 位,判别是否已发送完,并加以计数限制,以保证规定时间到时,记录故障标志并退出此模块

思路解析

FMECA (Failure Mode, Effects, and Criticality Analysis) 是在 FMEA 的基础上加入定量的关键度分析,常用指标是 SRPN (软件风险优先数),其计算公式为: $SRPN = SESR \times SOPR \times SDDR$,即严酷度等级 $\times$ 发生概率等级 $\times$ 检测难度等级。

在本题中,针对 CSU01 通信模块,FMECA 表格列出了部分潜在故障模式。结合题干与表格:

故障模式 (1) 程序写 0C300H 地址单元不当: 这是典型的软件控制错误,可能导致数据缓冲区操作异常,从而影响数据收发。

故障影响 (2) 无法响应 INTO 中断: 由于写错寄存器地址,导致中断响应失效,CPU 无法及时处理数据,通信中断。

故障原因 (3) 线路误码: 当通信线路上出现比特误码时,会直接导致接收数据异常。

故障影响 (4) 通信错误: 线路误码会引发整个通信过程错误,数据无法正确传输。

故障模式 (5) 程序控制错误: 如果程序逻辑有缺陷,例如状态机切换错误,会导致通信过程不符合协议。

SRPN 值 (6) = 210: 通过 $SESR \times SOPR \times SDDR$ 计算得出。

最终影响 (7) 数据发送始终不成功: 该错误将导致 CSU01 模块无法完成数据传输,严重影响系统运行。

值得注意的是,在表中故障模式 (5) “程序控制错误”的 SRPN 高达 336,说明该模式的严重度和检测难度极高,必须在设计阶段重点关注。例如,可以通过形式化验证、代码静态检查、单元测试和集成测试来降低该类故障发生的可能性,并通过监控和日志机制提升可检测性。

答案-问题 (1)

- (1) 找出产品/过程中潜在的故障模式
- (2) 根据相应的评价体系对找出的潜在故障模式进行风险量化评估
- (3) 列出故障起因/机理,寻找预防或改进措施

- (4) 设计
- (5) 过程
- (6) 使用
- (7) 服务

答案-问题（2）

- (1) 根据通信协议，可按接收数据功能和发送数据功能分别确定故障模式
- (2) 故障原因分为总线通信控制器原因、对方发送的原因和自身程序的原因
- (3) 针对每个故障模式分析其对本模块直至整个DY系统造成的影响
- (4) 采用风险优先数RPN方法进行该通信模块的危害性分析
- (5) 根据以上故障模式、原因、影响及危害性的分析结果，综合考虑故障的影响及SRPN值等情况，对每个故障模式制定了相应的改进措施。

答案-问题（3）

- (1) 程序写0C300H地址单元不当
- (2) 无法响应INT0中断
- (3) 线路误码
- (4) 通信错误
- (5) 程序控制错误
- (6) 210
- (7) 数据发送始终不成功

第31章——项目进度计划
(2013年11月系统架构设计师真题-第2题)

阅读以下关于某项目开发计划的说明，在答题纸上回答问题1至问题4。

【说明】
 某软件公司拟开发一套电子商务系统，王工作为项目组负责人负责编制项目计划。由于该企业业务发展需要，CEO急于启动电子商务系统，要求王工尽快准备一份拟开发系统的时间和成本估算报告。
 项目组经过讨论后，确定出与项目相关的任务如表2-1所示。其中，根据项目组开发经验，分别给出了正常工作及加班赶工两种情况下所需的时间和费用。

任务名称	正常工作	加班工作	前置任务
A. 系统调研	4 天/7200 元	3 天/8400 元	-
B. 提交项目计划	2 天/1600 元	1 天/1900 元	A
C. 需求分析	6 天/9600 元	4 天/14200 元	B
D. 系统设计	12 天/22200 元	8 天/27600 元	C
E. 数据库开发	3 天/5100 元	2 天/5700 元	D
F. 网页开发	6 天/8700 元	5 天/10000 元	D
G. 报表开发	4 天/6000 元	任务外包无法赶工	D
H. 测试修改	7 天/9800 元	4 天/12800 元	E, F, G
I. 安装部署	4 天/4000 元	2 天/5000 元	H

问题（1）

请用400字以内文字说明王工拟编制的项目计划中应包括哪些内容。

思路解析

项目计划主要分为两大类内容：1. 项目总计划（围绕“做什么、怎么做、何时完成、花多少”）2. 项目辅助计划（围绕“如何确保质量、沟通、风险、人力、采购”）。

一、项目总计划包括

范围计划：明确电子商务系统的目标与交付边界。

工作分解结构（WBS）：将项目拆分为具体任务，如表中A至I各任务。

活动定义与排序：根据前置任务制定合理的任务执行顺序。

资源需求与资源计划：确定每项任务所需人力、技术与资金资源。

费用估算：基于正常/加班模式，测算每条路径的成本。

进度计划与进度压缩分析：计算关键路径，评估赶工方案下的工期最短可能性。

二、项目辅助计划包括

质量计划：确保开发成果符合需求与标准。

沟通计划：确保项目干系人之间信息传达顺畅。

人力资源计划：合理配置与分配开发人员。

风险计划：识别关键任务（如D、H）可能的进度与成本风险。

采购计划：若涉及外包或采购工具，也需纳入计划。

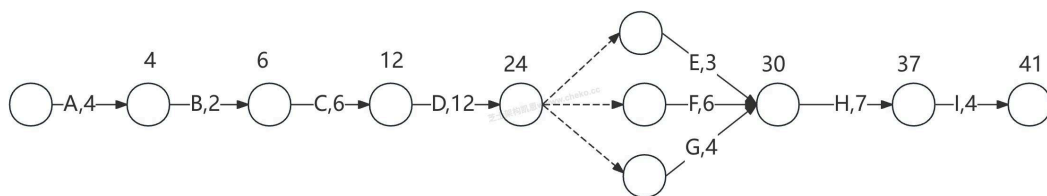
问题（2）

请根据表2-1，分别给出正常工作 and 最短工期（加班工作）两种情况下完成此项目所需的时间和费用。

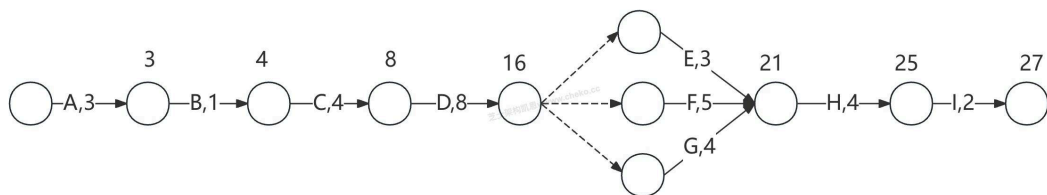
思路解析

正常工作成本=7200元+1600元+9600元+22200元+5100元+8700元+6000元+9800元+4000元=74200元。

正常工作工期=4+2+6+12+6+7+4=41天。



最短工期就是通过加班缩短关键路径，这里其他没有问题，关键是 E 需不需要加班。凯恩认为不需要，缩短 E 不会改变关键路径，因此最短工期成本=8400元+1900元+14200元+27600元+5100元+10000元+6000元+12800元+5000元=91000元。最短工期=3+1+4+8+5+4+2=27天。



问题（3）

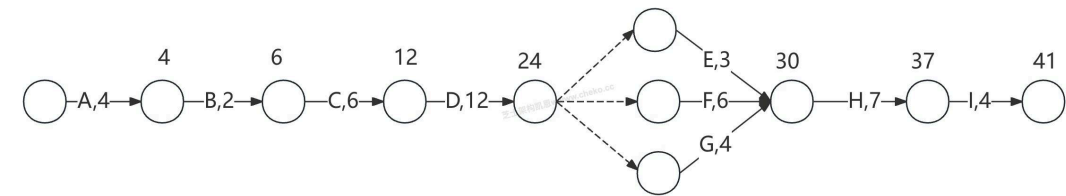
如果项目在系统调研阶段用了7天时间才完成，公司要求尽量控制成本，王工可在后续任务中采取什么措施来保证项目能按照正常工作进度完成？

思路解析

首先给出下面加班压缩一天的费用，然后用贪心算法选取费用最低的节点尝试来做压缩。压缩的时候一定要优先压缩关键路径上的工作，否则不会缩短工期，这个要注意。

任务名称	正常工作	加班工作	可压缩天数	压缩 1 天增加费用
A.系统调研	4 天/7200 元	3 天/8400 元	1	1200 元
B.提交项目计划	2 天/1600 元	1 天/1900 元	1	300 元
C.需求分析	6 天/9600 元	4 天/14200 元	2	2300 元
D.系统设计	12 天/22200 元	8 天/27600 元	4	1350 元
E.数据库开发	3 天/5100 元	2 天/5700 元	1	600 元
F.网页开发	6 天/8700 元	5 天/10000 元	1	1300 元
G.报表开发	4 天/6000 元	任务外包无法赶工	0	
H.测试修改	7 天/9800 元	4 天/12800 元	3	1000 元
I.安装部署	4 天/4000 元	2 天/5000 元	2	500 元

在"B提交项目计划"和"I安装部署"任务中采用加班工作措施，以使得能够按照正常工作进度完成。然后参照下图，对需要加班的节点做修改



问题（4）

如果企业CEO想在34天后系统上线，王工应该采取什么措施来满足这一要求？这种情况下完成项目所需的费用是多少？

思路解析

这里同样也要参照下表算出加班压缩一天的费用，然后用贪心算法选取费用最低的节点尝试来做压缩。压缩的时候一定要优先压缩关键路径上的工作，否则不会缩短工期，这个要注意。可以看到将A、B、H、I四个任务加班完成，这样正好弥补之前延误的7天工期。

任务名称	正常工作	加班工作	可压缩天数	压缩 1 天增加费用
A.系统调研	4 天/7200 元	3 天/8400 元	1	1200 元
B.提交项目计划	2 天/1600 元	1 天/1900 元	1	300 元
C.需求分析	6 天/9600 元	4 天/14200 元	2	2300 元
D.系统设计	12 天/22200 元	8 天/27600 元	4	1350 元
E.数据库开发	3 天/5100 元	2 天/5700 元	1	600 元
F.网页开发	6 天/8700 元	5 天/10000 元	1	1300 元
G.报表开发	4 天/6000 元	任务外包无法赶工	0	
H.测试修改	7 天/9800 元	4 天/12800 元	3	1000 元
I.安装部署	4 天/4000 元	2 天/5000 元	2	500 元

答案-问题（1）

王工在接到任务后，项目计划应包括：**一是项目总计划**：涵盖范围计划、工作分解与任务定义、任务排序、资源与人员需求、进度安排、费用估算与成本控制等关键内容。**二是项目辅助计划**：包括质量保障、沟通机制、人力资源配置、风险识别与应对策略、采购流程安排等。

答案-问题（2）

正常工作成本=74200元。正常工作工期=41天。最短工期成本=91000元。最短工期=27天。

答案-问题（3）

在"B提交项目计划"和"I安装部署"任务中采用加班工作措施，以使得能够按照正常工作进度完成。

答案-问题（4）

标准时长41天的任务，要34天完成，应赶工7天。具体赶工的任务包括：将A、B、H、I四个任务加班完成，这样正好弥补之前延误的7天工期，最终以79700元完成项目。

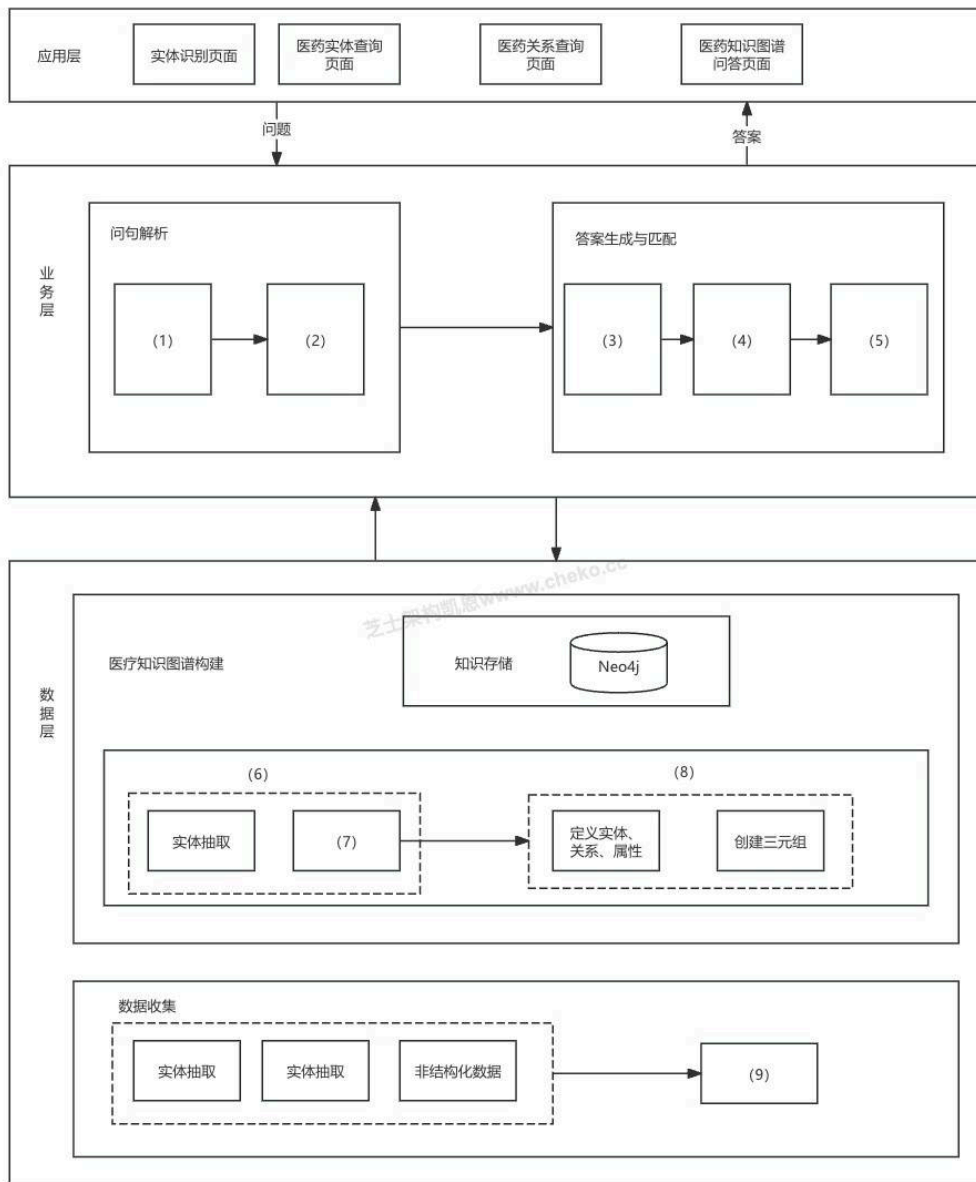
第32章——知识图谱与智能问答 (2025年5月系统架构设计师真题-第2题)

某研究团队基于医疗领域的大量文本与结构化数据，构建了一个面向医疗知识图谱的问答系统。该系统面向医学科研人员与临床医生，支持实体识别、关系抽取、知识检索与答案生成等功能。由于系统设计较为复杂，包含数据预处理、自然语言处理、知识图谱构建、后端服务、前端服务、机器学习与深度学习等多个模块，系统采用“自底向上”的方式构建总体系结构。该系统主要包括：爬取医疗相关数据并建立知识图谱、提供实体识别和关系抽取服务、实现多种类型问答接口（如实体知识问答、药物关系问答、医学关系查询等），并通过后端模块进行统一处理，生成答案。

问题（1）

根据题干描述的内容，将下列选项填入图中合适位置

- （a）知识检索（b）模板翻译（c）数据清洗（d）命名实体识别（e）知识表示（f）模板匹配
（g）问题分类（h）知识抽取（i）关系抽取



思路解析

整个系统是一个典型的**医疗问答系统+知识图谱构建架构**，从下到上分为：

数据层：包括原始数据收集、清洗、抽取实体和知识。构建结构化知识图谱，存储于 Neo4j。

业务层：接收用户问题后进行解析（问题分类、实体识别），然后生成结构化查询（模板翻译、匹配）并从图数据库检索答案。

应用层：提供不同功能页面，面向用户展示，如“实体说明页面”、“关系查询页面”等。

(1) g. 问题分类 问句处理的第一步通常是对问题进行分类，如判断是查实体、查关系等。

(2) d. 命名实体识别 在用户问题中识别出疾病、药物等实体，是问句理解中的关键步骤。

(3) a. 知识检索 匹配后执行查询，进入图数据库中检索答案。

(4) f. 模板匹配 将翻译后的模板与已有模板库匹配，确定合适的查询方式。

(5) b. 模板翻译 将结构化的语义解析结果转换为查询模板，比如转换成 Cypher 查询语句。

(6) h. 知识抽取 从原始数据中提取结构化信息，为知识图谱构建做准备。

(7) i. 关系抽取 从非结构化或半结构化的文本中，自动识别并提取 **实体之间存在的语义关系**。

(8) e. 知识表示 对抽取的知识进行结构化表达，例如 RDF 三元组，定义实体、关系和属性。

(9) c. 数据清洗 对原始数据（尤其是非结构化数据）进行预处理，包括去噪、去重、格式规范等。

问题（2）

在构建医药知识图谱系统中，选择哪类数据库更适合存储知识图谱数据？请明确说明数据库类型及推荐技术实现，并结合医药数据特性说明理由。

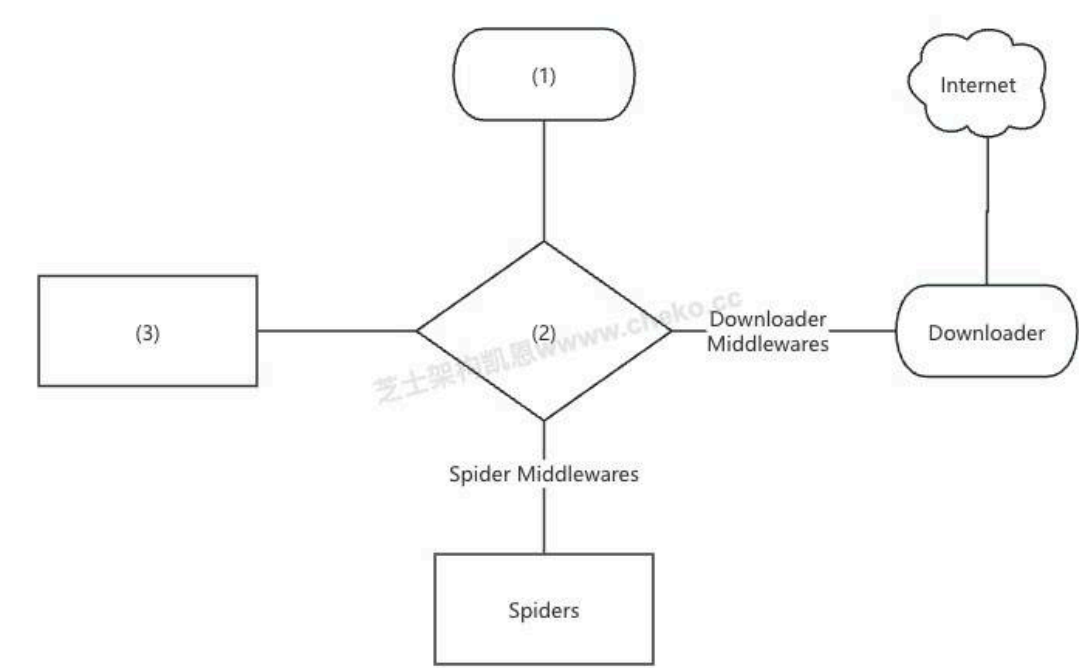
思路解析

在医药行业构建知识图谱时，推荐使用图数据库（Graph Database），如 Neo4j、Amazon Neptune 或 ArangoDB，这类数据库以“节点+边”的形式存储信息，天然适配知识图谱中的三元组结构，例如「阿司匹林」→（治疗）→「血栓性疾病」或「阿司匹林」→（副作用）→「胃肠道出血」，可直观表达药物、疾病、基因、症状等实体之间复杂的多对多关系。

图数据库支持高效的多跳关系查询，优于关系型数据库在复杂语义链路处理方面的性能，同时还支持图算法用于疾病传播路径分析、药品聚类 and 关键实体发现等高级任务。在 Neo4j 中，可通过类 SQL 的 Cypher 查询语言进行灵活的模式匹配与关系推理，其图形可视化界面适合医药专家理解图谱结构，且生态成熟，易于与 NLP、向量数据库和机器学习系统集成，广泛应用于药物发现、智能问答（如“哪些药物不能与华法林同时使用？”）和病例辅助分析等场景。

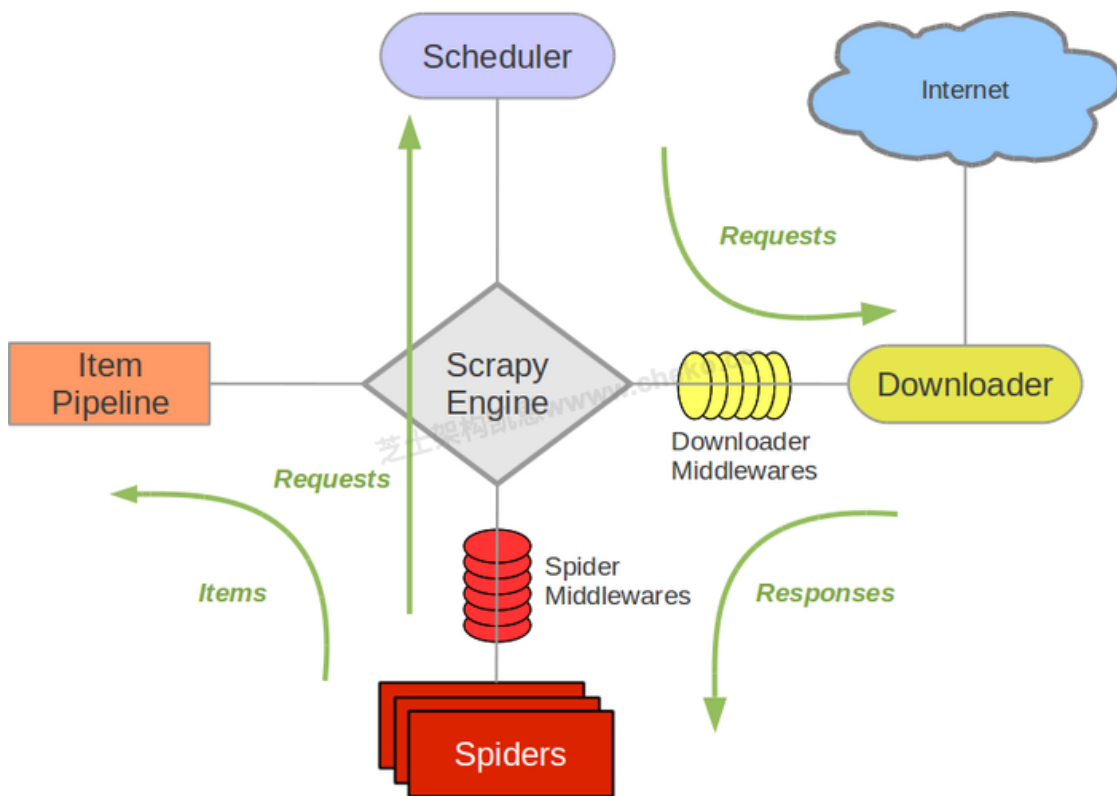
问题（3）

下图是 Scrapy 框架的核心结构图，其中挖空了三个关键组件，请根据图示结构，在图中标记的位置填入相应的模块名称，并请简要解释什么是“异步IO”。



思路解析

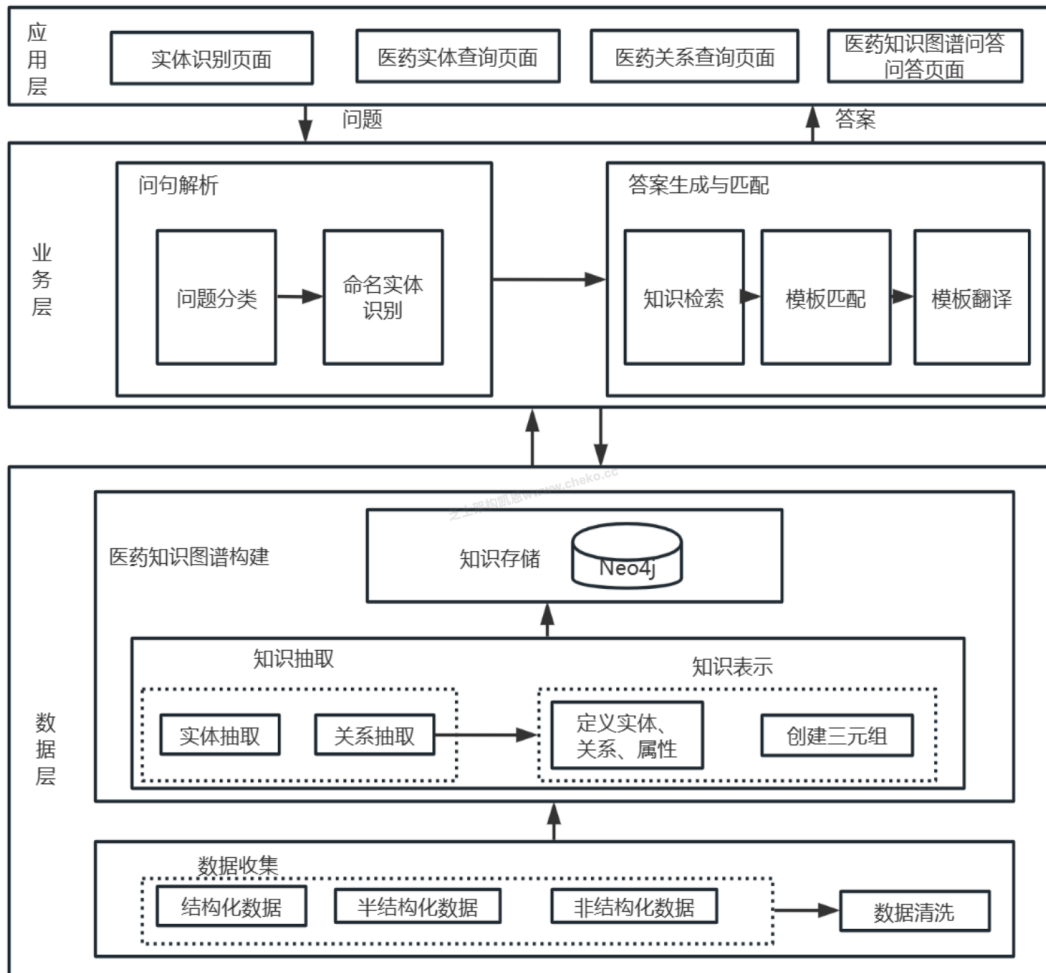
完整的 Scrapy 架构图如下图所示。Scrapy 是一个基于事件驱动和组件解耦设计的高效 Python 爬虫框架，其核心由 Scrapy Engine 驱动各模块协同工作，实现请求的调度、页面的下载、数据的解析与存储。其主要组件包括 Scheduler（管理请求队列）、Downloader（负责抓取网页）、Spider（解析网页数据）、Item Pipeline（处理和存储数据）及中间件（提供灵活的处理钩子）。整体数据流从 Spider 生成请求开始，经由 Engine 分发至 Downloader 并返回响应，再交由 Spider 提取数据，最终交由 Pipeline 存储，形成完整的爬虫闭环流程。该架构具有高扩展性、清晰的数据流向和良好的模块隔离性。



异步IO是一种“非阻塞 + 回调机制”的输入输出处理方式，程序在发起I/O操作后无需等待结果返回，而是将操作交由操作系统异步完成，待I/O结束后通过事件、信号或回调函数等机制通知程序继续处理结果。常见实现方式包括事件驱动模型（如Node.js的事件循环）、IO多路复用（如epoll）、信号/回调通知（如Windows IOCP）、以及协程与异步结合（如Python的asyncio）。这种机制能够显著提升并发性能，降低线程开销，尤其适合构建高吞吐量、高并发的网络系统。

答案-问题（1）

(1) g (2) d (3) a (4) f (5) b (6) h (7) i (8) e (9) c



答案-问题（2）

推荐使用图数据库（Graph Database），如 Neo4j、Amazon Neptune 等，来存储医药知识图谱数据。因为医药领域的知识结构普遍基于三元组，图数据库能以节点和边的形式自然建模实体及其复杂关系，尤其适合表示药物相互作用、疾病关联等语义网络。图数据库还支持高效的关系遍历、多跳查询和图算法分析，如最短路径、社区发现等，有助于深入挖掘潜在医学关联。此外，Neo4j 提供的 Cypher 查询语言便于灵活表达医学图谱中的复杂语义关系。

答案-问题（3）

（1）Scheduler（调度器）（2）Scrapy Engine（引擎）（3）Item Pipeline（项目管道）

在异步IO模式下，程序向操作系统发起I/O请求（如读文件、收网络数据）后，不会阻塞等待结果返回，而是立即返回并继续执行其他逻辑。当I/O操作完成后，操作系统会通过回调函数、事件通知或信号等方式通知程序，程序再去处理结果。

（2026年5月系统架构设计师真题-第4题）

阅读以下关于学习路径推荐与智能辅助系统的叙述，在答题纸上回答问题1-3。

【说明】

某在线教育企业计划建设一套智能学习系统，为软考、职业资格考试和企业内训用户提供个性化学习服务。系统需要根据用户的基础水平、学习目标、做题记录、课程学习行为和阶段测评结果，动态生成学习路径，并在学习过程中提供智能答疑、错因分析、课程推荐、学习提醒和效果反馈。

项目组在设计系统时，将用户的学习过程抽象为若干状态。用户进入系统后，首先处于“学习进行中”状态。在该状态下，系统持续记录用户的课程观看、做题情况、知识点掌握度和学习时长等信息。当用户主动结束本次学习，或系统检测到学习任务完成后，用户可以进入“退出学习”状态。

在学习过程中，系统会定期进行信息采集，包括用户画像、学习行为、测评结果、题目作答、错题记录和学习资源使用情况。系统根据采集到的数据，结合知识图谱、用户画像模型、掌握度模型和推荐模型，对用户当前学习情况进行分析，并向智能辅导模块发起课程推荐。智能辅导模块根据用户的薄弱知识点、前置依赖关系、目标考试要求和学习进度，推荐相应课程、练习题和复习资源。

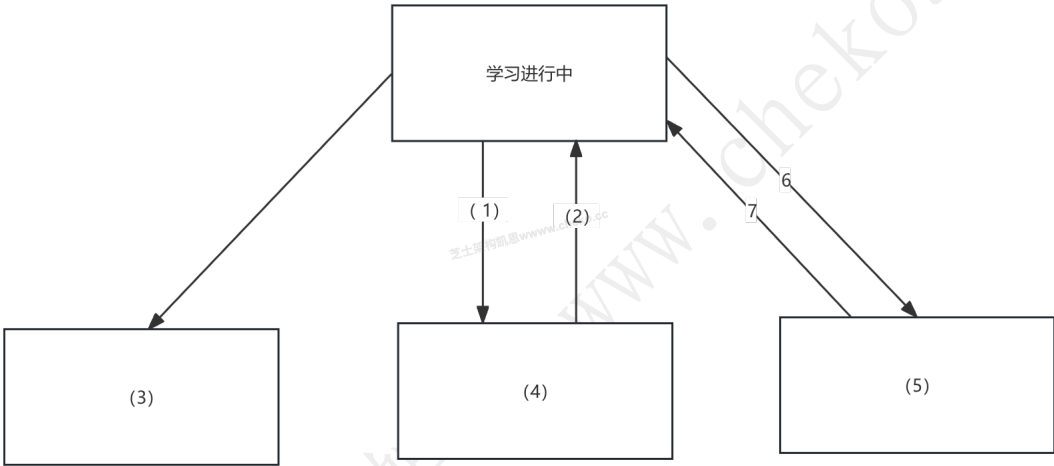
当用户在学习过程中遇到问题时，可以进入智能辅导状态。智能辅导模块支持智能答疑、错因分析、学习提醒、资源匹配和个性化讲解。完成辅导后，系统会根据用户新的学习行为和反馈结果，重新调整学习建议，并返回学习进行中状态。

系统还设置了效果评估环节。用户完成阶段学习后，系统会进入效果评估状态，对用户的阶段测评成绩、知识点掌握度、错题变化、学习完成率和课程吸收情况进行分析。效果评估结果一方面会形成效果反馈，用于调整后续学习策略；另一方面也会作为学习评估结果反馈给学习过程，用于判断用户是否需要继续巩固、重新学习或进入下一阶段内容。

系统试运行后发现两个问题：一是新注册用户缺少历史学习行为和测评数据，系统推荐准确率较低；二是部分基础薄弱学生被推荐了过多高级知识点，跳过了必要的前置基础内容，导致学习路径过难。项目组需要从冷启动、知识依赖推理、数据质量和推荐鲁棒性等角度分析原因并提出改进思路。

问题（1）

根据题干内容填写下面图片中空白部分。



思路解析

本问考查的是个性化学习系统的闭环设计，围绕“学习进行中”这个核心状态，不断完成信息采集、智能辅导、效果评估和反馈调整。

在学习进行中状态下，系统需要持续进行信息采集。这里的信息采集不是泛泛地收集用户数据，而是要覆盖用户画像、学习目标、基础水平、课程观看记录、练习作答情况、错题记录、阶段测评结果、知识点掌握度、学习时长、资源使用情况和反馈评价等内容。这些数据分别回答“用户是谁”“要学什么”“现在会什么”“学过什么”“哪里薄弱”“资源是否有效”等问题。若系统只采集点击次数和观看时长，就很难判断用户是否真正掌握了知识点，也难以支撑后续推荐。

系统采集到学习数据后，需要进入智能辅导环节。智能辅导中更关注学习过程中的即时支持，例如智能答疑、错因分析、相似题推荐、薄弱知识点提醒、课程推荐、学习计划提醒和阶段测评建议。它解决的是“用户当前卡在哪里、下一步该补什么、用什么资源补”的问题。比如用户在“软件架构风格”相关题目上连续出错，系统不应只推荐更多同类难题，而应先分析其是否没有掌握“构件、连接件、约束”等基础概念，再推荐管道-过滤器、事件驱动、黑板架构等进阶内容。

学习路径推荐则更偏路线生成，它不是普通的内容推荐。路径推荐需要结合目标知识点、前置依赖、难度梯度、掌握阈值、考试时间、用户基础和学习偏好，生成从基础到进阶的学习顺序，并随着新的作答数据和测评结果动态调整。对于基础薄弱的用户，系统应优先补齐先修知识，再逐步进入综合案例、论文素材和高阶应用，而不能直接推送高级知识点，否则容易造成学习路径过难。

效果评估是闭环中的关键环节。用户完成阶段学习后，系统需要根据阶段测评成绩、知识点掌握度变化、错题减少情况、课程完成率、学习活跃度和资源反馈，对学习效果进行判断。效果评估结果一方面形成效果反馈，用于调整后续辅导内容和推荐策略；另一方面形成学习评估，用于判断用户是否需要继续巩固、重新学习或进入下一阶段。这样，系统才能形成“信息采集—智能辅导—学习推荐—效果评估—反馈调整”的闭环。

问题（2）

新用户推荐准确率低的原因是什么？除用户冷启动外，还可能有哪些冷启动？

思路解析

冷启动的本质是缺少可用于建模和推荐的历史数据。新注册用户没有课程浏览、做题记录、错题分布、阶段测评和资源反馈，系统无法准确判断他的基础水平、学习目标、薄弱知识点和学习偏好，因此初始画像和掌握度估计不稳定，推荐准确率自然偏低。对软考或职业资格考试场景来说，用户可能有工作经验但缺少考试训练，也可能刚入门但点击了高级课程，如果没有入门测评和目标问卷，模型很容易把少量行为误判为真实能力。

除用户冷启动外，还可以写物品冷启动、知识点冷启动和领域/系统冷启动。物品冷启动是新课程、新题目、新资料缺少点击、作答、收藏、完课率和评价数据，系统不知道它适合哪类用户。知识点冷启动是新增考试大纲、新增技术主题或新建知识点缺少前置依赖、难度标注和题目映射，导致路径中位置不稳定。领域/系统冷启动是平台刚上线、切换到新学科或新考试类别时，整体用户行为和交互数据不足。

缓解思路包括入门诊断测评、目标和时间问卷、专家维护知识图谱、基于规则的初始路径、热门资源或基础资源兜底、相似用户迁移、内容特征推荐，以及适度探索曝光。答题时只写“数据少”不够，应说明少了哪些数据、影响了哪些模型，以及可以如何补数据或用规则兜底。

从系统设计角度看，冷启动不能完全依赖模型自己学习，因为初期数据不足时模型输出不稳定。更稳的做法是用“测评 + 规则 + 专家知识 + 少量探索”建立初始画像。例如新用户注册时选择考试科目、备考时间和基础水平，再做10到20道诊断题，系统根据知识点覆盖情况生成初始掌握度；新课程上线时由教研人员标注知识点、难度、适用阶段和先修要求；新知识点加入大纲时先由专家维护依赖关系，再用用户数据逐步校正。这样冷启动阶段有规则兜底，数据积累后再逐步交给推荐模型优化。

问题（3）

基础薄弱学生得到的路径过难，系统过度推荐高级知识。分别从路径推理和数据角度分析鲁棒性差的原因。

思路解析

本问要把路径推理问题和数据问题分开分析。路径推理角度看，基础薄弱学生被推荐大量高级知识，通常说明系统没有严格使用知识点前置依赖和拓扑顺序。例如在学习架构设计时，用户还没有掌握质量属性、架构风格和 UML 基础，系统却直接推荐 ATAM、DSSA 或微服务治理，就会造成路径过难。合理路径应先检查前置知识掌握阈值，再按难度梯度逐步推进，并在测评不达标时回退到基础知识或补充练习。若系统只按热度、相似用户点击或高级课程转化率推荐，就会把“别人常学”误当成“当前用户适合学”。

数据角度看，鲁棒性差可能来自作答数据稀疏、测评样本不足、知识点标签错误、题目难度标注不准、课程资源标签过粗、历史行为不能真实反映掌握度、训练样本偏向中高水平用户等问题。行为数据尤其容易误导模型，用户点开高级课程不代表能够学习，高观看时长也可能是反复听不懂。若错题、错因和阶段测评没有及时回流，模型会低估基础缺口。

改进可从两方面写：推理侧建立知识图谱和先修依赖约束，设置掌握阈值、难度梯度、路径回退和补救学习机制；数据侧提高知识点标注质量，增加诊断测评和错因标签，对低基础用户单独建模

，对异常行为降权，并用人工规则为关键基础知识设置硬约束。所谓鲁棒性，就是在数据稀疏、标签有噪声、用户基础很弱时，系统仍能给出保守、可完成、能逐步提升的学习路径。

这里的“鲁棒性”可以理解为推荐系统面对异常、稀疏或低质量输入时仍能给出合理结果。对基础薄弱用户，系统应宁可保守推荐基础补救内容，也不能因为点击了高级课程就跳过先修知识。工程上可设置硬性路径约束：关键前置知识未达阈值时，高级知识点只允许预览或少量探索，不进入主路径；连续错题时触发路径回退；测评样本不足时降低模型置信度，优先采用规则路径。数据侧则要做标签审核、题目难度校准、异常行为过滤和分层评估，分别观察低基础、中等基础和高基础用户的推荐效果，避免整体准确率掩盖低基础群体体验差的问题。

答案-问题（1）

- （1）信息采集
- （2）课程推荐
- （3）退出学习
- （4）智能辅导中
- （5）效果评估
- （6）效果反馈
- （7）学习评估

答案-问题（2）

新用户推荐准确率低的主要原因是缺少历史学习行为、测评数据、作答记录、偏好信息和目标约束，系统无法准确判断其基础水平、薄弱知识点和学习偏好，因此用户画像和掌握度模型不稳定。

除用户冷启动外，还可能存在物品冷启动，例如新课程、新题目、新资料缺少点击、作答和评价数据；知识点冷启动，例如新增考试大纲或新增知识点缺少依赖关系和标注样本；系统/领域冷启动，例如平台刚上线、换到新学科或新考试类别时，整体交互数据不足。改进思路包括入门测评、兴趣/目标问卷、基于规则的初始路径、利用相似用户迁移、热门资源兜底和专家标注知识图谱。

答案-问题（3）

路径推理角度：可能没有严格建模前置知识依赖和拓扑顺序，没有设置基础知识掌握阈值，也没有把知识点难度梯度纳入约束，导致系统在推荐高级知识点前没有检查用户是否掌握必要前置内容。还可能只按资源热度或相似用户行为推荐，忽略了个体基础差异。

数据角度：可能存在作答数据稀疏、测评样本不足、知识点标签错误、题目难度标注不准、历史行为不能真实反映掌握度、训练样本偏向中高水平用户等问题。如果错题和薄弱知识点没有及时回流，模型也会低估基础知识缺口。

改进可从两方面入手：推理侧建立知识图谱和前置依赖约束，设置掌握阈值和路径回退机制；数据侧提高知识点标注质量，增加诊断测评，对低基础用户单独建模，并将作答、错因和学习反馈及时回流。

第33章——Scrum 与敏捷项目管理 (2016年11月系统架构设计师真题-第5题)

阅读以下关于Scrum敏捷开发过程的叙述，在答题纸上回答问题1至问题3。

【说明】

Scrum是一个增量的、迭代的敏捷软件开发过程。某软件公司计划开发一个基于Web的Scrum项目管理系统，用于支持项目团队采用Scrum敏捷开发方法进行软件开发，辅助主管智能决策。此项目管理系统提供的主要服务包括项目团队的管理、敏捷开发过程管理和工件的管理。

Scrum敏捷开发中，项目团队由Scrum主管、产品负责人和开发团队人员三种不同的角色组成，其开发过程由若干个Sprint（短的迭代周期，通常为2到4周）活动组成。

Product Backlog是在Scrum过程初期产生的一个按照商业价值排序的需求列表，该列表条目的体现形式通常为用户故事。在每一个Sprint活动中，项目团队从Product Backlog中挑选最高优先级的用户故事进行开发。被挑选的用户故事在Sprint计划会议上经过细化分解为任务，同时初步估算每一个任务的预计完成时间，编写Sprint Backlog。

在Sprint活动期间，项目团队每天早晨需举行每日站立会议，重新估算剩余任务的预计完成时间，更新Sprint Backlog、Sprint燃尽图和Release燃尽图。在每个Sprint活动结束后，项目团队召开评审会议和回顾会议，交付产品增量，总结Sprint期间的工作情况和问题。此时，如果Product Backlog中还有未完成的用户故事，则项目团队将开始筹备下一个Sprint活动迭代。

为完成Scrum项目管理系统，考虑到系统的智能决策需求，公司决定使用MVC架构模式开发该项目管理系统。具体来说，系统采用轻量级J2EE架构和SSH框架进行开发，使用MySQL数据库作为底层存储。

问题（1）

Scrum项目管理系统需真实模拟Scrum敏捷开发流程，请根据你的理解完成图5-1给出的Scrum敏捷开发状态图，填写其中（1）~（5）的内容。

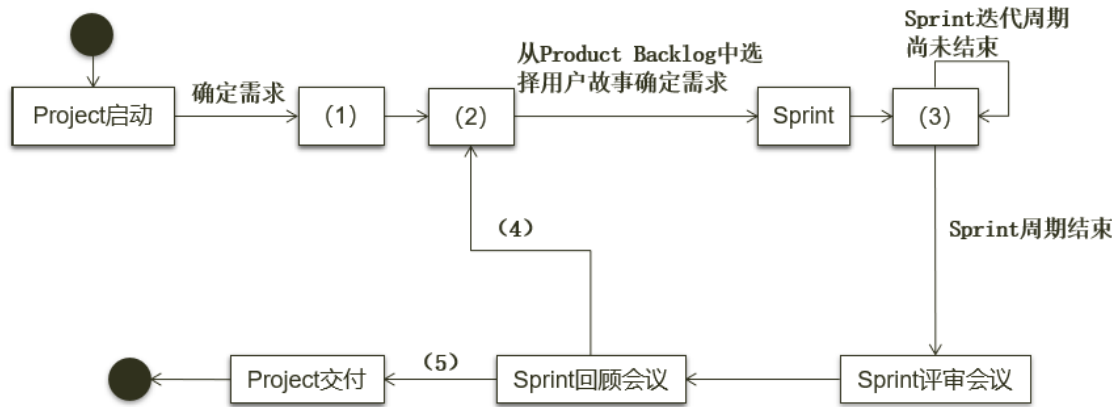


图5-1 Scrum敏捷开发状态图

思路解析

状态图是 UML（统一建模语言）中用来描述对象动态行为的重要建模工具，它通过“状态”和“事件”来表现一个对象在其生命周期内可能经历的变化。一个对象通常经历多个状态，这些状态通过触发事件相互转换。在软件工程案例题中，状态图经常用于描述业务流程或系统运行流程。

在题目中，Scrum 的敏捷开发过程本身就是一个迭代、循环的过程，因此非常适合用状态图来表现：

- （1）制定Product Backlog: 这是整个过程的起点，Scrum 项目必须先建立一个用户故事驱动的需求清单（Product Backlog），它决定了后续迭代要完成的内容。
- （2）Sprint计划会议: 这是 Product Backlog 向实际开发落地的第一步，团队挑选高优先级用户故事并分解为任务，形成 Sprint Backlog。
- （3）每日站立会议: 这是迭代周期中最关键的日常活动，每天更新任务剩余时间并调整 Sprint Backlog。
- （4）Product Backlog中还有未完成的用户故事: 这是一个条件事件，如果在某次 Sprint 完成后，仍然存在未完成的需求，系统将继续进入下一个 Sprint 迭代。
- （5）已交付Product Backlog中的所有用户故事: 这是终止条件事件，意味着系统需求已全部实现，Scrum 项目结束。

问题（2）

根据题干描述，本系统采用MVC架构模式，请从备选答案a~n中分别选出属于MVC架构模型中的模型（Model）、视图（View）和控制器（Controller）的相关内容描述填入表5-1的空（1）~（3）处。

架构模式	包含内容
模型（Model）	（1）
视图（View）	（2）
控制器（Controller）	（3）

备选答案：

a	Sprint燃尽图	h	用户
b	Project	i	交付产品增量
c	Product Backlog	j	新建项目
d	用户故事	k	Task
e	估算任务预计完成时间	l	Sprint
f	Release燃尽图	m	产品负责人
g	Sprint回顾会议	n	Sprint Backlog

思路解析

MVC（Model-View-Controller）是一种经典的架构模式，用于分离表示逻辑、业务逻辑和数据逻辑。软考经常考察考生对 MVC 三层之间的理解。

Model（模型层）：负责应用的业务逻辑和数据处理，包括实体类、业务对象、数据库交互类等。在题干的选项里，b、c、d、h、k、l、m、n都是数据对象或逻辑处理类，因此属于模型。

View（视图层）：负责数据展示，是用户看到的 UI 部分。一般与图形界面、报表、页面直接相关。a、f带有“界面”或“图”的词汇，显然属于视图层。

Controller（控制器层）：负责接收用户请求，并调用模型与视图完成交互。控制器通常带有动词特征，例如“管理”“处理”“调度”。因此，e、j最符合控制器层的特征。

解题技巧：

视图（View）→ 与“图形”“界面”直接相关。

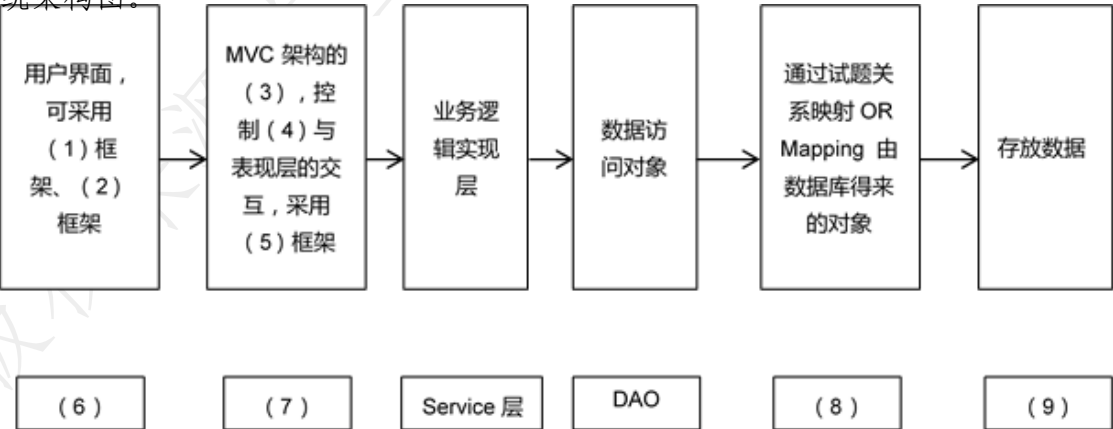
控制器（Controller）→ 动作 + 名词，如“会议调度器”“任务处理器”。

模型（Model）→ 实体类、数据、对象。

g 和 i 没被选入，是因为它们虽然看起来与流程相关，但题干中说明系统是**流程管理工具**，会议和交付属于过程事件，而不是系统要管理的具体对象，因此不属于 MVC 中的三部分。

问题（3）

根据项目组给出的系统设计方案，将备选答案a~l的内容填写在图5-2中的空（1）~（9），完成系统架构图。



备选答案：

左编号	左内容	右编号	右内容
a	Struts 2	g	模型层
b	Hibernate 持久层	h	控制层
c	数据库服务 (MySQL)	i	EJB
d	Sitemesh	j	Web 层
e	业务逻辑层	k	视图层
f	JQuery	l	PostgreSQL

思路解析

本题结合了 MVC 模式 + J2EE 分层架构 + SSH 框架 的应用，要求考生能够把架构图与框架组件一一对应。

在题干中已经明确：系统采用轻量级 J2EE 架构、SSH (Struts + Spring + Hibernate) 框架、MySQL 数据库。因此我们需要将备选项逐层填入系统架构图：

(1) = d Sitemesh: “用户界面，可采用 (1) 框架、(2) 框架。” Sitemesh 是典型的页面装饰/布局框架，负责视图拼装与统一外观，属于表现层可用的 UI 框架之一，放在最左侧“用户界面”恰当。

(2) = f JQuery: JQuery 用于前端交互与 DOM 操作，同样归属表现层 UI 能力，与 Sitemesh 搭配覆盖“外观+交互”。

(3) = h 控制层: “MVC 架构的 (3)，控制 (4) 与表现层的交互，采用 (5) 框架。”——这里 (3) 应是 MVC 的 Controller，承担“协调视图与后端”的职责。

(4) = e 业务逻辑层: 句子里“控制 (4) 与表现层的交互”中的 (4) 就是被 Controller 协调的一端，即业务逻辑层 (Service)。Controller 接收请求→调用业务→返回视图模型，语义完全吻合。

(5) = a Struts2: 在“采用 (5) 框架”位置放 Struts2，正是用于实现 MVC 的控制层 (Action/拦截器链等)，历史上与 Sitemesh、JQuery 常作组合。

(6) = k 视图层: 图下方有一排“分层标签”，紧贴最左“用户界面/表现层”方块的就是视图层。将 (6) 标为 View (视图层) 与上面的 Sitemesh、JQuery 对应。

(7) = j Web 层: 紧邻 MVC 方块下方的标签应是 Web 层 (承载 Controller、过滤器、拦截器等 Web 端职责)，因此 (7) 选 j Web 层最合适。

(8) = b Hibernate 持久层: 对应图中“通过对象关系映射 (OR-Mapping) 由数据库对象映射为领域对象”的方块，Hibernate 正是典型的 ORM/持久化实现。

(9) = c 数据库服务 (MySQL): 最右端“存放数据”为数据库层；在备选 MySQL/PostgreSQL 二选一，题面常见默认搭配是 Hibernate + MySQL，因此选 c。

答案-问题 (1)

- (1) 制定 Product Backlog
- (2) Sprint 计划会议
- (3) 每日站立会议
- (4) Product Backlog 中还有未完成的用户故事
- (5) 已交付 Product Backlog 中的所有用户故事

答案-问题 (2)

- (1) b、c、d、h、k、l、m、n
- (2) a、f
- (3) e、j

答案-问题 (3)

- (1) d
- (2) f
- (3) h
- (4) e
- (5) a
- (6) k
- (7) j
- (8) b
- (9) c